
APLICACIONES WEB

Compendio de tecnologías web del lado del cliente y del servidor: historia, fundamentos, prácticas paso a paso y buenas prácticas profesionales

Libro de texto académico

Edición en cuatro unidades — Capítulos 1 a 18

Unidad I: *Fundamentos y Diseño Web*

Unidad II: *Lenguajes, Frameworks y Seguridad del Servidor*

Unidad III: *Datos y Patrones Arquitectónicos*

Unidad IV: *Patrón MVC y Servicios Web*

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Quinto nivel — Período Académico 2026–2027 PPA

Autor docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Quevedo, Los Ríos — Ecuador

Junio de 2026

APLICACIONES WEB — Edición en cuatro unidades (Capítulos 1 a 18)

Universidad Técnica Estatal de Quevedo, 2026.

Facultad de Ciencias de la Computación.

Carrera de Ingeniería de Software (Rediseño).

Esta obra reúne las cuatro unidades de la asignatura en un único volumen. Incorpora un tratamiento profundo de la historia del desarrollo web, fundamentos teóricos extensos, ejercicios paso a paso plenamente reproducibles en IntelliJ IDEA Ultimate —y en Visual Studio Code para ASP.NET Core—, tablas comparativas de versiones de tecnologías y secciones dedicadas a buenas prácticas profesionales para cada tecnología abordada. Su contenido se ha alineado con el SWEBOK Guide v4.0 [68], las directrices del W3C [73], la guía OWASP Top 10:2021 [52] y las especificaciones HTML Living Standard del WHATWG.

Uso académico no comercial. Se permite la reproducción total o parcial citando la fuente.

Prefacio

Este libro responde a una reflexión pedagógica concreta: el desarrollo web moderno no se domina memorizando sintaxis, se domina comprendiendo el **porqué histórico** de cada decisión técnica, las **razones científicas** detrás de cada práctica y los **caminos prácticos** que llevan del concepto al artefacto funcional.

A diferencia de un manual de referencia, esta obra incorpora cuatro elementos pedagógicos sistemáticos en cada capítulo:

1. **Cajas de historia** que sitúan cada tecnología en su contexto histórico, mencionando autores, fechas, hitos y motivaciones originales que rara vez aparecen en los manuales técnicos pero que resultan indispensables para comprender por qué las cosas son como son.
2. **Tablas de versiones** que documentan la evolución de cada tecnología con las mejoras sustantivas que cada revisión aportó.
3. **Cajas de buenas prácticas** que destilan en pautas accionables el conocimiento profesional acumulado por la industria.
4. **Ejercicios paso a paso ampliados** con instrucciones tan detalladas que cualquier estudiante puede reproducir el resultado exacto descrito en el texto. Cada paso explica *qué* se hace, *por qué* se hace y *cómo verificar* que el resultado es correcto.

El entorno de referencia es **IntelliJ IDEA Ultimate** para HTML5, CSS3, JavaScript moderno, Perl, PHP, Python/Django y Java/Spring Boot, y **Visual Studio Code** para ASP.NET Core con C#, dado que el soporte de .NET no forma parte de IntelliJ IDEA. Todas las interfaces de usuario del libro se construyen con HTML5 semántico y CSS3, reutilizando una hoja de estilos base común (`estilos.css`) que se define en esta primera unidad; en los capítulos siguientes solo se muestran las reglas nuevas que cada ejercicio añade.

Organización del libro

El libro se organiza en cuatro unidades que se corresponden con las cuatro unidades curriculares de la asignatura:

Unidad	Capítulos
I — Fundamentos y Diseño Web	1–4
II — Lenguajes, Frameworks y Seguridad del Servidor	5–9
III — Datos y Patrones Arquitectónicos	10–13
IV — Patrón MVC y Servicios Web	14–18

Cada capítulo combina, de forma sistemática, una contextualización histórica, el desarrollo teórico riguroso del tema, prácticas reproducibles y un bloque de buenas prácticas profesionales, además de los ejercicios y, donde corresponde, las evaluaciones parciales y los hitos del Proyecto Final de Curso (PFC).

El autor docente.

Quevedo, junio de 2026.

Índice general

Prefacio	iii
Organización del libro	III
 I Fundamentos y Diseño Web	 1
1. Encuadre académico y fundamentos de las aplicaciones web	2
1.1. Encuadre académico de la asignatura	2
1.1.1. Ubicación curricular y prerequisites	2
1.1.2. Articulación con el SWEBOK Guide v4.0	3
1.1.3. Competencias por desarrollar	3
1.1.4. Sistema de evaluación	4
1.2. Historia de las aplicaciones web	4
1.2.1. Las tres eras de la web	5
1.2.2. Evolución temporal de los hitos web	6
1.3. Fundamentos de las aplicaciones web	7
1.3.1. Definición y clasificación de aplicaciones web	8
1.3.2. Características de las aplicaciones web modernas	9
1.3.3. Arquitectura cliente-servidor en aplicaciones web	9
1.3.4. Ventajas y desventajas de las aplicaciones web	14
1.4. Diseño web inclusivo y accesibilidad	14
1.4.1. ¿Qué es la accesibilidad web?	15
1.4.2. Las cuatro categorías de discapacidad relevantes para la web	16
1.4.3. Las Web Content Accessibility Guidelines 2.1 (WCAG 2.1)	17
1.4.4. Criterios de éxito clave de WCAG 2.1	19
1.5. Configuración del entorno con IntelliJ IDEA Ultimate	21
1.6. Buenas prácticas profesionales para HTML semántico	24
1.7. Tabla de referencia rápida: tags HTML5 más usados, atributos y valores	25
1.7.1. Atributos globales (aplican a casi todos los tags)	27
1.7.2. Tags estructurales y de contenido	27
 2. HTML y HTML5 para el desarrollo web	 29
2.1. Historia y evolución de HTML	29

2.2.	Fundamentos de HTML	30
2.2.1.	Estructura mínima de un documento HTML5 válido	30
2.2.2.	Las cinco categorías de contenido en HTML5	33
2.2.3.	Modelo de contenido y reglas de anidamiento	34
2.3.	Elementos semánticos de HTML5 en profundidad	35
2.3.1.	Catálogo de elementos semánticos	35
2.3.2.	¿<article> o <section>? Guía decisional con ejemplo real	35
2.4.	Formularios HTML5 y validación nativa	37
2.4.1.	Tipos de input modernos	37
2.4.2.	Atributos de validación	37
2.5.	Multimedia y APIs de HTML5	39
2.5.1.	La etiqueta <video> con accesibilidad completa	40
2.5.2.	El elemento <picture> para imágenes responsivas	41
2.6.	XML y familias de lenguajes de marcado	42
2.6.1.	Subdialectos de XML relevantes en aplicaciones web	42
2.6.2.	Formatos de intercambio de datos: XML, JSON, YAML, TOML y otros	43
2.7.	Buenas prácticas profesionales en HTML5	44
2.8.	Ejercicios	45
3.	CSS para el diseño y la presentación web	48
3.1.	Historia y evolución de CSS	48
3.1.1.	Tabla evolutiva de CSS	48
3.2.	Fundamentos de CSS	49
3.2.1.	Las tres formas de aplicar estilos	49
3.2.2.	Selectores principales	49
3.2.3.	Pseudo-clases CSS para feedback de validación	50
3.3.	Sistemas modernos de layout	50
3.3.1.	Flexbox: layout unidimensional	51
3.3.2.	CSS Grid: layout bidimensional	53
3.4.	Sass: el preprocesador profesional	53
3.5.	Ejercicios	54
3.6.	Buenas prácticas CSS profesional	54
3.7.	La hoja de estilos base del libro: <code>estilos.css</code>	54
4.	JavaScript moderno y programación asíncrona	57
4.1.	Historia de JavaScript	57
4.1.1.	Tabla evolutiva de ECMAScript	57
4.2.	Fundamentos modernos	58
4.2.1.	Variables: por qué <code>let/const</code> y nunca <code>var</code>	58
4.2.2.	Funciones flecha y funciones regulares	58
4.2.3.	Destructuring y spread	59
4.3.	Manipulación del DOM	59
4.4.	Programación asíncrona: <code>fetch</code> , <code>Promises</code> , <code>async/await</code>	61

4.5.	Ejercicios	62
4.6.	Buenas prácticas JavaScript	62
II	Lenguajes, Frameworks y Seguridad del Servidor	63
5.	Introducción a los lenguajes del lado del servidor	64
	Resultado de aprendizaje	64
5.1.	El lado del servidor y el ciclo petición–respuesta	64
5.2.	Perl	65
5.3.	PHP	65
5.4.	Python	66
5.5.	ASP.NET Core (C#)	67
5.6.	Java para la Web	68
6.	CRUD sin framework: acceso a datos a mano	70
	Resultado de aprendizaje	70
6.1.	El modelo de datos común	70
6.2.	Nuevo CSS para el listado	71
6.3.	CRUD en Perl con DBI	71
6.4.	CRUD en PHP con PDO	73
6.5.	CRUD en Python con psycopg (sin framework)	74
6.6.	CRUD en ASP.NET Core con ADO.NET (sin EF)	76
6.7.	CRUD en Java con JSP, Servlets, JSTL y EL (JDBC)	78
6.8.	Síntesis del capítulo	80
III	Datos y Patrones Arquitectónicos	81
10.	Gestión de estado y objetos integrados en aplicaciones web	82
10.1.	Introducción: el dilema fundamental de HTTP	82
	10.1.1. Las cinco estrategias de gestión de estado	83
	10.1.2. Los flags de seguridad de cookies en 2026	83
10.2.	El objeto Request: lectura segura de la petición	84
	10.2.1. Componentes del objeto Request	84
	10.2.2. Lectura del Request en Java Servlets / Spring Boot	86
	10.2.3. Lectura del Request en ASP.NET Core 10	87
10.3.	El objeto Response: control total de la respuesta HTTP	88
	10.3.1. Códigos de estado HTTP — la guía exhaustiva	88
	10.3.2. Errores estandarizados con RFC 7807 ProblemDetails	88
	10.3.3. Cabeceras de seguridad obligatorias en producción	89
10.4.	El objeto Session: estado entre peticiones	91
	10.4.1. Mecánica detallada de las sesiones de servidor	91
	10.4.2. Implementación comparada en las cuatro plataformas	91

10.4.3.	Sesiones en PHP 8.4 con configuración profesional	91
10.5.	Almacenamiento del cliente: localStorage, sessionStorage e IndexedDB	95
10.5.1.	Comparativa de las APIs de almacenamiento del cliente	95
10.5.2.	Ejemplos prácticos de almacenamiento	95
10.6.	Práctica integrada: carrito de compras con sesiones	97
10.7.	Buenas prácticas profesionales en gestión de estado	100
10.8.	Ejercicio resuelto adicional con resultado visual	101
11.	Acceso a datos desde aplicaciones web: SQL parametrizado y ORM	104
11.1.	Historia y panorama del acceso a datos	104
11.1.1.	Las cuatro estrategias de acceso a datos	105
11.1.2.	Tabla evolutiva de los ORM principales	105
11.2.	Consultas SQL desde aplicaciones web	105
11.2.1.	La amenaza fundamental: SQL Injection	105
11.2.2.	La defensa: sentencias preparadas en las tres plataformas	106
11.3.	El patrón Repository: encapsular el acceso a datos	112
11.3.1.	Repository típico en Spring Data JPA	112
11.4.	El problema N+1 y su solución	114
11.4.1.	Solución en cada ORM	114
11.5.	Migraciones de esquema con Flyway y Liquibase	115
11.6.	Procedimientos almacenados: cuándo aún tiene sentido	116
11.7.	Práctica integrada: CRUD ORM vs SQL puro	116
11.8.	Buenas prácticas profesionales en acceso a datos	121
11.9.	Ejercicio resuelto adicional con resultado visual	122
12.	Patrones de arquitectura para aplicaciones web	125
12.1.	Principios fundacionales de arquitectura de software	125
12.1.1.	Los principios SOLID aplicados a la arquitectura	125
12.1.2.	Atributos de calidad ISO/IEC 25010	126
12.2.	Patrón 1: Arquitectura por capas (3-capas y 4-capas)	126
12.2.1.	Estructura clásica	126
12.2.2.	Implementación canónica en Spring Boot	126
12.2.3.	Variante: Arquitectura Hexagonal (Ports and Adapters)	127
12.3.	Patrón 2: Arquitectura Orientada a Servicios (SOA)	128
12.4.	Patrón 3: Microservicios	128
12.4.1.	Comparativa SOA vs microservicios	128
12.4.2.	Cuándo NO usar microservicios	129
12.4.3.	Patrones esenciales de microservicios	129
12.5.	Patrón 4: Arquitectura dirigida por eventos (EDA)	129
12.6.	Patrón 5: Peer-to-peer (P2P)	130
12.7.	Documentación arquitectónica con el modelo C4 de Brown	130
12.7.1.	Los cuatro niveles del modelo C4	130
12.7.2.	Ejemplo: diagrama de contexto del PFC	130

12.7.3. Ejemplo: diagrama de contenedores del PFC	131
12.8. Architectural Decision Records (ADR)	131
12.9. Práctica integrada: documentar el PFC con C4 y ADRs	132
12.10. Buenas prácticas profesionales en arquitectura	135
12.11. Ejercicio resuelto adicional con resultado visual	135
13. Diseño de arquitecturas web escalables	138
13.1. Escalabilidad en aplicaciones web: vertical vs horizontal	138
13.1.1. Definiciones formales	138
13.1.2. El teorema CAP de Brewer	138
13.2. Estrategias de balanceo de carga	139
13.2.1. Algoritmos de balanceo	139
13.2.2. Implementaciones populares	140
13.3. Patrones de caché en aplicaciones web	141
13.3.1. Los cinco patrones canónicos de caché	141
13.3.2. Cache-aside con Redis: el patrón dominante	141
13.3.3. Estrategias de invalidación	143
13.4. Patrones avanzados: CDN, Edge caching y read replicas	143
13.4.1. CDN para contenido estático	143
13.4.2. Réplicas de lectura para BD	143
13.5. Métodos de evaluación arquitectónica	144
13.5.1. ATAM (Architecture Tradeoff Analysis Method)	144
13.5.2. Lightweight ATAM para equipos pequeños	144
13.6. Práctica integrada: implementar cache-aside con benchmark	144
13.7. Sumativa de la semana 13: Exposición Frameworks Backend	146
13.8. Buenas prácticas profesionales en escalabilidad	147
13.9. Contraste bibliográfico de los conceptos clave	148
13.10. Ejercicio resuelto adicional con resultado visual	148
IV Patrón MVC y Servicios Web	150
V Patrón MVC y Servicios Web	151
14. El patrón Modelo-Vista-Controlador en aplicaciones web modernas	152
14.1. Historia del patrón MVC	152
14.1.1. Tabla evolutiva del MVC en frameworks web	153
14.2. Anatomía del patrón MVC en la web	153
14.2.1. Adaptación del MVC original al contexto HTTP	153
14.2.2. Las tres capas en detalle	153
14.3. Ciclo de vida de una petición MVC en Spring Boot	154
14.4. Server-side MVC vs SPA + API REST	155
14.4.1. ¿Cuál usar en el PFC?	155

14.5.	Implementación canónica: MVC server-side completo	156
14.6.	Variantes del MVC	162
14.6.1.	MVP (Model-View-Presenter)	162
14.6.2.	MVVM (Model-View-ViewModel)	162
14.6.3.	Flux y Redux	162
14.6.4.	Signals (Angular 17+, SolidJS)	162
14.7.	Práctica integrada: CRUD MVC con debugging	162
14.8.	Buenas prácticas profesionales en MVC	165
14.9.	Contraste bibliográfico de los conceptos clave	166
14.10.	Ejercicio resuelto adicional con resultado visual	166
15.	Servicios web: arquitecturas REST, SOAP y diseño de APIs	169
15.1.	Historia de los servicios web	169
15.1.1.	Tabla evolutiva de los servicios web	170
15.2.	Las seis restricciones de la arquitectura REST	170
15.2.1.	Modelo de Madurez de Richardson	170
15.3.	Diseño de URIs y verbos HTTP	171
15.3.1.	Convenciones de nomenclatura	171
15.3.2.	Mapeo verbo-acción canónico	172
15.3.3.	Idempotencia y seguridad	172
15.4.	Implementación de APIs REST en las tres plataformas	172
15.4.1.	Spring Boot 4 con Spring Web	172
15.4.2.	ASP.NET Core 10	175
15.4.3.	PHP 8.4 con Slim Framework 4	177
15.5.	REST vs SOAP: la comparativa definitiva	178
15.5.1.	Cuándo aún tiene sentido SOAP en 2026	179
15.5.2.	Ejemplo SOAP mínimo	179
15.6.	Documentación con OpenAPI 3.0 (Swagger)	180
15.6.1.	¿Qué es OpenAPI?	180
15.6.2.	Configurar OpenAPI en Spring Boot	181
15.7.	Autenticación de APIs con JWT	181
15.7.1.	Anatomía de un JWT	181
15.7.2.	Flags estándar del payload (<i>claims</i>)	182
15.7.3.	JWT en el PFC: configuración Spring Security	182
15.8.	Práctica integrada: API REST consumiendo servicio externo	184
15.9.	Práctica integrada: PFC Entrega 2 (Semana 15)	190
15.10.	Buenas prácticas profesionales en APIs REST	191
15.11.	Contraste bibliográfico de los conceptos clave	191
15.12.	Ejercicio resuelto adicional con resultado visual	192
16.	Entrega final del PFC: pruebas, cobertura, seguridad y publicación	195
16.1.	Pruebas en aplicaciones web modernas	195
16.1.1.	Niveles de pruebas en aplicaciones web	195

16.1.2.	La pirámide de pruebas en detalle	196
16.2.	Pruebas unitarias con JUnit 5 y Mockito	196
16.3.	Pruebas de integración con MockMvc y Testcontainers	198
16.4.	Cobertura con JaCoCo	200
16.5.	Pruebas de carga con k6	201
16.6.	Auditoría de frontend con Lighthouse	202
16.6.1.	Las cuatro métricas core	202
16.6.2.	Cómo ejecutar Lighthouse	203
16.6.3.	CLI alternativa para CI/CD	203
16.7.	Auditoría de seguridad: 6 controles OWASP mínimos	203
16.8.	Práctica integrada: PFC Entrega Final v1.0.0	204
16.9.	Buenas prácticas profesionales en pruebas y entregas	205
16.10.	Contraste bibliográfico de los conceptos clave	205
16.11.	Ejercicio resuelto adicional con resultado visual	206
17.	Defensa del PFC y Evaluación Parcial Segundo Corte	209
17.1.	Cómo preparar la defensa oral del PFC	209
17.1.1.	Estructura recomendada de la presentación	209
17.1.2.	Las 20 preguntas más frecuentes del tribunal	210
17.1.3.	Errores comunes en defensas anteriores	210
17.2.	Estructura de la Evaluación Parcial Segundo Corte	211
17.2.1.	Temas garantizados en la EP2	211
17.2.2.	Ejemplos del tipo de preguntas que aparecen	212
17.3.	Preparación efectiva para la EP2	213
17.3.1.	Plan de estudio de 7 días previos al examen	213
17.3.2.	Estrategia durante el examen	213
17.4.	Reflexión sobre el segundo corte	214
17.5.	Contraste bibliográfico de los conceptos clave de la EP2	214
17.6.	Ejercicio resuelto adicional con resultado visual	214
18.	Tutoría de refuerzo final y cierre del curso	216
18.1.	Mapa integrador de la asignatura	216
18.1.1.	Las cuatro unidades como una historia coherente	216
18.1.2.	Tabla resumen de tecnologías dominadas	217
18.2.	Errores comunes detectados en las semanas 10–17	217
18.3.	Repaso intensivo de los temas más difíciles	218
18.3.1.	Tema 1: Diferencia REST vs SOAP	218
18.3.2.	Tema 2: JWT — estructura y uso correcto	218
18.3.3.	Tema 3: ORM y problema N+1	218
18.3.4.	Tema 4: Cache-aside con Redis	219
18.4.	Banco de ejercicios de consolidación	219
18.5.	Realimentación individual del PFC	220
18.5.1.	Criterios de excelencia observados	220

18.5.2.	Mejoras frecuentemente sugeridas	220
18.6.	El curso en perspectiva profesional	220
18.6.1.	Cómo se ve el desarrollo web profesional en 2026	221
18.6.2.	Próximos pasos en la formación	221
18.6.3.	Certificaciones que aportan valor profesional en 2026	221
18.7.	Conclusión del curso	221
18.8.	Cierre administrativo de la asignatura	222

Índice de figuras

1.1.	Las cuatro variantes principales de arquitectura cliente-servidor por número de niveles físicos. La elección depende del tamaño, presupuesto, requisitos de escalabilidad y disponibilidad. La mayoría de aplicaciones web modernas usan 3 o más niveles.	11
1.2.	Flujo simplificado de una petición web. Cada uno de estos seis pasos puede convertirse en cuello de botella; la disciplina de optimización web consiste en medir cada eslabón.	14
10.1.	Establecimiento de una sesión: el servidor genera un ID aleatorio, lo asocia con datos en su <i>Session Store</i> (memoria, Redis o BD) y lo envía al cliente como cookie con flags de seguridad.	91
12.1.	Los ocho atributos de calidad de la norma ISO/IEC 25010. Toda decisión arquitectónica supone un <i>trade-off</i> entre algunos de estos atributos.	126
12.2.	Arquitectura clásica de 4 capas para aplicaciones web. La dependencia siempre es descendente: la capa superior depende de la inferior, nunca al revés.	127
12.3.	Arquitectura hexagonal: el dominio (centro) define <i>ports</i> (interfaces) y los <i>adapters</i> (implementaciones) externos lo conectan con tecnologías concretas (REST, JPA, Redis). Permite cambiar el adaptador de persistencia sin tocar el dominio.	128
12.4.	Flujo en una arquitectura dirigida por eventos: los servicios publican eventos sin saber quién los consumirá; los consumers se suscriben a los eventos que les interesan.	130
12.5.	Los cuatro niveles del modelo C4. Cada nivel hace zoom sobre un elemento del nivel superior. La mayoría de equipos solo necesita los niveles 1, 2 y 3; el nivel 4 se genera automáticamente desde el código.	130
12.6.	Nivel 1 (Contexto) del modelo C4 para el PFC: usuarios y sistemas externos.	131
12.7.	Nivel 2 (Contenedores) del PFC: las unidades desplegadas y la tecnología de cada una.	131
12.8.	Diagrama C4 nivel 2 (contenedores) de TiendaUTEQ.	136
13.1.	Topología clásica con balanceador de carga: el LB recibe las peticiones del cliente y las distribuye entre N servidores de aplicación que comparten la BD.	139

13.2.	Patrón cache-aside: la aplicación consulta Redis primero; si hay HIT, devuelve. Si hay MISS, consulta PostgreSQL, guarda en Redis con TTL y devuelve. . .	141
14.1.	Ciclo de vida completo de una petición HTTP en Spring MVC. La ida (línea sólida): Dispatcher → HandlerMapping → Controller → Service → Repository → BD. El regreso (línea punteada): los datos suben hasta el Controller, que devuelve un <code>ModelAndView</code> ; el <code>ViewResolver</code> elige la plantilla y el HTML final llega al cliente.	155
16.1.	Pirámide de pruebas de Cohn: la base son muchas pruebas unitarias rápidas; la cumbre son pocas pruebas E2E lentas pero de alta confianza.	196
18.1.	Mapa integrador del curso: las cuatro unidades convergen en el Proyecto Fin de Curso. Las flechas verdes representan la dependencia secuencial entre unidades; las flechas doradas, la integración explícita hacia el PFC.	217

Índice de tablas

1.1.	Hitos seleccionados de la historia de las aplicaciones web.	7
1.2.	Características esenciales de las aplicaciones web modernas y sus implicaciones.	9
1.3.	Actores del flujo de una petición web — función, fallo típico y diagnóstico.	13
1.4.	Comparativa entre aplicaciones web, escritorio y móviles nativas.	14
1.5.	Tags HTML5 más usados: estructura, contenido y formularios.	25
1.6.	Atributos HTML5 globales.	27
2.1.	Hitos de la evolución de HTML.	30
2.2.	Elementos semánticos esenciales de HTML5 con su uso correcto e incorrecto.	35
2.3.	Tipos de <code><input></code> de HTML5 y su validación nativa.	37
2.4.	Formatos de intercambio de datos modernos.	43
2.5.	XML vs JSON — comparación detallada.	43
3.1.	Hitos en la evolución de CSS.	49
3.2.	Selectores CSS más usados.	50
4.1.	Versiones recientes de ECMAScript.	58
10.1.	Estrategias de gestión de estado en aplicaciones web modernas.	83
10.2.	Códigos de estado HTTP de uso obligado en aplicaciones web profesionales.	89
10.3.	Almacenamiento del cliente en navegadores modernos.	95
11.1.	Estrategias de acceso a datos en aplicaciones web (2026).	105
11.2.	Versiones actuales de ORM en 2026.	105
12.1.	SOLID, sus autores y significado en arquitectura.	126
12.2.	Patrones imprescindibles en arquitectura de microservicios.	129
13.1.	Comparativa: escalado vertical vs horizontal.	139
13.2.	Algoritmos de balanceo de carga.	139
13.3.	Patrones de caché y sus características.	141
14.1.	Hitos en la historia del MVC en frameworks web.	153
15.1.	Hitos en la historia de los servicios web.	170
15.2.	Modelo de Madurez de Richardson para APIs REST.	171

15.3.	Mapeo verbos HTTP a operaciones CRUD.	172
16.1.	Niveles de pruebas y tecnologías 2026.	196
16.2.	Métricas Core Web Vitals 2026.	202
16.3.	Los seis controles mínimos OWASP a verificar en el PFC.	203
17.1.	Composición de la Evaluación Parcial Segundo Corte.	211
18.1.	Tecnologías que el estudiante debe dominar al completar la asignatura. . . .	217
18.2.	Certificaciones de alto valor profesional para el desarrollador web 2026. . . .	221

Índice de listados de código

1.1.	Ejemplo de Shell	10
1.2.	index.html (Ejercicio 1.2)	22
1.3.	Ejemplo de HTML	27
1.4.	Ejemplo de HTML	27
2.1.	pagina-completa.html	31
2.2.	Ejemplo de HTML	34
2.3.	Ejemplo de HTML	34
2.4.	Ejemplo de HTML	34
2.5.	blog.html (extracto significativo)	35
2.6.	inscripcion.html (formulario funcional)	38
2.7.	Ejemplo de HTML	40
2.8.	bienvenida.es.vtt	41
2.9.	Ejemplo de HTML	41
2.10.	Ejemplo de XML	43
2.11.	Ejemplo de JavaScript	44
2.12.	Ejemplo de Código	44
2.13.	Ejemplo de Shell	44
2.14.	registro.html	45
3.1.	Ejemplo de HTML	49
3.2.	Ejemplo de HTML	49
3.3.	Ejemplo de CSS	50
3.4.	navbar.html	51
3.5.	navbar.css	51
3.6.	galeria.css (fragmento)	53
3.7.	ejemplo.scss	53
3.8.	estilos.css — parte 1: variables, reinicio y estructura	55
3.9.	estilos.css — parte 2: tablas, formularios, botones y responsivo	55
3.10.	Enlace de la hoja base en cualquier pagina	56
4.1.	Ejemplo de JavaScript	58
4.2.	Ejemplo de JavaScript	58
4.3.	Ejemplo de JavaScript	59
4.4.	tareas.html	59
4.5.	tareas.js	60

4.6.	api.js	61
5.1.	hola.pl — Hola Mundo en Perl (CGI)	65
5.2.	Estructura minima de un proyecto Perl/CGI	65
5.3.	index.php — Hola Mundo en PHP	65
5.4.	Estructura minima de un proyecto PHP	66
5.5.	app.py — Hola Mundo en Python (WSGI)	66
5.6.	Estructura minima de un proyecto Python/WSGI	67
5.7.	Creacion del proyecto ASP.NET Core	67
5.8.	Program.cs — Hola Mundo en ASP.NET Core	67
5.9.	Estructura creada por dotnet new web	67
5.10.	HolaServlet.java — Hola Mundo en Java web	68
5.11.	Estructura minima de un proyecto Java web	68
6.1.	Esquema compartido de la tabla estudiantes	70
6.2.	Reglas nuevas para la tabla del CRUD	71
6.3.	EstudianteDao.pm — parte 1: conexion y listar	71
6.4.	EstudianteDao.pm — parte 2: insertar, actualizar, eliminar	71
6.5.	lista.pl — vista de listado	72
6.6.	Estructura del CRUD Perl	72
6.7.	conexion.php — conexion PDO	73
6.8.	EstudianteDao.php — CRUD con PDO	73
6.9.	index.php — vista de listado	74
6.10.	Estructura del CRUD PHP	74
6.11.	dao.py — CRUD con psycpg	74
6.12.	app.py — vista de listado (WSGI)	75
6.13.	Estructura del CRUD Python	76
6.14.	EstudianteDao.cs — parte 1: conexion y listar	76
6.15.	EstudianteDao.cs — parte 2: insertar, actualizar, eliminar	77
6.16.	Program.cs — vista de listado	77
6.17.	Estructura del CRUD ASP.NET Core	78
6.18.	EstudianteDAO.java — parte 1: conexion y listar	78
6.19.	EstudianteDAO.java — parte 2: insertar, actualizar, eliminar	79
6.20.	lista.jsp — vista con JSTL y EL	79
6.21.	Estructura del CRUD Java web	80
10.1.	Ejemplo de Shell	83
10.2.	Ejemplo de Shell	83
10.3.	Ejemplo de PHP	84
10.4.	request-helpers.php	84
10.5.	RequestInspectorController.java	86
10.6.	Controllers/InspeccionController.cs	87
10.7.	Respuesta de error según RFC 7807	88
10.8.	Ejemplo de PHP	89
10.9.	Ejemplo de Java	90

10.10.	Ejemplo de C#	90
10.11.	config/session.php (incluir antes de session_start)	91
10.12.	login.php con todas las prácticas	92
10.13.	Ejemplo de JavaScript	95
10.14.	preferencias.js	96
10.15.	config.php	97
10.16.	index.php	98
10.17.	Ejemplo de Shell	99
10.18.	Program.cs	99
10.19.	application.yml	100
10.20.	Contador stateful con sesión segura	101
11.1.	Ejemplo de PHP	106
11.2.	Ejemplo de PHP	106
11.3.	UsuarioRepository.php	106
11.4.	Ejemplo de PHP	108
11.5.	UsuarioJdbcRepository.java	109
11.6.	UsuarioRepository.cs	111
11.7.	ProductoRepository.java	112
11.8.	Ejemplo de Java	114
11.9.	Ejemplo de PHP	114
11.10.	Ejemplo de C#	114
11.11.	Ejemplo de PHP	114
11.12.	Ejemplo de Shell	115
11.13.	V1__crear_usuarios.sql	115
11.14.	Ejemplo de YAML	115
11.15.	Ejemplo de Shell	117
11.16.	Ejemplo de Shell	117
11.17.	Ejemplo de SQL	117
11.18.	Ejemplo de YAML	118
11.19.	JdbcController.java	118
11.20.	Producto.java	119
11.21.	ProductoRepository.java + JpaController.java	120
11.22.	Ejemplo de Shell	121
11.23.	Entidad JPA con validaciones	122
11.24.	Repositorio Spring Data	122
11.25.	Endpoints REST	122
12.1.	Ejemplo de Shell	127
12.2.	Ejemplo de Shell	132
12.3.	Ejemplo de Shell	133
12.4.	Ejemplo de XML	133
12.5.	Ejemplo de Shell	134
12.6.	Ejemplo de YAML	134

12.7.	Ejemplo de Shell	135
13.1.	nginx.conf	140
13.2.	Ejemplo de YAML	141
13.3.	ProductoService.java	141
13.4.	Ejemplo de Shell	145
13.5.	Ejemplo de Shell	145
13.6.	Ejemplo de SQL	145
13.7.	Ejemplo de Shell	146
13.8.	Configuración Redis Spring Boot	148
13.9.	Servicio con anotaciones de caché	148
13.10.	Script k6 que evidencia el efecto del caché	149
14.1.	Ejemplo de Shell	156
14.2.	Producto.java	156
14.3.	ProductoForm.java	157
14.4.	ProductoRepository.java	157
14.5.	ProductoService.java	157
14.6.	ProductoController.java	158
14.7.	templates/productos/lista.html	160
14.8.	templates/productos/formulario.html	161
14.9.	Ejemplo de YAML	163
14.10.	Ejemplo de Java	163
14.11.	Ejemplo de Java	166
14.12.	Ejemplo de Java	166
14.13.	Ejemplo de HTML	167
15.1.	Ejemplo de Shell	171
15.2.	PedidoController.java	172
15.3.	GlobalExceptionHandler.java	174
15.4.	PedidosController.cs	175
15.5.	public/index.php	177
15.6.	Ejemplo de HTML	179
15.7.	consultar-ruc.php	180
15.8.	Ejemplo de XML	181
15.9.	Ejemplo de Java	181
15.10.	Ejemplo de Shell	181
15.11.	Ejemplo de JavaScript	182
15.12.	Ejemplo de JavaScript	182
15.13.	Ejemplo de Java	182
15.14.	Ejemplo de Java	183
15.15.	Ejemplo de Shell	184
15.16.	Ejemplo de YAML	185
15.17.	ExternalApiClient.java	185
15.18.	PostService.java	186

15.19.	PostController.java	187
15.20.	Ejemplo de Shell	189
15.21.	Ejemplo de Shell	190
15.22.	Dependencias Maven	192
15.23.	Configuración de Swagger	192
15.24.	Endpoint documentado	192
16.1.	ProductoServiceTest.java	196
16.2.	ProductoControllerIT.java	198
16.3.	Configuración de JaCoCo	200
16.4.	Ejemplo de Shell	201
16.5.	carga-productos.js	201
16.6.	Ejemplo de Shell	202
16.7.	Ejemplo de Shell	203
16.8.	Test unitario con Mockito	206
16.9.	Integración con MockMvc	207
17.1.	Ejemplo de PHP	212

Unidad I

Fundamentos y Diseño Web

Encuadre académico y fundamentos de las aplicaciones web

«La web no es un medio: es un ecosistema. Comprender sus fundamentos es comprender la infraestructura digital de la civilización contemporánea.»

— *Tim Berners-Lee, inventor de la World Wide Web*

Resultado de aprendizaje del capítulo

Al concluir el capítulo 1, el estudiante construye interfaces web semánticas, accesibles y responsivas **en su nivel introductorio**, integrando HTML5, CSS3 y JavaScript para crear aplicaciones del lado del cliente que cumplan con los estándares del W3C y los principios del diseño web inclusivo (WCAG 2.1) [73].

1.1 Encuadre académico de la asignatura

Antes de adentrarnos en los aspectos técnicos del desarrollo web, conviene situar la asignatura dentro del plan de estudios y aclarar los acuerdos que regirán el trabajo durante las dieciocho capítulos. Esta sección presenta tres aspectos institucionales fundamentales: la ubicación curricular de la materia y los conocimientos previos que se asumen; las modalidades formales de evaluación según la norma académica de la UTEQ; y las políticas profesionales que se aplicarán en clase (asistencia, entregas, integridad académica). La claridad sobre estos acuerdos al inicio del curso evita malentendidos posteriores y permite al estudiante planificar su esfuerzo con realismo.

1.1.1 Ubicación curricular y prerrequisitos

La asignatura Aplicaciones Web pertenece al campo de estudio de *Diseño de Software (DES.dd)* en la Carrera de Ingeniería de Software, ubicada en el quinto período académico de la malla curricular vigente dentro de la Unidad de Organización Curricular Profesional. Posee tres créditos académicos distribuidos en 144 horas totales, de las cuales 48 corresponden a clases presenciales

teóricas y prácticas, 48 al aprendizaje autónomo y 48 al contacto directo con el docente.

Sus prerequisites formales son *Base de Datos* y *Estructura de Datos*, ambos de tercer nivel. El primero garantiza que el estudiante maneje el modelo relacional y el lenguaje SQL como insumo indispensable para el acceso a datos del bloque servidor; el segundo asegura el dominio de listas, árboles, mapas hash y la complejidad algorítmica $O(n \log n)$ y $O(n^2)$, elementos sin los cuales no es posible diseñar APIs eficientes ni razonar sobre escalabilidad. La ausencia de cualquiera de estos prerequisites compromete severamente el aprovechamiento del curso.

1.1.2 Articulación con el SWEBOK Guide v4.0

El *Software Engineering Body of Knowledge* en su cuarta versión [68] reconoce explícitamente al desarrollo web como una de las áreas transversales de la práctica profesional. La asignatura articula los siguientes núcleos del SWEBOK: requisitos de software (KA02), diseño de software (KA03), construcción de software (KA04), calidad del software (KA10) y seguridad de software (KA15). El estudiante no construye únicamente código que «funciona»; construye software que satisface estándares internacionales de calidad, seguridad y accesibilidad.

1.1.3 Competencias por desarrollar

Esta asignatura forma al estudiante en cuatro frentes complementarios: la **capacidad técnica** de implementar interfaces y servicios web modernos; el **razonamiento arquitectónico** para tomar decisiones de diseño justificadas; la **conciencia de calidad** (seguridad, rendimiento, accesibilidad); y la **disciplina profesional** (versionado, pruebas, documentación). La caja siguiente recoge la formulación oficial de la competencia general.

Competencia general de la asignatura

Analizar, diseñar, implementar y evaluar aplicaciones web mediante tecnologías de diseño, lenguajes de programación cliente y servidor, patrones de arquitectura y servicios web, aplicando estándares de calidad, accesibilidad e inclusión, con el fin de construir *soluciones informáticas robustas, escalables y pertinentes* que respondan a las necesidades productivas, sociales y tecnológicas del Ecuador, de la región y del mundo.

Esta competencia general se descompone en cuatro resultados de aprendizaje unitarios (RAU):

- RAU 1.** Construir interfaces web semánticas, accesibles y responsivas con HTML5, CSS3 y JavaScript siguiendo W3C y WCAG 2.1.
- RAU 2.** Desarrollar aplicaciones web dinámicas del lado del servidor con PERL, PHP, ASP.NET y Java/JSP, implementando lógica de negocio, procesamiento de formularios y mecanismos de seguridad.
- RAU 3.** Diseñar e implementar soluciones web que integren gestión de estado, acceso a datos relacionales mediante SQL parametrizado y ORM, y patrones arquitectónicos escalables (multicapa, SOA, EDA, P2P).

RAU 4. Construir una aplicación web completa bajo el patrón MVC con un *framework* moderno, exponiendo e integrando servicios web REST y SOAP, y aplicando criterios de calidad, seguridad e interoperabilidad.

1.1.4 Sistema de evaluación

La calificación final se construye a partir de tres tipos de actividades ponderadas según la Norma Académica de la UTEQ: *Gestión del Aprendizaje (GA)*, *Tareas y Proyectos Autónomos (TA)* y *Evaluaciones (EV)*. El curso tiene una duración de 20 semanas, de las cuáles 18 son de clases y dos semanas de exámenes (19 y 20). El programa de la asignatura se desarrolla en 16 semanas, organizado en dos cortes evaluativos (semana 9 y 17) y un Proyecto Fin de Curso entregado en tres hitos parciales (semanas 5, 9 y 12) y uno final o completo en la semana 16.

1.2 Historia de las aplicaciones web

Comprender la web actual exige conocer su origen: las decisiones arquitectónicas tomadas en los años noventa (HTTP sin estado, URLs opacas, hipertexto distribuido) condicionan todavía hoy la forma en que diseñamos aplicaciones. La siguiente caja presenta los hitos fundacionales que conviene tener presente como contexto histórico permanente.

Los orígenes — del hipertexto a la World Wide Web

La idea del hipertexto precede a la web por décadas. Vannevar Bush concibió en 1945 el sistema *Memex* en el artículo «*As We May Think*» publicado en *The Atlantic Monthly*, una máquina capaz de almacenar libros y vincularlos mediante asociaciones siguiendo el modo en que opera la mente humana [10]. Ted Nelson acuñó el término *hipertexto* en 1965 dentro de su proyecto Xanadu, que aspiraba a crear un repositorio universal de documentos enlazados [42].

El paso decisivo lo dio Tim Berners-Lee en marzo de 1989 cuando, en el laboratorio del CERN en Ginebra, presentó el documento titulado «*Information Management: A Proposal*» [5]. Su intención era resolver un problema interno: la fuga de información cuando los investigadores abandonaban el centro. La propuesta combinó tres ingredientes existentes (el hipertexto, los protocolos de internet y el sistema de nombres de dominio) en un sistema nuevo: la World Wide Web.

A finales de 1990, Berners-Lee había desarrollado en una computadora NeXT los tres pilares fundacionales de la web: el primer servidor (CERN `httpd`), el primer navegador (WorldWideWeb, luego renombrado Nexus para evitar confusión con la red) y el primer documento HTML. El 6 de agosto de 1991 se publicó el primer sitio web público, alojado en `info.cern.ch`, que aún puede visitarse en su versión preservada en <http://info.cern.ch/hypertext/WWW/TheProject.html> [11].

1.2.1 Las tres eras de la web

La World Wide Web no es un fenómeno monolítico: ha atravesado tres grandes eras evolutivas que reflejan tanto cambios tecnológicos como transformaciones sociales profundas en cómo las personas interactúan con la información digital. Comprender estas eras es indispensable para situar el desarrollo web contemporáneo dentro de una trayectoria histórica significativa, y para anticipar hacia dónde se dirige.

Cada era se caracteriza por una combinación distintiva de: *(i)* el modelo de participación dominante (quién produce y quién consume el contenido), *(ii)* las tecnologías facilitadoras que hicieron posible esa participación, *(iii)* los modelos de negocio que emergieron y *(iv)* las consecuencias sociales y políticas observables. Berners-Lee, en su revisión retrospectiva de 2019, describió esta evolución como un «doloroso aprendizaje» porque ninguna era ha estado exenta de patologías propias: la Web 1.0 fue limitada en participación, la Web 2.0 concentró poder en plataformas, y la Web 3.0 enfrenta el desafío de la descentralización genuina [7].

Web 1.0 — la web de los documentos (1989–2004)

Esta primera era se caracterizó por el modelo *de solo lectura* (*read-only web*): una minoría de autores con conocimientos técnicos publicaba contenido, y la mayoría lo consumía pasivamente. Las páginas eran fundamentalmente estáticas, escritas a mano en HTML sin interactividad significativa. O'Reilly y Battelle [48] caracterizaron esta era como la del «sitio web folleto»: una transposición digital del medio impreso.

Los navegadores dominantes evolucionaron desde el pionero Mosaic (1993), liderado por Marc Andreessen y Eric Bina en el NCSA, que fue el primero en mostrar imágenes *inline* junto al texto, hasta Netscape Navigator (1994), que popularizó la web entre el público general. La respuesta de Microsoft fue Internet Explorer (1995), desencadenando la *primera guerra de los navegadores* que duró hasta aproximadamente 2003 con la victoria temporal de IE6 (alcanzó 95 % de cuota de mercado en 2003) y la consiguiente década de estancamiento en estándares web [74].

Web 2.0 — la web social (2004–2014)

El término *Web 2.0* lo popularizó Tim O'Reilly durante una conferencia homónima en octubre de 2004 [47]. Su característica distintiva: la *participación*. Los usuarios pasaron de consumidores a productores de contenido mediante blogs, wikis, redes sociales y plataformas de video. O'Reilly resumió la filosofía como «la web como plataforma», donde el valor reside en los datos generados por usuarios y los efectos de red.

Tecnológicamente esta era se sostuvo en cuatro pilares:

- **AJAX** (Asynchronous JavaScript And XML), formalizado por Jesse James Garrett en su artículo seminal de febrero de 2005 [24], que permitió actualizar partes de una página sin recargarla. Google Maps (lanzado ese mismo mes) fue la demostración pública más espectacular del paradigma.
- **Frameworks JavaScript de primera generación**: Prototype (2005), MooTools (2006)

y especialmente jQuery (2006), creado por John Resig, que para 2013 estaba presente en más del 60 % de los sitios web del mundo.

- **Plataformas sociales:** Facebook (2004), YouTube (2005), Twitter (2006), GitHub (2008), Instagram (2010), Pinterest (2010).
- **APIs públicas:** Google Maps API (2005), Twitter API (2006), Flickr API, abriendo el camino a los *mashups* y al ecosistema de aplicaciones de terceros.

Esta era también vio surgir las críticas: van Dijck [65] analizó cómo las «plataformas sociales» construyeron una nueva forma de capitalismo informacional basada en la captura y monetización de datos personales, fenómeno que Zuboff [75] bautizó como *capitalismo de vigilancia*.

Web 3.0 — la web semántica, descentralizada e inteligente (2014–presente)

La tercera era convive con tres significados parcialmente solapados:

1. La *Semantic Web* originalmente propuesta por Berners-Lee, Hendler y Lassila en *Scientific American* en 2001 [8], basada en RDF, OWL y SPARQL, que busca representar el significado de la información para que las máquinas razonen sobre ella. Su realización plena ha sido más lenta de lo previsto, pero subyace en proyectos como *schema.org* (consorcio de Google, Microsoft, Yahoo y Yandex desde 2011), Wikidata y los *Knowledge Graphs* de los buscadores modernos.
2. La *web descentralizada* (Web3 en sentido estricto), emergente desde 2017, que reposa sobre blockchain, IPFS y contratos inteligentes. Wood [70] acuñó el término con la publicación del whitepaper de Ethereum. Sus aplicaciones (dApps) incluyen finanzas descentralizadas (DeFi), tokens no fungibles (NFT) y identidad soberana.
3. La *web inteligente* potenciada por IA generativa, emergente desde 2023 con ChatGPT, Claude y GitHub Copilot. Sus implicaciones en el desarrollo web son tan profundas que algunos autores hablan ya de una *cuarta era* en gestación [33].

En paralelo, las aplicaciones SPA (Single Page Applications) y PWA (Progressive Web Applications) desplazan progresivamente al modelo *server-rendered* clásico, y la integración de IA generativa (2023+) está redefiniendo nuevamente el panorama del desarrollo profesional.

1.2.2 Evolución temporal de los hitos web

La tabla 1.1 sintetiza los hitos cronológicos más relevantes de la trayectoria de la web. No se trata de una lista exhaustiva sino de los eventos que cualquier profesional del desarrollo web debe conocer porque cada uno de ellos transformó la manera de construir software para la web. Los hitos están organizados en cinco grandes movimientos: (a) la fundación (1989–1995), (b) la estandarización (1995–2004), (c) la era social (2004–2014), (d) la madurez técnica (2014–2022) y (e) la era de la inteligencia generativa (2023–presente).

Tabla 1.1: Hitos seleccionados de la historia de las aplicaciones web.

Año	Hito
<i>(a) Fundación</i>	
1989	Tim Berners-Lee propone la WWW en el CERN.
1990	Primer servidor (CERN <code>httpd</code>) y navegador (WorldWideWeb).
1991	Publicación del primer sitio web (<code>info.cern.ch</code>).
1993	Mosaic populariza la navegación con imágenes inline.
1993	Especificación CGI 1.0 para programación del servidor.
1994	Fundación del W3C en MIT; nace Netscape Navigator.
<i>(b) Estandarización</i>	
1995	Brendan Eich crea JavaScript en 10 días para Netscape.
1995	PHP/FI (Personal Home Page Tools) de Rasmus Lerdorf.
1996	CSS1 publicado como recomendación W3C.
1996	Macromedia Flash y los <i>cookies</i> de Netscape.
1998	XML 1.0 estandarizado; nace Google.
1999	ECMAScript 3, base estable de JavaScript por más de una década.
2003	WordPress y MediaWiki marcan la era de los CMS.
<i>(c) Era social</i>	
2004	Web 2.0 (O'Reilly); nace Facebook.
2005	AJAX formalizado; lanzamiento de YouTube; Google Maps.
2006	jQuery; Twitter; Amazon Web Services (S3, EC2).
2008	Google Chrome con motor V8; Android SDK 1.0.
2009	HTML5 borrador funcional WHATWG; Node.js (Ryan Dahl).
2010	Responsive Web Design (Ethan Marcotte).
2011	Laravel (Taylor Otwell); React (Jordan Walke, 2013).
<i>(d) Madurez técnica</i>	
2014	HTML5 oficial W3C; ECMAScript 6 anunciado (publicado en 2015).
2015	ES6 (ES2015); Angular 2 reescrito desde AngularJS.
2018	GDPR; Web Components estables; PWA en producción.
2019	WebAssembly 1.0 estandarizado.
2020	.NET 5 unifica .NET Framework y .NET Core.
2021	OWASP Top 10:2021; HTTP/3 (RFC 9114).
2022	ECMAScript 2022 (ES13); Angular 14 con standalone components.
<i>(e) Era de la IA generativa</i>	
2023	GitHub Copilot, ChatGPT, Claude transforman el desarrollo.
2024	SWEBOK Guide v4.0; PHP 8.3; Spring Boot 3.2; .NET 8 LTS.
2026	ECMAScript 2025; HTML Living Standard al día; IA agentic en IDEs.

1.3 Fundamentos de las aplicaciones web

Los fundamentos de las aplicaciones web no son meros tecnicismos introductorios: constituyen el marco conceptual sobre el que se edifica todo el resto del curso. Sin una comprensión sólida de qué es exactamente una aplicación web, cómo se distingue de otros tipos de software, qué arquitecturas la sostienen y cuáles son sus ventajas y limitaciones, el aprendizaje posterior se vuelve mecánico y superficial. Como observa Shklar y Rosen [61], las aplicaciones web son hoy «el género dominante del software»: la mayoría de las personas interactúa más horas al día con aplicaciones web (Gmail, redes sociales, banca en línea, sistemas educativos) que con cualquier otro tipo de software.

Esta sección establece las definiciones operativas, la tipología y los modelos arquitectónicos esenciales que el estudiante usará como referencia durante las 18 capítulos del curso.

1.3.1 Definición y clasificación de aplicaciones web

Una **aplicación web** es un programa de software cuya lógica se ejecuta en uno o varios servidores remotos y cuya interfaz de usuario se distribuye al cliente a través del Hypertext Transfer Protocol (HTTP) y se renderiza en un navegador o agente de usuario. A diferencia de una aplicación de escritorio, no requiere instalación local; a diferencia de una página web estática, su contenido se genera dinámicamente en respuesta a las acciones del usuario y al estado del servidor [61].

Siguiendo la clasificación de Ganzábal García [14] y la actualización de Nixon [45], las aplicaciones web modernas se agrupan en seis tipologías principales que conviene distinguir:

1. **Aplicaciones web estáticas.** Sirven HTML, CSS y JavaScript pre-compilado sin procesamiento de servidor más allá del envío de archivos. Su contenido cambia solo cuando el desarrollador publica una nueva versión. Ejemplo canónico: un currículum vitae publicado en GitHub Pages, o un blog personal generado con Jekyll o Hugo. **Ventaja:** rendimiento extremo (respuesta en milisegundos desde un CDN). **Limitación:** incapacidad de personalización por usuario.
2. **Aplicaciones web dinámicas.** Generan el HTML en el servidor en cada petición. La lógica reside en lenguajes como PHP, Java, Python, Ruby o C#. Cada usuario ve una versión personalizada del contenido según su sesión, sus permisos, los datos almacenados en bases de datos. **Ejemplo:** el portal de matrícula de la UTEQ; un panel de administración construido con Laravel; cualquier sitio WordPress.
3. **Aplicaciones de página única (SPA).** Cargan un único HTML inicial y a partir de ahí toda la interacción ocurre vía JavaScript que consume APIs REST o GraphQL. La navegación entre «páginas» ocurre sin recargar el navegador, gracias a la History API. **Ejemplos:** Gmail, Trello, Notion. Frameworks habituales en 2026: React 19, Angular 19, Vue 3.5, Svelte 5.
4. **Aplicaciones web progresivas (PWA).** Combinan SPA con capacidades nativas: instalación en pantalla de inicio, notificaciones push, funcionamiento offline, acceso a sensores del dispositivo. **Tecnologías:** Service Workers, Web App Manifest, IndexedDB. Twitter Lite fue uno de los primeros casos de éxito; en Latinoamérica, Pinterest PWA y MercadoLibre han demostrado mejoras del 60 % en retención de usuarios [57].
5. **Aplicaciones de comercio electrónico.** Subgénero especializado con flujos transaccionales: catálogo, carrito, pasarela de pago (PayPal, Stripe, Datafast en Ecuador), gestión de pedidos, post-venta. Plataformas dominantes: Shopify, WooCommerce, Magento/Adobe Commerce.
6. **Portales corporativos y e-government.** Aplicaciones con altos requisitos de seguridad, trazabilidad e integración con sistemas heredados. Ejemplos nacionales: el portal del SRI, el sistema de compras públicas SOCE, la Ventanilla Única Electrónica del SENA. Suelen requerir autenticación con certificado digital y firma electrónica.

1.3.2 Características de las aplicaciones web modernas

Antes de profundizar en arquitectura y ventajas comparativas, conviene sintetizar las características esenciales que distinguen a una aplicación web de cualquier otro tipo de software. Estas características no son meras curiosidades técnicas: cada una tiene implicaciones directas en cómo se desarrollan, despliegan, usan, escalan y aseguran las aplicaciones web. Newman [44] y Kleppmann [32] dedican capítulos enteros a discutir las consecuencias arquitectónicas de cada característica.

Tabla 1.2: Características esenciales de las aplicaciones web modernas y sus implicaciones.

Dimensión	Característica
<i>Desarrollo</i>	
Lenguajes	Múltiples coexistentes: HTML+CSS+JS en cliente; PHP/Java/C#/Node en servidor.
Herramientas	IDE (IntelliJ IDEA Ultimate, VS Code), Git, Docker, gestores de paquetes.
Metodología	Predominantemente ágil (Scrum, Kanban) con CI/CD.
<i>Despliegue</i>	
Empaquetado	Contenedores Docker, archivos JAR/WAR, paquetes <code>npm</code> .
Hosting	Cloud (AWS, Azure, GCP), VPS, on-premise; orquestación con Kubernetes.
Actualización	<i>Rolling updates</i> sin interrumpir el servicio; <i>blue-green deployment</i> .
<i>Uso</i>	
Acceso	Universal vía URL, sin instalación local.
Multiplataforma	Mismo código accesible desde Windows, macOS, Linux, iOS, Android.
Conectividad	Requiere red; PWAs ofrecen modo offline limitado.
<i>Operación</i>	
Monitoreo	Métricas (Prometheus), logs (ELK), traces (OpenTelemetry, Tempo).
Escalabilidad	Horizontal preferida sobre vertical en arquitecturas modernas.
Disponibilidad	Objetivo común 99.9% (8.7 h/año de downtime aceptable).
<i>Seguridad</i>	
Superficie de ataque	Pública en internet: requiere defensas multinivel.
Estándares	OWASP Top 10:2021, ISO 27001, GDPR, LOPDP (Ecuador).
Identidad	OAuth 2.0, OpenID Connect, SAML, JWT.

Las implicaciones prácticas de estas características son enormes. Por ejemplo, la naturaleza multiplataforma elimina el costo histórico de mantener versiones separadas para Windows, macOS y Linux, pero introduce el reto de la consistencia entre navegadores. La actualización *rolling* elimina la fricción del «día de instalación» del software clásico, pero exige diseñar el código para que dos versiones coexistan durante el despliegue. La superficie pública de ataque obliga a integrar la seguridad desde el día uno; no se puede «agregar seguridad» al final como en un sistema interno.

1.3.3 Arquitectura cliente-servidor en aplicaciones web

La arquitectura cliente-servidor es el modelo de comunicación distribuida que organiza prácticamente la totalidad del tráfico web actual. Comprenderla a profundidad es indispensable porque cada decisión técnica posterior (desde qué lenguaje usar en el navegador hasta cómo escalar el servidor) se apoya en sus principios. En esta sección revisaremos primero su concepto fundamental, después la diferencia frecuentemente confundida entre *capas* y *niveles*, y finalmente las arquitecturas de 2, 3 y N niveles con ejemplos concretos y diagramas.

Concepto fundamental

La arquitectura cliente-servidor es el paradigma fundamental sobre el que se erige la World Wide Web. En este modelo, un proceso *cliente* emite una solicitud (*request*) a un proceso *servidor*, el cual procesa la solicitud y devuelve una respuesta (*response*). El protocolo que estandariza este intercambio es HTTP/1.1 (RFC 7230–7235), HTTP/2 (RFC 7540) o HTTP/3 (RFC 9114), este último ya dominante en 2026 gracias a su desempeño superior en redes móviles.

Anatomía de una petición HTTP

```
GET /productos?categoria=cafe HTTP/1.1
Host: tienda.uteq.edu.ec
Accept: text/html
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Accept-Language: es-EC,es;q=0.9,en;q=0.8
Cookie: SESSID=ab39e1ff7e1c4
```

Listado 1.1: Ejemplo de Shell

La primera línea es la *línea de petición*, compuesta por método, ruta y versión del protocolo. Las líneas siguientes son *cabeceras*; tras una línea en blanco puede ir el cuerpo (en métodos POST, PUT y PATCH).

Niveles vs capas: una distinción frecuentemente confundida

Uno de los conceptos más confusos para el estudiante principiante es la distinción entre *capas* y *niveles*. Aunque a menudo se usan como sinónimos, en arquitectura de software profesional **tienen significados diferentes** que conviene precisar desde el inicio [21, 57]:

Capa (*layer*) vs Nivel (*tier*)

Capa (*layer*): agrupación *lógica* de responsabilidades dentro del software. Es una abstracción conceptual; varias capas pueden ejecutarse en el mismo proceso físico. Ejemplo: en una aplicación monolítica, las capas «Presentación», «Negocio» y «Datos» pueden coexistir en el mismo `.jar`.

Nivel (*tier*): unidad *física* de despliegue. Cada nivel corre típicamente en su propio servidor o contenedor, y los niveles se comunican por red. Ejemplo: «servidor de aplicación» y «servidor de BD» son dos niveles distintos aunque la app sea un monolito de varias capas internas.

Un sistema puede tener **3 capas pero 2 niveles** (capas presentación, negocio y datos en un mismo proceso que se conecta a una BD en otro servidor), o **3 capas y 3 niveles** (cada capa en su propio servidor).

Arquitecturas N-niveles: panorama visual

Las arquitecturas web se clasifican por la cantidad de *niveles físicos* (máquinas o procesos separados) en que se despliega el sistema. Comprender visualmente esta diferencia entre 1, 2, 3 y

Niveles es esencial antes de hablar de patrones más complejos como microservicios o serverless. La siguiente figura contrasta los cuatro casos canónicos.

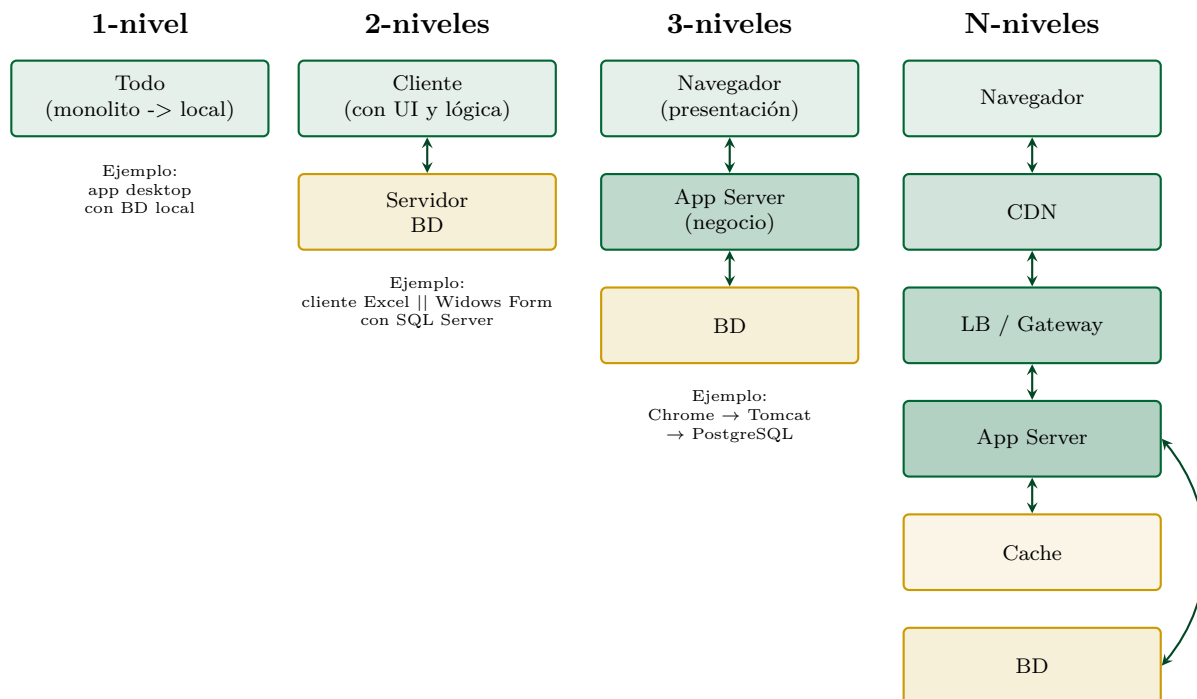


Figura 1.1: Las cuatro variantes principales de arquitectura cliente-servidor por número de niveles físicos. La elección depende del tamaño, presupuesto, requisitos de escalabilidad y disponibilidad. La mayoría de aplicaciones web modernas usan 3 o más niveles.

A continuación se describe cada variante:

Arquitectura 1-nivel (monolito local). Todo el software (UI, lógica, datos) reside en una sola máquina sin red. No aplica a la web. Ejemplos: una aplicación de escritorio con SQLite embebido, Microsoft Access en modo monousuario.

Arquitectura 2-niveles (cliente-servidor tradicional). El cliente contiene la UI y parte de la lógica de negocio; el servidor expone solo la BD. Modelo dominante en los 90 con clientes *fat-client* (Visual Basic + Oracle, Delphi + Sybase). Para la web no es óptimo: la lógica de negocio expuesta al cliente compromete la seguridad. Ejemplos vigentes: algunas aplicaciones legadas de contabilidad con cliente Windows + SQL Server.

Arquitectura 3-niveles (la clásica web). Separa explícitamente presentación (navegador), lógica de negocio (servidor de aplicación: Tomcat, IIS, Apache+PHP) y datos (SGBD). Es el modelo canónico de la web desde 1998 y sigue siendo el más común. Ejemplo: Chrome → Spring Boot → PostgreSQL.

Arquitectura N-niveles. Introduce niveles adicionales para abordar requisitos no funcionales: CDN para contenido estático, balanceador de carga para distribución, gateway API para autenticación cruzada, caché distribuida (Redis), sistemas de mensajería (Kafka). Es el modelo dominante en sistemas a gran escala [3, 32]. Ejemplo: Netflix opera con decenas de niveles

distintos.

Anatomía del flujo de una petición web

Comprender qué ocurre, paso a paso, desde que la persona usuaria escribe una dirección hasta que ve la página renderizada es la base de toda la ingeniería de rendimiento y del diagnóstico en producción. La latencia total percibida no la produce un único componente, sino que se *acumula* a lo largo de una cadena de eslabones; mejorar un sistema real exige medir cada eslabón por separado y atacar el que domina el tiempo de respuesta, no el que resulta más cómodo de modificar [25, 32]. Una aplicación web de producción involucra al menos cinco actores físicos o lógicos cuya función conviene precisar:

1. **Navegador del usuario** (Chrome, Firefox, Safari, Edge). Es un entorno de ejecución completo, no un simple visor: descarga el documento, construye el árbol DOM a partir del HTML y el árbol CSSOM a partir de las hojas de estilo, ejecuta el JavaScript y, finalmente, calcula el diseño (*layout*) y pinta los píxeles en pantalla. Además gestiona las conexiones de red —reutilizándolas mediante *keep-alive* en HTTP/1.1, multiplexando varias peticiones sobre una sola conexión en HTTP/2 o sobre QUIC en HTTP/3— y administra el estado del lado del cliente: cookies, caché de recursos, `localStorage` y *Service Workers* [25, 18]. Cada una de estas etapas puede convertirse en un cuello de botella perceptible por la persona usuaria.
2. **Servidor DNS**. Traduce el nombre de dominio (`uteq.edu.ec`) a una o varias direcciones IP. La resolución no la realiza un único servidor: un *resolver* recursivo consulta, en cascada, a los servidores raíz, a los del dominio de primer nivel y a los autoritativos del dominio, y conserva el resultado en caché durante el tiempo indicado por el campo TTL [40, 41]. Por ello, una resolución que ya está en caché es prácticamente instantánea, mientras que una resolución *en frío* —sin entrada previa— añade típicamente desde unas pocas decenas hasta algunos cientos de milisegundos a la primera petición, según la distancia a los servidores y el número de saltos necesarios [25, 34]. Mecanismos como DNSSEC, DNS sobre TLS (DoT) y DNS sobre HTTPS (DoH) añaden integridad y privacidad a este intercambio [64].
3. **Balanceador de carga o proxy inverso** (Nginx, HAProxy, AWS ELB). Es la puerta de entrada del centro de datos. Distribuye las peticiones entrantes entre varios servidores de aplicación según políticas como *round-robin*, *least-connections* o reparto por peso, y retira de la rotación los nodos que no superan sus chequeos de salud (*health checks*). Suele terminar también el cifrado TLS —descifra la conexión segura una sola vez para liberar de ese costo a los servidores internos— y sirve directamente el contenido estático [56, 32]. Distinguir un balanceo de nivel de transporte (L4) de uno de nivel de aplicación (L7) es clave: solo el segundo puede enrutar según la ruta de la URL o las cabeceras HTTP [64].
4. **Servidor de aplicación** (Apache HTTPD, Apache Tomcat, IIS, Node.js, PHP-FPM). Ejecuta el código de la aplicación: recibe la petición HTTP, invoca la lógica de negocio y construye la respuesta. Lo que diferencia a estos entornos es su *modelo de concurrencia*: Tomcat asigna un hilo de un *pool* a cada petición; PHP-FPM mantiene un conjunto de

procesos trabajadores; y Node.js emplea un único bucle de eventos no bloqueante. Ese modelo determina cómo se comporta el servidor bajo carga y qué clase de operaciones (de CPU o de E/S) lo saturan primero [32, 25].

5. **Sistema gestor de bases de datos** (PostgreSQL, MySQL, MariaDB, SQL Server, Oracle). Persiste el estado de la aplicación y, con frecuencia, es el eslabón que domina la latencia de cola (los percentiles más altos del tiempo de respuesta). Su rendimiento depende de factores como el uso de un *pool* de conexiones reutilizables, la presencia de índices adecuados, la planificación de consultas y las garantías transaccionales (ACID) que ofrece [32, 54]. Una consulta sin índice sobre una tabla grande puede, por sí sola, multiplicar el tiempo total de la petición.

La Tabla 1.3 resume, para cada actor, su función principal, un fallo característico en producción y una herramienta de diagnóstico habitual.

Tabla 1.3: Actores del flujo de una petición web — función, fallo típico y diagnóstico.

Actor	Función principal	Fallo típico	Diagnóstico
Navegador	Renderiza HTML/CSS/JS y gestiona la red del cliente	JavaScript que bloquea el renderizado	DevTools / Lighthouse
DNS	Traduce el dominio a dirección IP	Resolución en frío o TTL mal configurado	dig , nslookup
Proxy / LB	Reparte carga y termina TLS	Nodo caído no retirado de la rotación	Logs de acceso, métricas L7
App Server	Ejecuta la lógica de negocio	Agotamiento del <i>pool</i> de hilos o procesos	APM, perfilado del runtime
SGBD	Persiste el estado	Consulta lenta sin índice	EXPLAIN , registro de consultas lentas

Este recorrido de forma simplificada está representado en la Figura 1.2. Conviene leerla recordando que cada flecha es una posible fuente de retardo y que la optimización web consiste, esencialmente, en medir cada salto antes de intentar mejorarlo.

No optimizar a ciegas

La intuición es una mala guía para el rendimiento. Antes de reescribir código, instrumente la aplicación y mida el tiempo de cada eslabón —resolución DNS, establecimiento de TLS, tiempo en el servidor de aplicación y tiempo en la base de datos— preferiblemente con percentiles (p50, p95, p99) y no solo con promedios, que ocultan los casos peores que sí percibe la persona usuaria [25, 32].

Una petición web se puede resumir en seis pasos como se muestra en la Figura 1.2, desde que el usuario pulsa Enter hasta que ve la página. En el paso 1 (*resolver*), el navegador todavía no sabe a qué máquina conectarse: entrega el nombre de dominio —por ejemplo, `uteq.edu.ec`— a un servidor DNS, que lo traduce a una dirección IP. Con esa IP, en el paso 2 (*HTTPS*) el navegador abre una conexión cifrada y envía la petición HTTP al punto de entrada del sitio, que rara vez es el servidor de aplicación directamente, sino un proxy inverso o balanceador de carga. En el paso 3 (*enrutar*), ese proxy termina el cifrado TLS y decide a cuál de los servidores de aplicación entrega la petición, repartiendo así la carga entre varios. El servidor de aplicación ejecuta entonces la

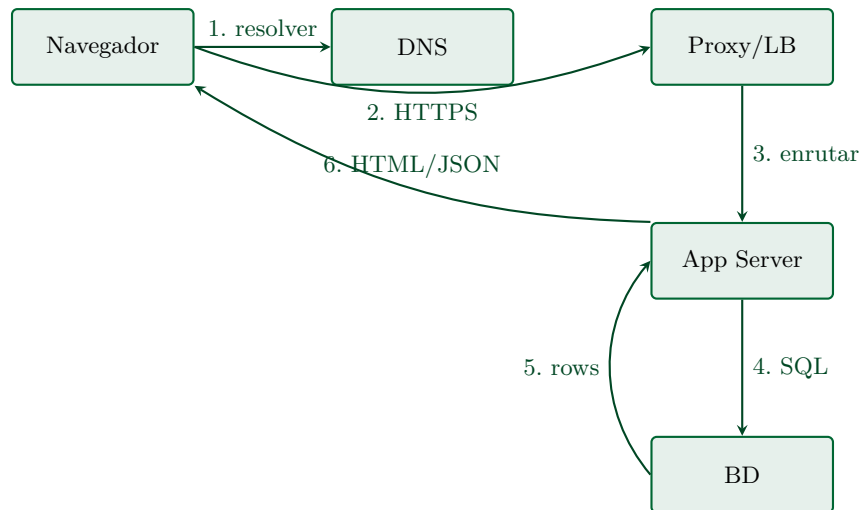


Figura 1.2: Flujo simplificado de una petición web. Cada uno de estos seis pasos puede convertirse en cuello de botella; la disciplina de optimización web consiste en medir cada eslabón.

lógica del programa y, cuando necesita datos persistentes, realiza el paso 4 (*SQL*): envía una consulta al sistema gestor de base de datos. En el paso 5 (*rows*), la base de datos responde con las filas solicitadas, que el servidor convierte en objetos para construir la respuesta. Por último, en el paso 6 (*HTML/JSON*), el servidor devuelve esa respuesta al navegador —HTML si es una página, JSON si es una API— y el navegador la presenta al usuario. Cada uno de estos eslabones puede convertirse en un cuello de botella, de modo que optimizar una aplicación web consiste, ante todo, en medir el tiempo que consume cada paso.

1.3.4 Ventajas y desventajas de las aplicaciones web

Toda decisión arquitectónica implica trade-offs: las aplicaciones web ofrecen ventajas operativas significativas frente a las nativas, pero también limitaciones que debemos conocer antes de comprometernos con la plataforma. La tabla siguiente resume el balance.

Tabla 1.4: Comparativa entre aplicaciones web, escritorio y móviles nativas.

Criterio	Web	Escritorio	Móvil nativa
Despliegue	Inmediato sin instalación	Empaquetado e instalador	Tienda de apps (revisión)
Compatibilidad	Multiplataforma	Sistema-dependiente	iOS y Android (separadas)
Acceso a hardware	Limitado vía Web APIs	Total	Total
Costo de desarrollo	Bajo (un <i>stack</i>)	Medio	Alto (dos <i>stacks</i>)
Costo de mantenimiento	Muy bajo (deploy único)	Medio	Alto (dos versiones)
Rendimiento bruto	Medio	Alto	Alto
Modo <i>offline</i>	Limitado (PWA)	Total	Total
Distribución	Instantánea (URL)	Descarga manual	Tienda de apps
Actualizaciones	Automáticas	Manuales o semi	Aceptación del usuario

1.4 Diseño web inclusivo y accesibilidad

La web nació con vocación universal: cualquier persona, desde cualquier dispositivo, en cualquier lugar del mundo, debería poder acceder a la información que contiene. Sin embargo, la realidad

muestra que millones de personas con discapacidad encuentran barreras infranqueables cada día al navegar sitios que ignoraron sus necesidades. La accesibilidad web no es un tema marginal ni un añadido decorativo: es una responsabilidad profesional, una obligación legal en cada vez más países (incluido Ecuador) y un factor de competitividad comercial cuantificable. Esta sección introduce el concepto, presenta las cuatro categorías de discapacidad que afectan la experiencia web, expone las pautas WCAG 2.1 del W3C con sus principios POUR, y termina con los criterios de éxito que todo desarrollador debe dominar.

1.4.1 ¿Qué es la accesibilidad web?

La accesibilidad web es la práctica de diseñar y construir sitios y aplicaciones que puedan ser percibidos, comprendidos, navegados y utilizados por *todas las personas*, independientemente de sus capacidades físicas, sensoriales o cognitivas. La definición canónica del W3C [73] la formula así: «accesibilidad significa que personas con discapacidades pueden percibir, comprender, navegar e interactuar con la Web, y que pueden contribuir a la Web».

Sin embargo, la accesibilidad web no es solo una buena causa moral: es un imperativo legal, comercial y técnico. García-Holgado et al. [26] demuestran en un estudio sistemático sobre instituciones de educación superior que solo el 23 % de los sitios analizados cumplen el nivel mínimo de WCAG 2.1, lo que constituye un incumplimiento normativo en la mayoría de jurisdicciones.

Marco legal de la accesibilidad

La accesibilidad no es solo una buena práctica: en muchas jurisdicciones es una obligación legal con sanciones concretas. La siguiente enumeración recopila los marcos normativos vigentes que un desarrollador web debe conocer antes de publicar un sitio.

- **Ecuador.** La Ley Orgánica de Discapacidades (Registro Oficial 2012) exige conformidad mínima AA en sitios públicos a partir de su Reglamento de 2017. CONADIS supervisa el cumplimiento.
- **Estados Unidos.** Section 508 del Rehabilitation Act (enmienda de 1998, refundición de 2017) obliga a las agencias federales a la accesibilidad. Casos como *Robles v. Domino's Pizza* (2019) extendieron la jurisprudencia al sector privado bajo el ADA.
- **Unión Europea.** Directiva 2016/2102 sobre accesibilidad de los sitios y aplicaciones móviles de los organismos del sector público. EN 301 549 V3.2.1 (2021) define los requisitos técnicos.
- **España, México, Argentina, Brasil, Colombia, Chile.** Todos cuentan con normativa específica derivada de los lineamientos internacionales.

Impacto comercial cuantificable

Más allá del cumplimiento legal, la accesibilidad genera retorno medible:

- **15 % de la población mundial** vive con alguna forma de discapacidad según la OMS (2024).
- Mejorar la accesibilidad mejora el SEO: Google penaliza sitios con baja accesibilidad mediante Core Web Vitals.
- Microsoft estimó que sus inversiones en accesibilidad generaron un retorno de inversión del 1.000 % al ampliar mercados [39].

1.4.2 Las cuatro categorías de discapacidad relevantes para la web

Antes de profundizar en los criterios técnicos, es indispensable comprender qué tipos de barreras enfrentan los usuarios con discapacidad y qué tecnologías de asistencia emplean. Esta comprensión es la base ética y empática de la accesibilidad: no se trata de cumplir una norma abstracta sino de habilitar a personas reales con necesidades específicas.

Las categorías que se presentan a continuación siguen la taxonomía clásica de Henry [26] y el W3C WAI. Importante: una misma persona puede tener *múltiples* discapacidades simultáneas (por ejemplo, baja visión y artritis), o *discapacidades temporales* (un brazo enyesado, una lesión ocular) o *situacionales* (usar el móvil bajo luz solar intensa, manejar con manos ocupadas). El diseño accesible beneficia a todos estos casos.

1. **Discapacidad visual.** Incluye ceguera total (1.5 % de la población mundial), baja visión y daltonismo (8 % de hombres, 0.5 % de mujeres). Los usuarios con ceguera utilizan lectores de pantalla (NVDA gratuito en Windows, JAWS comercial, VoiceOver en Apple, TalkBack en Android) que recorren el árbol de accesibilidad del DOM y convierten la información a voz. Los daltónicos requieren que la información no se transmita exclusivamente por color (el rojo y verde son indistinguibles para la mayoría).
2. **Discapacidad auditiva.** Sordera total o parcial (5 % de la población mundial). Requiere subtítulos en contenido multimedia (WCAG 1.2.2), transcripciones de podcasts, lengua de señas intérprete en videos institucionales críticos, y notificaciones visuales además de sonoras.
3. **Discapacidad motora.** Imposibilidad de usar mouse o teclado de forma estándar. Causas incluyen parálisis, temblores, artritis reumatoide, distrofia muscular, amputaciones. Requiere navegación completa por teclado (sin depender del mouse), áreas de clic amplias (mínimo 44×44 px según WCAG 2.5.5), ausencia de tiempos límite estrictos, soporte para entrada por voz, switches adaptados.
4. **Discapacidad cognitiva.** La categoría más amplia y diversa: incluye dislexia (10 % de la población), TDAH (5 %), trastornos del espectro autista (1–2 %), discapacidad intelectual, demencia. Requiere lenguaje claro y conciso, estructura predecible, ausencia de elementos parpadeantes (riesgo de crisis epilépticas fotosensibles entre 3–65 Hz), feedback claro de errores, opciones para ajustar tipografía y espaciado.

1.4.3 Las Web Content Accessibility Guidelines 2.1 (WCAG 2.1)

WCAG 2.1 [73] es la norma internacional de referencia, publicada por el W3C en junio de 2018 y vigente. Está organizada en torno a cuatro principios fundamentales conocidos por el acrónimo **POUR** (Perceivable, Operable, Understandable, Robust). Cada principio define directrices, cada directriz se materializa en *criterios de éxito* comprobables, y cada criterio se clasifica en uno de tres niveles de conformidad: A (mínimo), AA (recomendado), AAA (excepcional).

Esta sección no se limita a enunciar los principios: para cada uno explica *qué tecnologías* permiten implementarlo y *cómo* medir el cumplimiento de forma objetiva.

Principio 1 — Perceptible

Enunciado: La información y los componentes de la interfaz deben presentarse de modo que los usuarios puedan percibirlos.

Cómo lograrlo en HTML:

- Atributo `alt` descriptivo en toda `<img>`.
- Subtítulos `<track kind="captions">` en `<video>`.
- Transcripciones para podcasts publicadas junto al audio.
- `aria-label` para iconos clicables sin texto.

Cómo lograrlo en CSS:

- Contraste mínimo 4.5:1 (texto normal) y 3:1 (texto grande).
- No usar solo color para transmitir información crítica.
- Diseño responsive que soporte zoom hasta 200 % sin pérdida funcional.

Cómo medir cumplimiento:

- **WebAIM Contrast Checker** (<https://webaim.org/resources/contrastchecker/>): ingresa colores hex y devuelve la ratio exacta.
- **axe DevTools** (extensión Chrome/Firefox): detecta imágenes sin `alt`, contraste insuficiente, audio sin transcripción.
- **Pa11y CLI** (<https://pa11y.org>): herramienta de línea de comandos para integrar auditoría en CI/CD.
- **Lighthouse Accessibility** (incluido en Chrome DevTools): score 0–100 con recomendaciones priorizadas.

Principio 2 — Operable

Enunciado: Los componentes y la navegación deben ser operables por todos los usuarios.

Cómo lograrlo en HTML:

- Elementos interactivos nativos (`<button>`, `<a>`) en lugar de `<div onclick>`.
- Atributo `tabindex` solo cuando estrictamente necesario; nunca `tabindex="2"` u otros valores positivos.
- `skip-link` al inicio de cada página para saltar a contenido principal.

Cómo lograrlo en CSS:

- `:focus-visible` con outline claramente visible.
- `prefers-reduced-motion` respetado en animaciones.
- Áreas de clic mínimo 44×44 px.

Cómo lograrlo en JavaScript:

- Sin tiempo límite ajustable o que pueda extenderse.
- Manejo de teclado en componentes custom: **Enter**, **Space**, **Escape**, flechas direccionales.
- Sin contenido parpadeante en rango 3–55 Hz.

Cómo medir cumplimiento:

- Navegación completa solo con **Tab** y **Enter** sin mouse.
- Recorrido visual del foco siguiendo orden lógico.
- Lighthouse y axe DevTools detectan tabbing roto.

Principio 3 — Comprensible

Enunciado: La información y el manejo de la interfaz deben ser comprensibles.

Cómo lograrlo en HTML:

- Atributo `lang="es-EC"` en `<html>`; cambiar con `<span lang="en">` en fragmentos en otros idiomas.
- `<label for="id">` asociado a cada `<input>`.
- Atributo `autocomplete` con valores estándares.
- `aria-describedby` para descripción extendida.

Cómo lograrlo en lenguaje natural:

- Nivel de lectura grado 8–9 según escala Flesch-Kincaid.
- Frases cortas (máx. 25 palabras).
- Voz activa preferida a pasiva.
- Glosario de términos técnicos accesible desde cada uso.

Cómo medir cumplimiento:

- **Hemingway Editor** (<https://hemingwayapp.com>): evalúa legibilidad.
- Pruebas de usabilidad con usuarios reales de baja alfabetización digital.

Principio 4 — Robusto

Enunciado: El contenido debe ser robusto suficiente para que pueda ser interpretado por una amplia variedad de agentes de usuario, incluyendo tecnologías asistivas.

Cómo lograrlo en HTML:

- HTML válido sin errores (verificar en <https://validator.w3.org/nu/>).
- Roles ARIA solo cuando los elementos semánticos nativos no basten.
- IDs únicos en todo el documento.
- Nombres accesibles para todos los controles (`aria-label` o texto visible asociado).

Cómo medir cumplimiento:

- W3C Markup Validator: 0 errores.
- NVDA o VoiceOver: recorrido auditivo manual.
- axe DevTools: 0 violaciones serias o críticas.

Niveles de conformidad y herramientas integradas de auditoría

WCAG 2.1 establece tres niveles de conformidad: **A** (mínimo imprescindible), **AA** (recomendado para sector público y privado serio) y **AAA** (excepcional, raro de alcanzar íntegramente). En 2023 se publicó WCAG 2.2 que añadió nueve criterios de éxito adicionales y previsiblemente WCAG 3.0 (*Silver*, borrador desde 2021) será la próxima revolución mayor, reemplazando el sistema de niveles A/AA/AAA por un puntaje continuo.

1.4.4 Criterios de éxito clave de WCAG 2.1

Esta subsección presenta los diez criterios de éxito más importantes que todo desarrollador web debe interiorizar. Para cada criterio se explica: (a) qué exige exactamente la norma, (b) cómo

implementarlo en código real, y (c) un ejemplo concreto.

Diez criterios WCAG imprescindibles — detallados

1.1.1 Contenido no textual (Nivel A).

Qué exige: todo contenido no textual tiene una alternativa textual equivalente.

Cómo lograrlo: `alt` en `<img>`; `aria-label` en iconos clicables; `<title>` en SVGs significativas; `alt=""` (vacío) si la imagen es decorativa.

Ejemplo: ``.

1.3.1 Información y relaciones (Nivel A).

Qué exige: la estructura semántica del HTML refleja la jerarquía visual y las relaciones de la información.

Cómo lograrlo: usar encabezados correctos `<h1>-<h6>`, listas `<ul>/<ol>` para listas, `<table>` con `<th scope>` para datos tabulares, `<fieldset>+<legend>` para agrupar campos relacionados.

Anti-ejemplo: simular tabla con `<div>` y CSS Grid sin roles ARIA, rompe lectores de pantalla.

1.4.3 Contraste mínimo (Nivel AA).

Qué exige: ratio de contraste $\geq 4.5:1$ para texto normal; $\geq 3:1$ para texto grande (18pt+ o 14pt+ negrita).

Cómo lograrlo: verificar con WebAIM Contrast Checker; usar paletas pre-validadas.

Ejemplo de combinaciones AA conformes sobre fondo blanco: #006633 (verde UTEQ) ratio 6.27:1; negro #000000 ratio 21:1.

1.4.10 Reflujo (Nivel AA).

Qué exige: el contenido reflowea sin pérdida de información ni funcionalidad y sin scroll horizontal en 320 px de ancho.

Cómo lograrlo: CSS responsive mobile-first; `viewport meta`; unidades relativas (rem, em, %).

1.4.11 Contraste no textual (Nivel AA).

Qué exige: componentes UI (botones, iconos de estado, bordes de campos) con contraste $\geq 3:1$ con sus alrededores.

Cómo lograrlo: bordes visibles en inputs; iconos de estado con fondo o borde definido.

2.1.1 Teclado (Nivel A).

Qué exige: toda funcionalidad disponible solo desde teclado.

Cómo lograrlo: usar elementos interactivos nativos; manejar **Enter** y **Space** en componentes custom; no atrapar el foco.

Anti-ejemplo: dropdown que solo abre con hover de mouse.

2.4.7 Foco visible (Nivel AA).

Qué exige: indicador visible cuando un elemento recibe foco.

Cómo lograrlo: no quitar el outline por defecto, o reemplazarlo con `:focus-visible` de mayor visibilidad (≥ 2 px, contraste $\geq 3:1$).

3.1.1 Idioma de la página (Nivel A).

Qué exige: elemento `<html>` tiene atributo `lang` correcto.

Cómo lograrlo: `<html lang="es-EC">`.

3.3.1 Identificación de errores (Nivel A).

Qué exige: si el sistema detecta error en entrada del usuario, lo identifica y describe en texto.

Cómo lograrlo: `<input aria-invalid="true"aria-describedby="err-email">` y mensaje en `<span id="err-email">Email inválido</span>`.

4.1.2 Nombre, función, valor (Nivel A).

Qué exige: para todos los componentes de interfaz, nombre, función y valor son determinables programáticamente.

Cómo lograrlo: usar elementos nativos siempre que sea posible; si se construye un componente custom, declarar `role`, `aria-label`, `aria-checked` etc.

1.5 Configuración del entorno con IntelliJ IDEA Ultimate

Antes de continuar con el resto de la asignatura, el estudiante debe disponer de un entorno IDE funcional. El siguiente ejercicio resuelto guía paso a paso la instalación y verificación de IntelliJ IDEA Ultimate con la licencia educativa UTEQ.

Ejercicio 1.1 (RESUELTO) — Instalación y verificación de IntelliJ IDEA Ultimate

Objetivo: configurar el entorno integrado de desarrollo que se utilizará durante todo el semestre y verificar que los plugins necesarios para HTML, CSS y JavaScript están operativos.

Paso 1 — Obtener la licencia educativa. Ingresar a <https://www.jetbrains.com/community/education> y crear una cuenta con el correo institucional `<usuario>@uteq.edu.ec`. Al verificar el dominio, JetBrains envía un correo de confirmación con una clave educativa renovable anualmente. Sin esta clave, IntelliJ IDEA Ultimate funciona únicamente 30 días en modo evaluación.

Paso 2 — Descargar e instalar. Descargar IntelliJ IDEA Ultimate desde <https://www.jetbrains.com/idea/download>. El instalador pesa aproximadamente 850 MB.

Paso 3 — Activar la licencia. En la primera ejecución elegir «Activate License → JB Account». Verificar en `Help → Register` que la licencia es de tipo *Educational License*.

Paso 4 — Crear el primer proyecto. En la pantalla *Welcome* pulsar `New Project`. Seleccionar la plantilla HTML, establecer **Name:** `appweb-cap01`, y pulsar `Create`.

Paso 5 — Verificar plugins esenciales en `File → Settings → Plugins`: HTML Tools, CSS, JavaScript and TypeScript, Markdown, Live Edit.

Paso 6 — Probar. Abrir el archivo `index.html` que el IDE generó automáticamente. Posicionar el cursor sobre el código y pulsar `Alt+F2`; elegir Chrome.

Resultado esperado (verificación):

[Ventana del navegador Chrome]

URL: `http://localhost:63342/appweb-cap01/index.html`

[Pestaña: `index.html`]

Hello, World!

(Página HTML mínima generada por IntelliJ con el texto por defecto, servida desde el servidor de desarrollo embebido del IDE en el puerto 63342.)

Indicadores de éxito verificables:

- La barra de URL muestra una dirección `localhost:63342/...`
- El título de la pestaña coincide con `index.html`.
- En IntelliJ, el panel *Run* (parte inferior) muestra el servidor en estado «Running».

Ejercicio 1.2 (RESUELTO) — Primera página HTML5 semántica y accesible

Objetivo: crear una página HTML5 estructuralmente correcta, semánticamente significativa y accesible. Verificar su validez en el W3C Validator y la ausencia de violaciones críticas en axe DevTools.

Código completo (Listado 1.2):

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7   <title>Mi primera pagina semantica - UTEQ</title>
8   <meta name="description" content="Pagina de practica
9     para Aplicaciones Web - UTEQ 2026">
10 </head>
11 <body>
12   <header role="banner">
13     <h1>Universidad Tecnica Estatal de Quevedo</h1>
14     <p>La primera universidad agropecuaria del Ecuador</p>
15     <nav role="navigation" aria-label="Menu principal">
16       <ul>
17         <li><a href="#inicio">Inicio</a></li>
18         <li><a href="#carreras">Carreras</a></li>
19         <li><a href="#contacto">Contacto</a></li>
20       </ul>
21     </nav>
22   </header>
23   <main role="main">
24     <article>
25       <h2 id="inicio">Bienvenido al curso de Aplicaciones Web</h2>
26       <p>Este es el primer ejercicio del semestre. La fecha de hoy es
27         <time datetime="2026-05-04">4 de mayo de 2026</time>.</p>
28       <figure>
29         

```

```

31     <figcaption>Escudo institucional UTEQ</figcaption>
32 </figure>
33 </article>
34 <aside aria-label="Informacion adicional">
35     <h2>Datos de contacto</h2>
36     <address>
37         Av. Quito km. 1.5 via a Santo Domingo<br>
38         Quevedo, Los Rios, Ecuador<br>
39         Telefono: <a href="tel:+59353702220">(+593) 5 3702-220</a>
40     </address>
41 </aside>
42 </main>
43 <footer role="contentinfo">
44     <small>&copy; 2026 UTEQ. Todos los derechos reservados.</small>
45 </footer>
46 </body>
47 </html>

```

Listado 1.2: index.html (Ejercicio 1.2)

Explicación condensada: `<!DOCTYPE html>` declara HTML5; `lang="es-EC"` permite al lector de pantalla elegir voz española; `<meta charset>` evita corrupción de tildes; `<meta viewport>` habilita responsive en móvil. Los roles ARIA (`banner`, `navigation`, `main`, `contentinfo`) refuerzan la semántica para tecnologías asistivas antiguas. `<time datetime>` hace la fecha legible por máquinas. `alt` en la imagen cumple WCAG 1.1.1.

Resultado esperado en el navegador:

[Vista renderizada en Chrome - sin estilos CSS]

Universidad Tecnica Estatal de Quevedo

La primera universidad agropecuaria del Ecuador

• Inicio • Carreras • Contacto

Bienvenido al curso de Aplicaciones Web

Este es el primer ejercicio del semestre. La fecha de hoy es 4 de mayo de 2026.



Escudo institucional UTEQ

Datos de contacto

Av. Quito km. 1.5 via a Santo Domingo

Quevedo, Los Rios, Ecuador

Telefono: (+593) 5 3702-220

© 2026 UTEQ. Todos los derechos reservados.

Verificación obligatoria del éxito:

1. W3C Validator (<https://validator.w3.org/nu/>): copiar el código completo, pegar

en «Validate by direct input», pulsar Check. Debe aparecer en verde: «Document checking completed. No errors or warnings to show.»

2. *axe DevTools*: instalar la extensión desde Chrome Web Store. F12 → pestaña axe DevTools → «Scan ALL of my page». Resultado esperado: 0 violaciones críticas.
3. *Navegación con teclado*: pulsar Tab repetidamente desde la barra de URL. El foco debe recorrer en orden los tres enlaces del menú y el enlace del teléfono.

Ejercicio 1.3 (PROPUESTO) — Página de presentación personal accesible

Objetivo: construir una página de presentación personal que integre todos los conceptos vistos: estructura semántica, accesibilidad, metadatos correctos, navegación con teclado.

Especificaciones obligatorias:

1. Crear un nuevo proyecto en IntelliJ: `appweb-presentacion-<TuApellido>`.
2. El archivo `index.html` debe contener:
 - DOCTYPE HTML5 y `lang="es-EC"`.
 - Metadatos: `charset`, `viewport`, `description` (máx. 160 caracteres), `author`, `keywords` (5–7 términos).
 - `<header>` con nombre completo (`h1`), foto con `alt` descriptivo, y un `<nav>` con cinco enlaces internos.
 - Cinco `<section>` (cada una con su `<h2>`): «Sobre mí», «Estudios», «Habilidades técnicas», «Proyectos» y «Contacto».
 - En «Habilidades técnicas», una tabla `<table>` con cuatro filas (HTML, CSS, JavaScript, otra de elección) y tres columnas (tecnología, nivel 1–5, años de experiencia).
 - En «Proyectos», tres `<article>` con `<figure>+<figcaption>` cada uno.
 - `<footer>` con copyright y fecha `<time datetime="2026">`.
3. La página debe ser *válida* en <https://validator.w3.org/nu/> sin errores ni advertencias.
4. axe DevTools debe arrojar cero violaciones de cualquier severidad.
5. La navegación por teclado debe permitir alcanzar todos los enlaces y secciones.

Entrega: `index.html` + captura del W3C Validator mostrando «No errors» + captura de axe DevTools mostrando «No issues found».

Esta práctica forma parte de la *Sumativa GA-Taller: Encuadre académico*.

1.6 Buenas prácticas profesionales para HTML semántico

Las buenas prácticas profesionales no son convenciones arbitrarias: cada una tiene una justificación técnica, de mantenibilidad, de accesibilidad o de SEO documentada en la literatura especializada [59, 15]. La adherencia disciplinada a estas prácticas es lo que separa al código profesional del código amateur.

Doce reglas de oro del HTML semántico profesional

1. **Un solo <h1> por página**, que coincide con el título principal y suele ser similar al del <title>.
2. **Jerarquía de encabezados sin saltos**: h1 → h2 → h3, nunca h1 → h3 saltando h2.
3. **Ningún encabezado vacío y ningún encabezado meramente decorativo**: si un texto necesita estilo de título pero no es título, usar <p> con clase CSS.
4. **Listas para listas**: cualquier serie de elementos similares debe ir en , o <dl>, nunca en una serie de <div>.
5. **Tablas solo para datos tabulares**, nunca para maquetación.
6. **Botones para acciones, enlaces para navegación**.
7. **Imágenes con alt descriptivo**; si la imagen es puramente decorativa, alt="" (vacío).
8. **Formularios con <label> asociado a cada <input>**.
9. **Validar siempre** con <https://validator.w3.org/nu/> antes de cada commit.
10. **Atributo lang obligatorio** en <html>.
11. **Evitar
 para crear espacios**: usar CSS.
12. **Mantener IDs únicos**: dos elementos con el mismo id son HTML inválido y rompen tecnologías asistivas.

1.7 Tabla de referencia rápida: tags HTML5 más usados, atributos y valores

La tabla 1.5 compendia los tags HTML5 más frecuentes en el desarrollo profesional, sus atributos principales, los valores admitidos y el efecto esperado. Algunos atributos (id, class, lang, title, hidden, tabindex, eventos onclick etc.) son **globales**: aplican a casi todos los elementos. Estos se listan una sola vez al inicio para evitar repetición.

Tabla 1.5: Tags HTML5 más usados: estructura, contenido y formularios.

Tag	Atributo	Valores / efecto	Ejemplo
<i>Estructura semántica (HTML5)</i>			
<header>	—	Cabecera de página o sección	<header>...</header>
<nav>	—	Bloque de navegación	<nav>...</nav>
<main>	—	Contenido principal (único por página)	<main>...</main>
<article>	—	Contenido autónomo reutilizable	<article>...</article>
<section>	—	Sección temática con encabezado	<section>...</section>
<aside>	—	Contenido relacionado pero secundario	<aside>...</aside>
<footer>	—	Pie de página o sección	<footer>...</footer>
<i>Contenido textual</i>			
<h1>-<h6>	—	Encabezados de niveles 1 a 6	Titulo
<p>	—	Párrafo	Texto.
	—	Énfasis fuerte (importancia)	Atencion
	—	Énfasis (entonación)	realmente
<mark>	—	Texto destacado contextualmente	<mark>resaltado</mark>

Continúa en la siguiente página. . .

(Continuación de la Tabla 1.5)

Tag	Atributo	Valores / efecto	Ejemplo
<time>	datetime	Fecha/hora ISO 8601 legible máquina	<time datetime="2026-05-04">
<code>	—	Fragmento de código	<code>let x=1</code>
<blockquote>	cite	URL fuente de la cita	<blockquote cite="...">
<i>Hipervínculos y enlaces</i>			
<a>	href	URL destino	href="pagina.html"
	target	_blank, _self, _parent	target="_blank"
	rel	noopener, noreferrer, me	rel="noopener"
	download	Fuerza descarga	download="archivo.pdf"
<i>Imágenes y multimedia</i>			
	src	URL de la imagen	src="logo.png"
	alt	Texto alternativo (obligatorio)	alt="Logo UTEQ"
	width, height	Dimensiones (px o %)	width="200"
	loading	lazy, eager	loading="lazy"
<video>	srcset	Múltiples resoluciones	srcset="img-2x.png 2x"
	src	URL del video	src="video.mp4"
	controls	Booleano: controles nativos	controls
	poster	Imagen previa	poster="prev.jpg"
<preload>	preload	none, metadata, auto	preload="metadata"
<i>Listas</i>			
,	—	Lista no/ordenada	...
	—	Ítem de lista	texto
<dl>, <dt>, <dd>	—	Lista de definiciones	—
<i>Tablas</i>			
<table>	—	Datos tabulares	<table>...</table>
<thead>, <tbody>, <tfoot>	—	Secciones	—
<tr>	—	Fila	<tr>...</tr>
<th>	scope	col, row, colgroup	<th scope="col">
<td>	colspan, rowspan	N de celdas a abarcar	colspan="2"
<i>Formularios</i>			
<form>	action	URL del handler	action="/api/users"
	method	GET, POST	method="POST"
	enctype	MIME para uploads	multipart/form-data
	type	text, email, password, number, date, tel, url, checkbox, radio, file, hidden, range, color	type="email"
<input>	name	Nombre del campo	name="email"
	value	Valor inicial	value="default"
	required	Booleano: obligatorio	required
	pattern	Regex de validación	pattern="[0-9]{10}"
<min/max/step>	min/max/step	Rango numérico/fecha	min="0"
	minlength/maxlength	Longitud texto	maxlength="80"
	autocomplete	on, off, name, email...	autocomplete="email"
	for	ID del input asociado	<label for="em">
<label>	multiple	Booleano: múltiple selección	multiple
<select>	value, selected	Valor; preselección	<option value="EC"selected>
<option>	rows, cols	Tamaño visible	rows="5"
<textarea>	type	submit, button, reset	type="submit"
<button>			

Esta tabla está organizada para servir como *referencia rápida* durante la práctica. Para profundizar en los atributos específicos de formularios (tipos de input, validación nativa, accesibilidad de controles) el capítulo 2 dedica una sección completa al tema.

1.7.1 Atributos globales (aplican a casi todos los tags)

Los atributos globales son aquellos que pueden aplicarse a *cualquier* elemento HTML5, no solo a unos pocos. Comprenderlos en profundidad evita reaprenderlos para cada nuevo tag. La tabla 1.6 cataloga los más importantes con su efecto y un ejemplo breve.

Tabla 1.6: Atributos HTML5 globales.

Atributo	Efecto / valores	Ejemplo
id	Identificador único en el documento	id="hero"
class	Una o varias clases CSS separadas por espacio	class="btn primario"
style	CSS inline (uso desaconsejado)	style="color:red"
title	Tooltip flotante al pasar el mouse	title="Cerrar"
lang	Idioma del contenido (ISO 639-1 y 3166-1)	lang="es-EC"
dir	Dirección texto: ltr , rtl , auto	dir="rtl"
hidden	Booleano: oculta el elemento	hidden
tabindex	Orden de tabulación; 0 (natural), -1 (programático)	tabindex="0"
role	Rol ARIA explícito	role="navigation"
aria-label	Etiqueta accesible cuando no hay texto visible	aria-label="Cerrar"
aria-hidden	true/false : oculta para lectores	aria-hidden="true"
data-*	Datos custom para JavaScript	data-id="42"
contenteditable	true/false	contenteditable="true"
draggable	Booleano: arrastrable	draggable="true"

Ejemplo corto comentado (atributos globales en acción) (Listado 1.3):

Listado 1.3: Ejemplo de HTML

```

<!-- 'id' identifica univocamente al elemento (uno por documento) -->
<!-- 'class' agrupa elementos para aplicar el mismo estilo CSS -->
<!-- 'title' muestra un tooltip al pasar el cursor -->
<!-- 'lang' especifica el idioma del contenido (importante a11y) -->
<!-- 'data-*' guarda datos personalizados accesibles desde JS -->
<article id="post-42" class="card_destacado"
        title="Noticia_destacada" lang="es"
        data-autor="ana" data-categoria="tecnologia">
    Contenido del artículo
</article>

```

1.7.2 Tags estructurales y de contenido

HTML5 introduce un conjunto de tags semánticos pensados para describir el *significado* del contenido, no solo su apariencia. La tabla 1.5 cataloga los tags estructurales y de contenido más usados, con sus atributos específicos y un ejemplo de uso para cada uno.

Ejemplo corto comentado (estructura semántica completa) (Listado 1.4):

Listado 1.4: Ejemplo de HTML

```

<!-- <header> contiene la cabecera del sitio o de una sección -->
<header>
  <!-- <nav> agrupa enlaces de navegación principal -->
  <nav>
    <a href="/inicio">Inicio</a>
    <a href="/contacto">Contacto</a>
  </nav>
</header>
<!-- <main> envuelve el contenido único de la página (uno por doc) -->
<main>
  <!-- <article> es contenido autónomo (post, noticia, comentario) -->
  <article>

```

```
<!-- <h1>...<h6> jerarquia de encabezados (h1 = mas importante) -->
<h1>Titulo del articulo</h1>
<!-- <section> agrupa contenido tematicamente relacionado -->
<section>
  <p>Parrafo de texto en la seccion.</p>
</section>
</article>
</main>
<!-- <footer> contiene pie de pagina (copyright, contacto, sitemap) -->
<footer><p>(c) 2026 UTEQ</p></footer>
```

HTML y HTML5 para el desarrollo web

«HTML no es código fuente: es un contrato semántico entre el autor del documento y el ecosistema de agentes que lo interpretan.»

— Ian Hickson, editor de la especificación HTML5

Resultado de aprendizaje del capítulo

Al concluir el capítulo 2, el estudiante construye documentos HTML5 estructuralmente correctos, semánticamente significativos y validables ante el W3C, integrando formularios con validación nativa, contenido multimedia accesible, APIs nativas del lenguaje y entendiendo el lugar de XML en el ecosistema de marcado.

2.1 Historia y evolución de HTML

El lenguaje HTML ha evolucionado durante tres décadas adaptándose a las demandas cambiantes de la web: del documento estático al *single-page application* interactivo. Conocer las etapas de esta evolución ayuda a entender por qué algunos tags existen, por qué otros se deprecaron y por qué HTML5 adoptó las formas actuales. La siguiente caja narra esa historia.

De SGML a HTML Living Standard

La historia de HTML no comienza con HTML sino con SGML (Standard Generalized Markup Language), estándar ISO 8879 publicado en 1986. SGML era un meta-lenguaje complejo que permitía definir lenguajes de marcado específicos para documentos de gobierno, ejército y empresa. Tim Berners-Lee, al inventar la web en 1989, tomó SGML como punto de partida pero buscó una versión radicalmente simplificada: HTML.

La primera descripción pública de HTML data de 1991 en una nota informal titulada «*HTML Tags*» que listaba apenas 18 etiquetas [6]. HTML 2.0 (1995, RFC 1866) fue el primer estándar formal, manteniendo la simplicidad pero añadiendo formularios. HTML 3.2 (W3C, 1997) incorporó tablas, applets y *scripting*. HTML 4.01 (1999) trajo la separación entre estructura y presentación al recomendar el uso de hojas de estilo CSS [55].

A finales de los noventa el W3C decidió reemplazar HTML por XHTML, una versión XML-estricta del lenguaje. Esta decisión, aunque técnicamente elegante, ignoró que la mayoría del HTML real en la web era *tag soup*: marcado mal formado pero que funcionaba en los navegadores. La presión por estandarizar las prácticas reales llevó a la formación, en 2004, del *Web Hypertext Application Technology Working Group* (WHATWG) por parte de Apple, Mozilla y Opera [27]. El WHATWG se desvinculó del proceso XHTML 2.0 y comenzó a desarrollar HTML5.

En 2014 el W3C publicó HTML5 como Recomendación oficial. En 2019 W3C y WHATWG firmaron un memorando reconociendo que el WHATWG sería el único editor del HTML Living Standard [69], abandonando la idea de versiones numeradas. Desde entonces, HTML evoluciona como *Living Standard*: cambios incrementales continuos

sin números de versión formales. Por convención y comodidad, seguimos hablando de «HTML5» para referirnos al HTML moderno.

La evolución de HTML refleja la transformación de la web misma: lo que comenzó como un sistema simple para compartir documentos científicos hipertextuales en el CERN se convirtió en la plataforma de software más ubicua del planeta. Cada versión documenta no solo adiciones técnicas sino también cambios profundos en cómo se entendía la naturaleza del documento web.

La tabla 2.1 resume los hitos. Conviene contextualizarla: las versiones *numeradas* de HTML (HTML 2.0, 3.2, 4.01) corresponden a una etapa en la que el lenguaje se actualizaba mediante *snapshots* formales del W3C. A partir de HTML5, y especialmente desde el memorando de 2019 entre WHATWG y W3C [69], el modelo cambió a *Living Standard*: la especificación se actualiza continuamente sin saltos de versión. Esto refleja la madurez del lenguaje: no necesita grandes revisiones disruptivas, solo evoluciones incrementales.

Tabla 2.1: Hitos de la evolución de HTML.

Versión	Año	Aportes principales
HTML «Tags»	1991	18 etiquetas iniciales (<a>, <p>, <h1>...).
HTML 2.0	1995	Primera estandarización formal (RFC 1866); formularios.
HTML 3.2	1997	Tablas, applets, fuentes, alineación.
HTML 4.01	1999	Marcos, scripts, separación de presentación (CSS).
XHTML 1.0	2000	Reformulación XML de HTML 4.01.
XHTML 1.1	2001	Modularización; estricta sintaxis XML.
HTML5 borrador	2008	WHATWG inicia HTML5; nuevos elementos semánticos.
HTML5 W3C Rec.	2014	Recomendación oficial; <video>, <canvas>, APIs.
HTML 5.1	2016	Mejoras menores (<picture>, <details>).
HTML 5.2	2017	<dialog>, modal nativo.
HTML 5.3	—	Cancelado en favor del Living Standard.
HTML Living Standard	desde 2019	Estado actual; cambios continuos por WHATWG.

La trayectoria muestra tres etapas claramente diferenciadas: (a) la etapa fundacional (1991–1999) donde HTML pasa de prototipo a estándar formal; (b) el periodo de transición XML (2000–2008) en el que el W3C intentó migrar HTML al ecosistema XML con XHTML 1.0 y 2.0, esfuerzo que finalmente fracasó porque la web real era demasiado tolerante a errores [28]; y (c) la era moderna de HTML5 (desde 2008) en la que el lenguaje recupera su pragmatismo original mientras incorpora capacidades multimedia, APIs avanzadas y semántica rica.

2.2 Fundamentos de HTML

HTML (*HyperText Markup Language*) es un lenguaje de marcado declarativo cuyo propósito es describir la estructura semántica de un documento hipertextual. A diferencia de los lenguajes procedurales o imperativos, HTML no «hace» nada: describe *qué es* cada parte del contenido y deja al navegador la responsabilidad de renderizarlo. Como argumentan Duckett [15] y Robbins [59], esta naturaleza declarativa es lo que ha permitido que HTML sobreviva 35 años casi sin cambios sintácticos: agentes muy diversos (navegadores, lectores de pantalla, motores de búsqueda, crawlers, extractores de datos, asistentes de IA) pueden interpretar el mismo documento de maneras distintas pero coherentes.

Sus tres pilares son:

- **Etiquetas (tags).** Construyen la estructura. Vienen en pares de apertura y cierre: <p>texto</p>. Algunas son *void* (no contienen contenido): ,
, <input>.
- **Atributos.** Aportan información adicional sobre los elementos: `class`, `id`, `lang`, `aria-*`. Se escriben dentro de la etiqueta de apertura como pares `nombre="valor"`.
- **Contenido.** El texto y los elementos hijos que se anidan dentro de las etiquetas.

2.2.1 Estructura mínima de un documento HTML5 válido

Todo documento HTML5 válido debe contener cinco elementos imprescindibles: (i) la declaración <!DOCTYPE html> en la primera línea; (ii) un elemento raíz <html> con atributo `lang`; (iii) una sección <head> con metadatos (al menos <meta

`charset` y `<title>`); (iv) una sección `<body>` con el contenido visible; y (v) idealmente, una estructura semántica que separe cabecera, navegación, contenido principal y pie.

El siguiente código presenta una página HTML5 completa que ilustra el uso de **todos** los tags principales: tanto los nuevos tags semánticos de HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<time>`) como los tags clásicos que siguen siendo fundamentales (`<div>`, `<table>`, `<form>`, `<ul>`, `<a>`, `<img>`). Este código sirve como **patrón canónico** de referencia para construir cualquier página web.

Ejemplo 2.1 — Documento HTML5 completo con tags clásicos y semánticos

Enunciado. Construir un documento HTML5 mínimo pero completo que incluya todos los tags semánticos esenciales (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`) y los tags clásicos de organización (`<div>`, `<table>`). Servirá como página de referencia para el resto de los ejercicios.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <meta name="description" content="Pagina demo con todos los tags
   clave">
7   <meta name="author" content="UTEQ - Aplicaciones Web">
8   <title>Pagina completa con tags clave - UTEQ</title>
9 </head>
10 <body>
11
12 <!-- Tags HTML5 semanticos: estructura -->
13 <header>
14   <h1>Universidad Tecnica Estatal de Quevedo</h1>
15   <p>La primera universidad agropecuaria del Ecuador</p>
16   <nav aria-label="Menu principal">
17     <ul>
18       <li><a href="#inicio">Inicio</a></li>
19       <li><a href="#carreras">Carreras</a></li>
20       <li><a href="#contacto">Contacto</a></li>
21     </ul>
22   </nav>
23 </header>
24
25 <main>
26   <article id="inicio">
27     <h2>Bienvenidos al sitio web</h2>
28     <p>Publicado el <time datetime="2026-05-04">4 de mayo
29     de 2026</time>.</p>
30     <figure>
31       
34       <figcaption>Campus central UTEQ - Quevedo</figcaption>
35     </figure>
36   </article>
37
38   <section id="carreras">
39     <h2>Carreras de la Facultad</h2>

```

```

40
41 <!-- Tag clasico: tabla para datos tabulares -->
42 <table>
43   <caption>Carreras de Ciencias de la Computacion</caption>
44   <thead>
45     <tr><th scope="col">Carrera</th>
46       <th scope="col">Duracion</th>
47       <th scope="col">Titulo</th></tr>
48   </thead>
49   <tbody>
50     <tr><td>Ingenieria de Software</td>
51       <td>9 semestres</td>
52       <td>Ing. de Software</td></tr>
53     <tr><td>Ingenieria en Sistemas</td>
54       <td>9 semestres</td>
55       <td>Ing. de Sistemas</td></tr>
56   </tbody>
57 </table>
58 </section>
59
60 <section id="contacto">
61   <h2>Contactenos</h2>
62
63   <!-- Tag clasico: formulario -->
64   <form action="/api/contacto" method="POST">
65     <!-- Tag clasico: div como contenedor generico -->
66     <div>
67       <label for="nombre">Nombre:</label>
68       <input id="nombre" name="nombre" type="text" required>
69     </div>
70     <div>
71       <label for="email">Correo:</label>
72       <input id="email" name="email" type="email" required>
73     </div>
74     <div>
75       <label for="msg">Mensaje:</label>
76       <textarea id="msg" name="msg" rows="4"></textarea>
77     </div>
78     <button type="submit">Enviar</button>
79   </form>
80 </section>
81
82 <aside aria-label="Datos adicionales">
83   <h2>Datos rapidos</h2>
84   <ul>
85     <li>Fundacion: <strong>1984</strong></li>
86     <li>Estudiantes: <strong>13.000+</strong></li>
87     <li>Programas: <strong>22</strong></li>
88   </ul>
89 </aside>
90 </main>

```

```

91
92     <footer>
93         <small>&copy; 2026 UTEQ. Todos los derechos reservados.</small>
94         <address>
95             Av. Quito km 1.5 via a Santo Domingo<br>
96             Telefono: <a href="tel:+59353702220">(+593) 5 3702-220</a>
97         </address>
98     </footer>
99
100 </body>
101 </html>

```

Listado 2.1: pagina-completa.html

Explicación de las decisiones tomadas:

- `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`: la estructura semántica de HTML5 reemplaza el uso clásico de `<div class="header">`, `<div id="content">`.
- `<table>`: usada *solo* para datos tabulares, con `<caption>`, `<thead>`, `<tbody>` y atributo `scope` en los `<th>`, conforme a las prácticas accesibles [59].
- `<form>`, `<input>`, `<label>`: cada `<input>` tiene su `<label for>` asociado (criterio WCAG 1.3.1).
- `<div>`: usado solo cuando no hay un elemento semántico apropiado, como contenedor neutro para cada grupo `<label>+<input>`.
- `<time datetime>`: fecha en formato ISO 8601 legible por máquinas.

2.2.2 Las cinco categorías de contenido en HTML5

HTML5 abandona la dicotomía clásica *block-level* / *inline* (propia de HTML 4) y la sustituye por las **categorías de contenido**, que se solapan y permiten razonar con mayor precisión sobre qué puede contener qué [28]. Este cambio conceptual es significativo: antes, los desarrolladores aprendían reglas mecánicas («un `<p>` no puede ir dentro de un `<a>`»); ahora aprenden categorías («un elemento de *phrasing content* no admite *flow content*»), lo que generaliza mucho mejor.

A continuación se presentan las cinco categorías principales con ejemplos de uso en una página web real:

1. **Flow content** (la mayoría del contenido del cuerpo): texto, párrafos, listas, encabezados, imágenes, formularios, secciones. Casi todo lo que puede ir en `<body>` es flow content. *Ejemplo de uso*: dentro de un `<section>` podemos poner `<h2>`, `<p>`, `<ul>`, `<table>`, `<form>`: todos son flow content.
2. **Phrasing content** (texto y sus marcas en línea): `<em>`, `<strong>`, `<a>`, `<span>`, `<code>`, `<time>`, `<mark>`. Es un subconjunto de *flow*. *Ejemplo de uso*: dentro de un párrafo podemos escribir `<p>El libro <em>1984</em> fue publicado en <time datetime="1949">1949</time>.</p>`.
3. **Embedded content** (contenido externo o interactivo incrustado): `<img>`, `<video>`, `<audio>`, `<canvas>`, `<iframe>`, `<object>`, `<svg>`. *Ejemplo de uso*: dentro de un `<figure>` incrustamos `<img>` y agregamos `<figcaption>`.
4. **Interactive content** (elementos con los que el usuario interactúa): `<a>` con `href`, `<button>`, `<input>` (excepto `type="hidden"`), `<select>`, `<textarea>`, `<details>`, `<label>`. *Ejemplo de uso*: dentro de un `<form>` ponemos varios `<input>` interactivos.
5. **Sectioning content** (secciones del documento): `<article>`, `<section>`, `<nav>`, `<aside>`. Cada uno define un nuevo «contexto de outline» que afecta la jerarquía de encabezados. *Ejemplo de uso*: la página de la sección anterior tiene un `<article>` (sectioning content) que contiene un `<h2>` de bienvenida.

Existen además categorías menos comunes: *metadata content* (`<meta>`, `<link>`, `<style>`, `<title>`), *heading content* (`<h1>`-`<h6>`), *form-associated content* (`<input>`, `<button>`, etc.). Un mismo elemento puede pertenecer a varias categorías simultáneamente: por ejemplo, `<a>` es flow, phrasing e interactive.

2.2.3 Modelo de contenido y reglas de anidamiento

Cada elemento HTML define qué puede contener y qué no, mediante un **modelo de contenido** declarado en la especificación. Estas reglas no son arbitrarias: derivan de la semántica del elemento y de limitaciones técnicas históricas. Comprenderlas evita errores sutiles que pasan desapercibidos a primera vista pero rompen la accesibilidad o el SEO.

Reglas fundamentales con ejemplos:

1. **<p> solo admite phrasing content.** Por eso un párrafo no puede contener un `<div>` o un `<table>`. **Anti-ejemplo (inválido) (Listado 2.2):**

```
<p>Aquí hay una tabla:
  <table><tr><td>X</td></tr></table>
</p>
```

Listado 2.2: Ejemplo de HTML

Correcto: cerrar el `<p>` antes de la tabla.

2. **<a> acepta cualquier contenido excepto interactive content.** Por eso un enlace no puede contener otro enlace, ni un `<button>`, ni un `<input>`. **Anti-ejemplo (inválido) (Listado 2.3):**

```
<a href="/x"><button>Click</button></a>
```

Listado 2.3: Ejemplo de HTML

Correcto: usar solo `<a>` con texto y CSS para darle apariencia de botón, o solo `<button>` con `onclick` que redirija.

3. **<button> no admite interactive content.** Por eso no se puede tener un `<a>` dentro de un `<button>`.
4. ** y solo aceptan (y opcionalmente <script>/<template>).** **Anti-ejemplo (inválido) (Listado 2.4):**

```
<ul>
  <h2>Items</h2>
  <li>Uno</li>
  <li>Dos</li>
</ul>
```

Listado 2.4: Ejemplo de HTML

Correcto: el encabezado va *antes* de la `<ul>`.

5. **<table> requiere estructura específica:** solo admite `<caption>`, `<colgroup>`, `<thead>`, `<tbody>`, `<tfoot>`, `<tr>` (este último solo sin `<thead>/<tbody>`). Las filas `<tr>` solo admiten `<td>` y `<th>`.
6. **<form> no puede contener otro <form>.** Pero sí puede tener `<input form="otroForm">` que asocia un input con un form externo (técnica avanzada).

¿Por qué el navegador es indulgente? Los navegadores intentan recuperarse de HTML inválido por compatibilidad histórica. Sin embargo:

- Los *validadores* (W3C) reportan errores.
- Los *lectores de pantalla* pueden anunciar contenido incoherente.
- Los *motores de SEO* penalizan estructuras inválidas.
- Las *herramientas automatizadas* (extracción de datos, asistentes de IA) producen resultados impredecibles.

Por estas razones, todo HTML profesional debe pasar el W3C Markup Validator [71] sin errores antes de ir a producción.

2.3 Elementos semánticos de HTML5 en profundidad

Antes de HTML5 la mayoría de la maquetación se hacía con `<div>` genéricos diferenciados por clases CSS. Este enfoque *semánticamente ciego* tenía varios problemas documentados [35]: los motores de búsqueda no entendían la estructura del contenido, los lectores de pantalla no podían ofrecer navegación rápida entre regiones, y el código se volvía ilegible para humanos. HTML5 introdujo un vocabulario semántico explícito que mejora la accesibilidad, el SEO, la mantenibilidad y la interoperabilidad con herramientas de *reader mode*, extracción automatizada y asistentes de IA.

2.3.1 Catálogo de elementos semánticos

La tabla 2.2 sintetiza los elementos semánticos esenciales de HTML5 con su uso correcto e incorrecto. La razón para incluir ambas columnas es pedagógica: en los entregables del curso, los errores más frecuentes consisten en usar elementos semánticos para fines genéricos (por ejemplo, `<section>` como contenedor visual sin encabezado), lo que invalida su semántica.

Tabla 2.2: Elementos semánticos esenciales de HTML5 con su uso correcto e incorrecto.

Elemento	Uso correcto	Uso incorrecto
<code><header></code>	Cabecera de página o sección	Como contenedor genérico
<code><nav></code>	Bloque de navegación principal	Para listas de enlaces secundarios
<code><main></code>	Contenido principal único	En múltiples instancias
<code><article></code>	Contenido autónomo y reutilizable	Para cualquier párrafo
<code><section></code>	Agrupación temática con encabezado	Sin <code><h2>/<h3></code> dentro
<code><aside></code>	Contenido relacionado pero secundario	Para barras laterales decorativas
<code><footer></code>	Pie de página o sección	Para contenido principal
<code><figure></code>	Imagen, código o tabla autocontenida	Para cualquier imagen
<code><figcaption></code>	Pie de figura	Fuera de <code><figure></code>
<code><time></code>	Fecha u hora marcada para máquinas	Sin atributo <code>datetime</code>
<code><mark></code>	Texto destacado contextualmente	Como sinónimo de <code></code>
<code><details>+<summary></code>	Bloque colapsable	Para acordeones complejos
<code><dialog></code>	Modal nativo	Para tooltips simples
<code><address></code>	Datos de contacto	Para direcciones postales arbitrarias

Para cada elemento, la regla mnemotécnica es: «*si puedo explicar en una frase qué significa este bloque, debo poder elegir el elemento semántico apropiado; si solo puedo describirlo por su apariencia visual, probablemente debo usar un `<div>`*».

2.3.2 ¿`<article>` o `<section>`? Guía decisonal con ejemplo real

La confusión más común es cuándo usar `<article>` y cuándo `<section>`. Ambos son *sectioning content*, ambos crean un nuevo contexto de outline, pero tienen semánticas distintas [35, 59].

Regla heurística canónica:

- `<article>`: si el bloque de contenido podría *publicarse de manera independiente* en otro contexto y seguir teniendo sentido, es un artículo. La «prueba RSS»: ¿tendría sentido incluir este bloque en un feed RSS aislado de su contexto?

Ejemplos canónicos: una entrada de blog, una noticia, un comentario en un foro, una tarjeta de producto, un *tweet*, una reseña.

- `<section>`: si el bloque es una agrupación temática que solo tiene sentido como parte del documento mayor.

Ejemplos: la sección «Estudios» de un currículum, el capítulo de un libro, la sección «Características» de una landing page.

Ejemplo real con código completo: blog con múltiples entradas.

Ejemplo 2.2 — Uso correcto de `<article>` y `<section>`

```

1 <main>
2   <h1>Blog de Aplicaciones Web - UTEQ</h1>
3
```

```

4  <!-- ARTICLE: entrada que puede publicarse en RSS independiente -->
5  <article>
6      <header>
7          <h2>Por que HTML5 cambio el desarrollo web</h2>
8          <p>Por <a href="/autor/jdoe" rel="author">Juan Doe</a>
9              el <time datetime="2026-05-04">4 mayo 2026</time></p>
10         </header>
11
12         <p>Introduccion al articulo...</p>
13
14         <!-- SECTION dentro de article: agrupacion tematica interna -->
15         <section>
16             <h3>Elementos semanticos nuevos</h3>
17             <p>HTML5 introdujo header, nav, main...</p>
18         </section>
19
20         <section>
21             <h3>APIs multimedia nativas</h3>
22             <p>Las etiquetas video y audio reemplazaron Flash...</p>
23         </section>
24
25         <footer>
26             <p>Etiquetas:
27                 <a href="/tag/html5">HTML5</a>,
28                 <a href="/tag/web">Web</a>
29             </p>
30         </footer>
31     </article>
32
33     <!-- Otro ARTICLE independiente -->
34     <article>
35         <header>
36             <h2>CSS Grid: el sistema de maquetacion definitivo</h2>
37             <p>Por <a href="/autor/asmith">Ana Smith</a>
38                 el <time datetime="2026-04-28">28 abril 2026</time></p>
39         </header>
40         <p>CSS Grid revoluciono la maquetacion web...</p>
41     </article>
42
43     <!-- SECTION: agrupacion que NO tendria sentido fuera del blog -->
44     <section aria-label="Articulos relacionados">
45         <h2>Tambien te puede interesar</h2>
46         <ul>
47             <li><a href="/post/1">Una guia rapida de Flexbox</a></li>
48             <li><a href="/post/2">JavaScript moderno en 2026</a></li>
49         </ul>
50     </section>
51 </main>

```

Listado 2.5: blog.html (extracto significativo)

Razonamiento de las decisiones:

- Cada entrada del blog es `<article>`: si se extrae y publica en otro sitio o en un feed, tiene sentido por sí misma.
- Dentro de cada artículo, las `<section>` agrupan subtemas pero *no* podrían publicarse independientemente: «APIs multimedia nativas» sin el contexto del artículo carece de significado.
- «Artículos relacionados» es una `<section>` porque es una agrupación temática (lista de enlaces) que pertenece a la página del blog pero no es *contenido* autónomo.

2.4 Formularios HTML5 y validación nativa

Los formularios constituyen el principal mecanismo de entrada de datos del usuario hacia el servidor. HTML5 amplió drásticamente las capacidades de validación nativa, reduciendo la dependencia de JavaScript para tareas básicas y mejorando significativamente la experiencia del usuario en dispositivos móviles [45]. Con HTML5 podemos confiar en el navegador para verificar formato de email, rangos numéricos, longitudes mínimas y máximas, y patrones complejos mediante expresiones regulares.

Es importante destacar: **la validación HTML5 es para usabilidad, no para seguridad**. Un atacante puede saltarse fácilmente las validaciones del cliente; por eso la validación robusta debe duplicarse en el servidor (capítulos 5–9). Sin embargo, la validación HTML5 mejora la experiencia ofreciendo retroalimentación inmediata al usuario *antes* de enviar el formulario.

2.4.1 Tipos de input modernos

A los tipos clásicos de input que existían desde HTML 2.0 (`text`, `password`, `checkbox`, `radio`, `submit`, `hidden`, `file`), HTML5 añadió **trece tipos modernos** que tres ventajas operacionales fundamentales: (i) validación automática del formato (correo, URL, número), (ii) interfaces de entrada especializadas (calendarios, selectores de color, sliders) y (iii) en móviles, invocan automáticamente el teclado virtual más apropiado para el tipo de dato (numérico para `tel`, con `@` para `email`, etc.).

La tabla 2.3 cataloga estos tipos modernos junto con su uso recomendado y la validación nativa que cada uno implementa. Conviene memorizar esta tabla: usar el tipo correcto de input es una de las maneras más simples y efectivas de mejorar la usabilidad de un formulario.

Tabla 2.3: Tipos de `<input>` de HTML5 y su validación nativa.

Tipo	Uso	Validación nativa
<code>email</code>	Correo electrónico	Formato <code>x@y.z</code>
<code>tel</code>	Teléfono	No valida formato (varía por país); usar <code>pattern</code>
<code>url</code>	URL	Formato URL absoluto
<code>number</code>	Número	Solo dígitos; <code>min/max/step</code>
<code>range</code>	Rango (slider)	Numérico; sin teclado de entrada
<code>date</code>	Fecha	Calendario nativo
<code>time</code>	Hora	Selector horario
<code>datetime-local</code>	Fecha y hora local	Calendario + reloj
<code>month</code>	Año-mes	Selector de mes
<code>week</code>	Semana del año	Selector de semana
<code>color</code>	Color	Selector de color
<code>search</code>	Búsqueda	Cosmética: borrar con X

En móviles, además, cada tipo de input invoca el teclado virtual adecuado: `email` muestra el carácter `@`; `number` y `tel` muestran el teclado numérico; `url` muestra `.com` y `/`. Esto reduce drásticamente los errores de tipeo en pantallas pequeñas.

2.4.2 Atributos de validación

Los siguientes atributos permiten especificar restricciones que el navegador valida automáticamente antes de enviar el formulario:

- **required**: el campo no puede enviarse vacío.

- `minlength` / `maxlength`: rango de longitud de texto.
- `min` / `max`: rango numérico o temporal.
- `pattern`: expresión regular (sintaxis JavaScript) que debe cumplir el valor.
- `step`: incremento permitido en numéricos y fechas (`step="0.01"` para dinero, `step="60"` para minutos).
- `autocomplete`: directrices al navegador (`on`, `off`, `name`, `email`, `tel`, `cc-number`, `street-address`, `country`, `bday`...).
- `novalidate` (en `<form>`): deshabilita la validación nativa.
- `multiple`: permite múltiples valores (`<input type="email" multiple>`).
- `accept`: tipos MIME permitidos en `<input type="file">`.

Ejemplo de formulario completo con validación nativa:

Ejemplo 2.3 — Formulario de inscripción a curso con validación HTML5 completa

Enunciado. Construir un formulario de inscripción a curso con validación HTML5 nativa (`required`, `type`, `pattern`, `min/max`) que rechace datos inválidos antes de enviarlos al servidor.

```

1 <form action="/api/inscripcion" method="POST">
2   <fieldset>
3     <legend>Datos personales</legend>
4
5     <label for="cedula">Cedula (10 digitos):</label>
6     <input id="cedula" name="cedula" type="text"
7       pattern="[0-9]{10}" required
8       autocomplete="off"
9       aria-describedby="ayuda-cedula">
10    <small id="ayuda-cedula">Sin guiones ni espacios</small>
11
12    <label for="nombre">Nombre completo:</label>
13    <input id="nombre" name="nombre" type="text"
14      minlength="3" maxlength="80" required
15      autocomplete="name">
16
17    <label for="email">Correo institucional:</label>
18    <input id="email" name="email" type="email"
19      pattern=".+@uteq\.edu\.ec" required
20      autocomplete="email">
21
22    <label for="tel">Telefono celular:</label>
23    <input id="tel" name="tel" type="tel"
24      pattern="09[0-9]{8}" required
25      autocomplete="tel">
26
27    <label for="fnac">Fecha de nacimiento:</label>
28    <input id="fnac" name="fnac" type="date"
29      min="1980-01-01" max="2010-12-31" required
30      autocomplete="bday">
31  </fieldset>
32
33  <fieldset>

```

```

34     <legend>Curso</legend>
35
36     <label for="nivel">Nivel academico (1-10):</label>
37     <input id="nivel" name="nivel" type="number"
38           min="1" max="10" step="1" required>
39
40     <label for="prom">Promedio acumulado:</label>
41     <input id="prom" name="prom" type="number"
42           min="6.0" max="10.0" step="0.01" required>
43
44     <label for="hrs">Horas semanales de estudio:</label>
45     <input id="hrs" name="hrs" type="range"
46           min="0" max="40" step="1" value="20">
47
48     <label for="carrera">Carrera:</label>
49     <select id="carrera" name="carrera" required>
50       <option value="">-- Seleccione --</option>
51       <option value="SW">Ingenieria de Software</option>
52       <option value="SE">Ingenieria de Sistemas</option>
53     </select>
54 </fieldset>
55
56 <button type="submit">Inscribirme</button>
57 <button type="reset">Limpiar</button>
58 </form>

```

Listado 2.6: inscripcion.html (formulario funcional)

Validaciones automáticas que aplica el navegador:

- Si la cédula no tiene exactamente 10 dígitos: error nativo.
- Si el email no termina en @uteq.edu.ec: error nativo.
- Si el teléfono no comienza con 09: error nativo.
- Si el promedio está fuera de rango 6.0–10.0: error nativo.
- Si el campo carrera no se selecciona: error nativo.

Todo esto sin escribir una sola línea de JavaScript.

Nota didáctica: las pseudo-classes CSS para dar retroalimentación visual al estado de validación (:valid, :invalid, :required, :user-invalid) se estudiarán en detalle en el capítulo 3, cuando dispongamos del marco conceptual completo de CSS. Por ahora, basta con saber que existen y que podemos darles estilo al campo según esté correctamente lleno o no.

2.5 Multimedia y APIs de HTML5

HTML5 incorporó al estándar nativo cinco APIs de gran importancia práctica que reemplazaron tecnologías propietarias como Flash y Silverlight [36]. Su importancia no es solo técnica: políticamente representó la victoria de los estándares abiertos sobre los plugins propietarios. Apple inició la batalla en 2010 cuando Steve Jobs publicó la carta abierta «Thoughts on Flash» [30] declarando que iOS nunca soportaría Flash. La transición a HTML5 nativo se completó alrededor de 2020, cuando Adobe discontinuó oficialmente Flash Player.

Las cinco APIs nativas son:

1. <video> y <audio>: reproducción multimedia nativa.
2. <canvas>: gráficos 2D y 3D (con WebGL).

3. **Geolocation API**: posición geográfica del usuario con consentimiento.
4. **Web Storage**: `localStorage` y `sessionStorage`.
5. **Drag-and-Drop API**: arrastrar y soltar entre componentes.

2.5.1 La etiqueta `<video>` con accesibilidad completa

La etiqueta `<video>` es uno de los elementos más complejos de HTML5 porque integra múltiples preocupaciones: formato del video, controles de reproducción, accesibilidad para usuarios sordos (subtítulos), accesibilidad para usuarios ciegos (audiodescripciones), optimización de carga (`preload`, `poster`) y compatibilidad con navegadores distintos (múltiples `<source>`).

Para construir un video accesible profesional necesitamos conocer los siguientes atributos y elementos asociados:

- **controls**: muestra los controles nativos del navegador (reproducir, pausa, volumen, pantalla completa).
- **preload**: indica al navegador cuánto descargar antes de reproducir. Valores: `none` (nada), `metadata` (solo cabeceras: duración y dimensiones), `auto` (todo).
- **poster**: imagen que se muestra antes de iniciar la reproducción (mejora la percepción de carga).
- **autoplay**: reproducción automática (la mayoría de navegadores la bloquean si el video tiene audio, por buenas razones).
- **loop**: reproducción en bucle.
- **muted**: silenciado por defecto (necesario para autoplay).
- Múltiples `<source>`: el navegador usa el primer formato que reconoce. Buena práctica: WebM primero (Chrome/Firefox), MP4 después (Safari/Edge).
- `<track>`: pistas de subtítulos, descripciones, metadatos. Sus tipos (`kind`):
 - **captions**: subtítulos para usuarios sordos en el idioma del video (criterio WCAG 1.2.2).
 - **subtitles**: subtítulos traducidos a otro idioma.
 - **descriptions**: descripciones de audio para usuarios ciegos (criterio WCAG 1.2.5).
 - **chapters**: marcadores de capítulos.
 - **metadata**: datos para scripts.

Ejemplo 2.4 — Video accesible profesional

Enunciado. Mostrar la etiqueta `<video>` con todos los atributos de accesibilidad necesarios en producción: `controls`, `preload`, `poster`, pistas de subtítulos con `<track>` y fuentes alternativas con `<source>`.

```
1 <figure>
2   <video controls width="640"
3       preload="metadata"
4       poster="poster-uteq.jpg"
5       crossorigin="anonymous">
6     <source src="bienvenida.webm" type="video/webm">
7     <source src="bienvenida.mp4" type="video/mp4">
8     <track kind="captions" src="bienvenida.es.vtt"
9         srclang="es" label="Español" default>
10    <track kind="captions" src="bienvenida.en.vtt"
11        srclang="en" label="English">
12    <track kind="descriptions" src="bienvenida.desc.vtt">
```

```

13         srclang="es" label="Audiodescripcion">
14     <p>Su navegador no soporta video HTML5.
15         <a href="bienvenida.mp4">Descargar el video</a>.</p>
16 </video>
17 <figcaption>Mensaje de bienvenida del Rector --- Mayo
        2026</figcaption>
18 </figure>

```

Listado 2.7: Ejemplo de HTML

Aspectos clave de este código:

- Múltiples <source>: el navegador elige el primero soportado.
- Subtítulos en español y en inglés (default marca el predeterminado).
- <track kind="descriptions">: audiodescripciones para usuarios ciegos.
- Texto fallback dentro de <video>: navegadores muy antiguos.
- Envuelto en <figure>+<figcaption> para semántica correcta.

Formato del archivo .vtt (WebVTT) para subtítulos (Listado 2.8):

```

1 WEBVTT
2
3 00:00:00.000 --> 00:00:04.000
4 Bienvenidos a la Universidad Tecnica Estatal de Quevedo.
5
6 00:00:04.500 --> 00:00:08.000
7 Soy el Rector y les hablo desde nuestro campus central.

```

Listado 2.8: bienvenida.es.vtt

2.5.2 El elemento <picture> para imágenes responsivas

El elemento <picture> resuelve un problema concreto: servir *la imagen correcta para cada dispositivo*. Antes de <picture>, la web servía la misma imagen a todos los dispositivos, lo que penalizaba a usuarios móviles con redes lentas: una imagen optimizada para desktop (1200 px de ancho) cargaba también en un teléfono que solo necesitaba 400 px, gastando datos innecesarios y prolongando tiempos de carga.

<picture> permite al navegador elegir la imagen óptima según: (i) el tamaño del viewport (media queries), (ii) la densidad de pixels (Retina, etc.) y (iii) el formato soportado (WebP es 25–35 % más pequeño que JPEG, AVIF es 50 % más pequeño aún, pero no todos los navegadores los soportan).

Ejemplo 2.5 — Imagen optimizada para múltiples dispositivos

Enunciado. Implementar una imagen responsive con <picture> y <source> para servir el formato y tamaño más apropiados según el dispositivo y soporte del navegador.

```

1 <picture>
2 <!-- Para pantallas grandes con AVIF soportado -->
3 <source media="(min-width: 1200px)"
4         srcset="banner-grande.avif"
5         type="image/avif">
6 <source media="(min-width: 1200px)"
7         srcset="banner-grande.webp"
8         type="image/webp">
9 <source media="(min-width: 1200px)"
10        srcset="banner-grande.jpg"

```

```

11         type="image/jpeg">
12
13     <!-- Para tablets -->
14     <source media="(min-width: 600px)"
15           srcset="banner-medio.webp"
16           type="image/webp">
17
18     <!-- Fallback para todos -->
19     
23 </picture>

```

Listado 2.9: Ejemplo de HTML

Beneficios:

- El navegador descarga *solo* la versión adecuada al viewport y al formato soportado.
- AVIF/WebP reducen el peso 25–50 % respecto a JPEG con calidad equivalente.
- `loading="lazy"` difiere la descarga hasta que la imagen está cerca del viewport.
- Las dimensiones explícitas (`width`, `height`) evitan el *layout shift* durante la carga.

2.6 XML y familias de lenguajes de marcado

HTML pertenece a una familia mayor de lenguajes de marcado que descienden de SGML. XML (*eXtensible Markup Language*) es el otro gran descendiente de SGML, estandarizado por el W3C en 1998 [72]. La diferencia fundamental: HTML tiene un vocabulario fijo (los tags están predefinidos); XML es *extensible*, el autor define su propio vocabulario.

Esta extensibilidad permitió que XML se convirtiera en el formato de intercambio dominante en sistemas empresariales durante los 2000s. Aunque desde 2015 ha sido desplazado parcialmente por JSON en APIs REST modernas, XML sigue siendo esencial en muchos contextos: configuración de aplicaciones Java (Maven, Spring), servicios web SOAP, sindicación de contenidos, formatos de documentos (DOCX, ODT, EPUB son ZIP de archivos XML).

2.6.1 Subdialectos de XML relevantes en aplicaciones web

XML como tecnología generó múltiples subdialectos especializados que siguen vigentes en distintas áreas. Conocer cuáles están en uso en aplicaciones web reales permite reconocerlos cuando aparecen en bibliotecas, configuraciones o servicios externos. La lista siguiente enumera los más relevantes en 2026.

- **XHTML**: HTML reformulado como XML estricto. Hoy relegado pero existe en casos específicos (correos electrónicos, algunos sistemas legados).
- **SVG** (Scalable Vector Graphics): gráficos vectoriales, nativos en HTML5. Iconos, gráficos, diagramas que escalan sin pérdida de calidad.
- **MathML**: notación matemática. Usada en sitios académicos y educativos.
- **RSS** y **Atom**: sindicación de contenido.
- **SOAP**: servicios web XML (capítulo 15).
- **XSLT**: transformaciones XML a otros formatos.
- **DocBook**: documentación técnica.
- **XAML**: interfaces de WPF y WinUI.

2.6.2 Formatos de intercambio de datos: XML, JSON, YAML, TOML y otros

XML y JSON no son los únicos formatos de intercambio de datos relevantes en 2026. Para que el desarrollador profesional pueda elegir bien, conviene conocer el panorama completo. Cada formato tiene fortalezas y casos de uso óptimos.

Tabla 2.4: Formatos de intercambio de datos modernos.

Formato	Características	Casos de uso 2026
XML	Verboso; etiquetas anidadas con atributos; validación con XSD; comentarios soportados.	Maven <code>pom.xml</code> , SOAP, SVG, configuración legada (Spring XML).
JSON	Sintaxis tipo objeto JS; tipos básicos; sin comentarios; soporte nativo en JavaScript.	APIs REST, <code>package.json</code> , NoSQL (MongoDB).
YAML	Indentación significativa; muy legible; tipos inferidos; soporta comentarios.	Docker Compose, Kubernetes, GitHub Actions, Spring Boot <code>application.yml</code> .
TOML	Sintaxis INI mejorada; tipado explícito; secciones; pensado para configuración.	Rust (<code>Cargo.toml</code>), Python (<code>pyproject.toml</code>).
INI	Formato <code>clave=valor</code> con secciones; muy simple.	Configuración legada en Windows, PHP <code>php.ini</code> .
CSV	Tabular plano; valores separados por comas.	Exportación a Excel, ETL básico.
Protobuf	Binario; eficiente; requiere esquema <code>.proto</code> ; soporte multi-lenguaje.	gRPC, comunicación servicio-servicio de alto rendimiento.
MessagePack	Binario tipo JSON; más compacto.	Comunicación IoT, caches binarias.
Avro	Binario; esquema JSON embebido.	Apache Kafka, big data, streaming.
Parquet	Columnar binario; comprimido.	Análisis de datos a escala (Spark, BigQuery).
GraphQL	Lenguaje de consulta; respuesta JSON.	APIs con clientes que necesitan flexibilidad.

Comparativa XML vs JSON (la elección más frecuente en desarrollo web):

Tabla 2.5: XML vs JSON — comparación detallada.

Criterio	XML	JSON
Verbosidad	Alta (etiquetas de apertura y cierre)	Baja (sintaxis tipo objeto)
Tipos de datos	Solo strings, requiere XSD para tipado	Strings, números, booleanos, null, arrays, objetos
Validación	XSD, DTD, RelaxNG	JSON Schema
Soporte JavaScript	Requiere parsing manual	<code>JSON.parse()</code> nativo
Atributos	Sí (atributos en elementos)	No (todo son propiedades)
Comentarios	Sí (<code><!-- --></code>)	No (JSON5 permite)
Tamaño típico	100 % (referencia)	60–70 % del XML equivalente
Casos de uso 2026	Sistemas legados, configuración (Maven, Spring XML)	APIs REST, configuración (<code>package.json</code>)

Ejemplo comparativo: la misma información en cuatro formatos.

XML (Listado 2.10):

```
<usuario id="42">
  <nombre>Maria Lopez</nombre>
  <edad>23</edad>
  <activo>true</activo>
  <roles>
    <rol>USER</rol>
    <rol>EDITOR</rol>
  </roles>
</usuario>
```

Listado 2.10: Ejemplo de XML

JSON (Listado 2.11):

```
{  
  "id": 42,  
  "nombre": "Maria Lopez",  
  "edad": 23,  
  "activo": true,  
  "roles": ["USER", "EDITOR"]  
}
```

Listado 2.11: Ejemplo de JavaScript

YAML (Listado 2.12):

Listado 2.12: Ejemplo de Código

```
id: 42  
nombre: Maria Lopez  
edad: 23  
activo: true  
roles:  
  - USER  
  - EDITOR
```

TOML (Listado 2.13):

```
id = 42  
nombre = "Maria Lopez"  
edad = 23  
activo = true  
roles = ["USER", "EDITOR"]
```

Listado 2.13: Ejemplo de Shell

2.7 Buenas prácticas profesionales en HTML5

Conocer la sintaxis no basta para escribir HTML profesional: hay convenciones, criterios de mantenibilidad y restricciones de accesibilidad que distinguen el código de producción del código estudiantil. La caja siguiente compila quince reglas que el estudiante debe respetar al entregar HTML5 en cualquier proyecto.

Quince reglas profesionales de HTML5

1. Usa el doctype HTML5 simple `<!DOCTYPE html>`.
2. Define el charset al inicio del `<head>`, en los primeros 1024 bytes del archivo.
3. Define el viewport en toda página que pretenda ser mobile-friendly.
4. Cierra todas las etiquetas (incluso las que HTML5 permite no cerrar).
5. Usa comillas dobles en valores de atributos por consistencia.
6. No abuses de `<div>`: si existe un elemento semántico, úsalo.
7. Atributos booleanos sin valor.
8. Imágenes con alt siempre.
9. Imágenes con width y height para evitar *layout shift*.
10. Lazy loading en imágenes *below the fold*.
11. Formularios con `<label>` asociados.
12. Atributo autocomplete con valores estándares.
13. Headings sin saltos jerárquicos.

14. Validar en <https://validator.w3.org/nu/> antes de cada commit.
15. No uses CSS inline excepto en correos electrónicos.

2.8 Ejercicios

Los siguientes ejercicios consolidan los temas del capítulo 2. El primero está completamente resuelto con código y captura de pantalla del resultado; el segundo es propuesto para que el estudiante lo desarrolle.

Ejercicio 2.1 (RESUELTO) — Formulario de registro completo y accesible

Objetivo: construir un formulario completo con ocho tipos diferentes de `<input>`, validación HTML5, accesibilidad y feedback visual.

Paso 1 — Crear proyecto en IntelliJ. New Project → HTML → Name: appweb-formularios. Crear registro.html.

Paso 2 — Código completo.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Registro completo - UTEQ</title>
7   <style>
8     body { font-family: system-ui, sans-serif; max-width: 600px;
9           margin: 2rem auto; padding: 0 1rem; }
10    fieldset { margin-bottom: 1rem; padding: 1rem;
11             border: 1px solid #aaa; border-radius: 6px; }
12    legend { font-weight: bold; color: #006633; }
13    label { display: block; margin-top: .8rem; font-weight: 600; }
14    input, select, textarea {
15      width: 100%; padding: .5rem; box-sizing: border-box;
16      border: 1px solid #aaa; border-radius: 4px;
17      font-size: 1rem; margin-top: .25rem;
18    }
19    input:user-invalid { border-color: #b00020; }
20    input:user-valid { border-color: #2e7d32; }
21    small { display: block; color: #555; font-size: .85rem; }
22    button { margin-top: 1rem; padding: .7rem 1.2rem;
23           background: #006633; color: white; border: 0;
24           border-radius: 4px; font-size: 1rem; cursor: pointer; }
25  </style>
26 </head>
27 <body>
28 <main>
29   <h1>Registro de estudiantes UTEQ</h1>
30   <form action="/api/registro" method="POST">
31     <fieldset>
32       <legend>Datos personales</legend>
33
34       <label for="cedula">Cedula ecuatoriana:</label>
35       <input id="cedula" name="cedula" type="text"
36             pattern="[0-9]{10}" required>

```

```

37     <small>10 digitos sin guiones</small>
38
39     <label for="email">Correo institucional:</label>
40     <input id="email" name="email" type="email"
41           pattern=".+@uteq\.edu\.ec" required>
42
43     <label for="tel">Telefono celular:</label>
44     <input id="tel" name="tel" type="tel"
45           pattern="09[0-9]{8}" required>
46
47     <label for="fnac">Fecha de nacimiento:</label>
48     <input id="fnac" name="fnac" type="date"
49           min="1980-01-01" max="2010-12-31" required>
50
51     <label for="url">Sitio web (opcional):</label>
52     <input id="url" name="url" type="url">
53 </fieldset>
54
55 <fieldset>
56     <legend>Datos academicos</legend>
57
58     <label for="prom">Promedio (6.0-10.0):</label>
59     <input id="prom" name="prom" type="number"
60           min="6.0" max="10.0" step="0.01" required>
61
62     <label for="color">Color institucional preferido:</label>
63     <input id="color" name="color" type="color" value="#006633">
64
65     <label for="carrera">Carrera:</label>
66     <select id="carrera" name="carrera" required>
67         <option value="">-- Seleccione --</option>
68         <option value="SW">Ingenieria de Software</option>
69         <option value="SE">Ingenieria de Sistemas</option>
70     </select>
71 </fieldset>
72
73     <button type="submit">Registrarme</button>
74 </form>
75 </main>
76 </body>
77 </html>

```

Listado 2.14: registro.html

Resultado esperado en el navegador:

[Vista renderizada en Chrome]

Registro de estudiantes UTEQ

Datos personales

Cedula ecuatoriana:

10 dígitos sin guiones

Correo institucional:

Teléfono celular:

Fecha de nacimiento:

(calendario nativo)

Datos académicos

Promedio (6.0-10.0):

(controles arriba/abajo)

Color preferido: (selector)

Carrera:

Registrarme

Verificación de validaciones nativas:

1. Pulsar «Registrarme» con campos vacíos: el navegador detiene el envío y enfoca el primer campo inválido mostrando tooltip.
2. Escribir email con dominio diferente a `uteq.edu.ec`: el borde se vuelve rojo (gracias a `:user-invalid`).
3. En móvil (F12 → Toggle device toolbar): hacer clic en campo teléfono → aparece teclado numérico; en email → aparece tecla @; en fecha → aparece selector calendario.

Ejercicio 2.2 (PROPUESTO) — Página de portafolio profesional

Construir una página `portafolio.html` que demuestre dominio integral de HTML5 semántico. Requisitos:

- Estructura semántica completa: `<header>`, `<nav>`, `<main>`, `<article>` (al menos 3), `<aside>`, `<footer>`.
- Al menos un `<figure>`+`<figcaption>` con `<picture>` sirviendo múltiples resoluciones.
- Un `<video>` con al menos un `<track>` de subtítulos (crear archivo `.vtt` con 3-4 líneas).
- Un formulario de contacto con al menos 6 tipos diferentes de `<input>` HTML5: `text`, `email`, `tel`, `date`, `number`, `url`.
- Un elemento `<details>`+`<summary>` para una sección de preguntas frecuentes.
- Una tabla `<table>` con `<caption>`, `<thead>`, `<tbody>`, `<th scope>`.

Validar: cero errores en W3C, cero violaciones críticas en axe DevTools, navegación completa por teclado, y todos los tipos de input invocando teclado virtual correcto en móvil.

Entrega: archivo HTML + screenshots de validadores.

CSS para el diseño y la presentación web

«El diseño no es lo que se ve, es lo que funciona. CSS es el puente entre la intención del diseñador y la experiencia del usuario.»

— Adaptado de Steve Jobs

Resultado de aprendizaje

Al concluir el capítulo 3, el estudiante diseña hojas de estilo CSS3 modernas aplicando metodologías de organización (BEM, mobile-first), sistemas de *layout* (Flexbox, Grid), preprocesadores (Sass) y buenas prácticas profesionales [36, 22].

3.1 Historia y evolución de CSS

CSS pasó en 30 años de un lenguaje minimalista para colores y fuentes a un sistema robusto de diseño y *layout* con soporte nativo para grids, container queries y propiedades personalizadas (`-variables`). Conocer las etapas de su evolución revela por qué hoy podemos prescindir de frameworks pesados como Bootstrap en muchos proyectos. La caja siguiente narra esa historia.

De CSS1 al motor de los *design systems* modernos

En 1994, **Håkon Wium Lie**, trabajando junto a Tim Berners-Lee en el CERN, propuso un mecanismo para separar la presentación del contenido. La idea era radical en su simplicidad: que los autores y los usuarios pudieran proponer estilos en *cascada*, donde el navegador combinara las reglas según prioridades. La propuesta se convertiría en **CSS Level 1** (1996), el primer estándar W3C de hojas de estilo.

Durante quince años CSS evolucionó lentamente. CSS2 (1998) añadió posicionamiento absoluto y *media types*. La explosión llegó con **CSS3** (modular, desde 2011 en adelante), que introdujo gradientes, transiciones, animaciones, transformaciones, `border-radius`, `box-shadow` y dos sistemas revolucionarios: **Flexbox** (2012) y **Grid** (2017). Hoy CSS es el motor de los *design systems* modernos: Material Design (Google), Fluent (Microsoft), Tailwind CSS, Bootstrap.

3.1.1 Tabla evolutiva de CSS

La especificación de CSS ha avanzado en módulos independientes desde 2011, lo que significa que distintas partes del estándar maduran a velocidades distintas. La tabla siguiente resume los hitos más relevantes para el desarrollador web 2026.

Tabla 3.1: Hitos en la evolución de CSS.

Año	Versión	Aporte
1996	CSS 1	Selectores básicos, colores, fuentes, márgenes.
1998	CSS 2	Posicionamiento (absolute, relative), z-index.
2004	CSS 2.1	Revisión de CSS 2 que es la usada en navegadores antiguos.
2011	CSS 3 (modular)	Transiciones, transformaciones, animaciones, gradientes.
2012	Flexbox CR	Layout unidimensional con propiedades de eje.
2017	CSS Grid CR	Layout bidimensional con áreas nombradas.
2020	Custom Properties	Variables CSS nativas (<code>--color: #006633</code>).
2022	Container Queries	Estilos basados en el contenedor padre, no en el viewport.
2023	<code>:has()</code> selector	Selector «padre» tras décadas de espera.
2024	Cascade Layers	Capas declarativas para resolver especificidad.
2025	CSS Nesting nativo	Anidamiento estilo Sass sin preprocesador.

3.2 Fundamentos de CSS

Antes de discutir layouts modernos, conviene revisar los tres pilares de CSS: cómo se aplica el estilo, qué selectores existen y cómo funciona la cascada y la especificidad. Estas bases son la diferencia entre escribir CSS que mantiene el código bajo control y escribir CSS que se vuelve ingobernable.

3.2.1 Las tres formas de aplicar estilos

Existen tres mecanismos para aplicar estilo a elementos HTML, en orden de preferencia profesional. La enumeración siguiente los expone con su justificación.

Ejemplo corto comentado (las tres formas) (Listado 3.1):

Listado 3.1: Ejemplo de HTML

```

<!-- 1. EXTERNO: archivo .css separado. Es la forma profesional -->
<!-- porque permite cache del navegador y separa contenido/forma. -->
<link rel="stylesheet" href="estilos.css">

<!-- 2. INTERNO: <style> dentro de <head>. Util para una sola pagina -->
<!-- o para CSS critico que debe cargar sin peticion adicional. -->
<style>
  p { color: navy; } /* p = selector de tipo (todos los <p>) */
</style>

<!-- 3. INLINE: atributo style dentro del tag. Evitar en produccion -->
<!-- porque pulveriza la separacion de responsabilidades. -->
<p style="color:red">Texto rojo (inline, anti-patron).</p>

```

1. **Inline** (`style="..."`): evitar en producción.
2. **Interno** (`<style>` en `<head>`): útil para páginas pequeñas o casos especiales.
3. **Externo** (`<link rel="stylesheet">`): el modelo profesional. Permite caché del navegador y separación de responsabilidades.

3.2.2 Selectores principales

Un *selector* CSS es la expresión que decide a qué elementos del DOM se les aplica un conjunto de propiedades. CSS ofrece más de una docena de tipos de selectores con distinta *especificidad*. La tabla siguiente recoge los más usados.

Ejemplo corto comentado (selectores principales) (Listado 3.2):

Listado 3.2: Ejemplo de HTML

```

/* Selector de TIPO: todos los parrafos */

```

```

p          { line-height: 1.6; }
/* Selector de CLASE: elementos con class="destacado" */
.destacado { background: yellow; }
/* Selector de ID: el unico elemento con id="logo" */
#logo      { width: 120px; }
/* Selector DESCENDIENTE: <a> dentro de <nav> */
nav a      { text-decoration: none; }
/* Selector de ATRIBUTO: inputs de tipo email */
input[type="email"] { border: 1px solid blue; }
/* PSEUDO-CLASE: estado del elemento */
a:hover    { color: red; } /* Cuando el cursor pasa encima */

```

Tabla 3.2: Selectores CSS más usados.

Selector	Aplica a	Especificidad
*	Todos los elementos	0
p	Por tag	1
.clase	Por clase	10
#id	Por ID	100
a:hover	Pseudo-clase	11
p::first-letter	Pseudo-elemento	2
ul > li	Hijo directo	2
[type="email"]	Por atributo	11
:has(img)	Padre con descendiente (2023)	varía

3.2.3 Pseudo-clases CSS para feedback de validación

Las pseudo-clases relacionadas con formularios permiten dar feedback visual al usuario sin JavaScript:

- **:valid** — el campo cumple la validación.
- **:invalid** — el campo no cumple.
- **:user-valid** — el usuario ya interactuó y es válido (no marca el campo vacío al cargar).
- **:user-invalid** — el usuario interactuó y es inválido.
- **:required** — el campo es obligatorio.
- **:placeholder-shown** — mientras el placeholder es visible.
- **:focus-visible** — foco visible (solo navegación con teclado).

Ejemplo aplicable a formularios HTML5 (los formularios fueron vistos en el capítulo 2) (Listado 3.3):

```

input:user-invalid { border-color: red; }
input:user-valid   { border-color: green; }
input:focus-visible { outline: 3px solid #0066CC; }

```

Listado 3.3: Ejemplo de CSS

3.3 Sistemas modernos de layout

Históricamente, maquetar páginas con CSS fue una pesadilla: floats, posicionamientos hack, márgenes negativos. Desde 2017, Flexbox y CSS Grid resuelven la mayoría de los layouts de forma limpia y declarativa. Las dos subsecciones siguientes los explican con ejemplos visuales.

3.3.1 Flexbox: layout unidimensional

Flexbox organiza elementos en una dimensión (fila o columna) y resuelve problemas históricos de CSS como centrado vertical, distribución de espacio y orden visual independiente del HTML [59].

Ejemplo 3.1 (RESUELTO) — Navbar responsivo con Flexbox

Objetivo: crear una barra de navegación que se adapte a pantalla grande (horizontal) y pequeña (vertical).

IDE: IntelliJ IDEA Ultimate.

Paso 1: Crear archivo `navbar.html`. Véase el Listado 3.4.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7   <title>Navbar Flexbox</title>
8   <link rel="stylesheet" href="navbar.css">
9 </head>
10 <body>
11   <nav class="navbar" aria-label="Principal">
12     <div class="logo">UTEQ</div>
13     <ul class="menu">
14       <li><a href="#">Inicio</a></li>
15       <li><a href="#">Carreras</a></li>
16       <li><a href="#">Investigacion</a></li>
17       <li><a href="#">Contacto</a></li>
18     </ul>
19   </nav>
20   <main><p>Contenido de la pagina.</p></main>
21 </body>
22 </html>

```

Listado 3.4: navbar.html

Paso 2: Crear `navbar.css`. Véase el Listado 3.5.

```

1 * { margin: 0; padding: 0; box-sizing: border-box; }
2 body { font-family: system-ui, sans-serif; }
3
4 .navbar {
5   /* activa Flexbox (layout unidimensional) */
6   display: flex;                                     /* activa
7     Flexbox (layout unidimensional) */
8   /* alineacion principal en el eje horizontal */
9   justify-content: space-between;                     /*
10     alineacion principal en el eje horizontal */
11   /* alineacion en el eje secundario (vertical) */
12   align-items: center;                                /*
13     alineacion en el eje secundario (vertical) */
14   background: #006633;
15   padding: 1rem 2rem;
16   /* permite saltos de linea en flex */
17   flex-wrap: wrap;                                    /* permite
18     saltos de linea en flex */

```

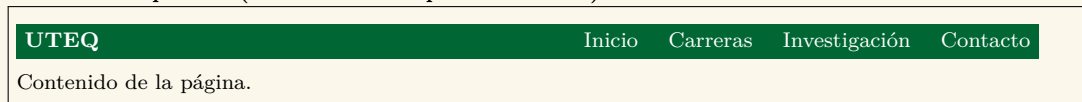
```

15 }
16
17 .logo {
18     color: white;
19     font-size: 1.5rem;
20     font-weight: bold;
21 }
22
23 .menu {
24     /* activa Flexbox (layout unidimensional) */
25     display: flex;                                /* activa
26     Flexbox (layout unidimensional) */
27     list-style: none;
28     /* espacio entre elementos hijos */
29     gap: 1.5rem;                                /* espacio
30     entre elementos hijos */
31 }
32
33 .menu a {
34     color: white;
35     text-decoration: none;
36     padding: 0.5rem 0;
37 }
38
39 .menu a:hover { color: #CC9900; }
40
41 /* media query (responsive) */
42 @media (max-width: 600px) {                      /* media
43     query (responsive) */
44     /* direccion principal del flex container */
45     /* direccion principal del flex container */
46     .menu { flex-direction: column; width: 100%; gap: 0.5rem; }
47 }
48
49 main { padding: 2rem; }

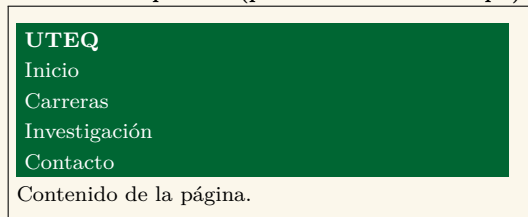
```

Listado 3.5: navbar.css

Resultado esperado (renderizado en pantalla ancha):



Resultado esperado (pantalla móvil < 600px):



3.3.2 CSS Grid: layout bidimensional

Grid permite layouts en dos dimensiones simultáneas. Es ideal para maquetar páginas completas, dashboards y galerías [36].

Ejemplo 3.2 (RESUELTO) — Galería de productos con Grid

Enunciado. Construir una galería de productos responsive con CSS Grid que se adapte automáticamente al ancho del viewport sin media queries explícitos.

```

1 .galeria {
2   /* activa CSS Grid (layout bidimensional) */
3   display: grid;                                /* activa
        CSS Grid (layout bidimensional) */
4   /* define columnas del grid */
5   /* define columnas del grid */
6   grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
7   /* espacio entre elementos hijos */
8   gap: 1rem;                                    /* espacio
        entre elementos hijos */
9 }
10 .producto {
11   border: 1px solid #ddd;
12   padding: 1rem;
13   border-radius: 8px;
14 }

```

Listado 3.6: galeria.css (fragmento)

Resultado esperado: una cuadrícula que ajusta automáticamente el número de columnas según el ancho del viewport (4 columnas en desktop, 2 en tablet, 1 en móvil).

Producto 1	Producto 2	Producto 3	Producto 4
\$2.50	\$3.20	\$4.50	\$1.80

3.4 Sass: el preprocesador profesional

Sass añade variables, mixins, anidamiento y módulos a CSS. Aunque CSS moderno ya tiene variables nativas, Sass sigue siendo el estándar en grandes proyectos por su capacidad de *computación*. Véase el Listado 3.7.

```

$verde: #006633;
$dorado: #CC9900;

@mixin boton($bg) {                                // definicion de
    mixin (codigo reutilizable)
    background: $bg;
    color: white;
    padding: 0.6rem 1.2rem;
    border: 0;
    border-radius: 4px;
    &:hover { filter: brightness(1.1); }             // referencia al
    selector padre
}

.btn-primario { @include boton($verde); }
.btn-acento   { @include boton($dorado); }

```

Listado 3.7: ejemplo.scss

3.5 Ejercicios

Los siguientes ejercicios consolidan Flexbox, Grid y Sass. El estudiante debe entregar el código fuente más una captura del resultado renderizado en el navegador.

Ejercicio 3.1 (PROPUESTO) — Card responsive con identidad UTEQ

Objetivo: construir un componente *card* reutilizable con HTML semántico + CSS Flexbox + Grid.

Requisitos:

- Tres cards alineadas con CSS Grid.
- Cada card tiene imagen (placeholder), título, descripción y botón.
- Colores institucionales (uteqverde #006633, uteqdorado #CC9900).
- Responsive: 3 columnas en desktop, 2 en tablet, 1 en móvil.
- Estado `:hover` con elevación (box-shadow).
- Score Lighthouse Accessibility ≥ 95 .

Entrega: archivos `cards.html` + `cards.css` con capturas en escritorio, tablet y móvil (3 imágenes).

Ejercicio 3.2 (PROPUESTO) — Dashboard básico con CSS Grid

Enunciado. Construir un dashboard básico con CSS Grid de 3 columnas que se reorganice a 1 columna en móviles. Crear un dashboard con cuatro tarjetas de estadísticas alineadas con Grid, una gráfica simulada con `<div>`+CSS y un menú lateral fijo. Aplicar mobile-first y validar accesibilidad.

3.6 Buenas prácticas CSS profesional

Conocer CSS no basta: la diferencia entre un proyecto mantenible y uno caótico está en aplicar disciplina al escribir estilos. La caja siguiente recopila doce reglas profesionales que el estudiante debe seguir desde el inicio.

Doce reglas CSS profesionales 2026

1. `box-sizing: border-box` global.
2. Mobile-first siempre.
3. Variables CSS para colores y tipografía (`--primario`).
4. BEM o similar para nombres de clase (`.card__titulo`).
5. Unidades relativas (`rem`, `em`, `%`) sobre `px`.
6. Sin `!important` excepto utilidades muy específicas.
7. No usar IDs como selectores (alta especificidad).
8. Flexbox para 1D, Grid para 2D.
9. Animaciones con `transform` y `opacity` (no afectan layout).
10. Respetar `prefers-reduced-motion` para usuarios sensibles a animaciones.
11. Lighthouse Performance + Accessibility ≥ 90 .
12. Minificación y autoprefixer en pipeline de build.

3.7 La hoja de estilos base del libro: `estilos.css`

Todas las interfaces de este libro —desde los formularios estáticos de esta unidad hasta los CRUD generados por Spring Boot, Django, Laravel o ASP.NET Core en las unidades siguientes— comparten una misma hoja de estilos: `estilos.css`. Se define una sola vez aquí y se reutiliza sin cambios; en los capítulos posteriores solo se mostrarán las reglas *nuevas* que cada ejercicio agregue. Así el lector concentra su atención en lo que cambia y no en repetir el andamiaje visual.

La hoja emplea recursos propios de CSS3: **propiedades personalizadas** (variables `--nombre`), **Flexbox** y **Grid** para la disposición, `nth-child` para el cebrado de filas, `transition` para los estados interactivos y una **consulta de medios** para la adaptación a pantallas pequeñas. La paleta corresponde a la identidad de la UTEQ (verde institucional y dorado).

Crear en la raíz del sitio: `estilos.css` — parte 1 de 2 (variables, reinicio y estructura semántica)

```

1  /* estilos.css - hoja base compartida del libro Aplicaciones Web */
2  :root {
3      --verde: #006633;
4      --dorado: #CC9900;
5      --gris-fondo: #F4F6F5;
6      --gris-borde: #D9DDDB;
7      --texto: #222222;
8      --radio: 8px;
9      --espacio: 1rem;
10 }
11
12 * { box-sizing: border-box; margin: 0; padding: 0; }
13
14 body {
15     font-family: system-ui, "Segoe UI", Roboto, sans-serif;
16     color: var(--texto);
17     background: var(--gris-fondo);
18     line-height: 1.6;
19 }
20
21 header.principal { background: var(--verde); color: #fff; padding:
22     var(--espacio); }
23
24 header.principal h1 { font-size: 1.4rem; }
25
26 nav.principal { background: #00472B; }
27 nav.principal ul { list-style: none; display: flex; gap: 1rem; padding:
28     .5rem 1rem; }
29
30 nav.principal a { color: #fff; text-decoration: none; }
31 nav.principal a:hover { color: var(--dorado); }
32
33 main { max-width: 960px; margin: 0 auto; padding: var(--espacio); }
34
35 footer.principal {
36     background: #00472B; color: #fff; text-align: center;
37     padding: var(--espacio); margin-top: 2rem;
38 }

```

Listado 3.8: `estilos.css` — parte 1: variables, reinicio y estructura

Continúa en: `estilos.css` — parte 2 de 2 (tablas, formularios, botones, avisos y diseño responsivo)

```

1  /* Tarjetas de contenido */
2  .tarjeta {
3      background: #fff; border: 1px solid var(--gris-borde);
4      border-radius: var(--radio); padding: var(--espacio);
5      margin-bottom: var(--espacio);
6  }
7
8  /* Tablas de datos (listados CRUD) */

```

```

9  table.datos { width: 100%; border-collapse: collapse; background: #fff; }
10 table.datos caption { text-align: left; font-weight: 700; padding: .5rem 0; }
11 table.datos th, table.datos td {
12     border: 1px solid var(--gris-borde); padding: .6rem .8rem; text-align:
        left;
13 }
14 table.datos thead th { background: var(--verde); color: #fff; }
15 table.datos tbody tr:nth-child(even) { background: #F0F4F2; }
16
17 /* Formularios */
18 form.formulario { display: grid; gap: .8rem; max-width: 480px; }
19 form.formulario label { font-weight: 600; }
20 form.formulario input, form.formulario select, form.formulario textarea {
21     width: 100%; padding: .55rem; font: inherit;
22     border: 1px solid var(--gris-borde); border-radius: var(--radio);
23 }
24 form.formulario input:focus { outline: 2px solid var(--dorado); }
25
26 /* Botones y enlaces de accion */
27 .boton, button {
28     display: inline-block; cursor: pointer; border: none; font: inherit;
29     background: var(--verde); color: #fff; text-decoration: none;
30     padding: .55rem 1rem; border-radius: var(--radio);
31     transition: background .2s ease;
32 }
33 .boton:hover, button:hover { background: var(--dorado); }
34 .boton.peligro { background: #B00020; }
35
36 /* Avisos */
37 .aviso { padding: .8rem; border-radius: var(--radio); border-left: 4px
        solid; }
38 .aviso.exito { background: #E7F4EC; border-color: var(--verde); }
39 .aviso.error { background: #FDECEA; border-color: #B00020; }
40
41 /* Diseno responsivo */
42 @media (max-width: 600px) {
43     nav.principal ul { flex-direction: column; }
44     main { padding: .6rem; }
45 }

```

Listado 3.9: estilos.css — parte 2: tablas, formularios, botones y responsivo

Para usar la hoja, cada página HTML5 la enlaza en su <head>:

```
1 <link rel="stylesheet" href="estilos.css">
```

Listado 3.10: Enlace de la hoja base en cualquier pagina

Convención de CSS incremental para todo el libro

A partir de aquí, ningún ejercicio vuelve a copiar `estilos.css` completo. Cuando una práctica necesite un estilo particular —por ejemplo, resaltar una fila seleccionada o maquetar un panel— solo se mostrará el **bloque nuevo** que se añade a la hoja, precedido de un comentario que indique a qué ejercicio pertenece. El lector mantiene una única `estilos.css` que crece de forma acumulativa.

JavaScript moderno y programación asíncrona

«Cualquier aplicación que pueda escribirse en JavaScript, eventualmente se escribirá en JavaScript.»

— Jeff Atwood, 2007 (*Ley de Atwood*)

Resultado de aprendizaje

Al concluir el capítulo 4, el estudiante construye lógica interactiva del lado del cliente con JavaScript moderno (ES2024+), domina el DOM, los eventos, las APIs asíncronas (fetch, Promises, async/await), y aplica las buenas prácticas de la industria [19, 12].

4.1 Historia de JavaScript

JavaScript nació en 1995 como un lenguaje juguete creado en diez días por Brendan Eich. Hoy es la lengua franca del navegador, ejecuta servidores enteros (Node.js), corre en satélites y dispositivos IoT. Conocer su historia ayuda a entender por qué tiene particularidades sintácticas y por qué evoluciona tan rápido.

De 10 días de diseño a la lengua franca de la web

En mayo de 1995, Netscape contrató a **Brendan Eich** para crear un lenguaje de scripting que se ejecutara en el navegador. Le dieron *diez días*. El resultado, originalmente llamado **Mocha**, luego **LiveScript**, y finalmente **JavaScript** (1996, por razones de marketing aprovechando la popularidad de Java), tenía rasgos memorables: tipado dinámico, *first-class functions*, prototipos en lugar de clases, **undefined** y **null** como dos formas distintas de «nada» [12].

Microsoft respondió con JScript en Internet Explorer 3 (1996), provocando una guerra de incompatibilidades que se cerró con la estandarización ECMA en 1997 (**ECMAScript**). Durante una década, JavaScript fue considerado un lenguaje de juguete. El cambio llegó con **Google Chrome** y su motor V8 (2008), que ejecutaba JavaScript hasta 100x más rápido. Ese mismo año Ryan Dahl creó **Node.js** (2009), llevando JS al servidor. **ECMAScript 2015 (ES6)** formalizó clases, módulos, **let**, **const**, arrow functions, Promises. ECMAScript 2024 ya es estándar y 2026 [16] incorpora características como **Array.fromAsync**, **Promise.try**, y *records* y *tuples* inmutables.

4.1.1 Tabla evolutiva de ECMAScript

ECMAScript es el estándar formal que define el lenguaje JavaScript. Desde 2015, evoluciona anualmente añadiendo funcionalidades. La tabla siguiente lista las versiones más significativas y sus aportes.


```
};
```

Listado 4.2: Ejemplo de JavaScript

4.2.3 Destructuring y spread

Destructuring permite extraer valores de objetos y arrays con una sintaxis declarativa concisa; *spread* (...) permite expandir colecciones en distintos contextos. Ambas técnicas son omnipresentes en código moderno. Véase el Listado 4.3.

```
const usuario = { nombre: "Ana", edad: 22, rol: "admin" }; // constante (no
                    reassignable)
const { nombre, rol } = usuario; // destructuring objeto

const [primero, segundo] = [10, 20, 30]; // destructuring array

const copia = { ...usuario, edad: 23 }; // spread + override
```

Listado 4.3: Ejemplo de JavaScript

4.3 Manipulación del DOM

El DOM (*Document Object Model*) es la representación en memoria de la página HTML, expuesta como un árbol de objetos que JavaScript puede leer y modificar. Los siguientes ejemplos muestran las operaciones más frecuentes.

Ejemplo 4.1 (RESUELTO) — Lista de tareas dinámica

Objetivo: crear una pequeña aplicación de lista de tareas con agregar y eliminar.

Paso 1: En IntelliJ, crear `tareas.html`. Véase el Listado 4.4.

```
1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <title>Lista de tareas</title>
6   <style>
7     body { font-family: system-ui; max-width: 500px;
8           margin: 2rem auto; padding: 0 1rem; }
9     h1 { color: #006633; }
10    input, button { padding: 0.5rem; font-size: 1rem; }
11    button { background: #006633; color: white;
12           border: 0; cursor: pointer; }
13    ul { list-style: none; padding: 0; }
14    li { display: flex; justify-content: space-between;
15        padding: 0.5rem; border-bottom: 1px solid #eee; }
16  </style>
17 </head>
18 <body>
19   <h1>Mis tareas</h1>
20   <form id="form-tarea">
21     <input type="text" id="texto-tarea" required
22           placeholder="Nueva tarea" minlength="3">
23     <button type="submit">Agregar</button>
24   </form>
```

```
25 <ul id="lista"></ul>
26 <script src="tareass.js"></script>
27 </body>
28 </html>
```

Listado 4.4: tareas.html

Paso 2: Crear `tareass.js`. Véase el Listado 4.5.

```
1 "use strict";
2
3 const form = document.getElementById("form-tarea");           //
4   constante (no reassignable)
5 const input = document.getElementById("texto-tarea");         //
6   constante (no reassignable)
7 const lista = document.getElementById("lista");               //
8   constante (no reassignable)
9
10 form.addEventListener("submit", (e) => {                      //
11   registra manejador de evento
12   e.preventDefault();
13   const texto = input.value.trim();                           //
14     constante (no reassignable)
15   if (texto.length < 3) return;                                //
16     condicional
17
18   const li = document.createElement("li");                   //
19     constante (no reassignable)
20   li.textContent = texto;                                     // cambia
21     el texto (seguro vs XSS)
22
23   const btn = document.createElement("button");              //
24     constante (no reassignable)
25   btn.textContent = "X";                                     // cambia
26     el texto (seguro vs XSS)
27   btn.addEventListener("click", () => li.remove());           //
28     registra manejador de evento
29
30   li.appendChild(btn);
31   lista.appendChild(li);
32   input.value = "";
33   input.focus();
34 });
```

Listado 4.5: tareas.js

Resultado esperado:

Mis tareas

Estudiar JavaScript	X
Hacer ejercicios HTML	X
Practicar Flexbox	X

4.4 Programación asíncrona: fetch, Promises, async/await

La programación asíncrona es ineludible en el navegador: peticiones HTTP, lectura de archivos, temporizadores y eventos no deben bloquear la interfaz. JavaScript provee tres mecanismos encadenados históricamente: callbacks, Promises y `async/await`. El ejemplo siguiente muestra el patrón moderno.

Ejemplo 4.2 (RESUELTO) — Consumir API pública con fetch

Enunciado. Consumir una API REST pública con `fetch` usando `async/await` para mostrar los datos en la página, manejando correctamente los errores de red.

```

1  "use strict";
2
3  async function obtenerUsuarios() {                                // funcion
4      // asincrona (devuelve Promise)
5      try {
6          const res = await fetch("https://jsonplaceholder.typicode.com/users");
7          if (!res.ok) throw new Error(`HTTP ${res.status}`);        //
8          // condicional
9          const usuarios = await res.json();                          //
10         // constante (no reassignable)
11         return usuarios;                                            //
12         // devuelve valor de la funcion
13     } catch (e) {
14         console.error("Error:", e.message);
15         return [];                                                //
16         // devuelve valor de la funcion
17     }
18 }
19
20 obtenerUsuarios().then(lista => {                                //
21     // callback ejecutado cuando la Promise se resuelve
22     lista.forEach(u => console.log(u.name, "-", u.email));        // imprime
23     // en la consola del navegador
24 });

```

Listado 4.6: api.js

Cómo probarlo: abrir `tareas.html` (o cualquier archivo con un `<script>`), abrir la consola del navegador (F12 → Console), pegar el código. Aparecen 10 usuarios con su nombre y correo en la consola.

4.5 Ejercicios

Los siguientes ejercicios consolidan DOM, fetch y patrones asíncronos. El estudiante debe entregar código y capturas del resultado en el navegador.

Ejercicio 4.1 (PROPUESTO) — Calculadora simple

Enunciado. Implementar una calculadora simple en JavaScript puro con operaciones aritméticas básicas, capturando eventos de los botones.

Crear una calculadora HTML+CSS+JS con las cuatro operaciones básicas. Validar división por cero. Mostrar el resultado en pantalla. Aplicar buenas prácticas (`strict mode`, `const/let`, manejo de errores).

Ejercicio 4.2 (PROPUESTO) — Buscador de pokémon

Enunciado. Construir un buscador de Pokémon que consuma la PokeAPI pública y muestre imagen, tipos y estadísticas del Pokémon buscado.

Consumir la API <https://pokeapi.co/api/v2/pokemon/{nombre}> con `fetch` y `async/await`. Mostrar al usuario el nombre, la imagen y los tipos del pokémon buscado. Manejar errores 404. Aplicar la pseudo-clase `:focus-visible` aprendida en la capítulo 3.

4.6 Buenas prácticas JavaScript

Escribir JavaScript profesional es más que conocer la sintaxis: requiere disciplina con tipos, manejo de errores, inmutabilidad y modularidad. La caja siguiente reúne doce reglas profesionales que distinguen al desarrollador junior del senior.

Doce reglas profesionales 2026

1. `"use strict"` en todo módulo.
2. `const` por defecto; `let` solo cuando muta; `var` nunca.
3. `===` sobre `==` siempre (comparación estricta).
4. Manejo de errores con `try/catch` en código asíncrono.
5. Validar entrada de funciones públicas.
6. Funciones pequeñas, una responsabilidad.
7. Sin variables globales: usar módulos ES (`type="module"`).
8. Linter (ESLint) con configuración del equipo.
9. Tests con Jest, Vitest o Playwright.
10. No mezclar `Promise.then()` y `await`: elegir uno.
11. Optional chaining (`?.`) para acceso seguro a objetos.
12. Tipado con JSDoc o TypeScript en proyectos serios.

Unidad II

Lenguajes, Frameworks y Seguridad del Servidor

Introducción a los lenguajes del lado del servidor

«Conocer cinco lenguajes del servidor no es saber cinco sintaxis: es entender que todos resuelven el mismo ciclo petición–respuesta de maneras distintas.»

— *Principio pedagógico de la asignatura*

Resultado de aprendizaje

Al terminar este capítulo, el estudiante reconoce el papel del lado del servidor en una aplicación web, identifica cinco tecnologías de servidor —Perl, PHP, Python, ASP.NET Core (C#) y Java para la web— y escribe en cada una un programa mínimo «Hola Mundo» que responde por HTTP, conociendo además la estructura de archivos que genera cada proyecto.

5.1 El lado del servidor y el ciclo petición–respuesta

Cuando el navegador solicita una página dinámica, un programa en el servidor recibe la petición HTTP, ejecuta lógica (consulta datos, calcula, decide) y devuelve una respuesta —casi siempre HTML5— que el navegador presenta. Ese programa puede estar escrito en muchos lenguajes; este libro recorre cinco de uso real.

Concepto — Cliente y servidor

El *cliente* (navegador) ejecuta HTML5, CSS3 y JavaScript y se ocupa de la presentación. El *servidor* ejecuta el lenguaje de turno (Perl, PHP, Python, C# o Java) y se ocupa de la lógica y de los datos. La frontera entre ambos es el protocolo HTTP.

Punto de partida que asume este libro

Se asume que el lector ya programa en **Java** (Java SE: clases, métodos, colecciones), pero **no** conoce Perl, PHP, Python ni C#, ni el desarrollo web con Java. Por eso cada lenguaje se introduce desde cero, y Java para la web se presenta al final, como puente desde lo que ya sabe.

El IDE de referencia es **IntelliJ IDEA Ultimate** para Perl, PHP, Python y Java (mediante sus plugins). Para C# / ASP.NET Core se usa **Visual Studio Code**, porque IntelliJ IDEA no ofrece soporte de C#.

5.2 Perl

Historia — Perl

Creado por Larry Wall en 1987, Perl fue durante los años noventa el lenguaje dominante de la web a través de CGI. Su lema «hay más de una manera de hacerlo» y su potencia con expresiones regulares lo hicieron omnipresente en scripts de servidor. Hoy convive con lenguajes más nuevos, pero sigue vigente; la versión de referencia es Perl 5.42.

Perl genera páginas escribiendo la respuesta HTTP por la salida estándar. El siguiente «Hola Mundo» es un script CGI: imprime una cabecera `Content-Type` y luego el HTML5.

Crear en: `cgi-bin/hola.pl`

```

1  #!/usr/bin/perl
2  use strict;
3  use warnings;
4
5  print "Content-Type: text/html; charset=UTF-8\n\n";
6  print "<!DOCTYPE html>\n";
7  print "<html lang=\"es\"><head><meta charset=\"UTF-8\">";
8  print "<title>Hola</title></head>";
9  print "<body><h1>Hola Mundo desde Perl</h1></body></html>";

```

Listado 5.1: `hola.pl` — Hola Mundo en Perl (CGI)

Estructura del proyecto Perl

```

1  hola-perl/
2  |-- cgi-bin/
3  |   '-- hola.pl           (programador)
4  '-- public/
5      '-- estilos.css       (hoja base de la Unidad I)

```

Listado 5.2: Estructura mínima de un proyecto Perl/CGI

Se ejecuta con un servidor que soporte CGI (Apache con `mod_cgi`, o el servidor de pruebas `plackup`). La sintaxis clave: las variables escalares empiezan con `$`, `use strict` obliga a declararlas, y `print` envía texto al cliente.

5.3 PHP

Historia — PHP

PHP (hoy «PHP: Hypertext Preprocessor») nació en 1994 de la mano de Rasmus Lerdorf. Diseñado específicamente para la web, se incrusta dentro del HTML y se ejecuta en el servidor. Es uno de los lenguajes más usados del mundo (WordPress, Laravel); la versión de referencia es PHP 8.4.

En PHP el código va entre `<?php` y `?>`, intercalado con HTML. El intérprete ejecuta el bloque y deja el resto tal cual.

Crear en: `public/index.php`

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head><meta charset="UTF-8"><title>Hola</title>
4  <link rel="stylesheet" href="estilos.css"></head>
5  <body>

```

```

6   <h1><?php echo "Hola Mundo desde PHP " . phpversion(); ?></h1>
7 </body>
8 </html>

```

Listado 5.3: index.php — Hola Mundo en PHP

Estructura del proyecto PHP

```

1 hola-php/
2 '-- public/
3   |-- index.php           (programador)
4   '-- estilos.css        (hoja base de la Unidad I)

```

Listado 5.4: Estructura mínima de un proyecto PHP

Se ejecuta con el servidor incorporado: `php -S localhost:8000 -t public`. La sintaxis clave: las variables empiezan con \$, las cadenas se concatenan con el punto (.) y `echo` envía texto al cliente.

5.4 Python

Historia — Python

Creado por Guido van Rossum en 1991, Python destaca por su legibilidad. En la web se usa con frameworks (Django, Flask), pero también se puede servir HTTP con la biblioteca estándar mediante WSGI —la interfaz estándar entre un servidor y una aplicación Python—. La versión de referencia es Python 3.13.

Para mantener el capítulo «sin framework», el «Hola Mundo» usa WSGI puro: una función que recibe el entorno de la petición y devuelve la respuesta.

Crear en: `app.py`

```

1 from wsgiref.simple_server import make_server
2
3
4 def app(environ, start_response):
5     cuerpo = ("<!DOCTYPE html><html lang='es'><head>"
6              "<meta charset='UTF-8'><title>Hola</title></head>"
7              "<body><h1>Hola Mundo desde Python</h1></body></html>")
8     datos = cuerpo.encode("utf-8")
9     start_response("200 OK", [
10         ("Content-Type", "text/html; charset=UTF-8"),
11         ("Content-Length", str(len(datos))),
12     ])
13     return [datos]
14
15
16 if __name__ == "__main__":
17     with make_server("", 8000, app) as servidor:
18         print("Escuchando en http://localhost:8000")
19         servidor.serve_forever()

```

Listado 5.5: app.py — Hola Mundo en Python (WSGI)

Estructura del proyecto Python

```

1 hola-python/
2 |-- app.py                (programador)
3 '-- static/
4   '-- estilos.css        (hoja base de la Unidad I)

```

Listado 5.6: Estructura mínima de un proyecto Python/WSGI

Se ejecuta con `python app.py`. La sintaxis clave: la indentación define los bloques, no hay llaves; `def` declara funciones y las cadenas se codifican a bytes antes de enviarse.

5.5 ASP.NET Core (C#)

Historia — ASP.NET Core

Microsoft lanzó ASP.NET en 2002; en 2016 lo reescribió como **ASP.NET Core**, multiplataforma y de código abierto. C# es su lenguaje, de tipado fuerte y sintaxis cercana a Java. La versión de referencia es **.NET 10 (LTS)** con **C# 14** y ASP.NET Core 10.

ASP.NET Core usa el modelo de *Minimal API*: en pocas líneas se define una ruta y su respuesta. El proyecto se crea con la CLI de .NET.

Crear el proyecto (terminal de VS Code):

```

1 dotnet new web -o HolaAsp
2 cd HolaAsp
3 dotnet run

```

Listado 5.7: Creación del proyecto ASP.NET Core

Editar: Program.cs

```

1 var builder = WebApplication.CreateBuilder(args);
2 var app = builder.Build();
3
4 app.MapGet("/", () => Results.Content(
5     "<!DOCTYPE html><html lang=\"es\"><head>" +
6     "<meta charset=\"UTF-8\"><title>Hola</title></head>" +
7     "<body><h1>Hola Mundo desde ASP.NET Core</h1></body></html>",
8     "text/html"));
9
10 app.Run();

```

Listado 5.8: Program.cs — Hola Mundo en ASP.NET Core

Estructura del proyecto ASP.NET Core

```

1 HolaAsp/
2 |-- Program.cs            (dotnet; lo edita el programador)
3 |-- HolaAsp.csproj        (dotnet: proyecto y dependencias)
4 |-- appsettings.json      (dotnet: configuracion)
5 |-- Properties/
6 |   '-- launchSettings.json (dotnet: perfiles de ejecucion)

```

```
7 '-- wwwroot/                                (programador: estaticos, estilos.css)
```

Listado 5.9: Estructura creada por dotnet new web

La sintaxis clave: `C#` es de tipado fuerte; `var` infiere el tipo; los *lambdas* `() => ...` definen el manejador de la ruta. Se ejecuta con `dotnet run`.

5.6 Java para la Web

Como el lector ya domina Java SE, aquí solo se tiende el puente hacia la web. En Java, un **servlet** es una clase que atiende peticiones HTTP: hereda de `HttpServlet` y sobrescribe `doGet`. Se ejecuta dentro de un servidor de aplicaciones; el de referencia es **Apache Tomcat 11** (Jakarta Servlet 6.1), con **Java 21**.

Crear en: `src/main/java/ec/edu/uteq/HolaServlet.java`

```
1 package ec.edu.uteq;
2
3 import jakarta.servlet.annotation.WebServlet;
4 import jakarta.servlet.http.HttpServlet;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8
9 @WebServlet("/hola")
10 public class HolaServlet extends HttpServlet {
11     @Override
12     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
13         throws IOException {
14         resp.setContentType("text/html; charset=UTF-8");
15         resp.getWriter().println(
16             "<!DOCTYPE html><html lang=\"es\"><head>"
17             + "<meta charset=\"UTF-8\"><title>Hola</title></head>"
18             + "<body><h1>Hola Mundo desde Java"
19               (Servlet)</h1></body></html>");
20     }
21 }
```

Listado 5.10: HolaServlet.java — Hola Mundo en Java web

Estructura del proyecto Java web (Maven — war)

```
1 hola-java/
2 |-- pom.xml                                (programador: dependencias)
3 '-- src/main/
4     |-- java/ec/edu/uteq/
5     |   '-- HolaServlet.java                (programador)
6     '-- webapp/
7         |-- WEB-INF/web.xml                  (IntelliJ)
8         '-- estilos.css                      (hoja base de la Unidad I)
```

Listado 5.11: Estructura mínima de un proyecto Java web

La anotación `@WebServlet("/hola")` mapea la URL; no hace falta `web.xml` para registrar el servlet. El detalle completo de JSP/Servlets/JSTL/EL llega en el Capítulo 6.

Las cinco tecnologías de un vistazo

Tecnología	Cómo responde HTTP	Se ejecuta con
Perl	<code>print</code> a la salida (CGI)	Apache/ <code>plackup</code>
PHP	Código incrustado en HTML	<code>php -S</code>
Python	Función WSGI	<code>python app.py</code>
C#/ASP	<i>Minimal API</i> (<code>MapGet</code>)	<code>dotnet run</code>
Java	Servlet (<code>doGet</code>)	Tomcat 11

CRUD sin framework: acceso a datos a mano

«Antes de dejar que un framework escriba el SQL por ti, escríbelo tú: solo así sabrás qué hace —y qué deshacer— cuando falle.»

— Principio pedagógico de la asignatura

Resultado de aprendizaje

Al terminar este capítulo, el estudiante implementa una aplicación CRUD (crear, leer, actualizar y eliminar) sobre la tabla **estudiantes** en los cinco lenguajes, escribiendo a mano la conexión y el SQL parametrizado, sin emplear ningún framework ni ORM. En cada caso conoce la estructura del proyecto y reutiliza la hoja **estilos.css** de la Unidad I.

6.1 El modelo de datos común

Las cinco implementaciones operan sobre la misma base PostgreSQL **appweb2026ppa** y la misma tabla. Se crea una sola vez.

Crear en: **db/schema.sql**

```
1 CREATE TABLE IF NOT EXISTS estudiantes (
2     id          SERIAL          PRIMARY KEY,
3     nombre     VARCHAR(100) NOT NULL,
4     correo     VARCHAR(120) NOT NULL,
5     carrera    VARCHAR(80)  NOT NULL,
6     creado     TIMESTAMP       NOT NULL DEFAULT NOW()
7 );
```

Listado 6.1: Esquema compartido de la tabla estudiantes

Credenciales y SQL parametrizado

Servidor **localhost:5432**, base **appweb2026ppa**, usuario **postgres**, contraseña **P0\$7Gr3\$QL**. En las cinco tecnologías los datos del usuario viajan como **parámetros** enlazados, nunca concatenados en la cadena SQL: esa es la defensa contra la inyección SQL.

6.2 Nuevo CSS para el listado

La hoja `estilos.css` de la Unidad I ya define el reinicio, la tipografía y la estructura. Para las tablas del CRUD solo se agregan estas reglas *nuevas*; el resto se hereda.

Agregar a: `estilos.css`

```
1 .tabla-crud { width: 100%; border-collapse: collapse; margin: 1rem 0; }
2 .tabla-crud th, .tabla-crud td {
3     border: 1px solid var(--borde); padding: .5rem .75rem; text-align: left;
4 }
5 .tabla-crud th { background: var(--primario); color: #fff; }
6 .tabla-crud tr:nth-child(even) { background: var(--fondo-alt); }
7 .acciones a { margin-right: .5rem; }
```

Listado 6.2: Reglas nuevas para la tabla del CRUD

6.3 CRUD en Perl con DBI

Perl accede a PostgreSQL con el módulo **DBI** y el driver **DBD::Pg**. La conexión se centraliza en una subrutina; cada operación usa `prepare` y `execute` con marcadores `?`.

Crear en: `lib/EstudianteDao.pm` — parte 1 de 2 (conexion y lectura)

```
1 package EstudianteDao;
2 use strict;
3 use warnings;
4 use DBI;
5
6 sub conectar {
7     my $dsn = "dbi:Pg:dbname=appweb2026ppa;host=localhost;port=5432";
8     return DBI->connect($dsn, "postgres", 'P@$7Gr3$QL',
9         { RaiseError => 1, AutoCommit => 1 });
10 }
11
12 sub listar {
13     my $dbh = conectar();
14     my $sth = $dbh->prepare(
15         "SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY id");
16     $sth->execute();
17     my @lista;
18     while (my $fila = $sth->fetchrow_hashref) { push @lista, $fila; }
19     $dbh->disconnect;
20     return \@lista;
21 }
```

Listado 6.3: `EstudianteDao.pm` — parte 1: conexion y listar

Continúa en: `lib/EstudianteDao.pm` — parte 2 de 2 (escritura)

```
1 sub insertar {
2     my ($nombre, $correo, $carrera) = @_;
3     my $dbh = conectar();
4     $dbh->do("INSERT INTO estudiantes (nombre, correo, carrera) "
```

```

5         . "VALUES (?, ?, ?)", undef, $nombre, $correo, $carrera);
6     $dbh->disconnect;
7 }
8
9 sub actualizar {
10     my ($id, $nombre, $correo, $carrera) = @_;
11     my $dbh = conectar();
12     $dbh->do("UPDATE estudiantes SET nombre=?, correo=?, carrera=? "
13         . "WHERE id=?", undef, $nombre, $correo, $carrera, $id);
14     $dbh->disconnect;
15 }
16
17 sub eliminar {
18     my ($id) = @_;
19     my $dbh = conectar();
20     $dbh->do("DELETE FROM estudiantes WHERE id=?", undef, $id);
21     $dbh->disconnect;
22 }
23 1;

```

Listado 6.4: EstudianteDao.pm — parte 2: insertar, actualizar, eliminar

El script CGI usa el DAO y genera el listado HTML5 enlazando `estilos.css`.

Crear en: `cgi-bin/lista.pl`

```

1  #!/usr/bin/perl
2  use strict; use warnings; use lib "../lib"; use EstudianteDao;
3
4  print "Content-Type: text/html; charset=UTF-8\n\n";
5  print "<!DOCTYPE html><html lang='es'><head><meta charset='UTF-8'>";
6  print "<link rel='stylesheet' href='\"/estilos.css\"></head><body>";
7  print "<h1>Estudiantes</h1><table class='tabla-crud'>";
8  print "<tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th></tr>";
9  for my $e (@{ EstudianteDao::listar() }) {
10     print "<tr><td>$e->{id}</td><td>$e->{nombre}</td>";
11     print "<td>$e->{correo}</td><td>$e->{carrera}</td></tr>";
12 }
13 print "</table></body></html>";

```

Listado 6.5: lista.pl — vista de listado

Estructura del proyecto CRUD en Perl

```

1  crud-perl/
2  |-- db/schema.sql           (programador)
3  |-- lib/
4  |   '-- EstudianteDao.pm    (programador: acceso a datos)
5  |-- cgi-bin/
6  |   |-- lista.pl           (programador: vista)
7  |   |-- guardar.pl         (programador: alta/edición)
8  |   '-- eliminar.pl        (programador)
9  '-- public/estilos.css      (hoja base de la Unidad I)

```

Listado 6.6: Estructura del CRUD Perl

6.4 CRUD en PHP con PDO

PHP usa **PDO** (PHP Data Objects), una capa uniforme de acceso a datos. La conexión se crea una vez y se reutiliza; las consultas usan marcadores con nombre (:nombre).

Crear en: src/conexion.php

```

1 <?php
2 function conectar(): PDO {
3     $dsn = 'pgsql:host=localhost;port=5432;dbname=appweb2026ppa';
4     return new PDO($dsn, 'postgres', 'P@7Gr3$QL', [
5         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
6         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
7     ]);
8 }
```

Listado 6.7: conexion.php — conexión PDO

Crear en: src/EstudianteDao.php

```

1 <?php
2 require_once 'conexion.php';
3
4 class EstudianteDao {
5     public function listar(): array {
6         $sql = 'SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY
7             id';
8         return conectar()->query($sql)->fetchAll();
9     }
10    public function insertar(string $nombre, string $correo, string
11        $carrera): void {
12        $sql = 'INSERT INTO estudiantes (nombre, correo, carrera) '
13            . 'VALUES (:nombre, :correo, :carrera)';
14        $stmt = conectar()->prepare($sql);
15        $stmt->execute([':nombre'=>$nombre, ':correo'=>$correo,
16            ':carrera'=>$carrera]);
17    }
18    public function actualizar(int $id, string $nombre, string $correo,
19        string $carrera): void {
20        $sql = 'UPDATE estudiantes SET nombre=:nombre, correo=:correo, '
21            . 'carrera=:carrera WHERE id=:id';
22        $stmt = conectar()->prepare($sql);
23        $stmt->execute([':nombre'=>$nombre, ':correo'=>$correo,
24            ':carrera'=>$carrera, ':id'=>$id]);
25    }
26    public function eliminar(int $id): void {
27        $stmt = conectar()->prepare('DELETE FROM estudiantes WHERE id=:id');
28        $stmt->execute([':id'=>$id]);
29    }
30 }
```

26 }

Listado 6.8: EstudianteDao.php — CRUD con PDO

Crear en: public/index.php

```

1 <?php require_once '../src/EstudianteDao.php';
2 $lista = (new EstudianteDao())->listar(); ?>
3 <!DOCTYPE html>
4 <html lang="es"><head><meta charset="UTF-8">
5   <link rel="stylesheet" href="estilos.css"></head>
6 <body>
7   <h1>Estudiantes</h1>
8   <table class="tabla-crud">
9     <tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th></tr>
10    <?php foreach ($lista as $e): ?>
11      <tr><td><?= htmlspecialchars($e['id']) ?></td>
12        <td><?= htmlspecialchars($e['nombre']) ?></td>
13        <td><?= htmlspecialchars($e['correo']) ?></td>
14        <td><?= htmlspecialchars($e['carrera']) ?></td></tr>
15    <?php endforeach; ?>
16  </table>
17 </body></html>

```

Listado 6.9: index.php — vista de listado

Estructura del proyecto CRUD en PHP

```

1 crud-php/
2 |-- db/schema.sql           (programador)
3 |-- src/
4 |   |-- conexion.php       (programador: conexion)
5 |   '-- EstudianteDao.php   (programador: acceso a datos)
6 '-- public/
7     |-- index.php           (programador: vista)
8     '-- estilos.css         (hoja base de la Unidad I)

```

Listado 6.10: Estructura del CRUD PHP

La función `htmlspecialchars` escapa la salida: defensa anti-XSS, equivalente a `c:out` en JSTL.

6.5 CRUD en Python con psycopg (sin framework)

Para mantener el capítulo sin framework, Python accede a PostgreSQL con el adaptador **psycopg** (versión 3) y sirve HTTP por WSGI. El DAO ejecuta SQL parametrizado con marcadores `%s`.

Instalar: `pip install "psycopg[binary]"`

Crear en: dao.py

```

1 import psycopg
2
3 CADENA = ("host=localhost port=5432 dbname=appweb2026ppa "
4          "user=postgres password=P@$7Gr3$QL")

```



```

5
6
7 def listar():
8     with pycopg.connect(CADENA) as cn, cn.cursor() as cur:
9         cur.execute("SELECT id, nombre, correo, carrera "
10                     "FROM estudiantes ORDER BY id")
11     return cur.fetchall()
12
13
14 def insertar(nombre, correo, carrera):
15     with pycopg.connect(CADENA) as cn, cn.cursor() as cur:
16         cur.execute("INSERT INTO estudiantes (nombre, correo, carrera) "
17                     "VALUES (%s, %s, %s)", (nombre, correo, carrera))
18
19
20 def actualizar(id_est, nombre, correo, carrera):
21     with pycopg.connect(CADENA) as cn, cn.cursor() as cur:
22         cur.execute("UPDATE estudiantes SET nombre=%s, correo=%s, "
23                     "carrera=%s WHERE id=%s",
24                     (nombre, correo, carrera, id_est))
25
26
27 def eliminar(id_est):
28     with pycopg.connect(CADENA) as cn, cn.cursor() as cur:
29         cur.execute("DELETE FROM estudiantes WHERE id=%s", (id_est,))

```

Listado 6.11: dao.py — CRUD con pycopg

La aplicación WSGI construye el listado HTML5 y enlaza `estilos.css`.

Crear en: `app.py`

```

1 from wsgiref.simple_server import make_server
2 from html import escape
3 import dao
4
5
6 def app(environ, start_response):
7     filas = "".join(
8         f"<tr><td>{e[0]}</td><td>{escape(e[1])}</td>"
9         f"<td>{escape(e[2])}</td><td>{escape(e[3])}</td></tr>"
10        for e in dao.listar())
11    html = (f"<!DOCTYPE html><html lang='es'><head>"
12           f"<meta charset='UTF-8'>"
13           f"<link rel='stylesheet' href='/estilos.css'></head><body>"
14           f"<h1>Estudiantes</h1><table class='tabla-crud'>"
15           f"<tr><th>ID</th><th>Nombre</th><th>Correo</th>"
16           f"<th>Carrera</th></tr>{filas}</table></body></html>")
17    datos = html.encode("utf-8")
18    start_response("200 OK", [("Content-Type", "text/html; charset=UTF-8")])
19    return [datos]
20
21
22 if __name__ == "__main__":

```

```

23     with make_server("", 8000, app) as s:
24         s.serve_forever()

```

Listado 6.12: app.py — vista de listado (WSGI)

Estructura del proyecto CRUD en Python

```

1 crud-python/
2 |-- db/schema.sql           (programador)
3 |-- dao.py                 (programador: acceso a datos)
4 |-- app.py                 (programador: vista WSGI)
5 '-- static/estilos.css     (hoja base de la Unidad I)

```

Listado 6.13: Estructura del CRUD Python

Sobre «sin framework» en Python

Hacer un CRUD web sin ningún framework es inusual: por eso se usa la biblioteca estándar (WSGI) y `psycpg` a bajo nivel. En el Capítulo 7 se reescribe este mismo CRUD con **Django**, que aporta enrutamiento, plantillas y mucho más.

6.6 CRUD en ASP.NET Core con ADO.NET (sin EF)

C# accede a PostgreSQL con el proveedor **Npgsql** (ADO.NET). Para que sea parejo con los demás, no se usa Entity Framework: el SQL se escribe a mano con parámetros (@nombre).

Instalar: `dotnet add package Npgsql` (versión 10)

Crear en: Datos/EstudianteDao.cs — parte 1 de 2 (conexion y lectura)

```

1 using Npgsql;
2
3 namespace CrudAsp.Datos;
4
5 public class EstudianteDao
6 {
7     private const string Cadena =
8         "Host=localhost;Port=5432;Database=appweb2026ppa;" +
9         "Username=postgres;Password=P@$7Gr3$QL";
10
11     public List<(int, string, string, string)> Listar()
12     {
13         var lista = new List<(int, string, string, string)>();
14         using var cn = new NpgsqlConnection(Cadena);
15         cn.Open();
16         using var cmd = new NpgsqlCommand(
17             "SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY
18             id", cn);
19         using var rd = cmd.ExecuteReader();
20         while (rd.Read())
21             lista.Add((rd.GetInt32(0), rd.GetString(1),
22                 rd.GetString(2), rd.GetString(3)));
23         return lista;
24     }
25 }

```

24 }
}

Listado 6.14: EstudianteDao.cs — parte 1: conexion y listar

Continúa en: Datos/EstudianteDao.cs — parte 2 de 2 (escritura)

```

1  public void Insertar(string nombre, string correo, string carrera)
2  {
3      using var cn = new NpgsqlConnection(Cadena);
4      cn.Open();
5      using var cmd = new NpgsqlCommand(
6          "INSERT INTO estudiantes (nombre, correo, carrera) " +
7          "VALUES (@n, @c, @ca)", cn);
8      cmd.Parameters.AddWithValue("n", nombre);
9      cmd.Parameters.AddWithValue("c", correo);
10     cmd.Parameters.AddWithValue("ca", carrera);
11     cmd.ExecuteNonQuery();
12 }
13
14 public void Eliminar(int id)
15 {
16     using var cn = new NpgsqlConnection(Cadena);
17     cn.Open();
18     using var cmd = new NpgsqlCommand(
19         "DELETE FROM estudiantes WHERE id=@id", cn);
20     cmd.Parameters.AddWithValue("id", id);
21     cmd.ExecuteNonQuery();
22 }
23 }

```

Listado 6.15: EstudianteDao.cs — parte 2: insertar, actualizar, eliminar

El *Minimal API* expone la ruta que arma el listado HTML5.**Editar: Program.cs**

```

1  using CrudAsp.Datos;
2
3  var app = WebApplication.CreateBuilder(args).Build();
4  var dao = new EstudianteDao();
5
6  app.MapGet("/estudiantes", () =>
7  {
8      var filas = string.Join("", dao.Listar().Select(e =>
9          $"<tr><td>{e.Item1}</td><td>{e.Item2}</td>" +
10         $"<td>{e.Item3}</td><td>{e.Item4}</td></tr>"));
11     var html = "<!DOCTYPE html><html lang=\"es\"><head>" +
12         "<meta charset=\"UTF-8\">" +
13         "<link rel=\"stylesheet\" href=\"/estilos.css\"></head><body>" +
14         "<h1>Estudiantes</h1><table class=\"tabla-crud\">" +
15         "<tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th></tr>"
16         +
17         filas + "</table></body></html>";
18     return Results.Content(html, "text/html");
19 }

```

```

18 });
19
20 app.UseStaticFiles();
21 app.Run();

```

Listado 6.16: Program.cs — vista de listado

Estructura del proyecto CRUD en ASP.NET Core

```

1 CrudAsp/
2 |-- Program.cs                (programador: rutas/vista)
3 |-- CrudAsp.csproj            (dotnet: proyecto y paquetes)
4 |-- Datos/
5 |   '-- EstudianteDao.cs      (programador: acceso a datos)
6 |-- db/schema.sql             (programador)
7 '-- wwwroot/estilos.css       (hoja base de la Unidad I)

```

Listado 6.17: Estructura del CRUD ASP.NET Core

6.7 CRUD en Java con JSP, Servlets, JSTL y EL (JDBC)

Java reúne aquí las cuatro piezas: **Servlet** (control), **JDBC** (datos), **JSP** con **JSTL** y **EL** (vista). Es la versión ampliada del CRUD que se introdujo como práctica integradora; el detalle de cada etiqueta JSTL se trató en su material específico.

Crear en: .../dao/EstudianteDAO.java — parte 1 de 2 (conexion y lectura)

```

1 package ec.edu.uteq.appweb.dao;
2
3 import ec.edu.uteq.appweb.modelo.Estudiante;
4 import java.sql.*;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class EstudianteDAO {
9     private static final String URL =
10         "jdbc:postgres://localhost:5432/appweb2026ppa";
11
12     private Connection cn() throws SQLException {
13         return DriverManager.getConnection(URL, "postgres", "P@$7Gr3$QL");
14     }
15
16     public List<Estudiante> listar() throws SQLException {
17         String sql = "SELECT id, nombre, correo, carrera "
18             + "FROM estudiantes ORDER BY id";
19         List<Estudiante> lista = new ArrayList<>();
20         try (Connection c = cn();
21             PreparedStatement ps = c.prepareStatement(sql);
22             ResultSet rs = ps.executeQuery()) {
23             while (rs.next())
24                 lista.add(new Estudiante(rs.getInt("id"),
25                     rs.getString("nombre"), rs.getString("correo"),
26                     rs.getString("carrera")));
27         }
28     }
29 }

```

```

28         return lista;
29     }
30 }

```

Listado 6.18: EstudianteDAO.java — parte 1: conexion y listar

Continúa en: .../dao/EstudianteDAO.java — parte 2 de 2 (escritura)

```

1     public void insertar(Estudiante e) throws SQLException {
2         String sql = "INSERT INTO estudiantes (nombre, correo, carrera) "
3             + "VALUES (?, ?, ?)";
4         try (Connection c = cn();
5             PreparedStatement ps = c.prepareStatement(sql)) {
6             ps.setString(1, e.getNombre());
7             ps.setString(2, e.getCorreo());
8             ps.setString(3, e.getCarrera());
9             ps.executeUpdate();
10        }
11    }
12
13    public void eliminar(int id) throws SQLException {
14        try (Connection c = cn();
15            PreparedStatement ps =
16                c.prepareStatement("DELETE FROM estudiantes WHERE id=?")) {
17            ps.setInt(1, id);
18            ps.executeUpdate();
19        }
20    }
21 }

```

Listado 6.19: EstudianteDAO.java — parte 2: insertar, actualizar, eliminar

La vista JSP recorre la lista con `c:forEach` y escapa la salida con `c:out`.

Crear en: WEB-INF/vistas/lista.jsp

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <!DOCTYPE html>
4 <html lang="es"><head><meta charset="UTF-8">
5     <link rel="stylesheet" href="estilos.css"></head>
6 <body>
7     <h1>Estudiantes</h1>
8     <table class="tabla-crud">
9         <tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th></tr>
10        <c:forEach var="e" items="{lista}">
11            <tr><td>${e.id}</td><td><c:out value="${e.nombre}" /></td>
12                <td><c:out value="${e.correo}" /></td>
13                <td><c:out value="${e.carrera}" /></td></tr>
14        </c:forEach>
15    </table>
16 </body></html>

```

Listado 6.20: lista.jsp — vista con JSTL y EL

Estructura del proyecto CRUD en Java

```
1 crud-java/  
2 |-- pom.xml                                (programador)  
3 |-- db/schema.sql                          (programador)  
4 '-- src/main/  
5     |-- java/ec/edu/uteq/appweb/  
6         |-- modelo/Estudiante.java         (programador: dominio)  
7         |-- dao/EstudianteDAO.java         (programador: datos)  
8         |-- controlador/EstudianteServlet.java (programador: control)  
9     '-- webapp/  
10         |-- WEB-INF/vistas/lista.jsp       (programador: vista)  
11         '-- estilos.css                   (hoja base de la Unidad  
I)
```

Listado 6.21: Estructura del CRUD Java web

6.8 Síntesis del capítulo

Los cinco lenguajes resolvieron el mismo CRUD con la misma idea: *una conexión propia y SQL parametrizado*. Cambian la sintaxis y el marcador de parámetro —? en Perl y Java, :nombre en PHP, %s en Python, @nombre en C#—, pero el patrón es idéntico. En el Capítulo 7 se reescriben estos CRUD con el framework de cada tecnología, donde el ORM generará el SQL por nosotros.

Unidad III

Datos y Patrones Arquitectónicos

Gestión de estado y objetos integrados en aplicaciones web

«HTTP es el protocolo de la amnesia colectiva: cada petición nace y muere sin memoria. Programar la web es, en buena medida, el arte de simular una memoria que el protocolo nos negó.»

— Adaptado de Roy Fielding

Resultado de aprendizaje de la semana

Al concluir la semana 10, el estudiante diseña e implementa soluciones web que integran mecanismos robustos de gestión de estado — cookies, sesiones de servidor, almacenamiento del cliente y tokens — y domina los objetos integrados **Request**, **Response**, **Session**, **Application** y **Page** en PHP, Java y ASP.NET Core, aplicando los flags de seguridad estándar que la industria exige en 2026.

10.1 Introducción: el dilema fundamental de HTTP

HTTP es un protocolo *stateless*: cada petición se trata independientemente, sin recordar las anteriores. Esto es una fortaleza para escalar (cualquier servidor puede atender cualquier petición), pero un problema para construir aplicaciones que necesitan recordar al usuario entre clics. La resolución de esta tensión es el tema central de la semana.

HTTP — el protocolo amnésico que cambió el mundo

Cuando Tim Berners-Lee diseñó HTTP en 1989, lo hizo deliberadamente como un protocolo *stateless* (sin estado): cada petición es independiente, autocontenida, y el servidor no recuerda nada de la petición anterior. Esta decisión, tomada en una época de servidores con poca memoria y conexiones lentas, parecía limitada pero resultó genial: permitió que la web escalara a miles de millones de usuarios sin colapsar bajo el peso de la memoria de sesiones. Pero la web pronto necesitó *algo* que pareciera memoria: ¿cómo saber qué usuario está autenticado?, ¿cómo recordar los productos en el carrito?, ¿cómo personalizar la experiencia? La respuesta llegó en 1994 cuando Lou Montulli, ingeniero de Netscape, inventó las **cookies HTTP** originalmente como solución para un cliente de e-commerce. Las cookies fueron una idea simple: el servidor envía un identificador en la respuesta; el navegador se lo guarda; en cada petición posterior, el navegador devuelve ese identificador. Con eso, el servidor puede asociar peticiones del mismo usuario.

Sobre las cookies se construyeron las **sesiones de servidor**: el servidor guarda los datos del usuario en su propia memoria asociados a un *session ID* aleatorio, y solo el ID viaja en la cookie. Treinta años después, este modelo sigue siendo el sustrato de la mayoría de aplicaciones web tradicionales del mundo. Pero la arquitectura moderna — microservicios, escalado horizontal, SPAs — llevó a la emergencia de los **tokens stateless** (JWT) que trasladan el

estado de vuelta al cliente, permitiendo que el servidor permanezca verdaderamente sin memoria.

La literatura técnica reciente confirma que la gestión de estado sigue siendo uno de los aspectos más críticos de la arquitectura web: [17] en su tesis doctoral fundacional estableció que *statelessness* es una restricción arquitectónica deseable para escalabilidad; [32] discute en profundidad cómo el estado distribuido es la fuente de la mayoría de los bugs sutiles en sistemas modernos; y [31] (RFC 7519) estandarizó JWT como el mecanismo dominante para sesiones distribuidas, hoy ampliamente adoptado en APIs públicas [31].

10.1.1 Las cinco estrategias de gestión de estado

Hay cinco estrategias diferentes para preservar estado entre peticiones HTTP. Cada una tiene ventajas, desventajas y casos donde es la mejor opción. La tabla siguiente las contrasta.

Tabla 10.1: Estrategias de gestión de estado en aplicaciones web modernas.

Mecanismo	Ubicación del estado	Caso de uso ideal	Tamaño
Cookie clásica	Cliente (cookie HTTP)	Preferencias simples (idioma, tema)	4 KB
Sesión servidor	Servidor (memoria/Redis)	App tradicional multi-página	MB
<code>localStorage</code>	Cliente (navegador)	Datos persistentes locales	5–10 MB
<code>sessionStorage</code>	Cliente (pestaña)	Datos volátiles de pestaña	5–10 MB
JWT	Cliente (cookie/header)	APIs REST stateless, SPAs	1–4 KB
IndexedDB	Cliente (navegador)	Apps offline, gran volumen	GB

10.1.2 Los flags de seguridad de cookies en 2026

Antes de profundizar en cualquier mecanismo, hay que dominar los *flags* de seguridad que toda cookie debe llevar en producción:

Los cuatro flags imprescindibles

Secure. La cookie solo viaja por HTTPS, nunca por HTTP plano. *Sin este flag, la cookie puede ser interceptada en redes WiFi públicas.*

HttpOnly. JavaScript del navegador no puede leer la cookie a través de `document.cookie`. *Defensa principal contra XSS para robar la sesión.*

SameSite. Controla cuándo el navegador envía la cookie en peticiones de otros sitios. Tres valores: **Strict** (nunca desde otros sitios), **Lax** (solo en navegación de nivel superior), **None** (siempre, requiere **Secure**). *Defensa principal contra CSRF.*

Max-Age o Expires. Tiempo de vida de la cookie. Sin esto, es *session cookie* (se borra al cerrar el navegador).

Combinación correcta vs incorrecta

Correcto para sesiones de autenticación (Listado 10.1):

```
Set-Cookie: SESSID=abc123; Path=/; Secure; HttpOnly; SameSite=Strict;
Max-Age=1800
```

Listado 10.1: Ejemplo de Shell

Incorrecto (frecuentemente visto en código de aprendizaje) (Listado 10.2):

```
Set-Cookie: SESSID=abc123
```

Listado 10.2: Ejemplo de Shell

La segunda forma expone la cookie a HTTP plano (puede leerse en redes inseguras), permite que JavaScript la lea (vulnerabilidad XSS), y se envía en peticiones cross-site (vulnerabilidad CSRF).

10.2 El objeto Request: lectura segura de la petición

El objeto **Request** encapsula toda la información que el cliente envía al servidor. Su correcta lectura es la primera línea de defensa de la aplicación: nunca se debe confiar en datos del cliente sin filtrarlos.

10.2.1 Componentes del objeto Request

El objeto *Request* encapsula todo lo que el cliente envía: URL, método, cabeceras, cuerpo, cookies. Conocer sus componentes es prerequisite para programar cualquier endpoint profesional.

Ejemplo corto comentado (Listado 10.3):

Listado 10.3: Ejemplo de PHP

```
<?php
// Componentes principales del Request HTTP en PHP
$_SERVER['REQUEST_METHOD']; // 'GET', 'POST', 'PUT', 'DELETE'
$_SERVER['REQUEST_URI']; // '/api/v1/usuarios?id=42'
$_SERVER['HTTP_HOST']; // 'tienda.uteq.edu.ec'
$_SERVER['HTTP_USER_AGENT']; // navegador del cliente
$_GET['id']; // parametros de la URL
$_POST['email']; // datos del cuerpo (form-urlencoded)
$_COOKIE['SESSID']; // cookies enviadas por el cliente
file_get_contents('php://input'); // cuerpo crudo (JSON, XML)
?>
```

- **Método HTTP:** GET, POST, PUT, DELETE, PATCH.
- **URL completa:** ruta, query string, fragmento.
- **Encabezados:** Content-Type, Authorization, User-Agent, Accept-Language.
- **Cookies:** enviadas por el navegador.
- **Cuerpo** (en POST/PUT/PATCH): JSON, form-urlencoded, multipart, raw.
- **Metadatos del cliente:** IP origen, protocolo (HTTP/HTTPS).

Ejemplo 10.1 — Lectura segura del Request en PHP 8.4

Enunciado. Mostrar la lectura segura del objeto Request en PHP 8.4 obteniendo método, URL, cabeceras, query string y cuerpo, con validación básica de cada campo.

```
1 <?php
2 declare(strict_types=1);
3
4 /**
5  * Obtiene la IP real del cliente, considerando proxy reverso.
6  * Verifica las cabeceras X-Forwarded-For y X-Real-IP que un proxy
7  * Nginx o un balanceador de carga (HAProxy, AWS ELB) inyecta.
8  */
9 function obtenerIpCliente(): string {
10     $cabeceras = [
11         'HTTP_X_FORWARDED_FOR', // proxy estandar
12         'HTTP_X_REAL_IP', // Nginx
13         'HTTP_CF_CONNECTING_IP', // Cloudflare
14         'REMOTE_ADDR' // conexion directa
15     ];
16     foreach ($cabeceras as $cabecera) { // itera sobre coleccion
```

```

17         if (!empty($_SERVER[$cabecera])) { // condicional
18             // X-Forwarded-For puede tener varias IPs separadas por
19             // coma
20             $ip = explode(',', $_SERVER[$cabecera])[0];
21             $ip = trim($ip);
22             if (filter_var($ip, FILTER_VALIDATE_IP, // valida/sanea
23                 entrada con filtro
24                 FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE))
25             {
26                 return $ip; // retorna valor
27             }
28         }
29     }
30     return '0.0.0.0'; // retorna valor
31 }
32
33 /**
34  * Lectura segura de un parametro GET.
35  * NUNCA acceder directamente a $_GET['param'] sin filtrar.
36  */
37 function leerEntero(string $nombre, int $minimo = 1,
38                     int $maximo = PHP_INT_MAX, int $defecto = 1): int {
39     $val = filter_input(INPUT_GET, $nombre, FILTER_VALIDATE_INT,
40         ['options' => [
41             'default' => $defecto,
42             'min_range' => $minimo,
43             'max_range' => $maximo
44         ]]);
45     return $val ?? $defecto; // retorna valor
46 }
47
48 function leerCadena(string $nombre, int $maxLen = 100,
49                     string $defecto = ''): string {
50     $raw = $_REQUEST[$nombre] ?? $defecto;
51     $val = trim((string)$raw);
52     // condicional
53     // condicional
54     if (strlen($val) > $maxLen) $val = substr($val, 0, $maxLen);
55     return $val; // retorna valor
56 }
57
58 function exigirMetodo(string $metodo): void {
59     // condicional
60     // condicional
61     if ($_SERVER['REQUEST_METHOD'] !== strtoupper($metodo)) {
62         http_response_code(405);
63         header('Allow: ' . strtoupper($metodo)); // envia cabecera
64         HTTP al cliente
65         exit("Metodo no permitido"); // termina ejecucion
66     }
67 }

```

```

64
65 // Uso conjunto
66 exigirMetodo('GET');
67 $pagina = leerEntero('page', 1, 1000, 1);
68 $busca = leerCadena('q', 80);
69 $ip = obtenerIpCliente();
70
71 // Registrar acceso para auditoria
72 error_log "[" . date('c') . "] $ip - GET ?page=$pagina&q=$busca");

```

Listado 10.4: request-helpers.php

Por qué este código es importante:

- Centraliza la lectura segura en funciones helper reutilizables.
- Detecta proxy reverso correctamente (un error común en producción).
- Filtra IPs privadas y reservadas con flags de `filter_var`.
- Aplica límites estrictos a los enteros (rangos).
- Trunca cadenas demasiado largas para prevenir ataques DoS por memoria.
- Verifica el método HTTP esperado.

10.2.2 Lectura del Request en Java Servlets / Spring Boot

En Java/Spring Boot, el equivalente al `$_SERVER` de PHP es la interfaz `HttpServletRequest` o las anotaciones de Spring (`@RequestParam`, `@RequestHeader`, `@RequestBody`). El ejemplo siguiente las recorre.

Ejemplo 10.2 — Acceso al Request en Spring Boot 4

Enunciado. Mostrar el acceso al objeto Request en Spring Boot 4 usando las anotaciones idiomáticas: `@RequestParam`, `@RequestHeader`, `@RequestBody`.

```

1 package ec.edu.uteq.appweb.controller; // paquete del archivo
2
3 import jakarta.servlet.http.HttpServletRequest; // importacion de
   clase externa
4 import org.springframework.web.bind.annotation.*; // importacion de
   clase externa
5 import java.util.*; // importacion de clase externa
6
7 @RestController // expone endpoints REST con JSON
8 @RequestMapping("/api/inspeccion") // ruta base del controlador
9 public class RequestInspectorController { // clase publica
10
11     @GetMapping // endpoint HTTP GET
12     public Map<String, Object> inspeccionar( // metodo publico
13         HttpServletRequest req,
14         @RequestParam(defaultValue = "") String q, // extrae
           parametro de query string
15         @RequestParam(defaultValue = "1") int page, // extrae
           parametro de query string
16         @RequestHeader(value = "User-Agent", // extrae cabecera
           HTTP
           defaultValue = "?") String userAgent) {
17
18

```

```

19      // Headers selectivos
20      var headers = new LinkedHashMap<String, String>();
21      var nombres = req.getHeaderNames();
22      while (nombres.hasMoreElements()) { // bucle while
23          var n = nombres.nextElement();
24          headers.put(n, req.getHeader(n));
25      }
26
27      return Map.of( // retorna
28          "metodo",      req.getMethod(),
29          "url",         req.getRequestURL().toString(),
30          "remoteAddr",  obtenerIp(req),
31          "userAgent",   userAgent,
32          "queryParams", Map.of("q", q, "page", page),
33          "headersSize", headers.size()
34      );
35  }
36
37  private String obtenerIp(HttpServletRequest r) { // campo privado
38      var fwd = r.getHeader("X-Forwarded-For");
39      if (fwd != null && !fwd.isBlank()) { // condicional
40          return fwd.split(",")[0].trim(); // retorna
41      }
42      return r.getRemoteAddr(); // retorna
43  }
44  }

```

Listado 10.5: RequestInspectorController.java

10.2.3 Lectura del Request en ASP.NET Core 10

ASP.NET Core usa `HttpContext.Request` para acceder a los datos de la petición, con una API moderna basada en propiedades fuertemente tipadas. El ejemplo siguiente la ilustra.

Ejemplo 10.3 — Endpoint que inspecciona el Request en ASP.NET Core

Enunciado. Implementar un endpoint en ASP.NET Core que inspeccione el Request: método, ruta, cabeceras, query string. Se usará el objeto `HttpContext.Request`.

```

1  using Microsoft.AspNetCore.Mvc; // importa namespace
2
3  namespace AppwebApi.Controllers; // namespace del archivo
4
5  [ApiController] // controlador de Web API (validacion automatica)
6  [Route("api/[controller]")] // ruta base del controlador
7  public class InspeccionController : ControllerBase // clase publica
8  {
9      [HttpGet] // endpoint HTTP GET
10     public IActionResult Inspeccionar( // metodo o miembro publico
11         [FromQuery] string q = "", // mapea query string
12         [FromQuery] int page = 1) // mapea query string
13     {

```

```
14      // IP real considerando X-Forwarded-For
15      var ip = HttpContext.Request.Headers["X-Forwarded-For"]
16              .FirstOrDefault()?.Split(',')[0].Trim()
17              ?? HttpContext.Connection.RemoteIpAddress?.ToString()
18              ?? "0.0.0.0";
19
20      return Ok(new { // HTTP 200 con cuerpo
21          metodo      = Request.Method,
22          url          =
23              $"{{Request.Scheme}}://{{Request.Host}}{{Request.Path}}",
24          remoteAddr  = ip,
25          userAgent   = Request.Headers.UserAgent.ToString(),
26          queryParams = new { q, page },
27          cookieCount = Request.Cookies.Count
28      });
29  }
```

Listado 10.6: Controllers/InspeccionController.cs

10.3 El objeto Response: control total de la respuesta HTTP

El objeto `Response` permite controlar tres aspectos: código de estado, cabeceras y cuerpo.

10.3.1 Códigos de estado HTTP — la guía exhaustiva

Los códigos de estado HTTP son la *lengua común* entre cliente y servidor para indicar el resultado de una petición. Confundirlos (devolver 200 cuando hubo un error, por ejemplo) es uno de los defectos más comunes en APIs amateur.

10.3.2 Errores estandarizados con RFC 7807 ProblemDetails

El RFC 7807 [46] estandariza un formato JSON para errores HTTP. Es el estándar de la industria en 2026 y los frameworks modernos (ASP.NET Core, Spring Boot) lo soportan nativamente (Listado 10.7):

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://uteq.edu.ec/errors/validacion",
  "title": "Datos invalidos",
  "status": 422,
  "detail": "El campo email no cumple el formato esperado",
  "instance": "/api/usuarios",
  "errores": {
    "email": ["debe tener formato valido"],
    "edad": ["debe ser mayor a 0"]
  }
}
```

Listado 10.7: Respuesta de error según RFC 7807

Tabla 10.2: Códigos de estado HTTP de uso obligado en aplicaciones web profesionales.

Código	Nombre	Cuándo usarlo (con ejemplos)
2xx — Éxito		
200	OK	Operación exitosa con cuerpo en respuesta. GET, PUT exitosos.
201	Created	Recurso creado. POST exitoso. Incluir Location .
204	No Content	Operación exitosa sin cuerpo. DELETE típicamente.
3xx — Redirección		
301	Moved Permanently	Cambio definitivo de URL. SEO sigue al nuevo URL.
302	Found	Redirección temporal. POST → GET (patrón PRG).
304	Not Modified	Caché válida (responde a If-None-Match).
4xx — Error del cliente		
400	Bad Request	JSON mal formado, parámetro faltante crítico.
401	Unauthorized	No autenticado. Falta JWT o cookie de sesión.
403	Forbidden	Autenticado pero sin permiso. Rol insuficiente.
404	Not Found	Recurso inexistente. ID inválido.
405	Method Not Allowed	Endpoint existe pero no acepta ese verbo. Incluir Allow .
409	Conflict	Email duplicado, optimistic locking failure.
422	Unprocessable Entity	Validación de negocio falla (campos inválidos).
429	Too Many Requests	Rate limit excedido. Incluir Retry-After .
5xx — Error del servidor		
500	Internal Server Error	Excepción no controlada. NUNCA mostrar stack trace al cliente.
502	Bad Gateway	Proxy recibió respuesta inválida del upstream.
503	Service Unavailable	Mantenimiento programado. Incluir Retry-After .
504	Gateway Timeout	Upstream tardó demasiado.

10.3.3 Cabeceras de seguridad obligatorias en producción

En producción, además de las cabeceras de seguridad básicas vistas en la semana 9, conviene configurar otras que protegen contra ataques específicos. La tabla y los ejemplos siguientes las catalogan.

Las siete cabeceras de seguridad imprescindibles

1. **Strict-Transport-Security: max-age=31536000; includeSubDomains; preload**
Fuerza HTTPS por un año en todo el dominio.
2. **Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-XYZ'**
Mitiga XSS limitando fuentes de scripts y estilos.
3. **X-Frame-Options: DENY**
No permite que el sitio se cargue en <iframe>: anti-clickjacking.
4. **X-Content-Type-Options: nosniff**
Impide que el navegador deduzca el tipo MIME (anti-MIME-sniffing).
5. **Referrer-Policy: strict-origin-when-cross-origin**
Equilibrio entre privacidad y funcionalidad.
6. **Permissions-Policy: geolocation=(), microphone=(), camera=()**
Restringe APIs sensibles del navegador.
7. **Cross-Origin-Opener-Policy: same-origin**
Aísla el contexto de navegación de otros orígenes (mitigación Spectre).

Ejemplo 10.4 — Cabeceras de seguridad en las tres plataformas

Enunciado. Aplicar las cabeceras de seguridad obligatorias en producción (**Content-Security-Policy**, **X-Frame-Options**, **Strict-Transport-Security**, etc.) en las tres plataformas del curso.

En PHP (Listado 10.8):

```
<?php
// envia cabecera HTTP al cliente
// envia cabecera HTTP al cliente
```

```

header('Strict-Transport-Security: max-age=31536000;
includeSubDomains');
header("Content-Security-Policy: default-src 'self'"); // envia
cabecera HTTP al cliente
header('X-Frame-Options: DENY'); // envia cabecera HTTP al cliente
header('X-Content-Type-Options: nosniff'); // envia cabecera HTTP al
cliente
header('Referrer-Policy: strict-origin-when-cross-origin'); // envia
cabecera HTTP al cliente
// envia cabecera HTTP al cliente
// envia cabecera HTTP al cliente
header('Permissions-Policy: geolocation=(), microphone=(), camera=()');

```

Listado 10.8: Ejemplo de PHP

En Spring Boot 4 (Listado 10.9):

```

@Configuration // clase de configuracion Spring
public class SeguridadConfig { // clase publica
    @Bean
    // metodo publico
    // metodo publico
    public SecurityFilterChain seguridad(HttpSecurity http) throws
    Exception {
        http.headers(h -> h
            .httpStrictTransportSecurity(hsts -> hsts
                .maxAgeInSeconds(31536000).includeSubDomains(true))
            .frameOptions(f -> f.deny())
            .contentSecurityPolicy(csp -> csp
                .policyDirectives("default-src 'self'"))
            .referrerPolicy(rp -> rp.policy(
                ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
        );
        return http.build(); // retorna
    }
}

```

Listado 10.9: Ejemplo de Java

En ASP.NET Core 10 (Listado 10.10):

```

app.Use(async (ctx, next) => {
    ctx.Response.Headers["Strict-Transport-Security"]
        = "max-age=31536000; includeSubDomains";
    ctx.Response.Headers["X-Frame-Options"] = "DENY";
    ctx.Response.Headers["X-Content-Type-Options"] = "nosniff";
    ctx.Response.Headers["Referrer-Policy"]
        = "strict-origin-when-cross-origin";
    ctx.Response.Headers["Content-Security-Policy"]
        = "default-src 'self'";
    await next();
});

```

Listado 10.10: Ejemplo de C#

10.4 El objeto Session: estado entre peticiones

La *sesión* es el mecanismo más antiguo y aún ampliamente usado para mantener estado entre peticiones: el servidor guarda los datos del usuario en memoria (o en Redis) y envía al cliente solo un identificador.

10.4.1 Mecánica detallada de las sesiones de servidor

Comprender la mecánica detallada de las sesiones de servidor es clave porque la mayoría de fallos de seguridad ocurren por desconocimiento de cómo se genera, transporta y persiste el *Session ID*.

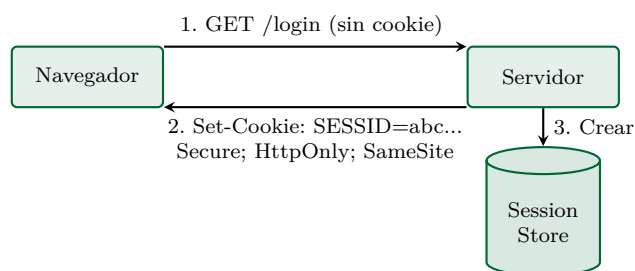


Figura 10.1: Establecimiento de una sesión: el servidor genera un ID aleatorio, lo asocia con datos en su *Session Store* (memoria, Redis o BD) y lo envía al cliente como cookie con flags de seguridad.

10.4.2 Implementación comparada en las cuatro plataformas

Cada plataforma implementa sesiones con su propia API, pero los conceptos son los mismos. La tabla siguiente las contrasta en el orden del libro —Spring Boot primero—: Spring Boot, Django, PHP y ASP.NET Core.

Comparativa de APIs de sesión				
Operación	Spring Boot 4	Django 5.2	PHP 8.4	ASP.NET Core 10
Iniciar	automático (Servlet)	<i>middleware</i> automático	<code>session_start()</code>	<code>UseSession()</code>
Guardar	<code>setAttribute(k,v)</code>	<code>session[k]=v</code>	<code>\$_SESSION[k]=\$v</code>	<code>Session.SetString</code>
Leer	<code>getAttribute(k)</code>	<code>session[k]</code>	<code>\$_SESSION[k]</code>	<code>Session.GetString</code>
Borrar	<code>removeAttribute(k)</code>	<code>del session[k]</code>	<code>unset(\$_SESSION[k])</code>	<code>Session.Remove</code>
Destruir	<code>invalidate()</code>	<code>session.flush()</code>	<code>session_destroy()</code>	<code>Session.Clear()</code>
Regenerar ID	<code>changeSessionId()</code>	<code>cycle_key()</code>	<code>session_regenerate_id()</code>	regenerar cookie
Cookie	JSESSIONID	sessionid	PHPSESSID	<code>AspNetCore.Session</code>
Almacén	JVM / Redis	BD / cache / archivo	archivos /tmp	Memoria / Redis / SQL
Timeout	30 min	2 semanas	24 min	20 min

10.4.3 Sesiones en PHP 8.4 con configuración profesional

PHP gestiona sesiones con la función nativa `session_start()`. El ejemplo siguiente la configura con todos los flags de seguridad de producción y muestra los anti-patrones comunes a evitar.

Ejemplo 10.5 — Sesión segura completa con login PHP

Enunciado. Construir una sesión segura completa con login PHP: hash BCrypt, cookies con todos los flags de seguridad, regeneración de ID y logout robusto.

```

1 <?php
2 declare(strict_types=1);
3
4 /**
5  * Configuración de sesión para producción.
6  * DEBE llamarse ANTES de session_start().
7  */
8 function configurarSesionSegura(): void {
9     // Solo cookies HTTPS
10    ini_set('session.cookie_secure', '1');
11    // JavaScript no puede leer la cookie
12    ini_set('session.cookie_httponly', '1');
13    // Anti-CSRF
14    ini_set('session.cookie_samesite', 'Strict');
15    // Forzar regeneración de ID si llega uno desconocido
16    ini_set('session.use_strict_mode', '1');
17    // Timeout 30 min de inactividad
18    ini_set('session.gc_maxlifetime', '1800');
19    // Cookie expira al cerrar navegador (=0) o tras 30 min (1800)
20    session_set_cookie_params([ // flags de cookie
21        'lifetime' => 1800,
22        'path'      => '/',
23        'secure'    => true,
24        'httponly'  => true,
25        'samesite'  => 'Strict',
26    ]);
27 }

```

Listado 10.11: config/session.php (incluir antes de session_start)

```

1 <?php
2 declare(strict_types=1);
3 require_once __DIR__ . '/config/session.php'; // incluye otro archivo
4     PHP (fatal si falla)
5
6 require_once __DIR__ . '/config/db.php'; // incluye otro archivo PHP
7     (fatal si falla)
8
9 configurarSesionSegura();
10 session_start(); // arranca o reanuda la sesión
11
12 // Si ya está autenticado, redirigir
13 if (!empty($_SESSION['user_id'])) { // condicional
14     header('Location: /dashboard.php'); // envía cabecera HTTP al
15         cliente
16     exit;
17 }
18
19 if ($_SERVER['REQUEST_METHOD'] === 'POST') { // condicional
20     // Validar token CSRF (anti-CSRF en formularios)
21     $tokenEnviado = $_POST['_csrf'] ?? '';
22     $tokenSesion = $_SESSION['_csrf'] ?? '';

```

```
19     if (!hash_equals($tokenSession, $tokenEnviado)) { // condicional
20         http_response_code(419);
21         exit('Token CSRF invalido'); // termina ejecucion
22     }
23
24     $email = filter_input(INPUT_POST, 'email',
25                           FILTER_VALIDATE_EMAIL);
26     $pass  = $_POST['password'] ?? '';
27
28     if (!$email || strlen($pass) < 8) { // condicional
29         http_response_code(422);
30         echo 'Datos invalidos'; // imprime al output del servidor
31         exit;
32     }
33
34     // Rate limiting en memoria (en produccion: Redis)
35     $intentosKey = 'intentos:' . $_SERVER['REMOTE_ADDR'];
36     if (($_SESSION[$intentosKey] ?? 0) >= 5) { // condicional
37         http_response_code(429);
38         header('Retry-After: 900'); // 15 min
39         exit('Demasiados intentos. Vuelva en 15 minutos.');// termina
40         // ejecucion
41     }
42
43     $pdo = obtenerPDO();
44     $stmt = $pdo->prepare( // prepara consulta SQL (defensa anti-SQLi)
45                          // hashea contrasena con BCrypt
46                          // hashea contrasena con BCrypt
47                          'SELECT id, password_hash, rol, activo, intentos_fallidos
48                          FROM usuarios WHERE email = :em LIMIT 1');
49     $stmt->execute([':em' => $email]); // ejecuta la consulta con
50     // parametros enlazados
51     $u = $stmt->fetch(PDO::FETCH_ASSOC); // recupera la siguiente fila
52     // como array
53
54     // password_verify usa tiempo constante (resistente a timing
55     // attacks)
56     // hashea contrasena con BCrypt
57     // hashea contrasena con BCrypt
58     if (!$u || !$u['activo'] || !password_verify($pass,
59         $u['password_hash'])) {
60         $_SESSION[$intentosKey] = ($_SESSION[$intentosKey] ?? 0) + 1;
61         http_response_code(401);
62         echo 'Credenciales invalidas'; // imprime al output del
63         // servidor
64         exit;
65     }
66
67     // CRITICAL: regenerar el ID para prevenir session fixation
68     session_regenerate_id(true); // cambia el ID para evitar session
69     // fixation
```

```

63
64     $_SESSION['user_id']      = (int)$u['id'];
65     $_SESSION['user_rol']    = $u['rol'];
66     $_SESSION['login_time']  = time();
67     $_SESSION['ip']          = $_SERVER['REMOTE_ADDR'];
68     $_SESSION['ua']          = $_SERVER['HTTP_USER_AGENT'] ?? '';
69     unset($_SESSION[$intentosKey]);
70
71     // Auditoria
72     error_log("[LOGIN] user_id={$u['id']}
73               ip={$_SERVER['REMOTE_ADDR']}");
74
75     header('Location: /dashboard.php'); // envia cabecera HTTP al
76     cliente
77     exit;
78 }
79
80 // Generar token CSRF para mostrar el formulario
81 if (empty($_SESSION['_csrf'])) { // condicional
82     $_SESSION['_csrf'] = bin2hex(random_bytes(32));
83 }
84 $csrf = $_SESSION['_csrf'];
85 ?>
86 <!DOCTYPE html>
87 <html lang="es-EC">
88 <head><meta charset="UTF-8"><title>Login UTEQ</title></head>
89 <body>
90 <form method="POST">
91     // escapa caracteres HTML (defensa anti-XSS)
92     // escapa caracteres HTML (defensa anti-XSS)
93     <input type="hidden" name="_csrf" value="<?= htmlspecialchars($csrf)
94     ?>">
95     <label>Email: <input type="email" name="email" required></label>
96     <label>Password: <input type="password" name="password" required
97         minlength="8"></label>
98     <button type="submit">Ingresar</button>
99 </form>
100 </body></html>

```

Listado 10.12: login.php con todas las prácticas

Defensas implementadas en este código (todas combinadas):

1. Cookies con Secure, HttpOnly, SameSite=Strict.
2. Token CSRF generado por sesión y validado con hash_equals (tiempo constante).
3. Validación email con FILTER_VALIDATE_EMAIL.
4. Rate limiting (5 intentos por IP).
5. password_verify (BCrypt, resistente a timing attacks).
6. session_regenerate_id(true): previene session fixation.
7. Auditoría con error_log.
8. htmlspecialchars al renderizar (anti-XSS).
9. Verificación de cuenta activa.

10.5 Almacenamiento del cliente: localStorage, sessionStorage e IndexedDB

Además del almacenamiento del servidor (cookies de sesión, JWT), el navegador ofrece tres APIs nativas para guardar datos del lado cliente: `localStorage`, `sessionStorage` e `IndexedDB`. Conocer cuándo usar cada una es esencial para SPAs modernas.

10.5.1 Comparativa de las APIs de almacenamiento del cliente

La tabla siguiente contrasta las tres APIs de almacenamiento cliente por capacidad, persistencia, tipo de datos y casos de uso recomendados.

Tabla 10.3: Almacenamiento del cliente en navegadores modernos.

Mecanismo	Características	Capacidad	API
Cookies	Enviadas con cada petición; <i>HttpOnly</i> oculta de JS	4 KB	<code>document.cookie</code>
<code>localStorage</code>	Persistente, mismo origen, síncrono, solo strings	5–10 MB	Síncrona
<code>sessionStorage</code>	Solo durante la pestaña, mismo origen	5–10 MB	Síncrona
<code>IndexedDB</code>	Persistente, transaccional, asíncrono, objetos	GB	Asíncrona
Cache API	Almacena objetos Response (Service Workers)	GB	Asíncrona

La regla de oro — ¿dónde guardar el JWT?

Nunca guarde el JWT en `localStorage` si su aplicación tiene cualquier riesgo de XSS (lo cual es prácticamente toda aplicación). El JavaScript puede leer `localStorage`; un solo script malicioso inyectado roba todos los tokens. **Solución correcta:** cookie `HttpOnly` + `Secure` + `SameSite=Strict`. JavaScript no puede leer cookies `HttpOnly`; el navegador las envía automáticamente al servidor en cada petición al mismo origen.

10.5.2 Ejemplos prácticos de almacenamiento

Los siguientes ejemplos breves muestran las tres APIs en acción para el caso más frecuente: persistir preferencias de UI.

Ejemplo corto comentado (`localStorage`) (Listado 10.13):

```
// localStorage persiste INDEFINIDAMENTE (hasta que el usuario lo borre)
// Solo guarda STRINGS, así que objetos hay que serializarlos con JSON
localStorage.setItem('tema', 'oscuro'); // escribe
const tema = localStorage.getItem('tema'); // lee
localStorage.removeItem('tema'); // borra una clave
localStorage.clear(); // borra todo

// Guardar un objeto: serializar con JSON.stringify
const prefs = { tema: 'oscuro', idioma: 'es' }; // constante (no reassignable)
// serializa objeto a string JSON
// serializa objeto a string JSON
localStorage.setItem('preferencias', JSON.stringify(prefs));
// Recuperar y parsear
// constante (no reassignable)
// constante (no reassignable)
const guardado = JSON.parse(localStorage.getItem('preferencias') || '{}');
```

Listado 10.13: Ejemplo de JavaScript

Ejemplo 10.6 — Persistencia de preferencias UI con localStorage

Enunciado. Persistir preferencias de UI (tema oscuro, idioma) en localStorage desde JavaScript, mostrando cómo serializar objetos con JSON y manejar errores de cuota.

```

1  "use strict";
2
3  const PREF_KEY = "uteq:preferencias:v1"; // constante (no reassignable)
4
5  const Preferencias = { // constante (no reassignable)
6    cargar() {
7      try {
8          const raw = localStorage.getItem(PREF_KEY); // constante (no
              reassignable)
9          return raw ? JSON.parse(raw) : this.defectos(); // devuelve
              valor de la funcion
10         } catch (e) {
11             console.warn("Error leyendo preferencias", e);
12             return this.defectos(); // devuelve valor de la funcion
13         }
14     },
15     guardar(p) {
16         try {
17             localStorage.setItem(PREF_KEY, JSON.stringify(p)); // serializa
              objeto a string JSON
18         } catch (e) {
19             console.error("Error guardando preferencias", e);
20         }
21     },
22     defectos() {
23         return { tema: "auto", idioma: "es-EC", // devuelve valor de la
              funcion
24                 fontSize: 1.0, animaciones: true };
25     },
26     aplicar(p) {
27         document.documentElement.dataset.tema = p.tema;
28         document.documentElement.lang = p.idioma;
29         document.documentElement.style.fontSize = `${p.fontSize}rem`;
30         if (!p.animaciones) { // condicional
31             document.documentElement.classList.add("sin-animaciones");
32         }
33     }
34 };
35
36 // Al cargar
37 document.addEventListener("DOMContentLoaded", () => { // registra
    manejador de evento
38     const p = Preferencias.cargar(); // constante (no reassignable)
39     Preferencias.aplicar(p);
40
41     // Boton de cambio de tema
42     // busca elemento por id
43     // busca elemento por id

```

```
44 document.getElementById("toggle-tema")?.addEventListener("click", ()  
    => {  
45     p.tema = p.tema === "claro" ? "oscuro" : "claro";  
46     Preferencias.guardar(p);  
47     Preferencias.aplicar(p);  
48 });  
49 });
```

Listado 10.14: preferencias.js

10.6 Práctica integrada: carrito de compras con sesiones

La práctica de la semana es construir un carrito de compras completo con sesiones de servidor, integrando todos los conceptos: Request, Response, Session, Cookie segura y validación.

Ejercicio 10.1 — Carrito de compras stateful en las tres plataformas (Sumativa GA-Práctica)

Objetivo: implementar un carrito de compras con sesiones de servidor en PHP 8.4, ASP.NET Core 10 y Spring Boot 4, evidenciando las diferencias de API y cómo cada plataforma maneja seguridad de cookies.

Especificación funcional:

- Página de listado con productos hardcodeados (nombre, precio).
- Botón «Agregar» para cada producto que lo añade al carrito.
- Página del carrito que muestra los productos agregados y total.
- Botón «Eliminar» por producto en el carrito.
- Botón «Vaciar carrito».
- La cookie de sesión debe tener `HttpOnly`, `Secure` y `SameSite=Strict`.

IDE recomendado

IntelliJ IDEA Ultimate con plugins:

- *PHP* (JetBrains, instalado por defecto en Ultimate).
- *Spring Boot* (incluido en Ultimate).
- *.NET Core* (Marketplace) o usar JetBrains Rider para C#.

Plataforma 1 — PHP 8.4 con XAMPP

Paso 1: verificar XAMPP. Iniciar el panel (`xampp-control`). Pulsar **Start** en Apache. Verificar en <http://localhost> que aparece el dashboard XAMPP.

Paso 2: crear carpeta `C:\xampp\htdocs\carrito-php`. En IntelliJ: **File** → **Open**, seleccionar la carpeta.

Paso 3: crear `config.php` (Listado 10.15):

```
1 <?php  
2 declare(strict_types=1);  
3  
4 ini_set('session.cookie_httponly', '1');  
5 ini_set('session.cookie_secure', '1');  
6 ini_set('session.cookie_samesite', 'Strict');  
7 ini_set('session.use_strict_mode', '1');  
8 ini_set('session.gc_maxlifetime', '1800');  
9  
10 session_name('CARRITO_PHP');  
11 session_start(); // arranca o reanuda la sesion
```

```

12
13 // Inicializar carrito si no existe
14 if (!isset($_SESSION['carrito'])) { // condicional
15     $_SESSION['carrito'] = [];
16 }
17
18 // Catalogo hardcodeado
19 const CATALOGO = [
20     1 => ['id'=>1, 'nombre'=>'Cafe galletti 500g', 'precio'=>4.50],
21     2 => ['id'=>2, 'nombre'=>'Cacao Pacari 70%', 'precio'=>6.00],
22     3 => ['id'=>3, 'nombre'=>'Te verde organico', 'precio'=>3.20],
23     4 => ['id'=>4, 'nombre'=>'Panela en bloque', 'precio'=>2.80],
24 ];

```

Listado 10.15: config.php

Paso 4: crear index.php (Listado 10.16):

```

1 <?php require_once 'config.php'; ?>
2 <!DOCTYPE html>
3 <html lang="es-EC">
4 <head><meta charset="UTF-8"><title>Tienda UTEQ</title>
5 <style>
6     body{font-family:system-ui;max-width:700px;margin:2rem
7         auto;padding:0 1rem}
8     h1{color:#006633} table{width:100%;border-collapse:collapse}
9     td,th{padding:.5rem;text-align:left;border-bottom:1px solid #eee}
10    .btn{padding:.4rem .8rem;background:#006633;color:white;
11        border:0;border-radius:4px;cursor:pointer}
12 </style>
13 </head>
14 <body>
15 <h1>Productos UTEQ</h1>
16 <p><a href="carrito.php">Ver carrito (<?= count($_SESSION['carrito'])
17     ?>)</a></p>
18 <table>
19     <thead><tr><th>Producto</th><th>Precio</th><th></th></tr></thead>
20     <tbody>
21     <?php foreach (CATALOGO as $p): ?>
22         <tr>
23             <td><?= htmlspecialchars($p['nombre']) ?></td> // escapa
24                 caracteres HTML (defensa anti-XSS)
25             <td>$<?= number_format($p['precio'], 2) ?></td>
26             <td>
27                 <form action="agregar.php" method="post" style="margin:0">
28                     <input type="hidden" name="id" value="<?= $p['id'] ?>">
29                     <button type="submit" class="btn">Agregar</button>
30                 </form>
31             </td>
32         </tr>
33     <?php endforeach; ?>
34 </tbody>

```



```

32 </table>
33 </body></html>

```

Listado 10.16: index.php

Paso 5: crear agregar.php, eliminar.php, vaciar.php, carrito.php (continuación obvia siguiendo el patrón).

Paso 6: probar.

1. Abrir <http://localhost/carrito-php/index.php>.
2. Agregar 3 productos diferentes.
3. Ir a Carrito: aparecen los 3.
4. Cerrar la pestaña, abrir nueva: el carrito persiste.
5. Cerrar el navegador completamente, abrirlo de nuevo: el carrito sigue ahí (porque `lifetime=1800`).

Paso 7: verificar la cookie. F12 → Application → Cookies → <http://localhost>. Debe aparecer CARRITO_PHP con: `HttpOnly = true, Secure = true, SameSite = Strict`.

Resultado esperado: carrito persistente entre páginas y entre sesiones del navegador, con cookie hardenizada visible en DevTools.

Plataforma 2 — ASP.NET Core 10

Paso 1: verificar .NET 8. `dotnet -version` debe devolver 8.0.x.

Paso 2: crear el proyecto desde la terminal de IntelliJ (Alt+F12) (Listado 10.17):

```

dotnet new mvc -n CarritoAsp -o CarritoAsp
cd CarritoAsp # cambia directorio

```

Listado 10.17: Ejemplo de Shell

Paso 3: configurar sesiones en Program.cs (Listado 10.18):

```

1 using Microsoft.AspNetCore.Builder; // importa namespace
2 using Microsoft.Extensions.DependencyInjection; // importa namespace
3
4 var builder = WebApplication.CreateBuilder(args); // crea el builder
   de la app
5
6 builder.Services.AddControllersWithViews(); // configuracion de
   servicios DI
7 builder.Services.AddDistributedMemoryCache(); // configuracion de
   servicios DI
8 builder.Services.AddSession(o => { // configuracion de servicios DI
9     o.IdleTimeout = TimeSpan.FromMinutes(30);
10    o.Cookie.Name      = "CARRITO_ASP";
11    o.Cookie.HttpOnly  = true;
12    o.Cookie.IsEssential = true;
13    o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
14    o.Cookie.SameSite   = SameSiteMode.Strict;
15 });
16
17 var app = builder.Build(); // construye la app a partir del builder
18 app.UseHttpsRedirection();
19 app.UseStaticFiles();
20 app.UseRouting(); // activa el routing del pipeline
21 app.UseSession();
22 app.UseAuthorization();

```

```

23 app.MapControllerRoute(
24     name: "default",
25     pattern: "{controller=Tienda}/{action=Index}/{id?}");
26 app.Run(); // arranca la aplicacion

```

Listado 10.18: Program.cs

Paso 4: crear Models/Producto.cs y Controllers/TiendaController.cs (con acciones Index, Agregar, Carrito, Eliminar, Vaciar usando HttpContext.Session.SetString con JSON).

Paso 5: ejecutar con dotnet run. Abrir <https://localhost:5001>.

Paso 6: verificar la cookie CARRITO_ASP en DevTools: debe tener HttpOnly, Secure, SameSite=Strict.

Plataforma 3 — Spring Boot 4

Paso 1: en IntelliJ Ultimate: File → New → Project → Spring Boot. Java 21, Spring Boot 4, dependencias Spring Web y Thymeleaf.

Paso 2: configurar src/main/resources/application.yml (Listado 10.19):

```

1 server: # configuracion del servidor embebido
2   port: 8080 # puerto donde escucha la aplicacion
3   servlet:
4     session:
5       timeout: 30m
6     cookie:
7       name: CARRITO_SPRING
8       http-only: true
9       secure: false # true en HTTPS de produccion
10      same-site: strict

```

Listado 10.19: application.yml

Paso 3: crear Producto.java (record Serializable), TiendaController.java con HttpSession.

Paso 4: ejecutar y probar en <http://localhost:8080>.

Tabla comparativa de la práctica (entrega)

Cada estudiante debe completar y entregar:

Aspecto	PHP	ASP.NET	Spring
Líneas de código del proyecto			
Tiempo de arranque			
Cookie tiene HttpOnly	¿sí/no?	¿sí/no?	¿sí/no?
Cookie tiene Secure			
Cookie tiene SameSite=Strict			
Carrito persiste entre pestañas			

Reflexión final

Responder por escrito en máximo 200 palabras: ¿qué plataforma le resultó más fácil? ¿Cuál considera más segura por defecto? ¿Cuál usaría en un proyecto real y por qué?

Esta es la *Sumativa GA-Práctica en clase* de la semana 10.

10.7 Buenas prácticas profesionales en gestión de estado

Las técnicas de gestión de estado tienen su propio conjunto de reglas profesionales. La caja siguiente las reúne en 15 puntos.

Quince reglas profesionales 2026

1. Cookies de sesión SIEMPRE con `Secure`, `HttpOnly`, `SameSite=Strict`.
2. Regenerar el ID de sesión al hacer login (anti-fixation).
3. Tiempo de inactividad razonable: 15–30 min para apps sensibles, 60 min para uso general.
4. Logout debe invalidar el ID de sesión, no solo limpiar variables.
5. Para APIs REST que sirven SPAs: preferir JWT en cookie `HttpOnly`.
6. NUNCA guardar JWT en `localStorage` en sitios con riesgo de XSS.
7. Validar token CSRF en todo POST/PUT/PATCH/DELETE.
8. En clusters: usar Redis o BD compartida para sesiones, no memoria local.
9. No almacenar información sensible en la cookie misma; solo el ID.
10. Auditar logins exitosos y fallidos con IP y User-Agent.
11. Implementar rate limiting (máx. 5–10 intentos por IP por minuto).
12. Cookies con `Domain` restrictivo (no `example.com` si solo se necesita `app.example.com`).
13. Nombres de cookies que NO revelen la tecnología (`SESS` en lugar de `PHPSESSID`).
14. En SPAs Angular/React: usar `credentials: 'include'` y configurar CORS con `Access-Control-Allow-Credentials: true`.
15. Documentar la estrategia de gestión de estado en la ADR-001 del proyecto.

10.8 Ejercicio resuelto adicional con resultado visual

Como cierre de la semana, el siguiente ejercicio adicional está completamente resuelto e incluye captura del resultado visual. Se complementa con un ejercicio propuesto para que el estudiante practique.

Ejercicio resuelto 10.1 — Contador de visitas con sesión PHP

Enunciado. Implementar una página PHP que muestre cuántas veces el usuario ha visitado el sitio en la sesión actual, con botones para incrementar manualmente y para reiniciar el contador. La cookie de sesión debe llevar todos los flags de seguridad de producción.

Código completo (contador.php):

Listado 10.20: Contador stateful con sesión segura

```
<?php
// Configuración ANTES de session_start()
session_set_cookie_params([ // flags de cookie
    'lifetime' => 1800,
    'path'     => '/',
    'domain'   => '',
    'secure'   => false, // true en HTTPS de producción
    'httponly' => true,
    'samesite' => 'Strict'
]);
session_name('VISITAS_SESS');
session_start(); // arranca o reanuda la sesión

// Inicializar contador
if (!isset($_SESSION['visitas'])) { // condicional
    $_SESSION['visitas'] = 1;
} else {
    // Solo incrementar en GET sin parametros (visita real)
    // condicional
    // condicional
    if ($_SERVER['REQUEST_METHOD'] === 'GET' && empty($_GET)) {
        $_SESSION['visitas']++;
    }
}
```

```
// Acciones
if (isset($_GET['accion'])) { // condicional
    // condicional
    // condicional
    if ($_GET['accion'] === 'incrementar') $_SESSION['visitas']++;
    // condicional
    // condicional
    if ($_GET['accion'] === 'reiniciar') $_SESSION['visitas'] = 0;
    header('Location: _contador.php'); // envia cabecera HTTP al cliente
    exit;
}
?>
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Contador de visitas</title></head>
<body>
    <h1>Contador de visitas</h1>
    <p>Visitas en esta sesion: <strong><?= $_SESSION['visitas'] ?></strong></p>
    // escapa caracteres HTML (defensa anti-XSS)
    // escapa caracteres HTML (defensa anti-XSS)
    <p>Session ID: <code><?= htmlspecialchars(session_id()) ?></code></p>
    <a href="?accion=incrementar">Incrementar +1</a> |
    <a href="?accion=reiniciar">Reiniciar</a>
</body>
</html>
```

Resultado visual en el navegador (simulación de cómo se verá la página en la pantalla del estudiante tras varias recargas e incrementos manuales):

Contador de visitas

Visitas en esta sesión: **7**

Session ID: a8f3d9c2e1b4...

[Incrementar +1](#) | [Reiniciar](#)

Verificación de los flags de seguridad con DevTools del navegador (pestaña *Application* → *Cookies*):

Cookie: VISITAS_SESS

Value: a8f3d9c2e1b4...

HttpOnly: ✓ **Secure:** × **SameSite:** Strict

Expires: Session

Discusión. El contador demuestra que la sesión persiste entre recargas (el navegador reenvía la cookie automáticamente) y que el servidor reconoce al mismo usuario. La cookie es invisible a JavaScript (**HttpOnly**), bloqueando ataques XSS de robo de sesión, y solo se envía desde el propio dominio (**SameSite=Strict**), bloqueando CSRF. En producción HTTPS, añadir **secure=true** bloquea su envío por HTTP plano.

Ejercicio propuesto 10.2 — Autenticación con login persistente «Recuérdame»

Enunciado. Implementar en cualquiera de las tres plataformas (PHP, Spring Boot o ASP.NET Core) un flujo de login con la opción «Recuérdame» que mantenga al usuario autenticado durante 30 días si la marca, o solo durante la sesión del navegador si no.

Requisitos técnicos:

1. Formulario de login con campos **usuario**, **contraseña** y checkbox **recordarme**.
2. Si **recordarme** está marcado: crear una cookie adicional **REMEMBER_TOKEN** con un valor aleatorio de 256 bits (**random_bytes(32)** en PHP, **SecureRandom** en Java), **Max-Age=2592000** (30 días), todos los flags de seguridad.

3. Persistir el hash del token (`password_hash()` o `BCrypt`) en una tabla `remember_tokens(user_id, token_hash, expires_at)`.
4. En cada nueva sesión, si el usuario no está autenticado pero existe `REMEMBER_TOKEN`, validar contra la tabla y autenticar automáticamente.
5. Al hacer logout, eliminar la fila correspondiente y borrar la cookie con `Max-Age=0`.
6. Implementar rotación del token tras cada uso (defensa contra robo).

Criterios de evaluación:

- Token nunca se almacena en plano (solo el hash).
- Cookies con todos los flags (`HttpOnly`, `Secure`, `SameSite=Strict`, `Max-Age`).
- Rotación correcta del token tras autenticación automática.
- Caso de logout limpia BD y navegador.
- Documentación con capturas de Postman o navegador mostrando las cookies generadas.

Lecturas recomendadas: [31] para comparar con JWT; [49] para defensas contra robo de sesión.

Acceso a datos desde aplicaciones web: SQL parametrizado y ORM

«Toda aplicación que persiste datos hereda, tarde o temprano, la disciplina o el caos de cómo accede a ellos. Elegir bien es proteger el sistema durante años.»

— Adaptado de Martin Fowler

Resultado de aprendizaje de la semana

Al concluir la semana 11, el estudiante diseña e implementa el acceso a datos relacionales desde aplicaciones web utilizando SQL parametrizado y mapeo objeto-relacional (ORM), aplicando los patrones Repository y Unit of Work, prevenir totalmente la inyección SQL y comprende el problema N+1 y su solución.

11.1 Historia y panorama del acceso a datos

El acceso a datos es probablemente la decisión arquitectónica con mayor impacto en el rendimiento y la mantenibilidad de una aplicación web. La elección entre SQL plano, JPA/Hibernate, Entity Framework Core o Eloquent no es de detalle: condiciona todo el código del backend.

Del SQL embebido a los ORM modernos — 50 años de iteración

La historia del acceso a datos desde aplicaciones web comienza con SQL en su forma más cruda: cadenas de texto concatenadas dentro del código de la aplicación. En 1995–2005 era habitual ver código PHP así: `$sql = "SELECT * FROM users WHERE name = '" . $_POST['name'] . "'";`. Este patrón — inocente en apariencia — destruyó la seguridad de miles de aplicaciones durante una década por SQL Injection.

La primera respuesta fueron las **sentencias preparadas** (*prepared statements*), formalizadas en SQL-92 y ampliamente disponibles desde principios de los 2000: el SQL se compila una vez con marcadores (`?` o `:nombre`), y los valores viajan separadamente sin interpretarse como código. Esto cerró la mayor puerta de entrada a los ataques.

Una segunda evolución fueron los **Query Builders**: APIs fluidas para construir SQL programáticamente sin escribirlo a mano (Doctrine DBAL en PHP, jOOQ en Java). Permiten componer consultas dinámicas sin riesgo de inyección.

El salto cualitativo vino con los **ORM** (Object-Relational Mappers), que abstraen completamente la base de datos detrás de objetos del lenguaje. Hibernate en Java (Gavin King, 2001), Active Record en Ruby on Rails (David Heinemeier Hansson, 2004), Doctrine en PHP (Roman Borschel, 2006), Entity Framework en .NET (Microsoft, 2008) y Eloquent en Laravel (Taylor Otwell, 2011) representan distintas filosofías sobre el mismo problema. Hoy un desarrollador web rara vez escribe SQL a mano; pero comprender lo que hace el ORM por debajo es indispensable.

La literatura técnica confirma que la elección del mecanismo de acceso a datos es una decisión arquitectónica de primer orden. [21] describe formalmente los patrones que hoy dominan (Data Mapper, Active Record, Unit of Work, Identity Map); [50] y [4] son las referencias canónicas para Hibernate; [62] para Entity Framework Core; [51] para Eloquent. El trabajo empírico de [60] compara performance entre los principales ORM y documenta que la diferencia de rendimiento respecto a SQL plano oscila entre 10% y 30%, justificándose ampliamente por la productividad ganada. Estudios más recientes [50] en revistas indexadas confirman que el problema N+1 sigue siendo la causa número uno de degradación de performance en aplicaciones Java EE y Spring Boot en producción.

11.1.1 Las cuatro estrategias de acceso a datos

En la industria conviven cuatro estrategias para acceder a datos desde una aplicación web. La tabla siguiente las contrasta con sus ventajas, desventajas y cuándo elegir cada una.

Tabla 11.1: Estrategias de acceso a datos en aplicaciones web (2026).

Estrategia	Características	Cuándo usarla
Driver nativo + SQL	Máximo control y rendimiento; máxima responsabilidad. PDO (PHP), JDBC (Java), ADO.NET (C#).	Reportes complejos, consultas analíticas, ETL.
Query Builder	SQL programático sin concatenación. Doctrine DBAL, jOOQ, knex.js.	Consultas dinámicas con muchos filtros opcionales.
ORM Active Record	La entidad es su unidad de persistencia: <code>User::find(1)->save()</code> . Eloquent, Rails AR.	CRUDs estándar; prototipado rápido.
ORM Data Mapper	Entidades de dominio agnósticas; mapper externo. Hibernate, Doctrine ORM, EF Core.	Aplicaciones empresariales medianas/grandes con DDD.

11.1.2 Tabla evolutiva de los ORM principales

Los ORM dominantes han evolucionado durante dos décadas hasta sus versiones actuales. La tabla siguiente recoge las versiones de referencia 2026 con su licencia y características notables.

Tabla 11.2: Versiones actuales de ORM en 2026.

ORM	Lenguaje	Versión 2026	Filosofía
Hibernate / JPA	Java	Hibernate 6.6 / JPA 3.2	Data Mapper
Spring Data JPA	Java	3.4	Repository sobre JPA
Eloquent	PHP	Laravel 11 incluido	Active Record
Doctrine ORM	PHP	3.x	Data Mapper
Entity Framework Core	C#	8.0	Code-First, Data Mapper
TypeORM	TypeScript	0.3	Híbrido AR/DM
Prisma	TypeScript	5.x	Schema-first generador
SQLAlchemy	Python	2.x	Data Mapper + AR
Active Record (Rails)	Ruby	8.x	Active Record canónico
GORM	Go	1.25	Active Record

11.2 Consultas SQL desde aplicaciones web

Aunque el ORM oculte el SQL, el desarrollador web profesional debe conocer cómo se ejecutan las consultas desde aplicaciones, qué amenazas existen y qué defensas debe aplicar siempre.

11.2.1 La amenaza fundamental: SQL Injection

SQL Injection no es una vulnerabilidad teórica: ha sido la causa de ataques contra Sony, LinkedIn, Yahoo y miles de organizaciones. Sigue siendo la *Top 3* del OWASP en 2021. Su mecánica es engañosamente simple.

Ejemplo corto comentado (anatomía de un SQL Injection) (Listado 11.1):

Listado 11.1: Ejemplo de PHP

```
<?php
// Codigo vulnerable: concatena directamente la entrada del usuario
$email = $_POST['email'];
$sql = "SELECT * FROM users WHERE email = '$email'";

// El atacante introduce en el campo email:
// ' OR '1'='1
// La consulta resultante se convierte en:
// SELECT * FROM users WHERE email = '' OR '1'='1'
// Que devuelve TODOS los usuarios. El atacante entra sin password.
?>
```

Anti-patrón — el código que NUNCA debe escribirse

```
// VULNERABLE - PHP
$sql = "SELECT * FROM users WHERE email = '" . $_POST['email'] . "'";
$res = $pdo->query($sql);
```

Listado 11.2: Ejemplo de PHP

Ejemplo de ataque trivial:

Si `$_POST['email'] = "admin' OR '1'='1' --"`, la consulta ejecutada será:

```
SELECT * FROM users WHERE email = 'admin' OR '1'='1' --'
```

que devuelve *todos* los usuarios. Variantes más sofisticadas permiten dropear tablas, exfiltrar contraseñas, leer archivos del servidor.

Estadística reveladora: en el *Verizon Data Breach Investigations Report 2025* [66], SQL Injection sigue siendo responsable del 14 % de las brechas de aplicaciones web, 30 años después de su descubrimiento.

11.2.2 La defensa: sentencias preparadas en las tres plataformas

La defensa universal contra SQLi es la *sentencia preparada*: el SQL y los datos viajan por canales separados, de modo que los datos no pueden alterar la estructura del SQL. Las tres plataformas del curso la implementan.

PHP con PDO

PHP dispone de PDO (*PHP Data Objects*), una capa de abstracción que soporta múltiples motores (MySQL, PostgreSQL, SQLite, SQL Server) con una API uniforme.

Ejemplo 11.1 — PDO con parámetros nominales y posicionales

Enunciado. Implementar el CRUD de la entidad **Libro** con Spring Data JPA y H2: definir entidad, repositorio y controlador REST. Se exponen las cuatro operaciones básicas.

```
1 <?php
2 declare(strict_types=1);
3
4 class UsuarioRepository // declara clase PHP
5 {
6     public function __construct(private PDO $pdo) {} // metodo publico
7
8     /** Búsqueda con parametros NOMINALES (recomendado). */
9     public function buscarPorEmail(string $email): ?array { // metodo
10         $sql = 'SELECT id, nombre, email, rol, activo
```



```

11         FROM usuarios
12         WHERE email = :em AND activo = 1';
13     $stmt = $this->pdo->prepare($sql); // prepara consulta SQL
14         (defensa anti-SQLi)
15     $stmt->execute([':em' => $email]); // ejecuta la consulta con
16         parametros enlazados
17     $u = $stmt->fetch(PDO::FETCH_ASSOC); // recupera la siguiente
18         fila como array
19     return $u ?: null; // retorna valor
20 }
21
22 /** Busqueda con parametros POSICIONALES. */
23 public function buscarPorId(int $id): ?array { // metodo publico
24     $stmt = $this->pdo->prepare( // prepara consulta SQL (defensa
25         anti-SQLi)
26         'SELECT id, nombre, email FROM usuarios WHERE id = ?');
27     $stmt->execute([$id]); // ejecuta la consulta con parametros
28         enlazados
29     return $stmt->fetch(PDO::FETCH_ASSOC) ?: null; // recupera la
30         siguiente fila como array
31 }
32
33 /** Busqueda con LIKE seguro - nunca concatenar el % */
34 // metodo publico
35 // metodo publico
36 public function buscarPorNombre(string $patron): array {
37     $stmt = $this->pdo->prepare( // prepara consulta SQL (defensa
38         anti-SQLi)
39         'SELECT id, nombre, email FROM usuarios
40         WHERE nombre LIKE :p
41         ORDER BY nombre LIMIT 50');
42     // El % se anade aqui, NO concatenado en el SQL
43     $stmt->execute([':p' => '%' . $patron . '%']); // ejecuta la
44         consulta con parametros enlazados
45     return $stmt->fetchAll(PDO::FETCH_ASSOC); // recupera todas
46         las filas restantes
47 }
48
49 /** INSERT con retorno del ID generado. */
50 public function crear(string $email, string $nombre, // metodo
51     publico
52     string $passwordPlano): int {
53     // hashea contrasena con BCrypt
54     // hashea contrasena con BCrypt
55     $hash = password_hash($passwordPlano, PASSWORD_BCRYPT,
56         ['cost' => 12]);
57     $stmt = $this->pdo->prepare( // prepara consulta SQL (defensa
58         anti-SQLi)
59         // hashea contrasena con BCrypt
60         // hashea contrasena con BCrypt
61         'INSERT INTO usuarios (email, nombre, password_hash, rol)

```

```

51         VALUES (:em, :nom, :hash, :rol)');
52     $stmt->execute([ // ejecuta la consulta con parametros
53         ':em'      => $email,
54         ':nom'     => $nombre,
55         ':hash'    => $hash,
56         ':rol'     => 'ROLE_USER',
57     ]);
58     return (int)$this->pdo->lastInsertId(); // retorna valor
59 }
60
61 /** Transaccion explicita para operaciones multi-tabla. */
62 // metodo publico
63 // metodo publico
64 public function transferirPuntos(int $de, int $a, int $monto):
65     bool {
66     try {
67         $this->pdo->beginTransaction();
68         $stmt1 = $this->pdo->prepare( // prepara consulta SQL
69             (defensa anti-SQLi)
70             'UPDATE saldos SET puntos = puntos - ? WHERE user_id =
71             ?');
72         $stmt1->execute([$monto, $de]); // ejecuta la consulta con
73             parametros enlazados
74         $stmt2 = $this->pdo->prepare( // prepara consulta SQL
75             (defensa anti-SQLi)
76             'UPDATE saldos SET puntos = puntos + ? WHERE user_id =
77             ?');
78         $stmt2->execute([$monto, $a]); // ejecuta la consulta con
79             parametros enlazados
80         $this->pdo->commit();
81         return true; // retorna valor
82     } catch (\Throwable $e) {
83         $this->pdo->rollBack();
84         error_log("Error transferir: " . $e->getMessage());
85         return false; // retorna valor
86     }
87 }

```

Listado 11.3: UsuarioRepository.php

Configuración inicial obligatoria del PDO (Listado 11.4):

```

$pdo = new PDO( // conecta a la base de datos via PDO
    'mysql:host=localhost;dbname=appweb;charset=utf8mb4',
    'usuario', 'password',
    [
        // Errores como excepciones (NO codigos de error silenciosos)
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        // FETCH_ASSOC por defecto (arrays con keys, sin duplicar)
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]
);

```

```

        // Prepared statements REALES (no emulados, mas seguros)
        PDO::ATTR_EMULATE_PREPARES    => false,
    ];
};

```

Listado 11.4: Ejemplo de PHP

Java con JDBC y try-with-resources

Java accede a bases de datos a través de JDBC. Desde Java 7, la sintaxis *try-with-resources* garantiza que las conexiones, Statements y ResultSets se cierren automáticamente, incluso si hay excepciones.

Ejemplo 11.2 — JDBC moderno con try-with-resources

Enunciado. Mostrar el patrón profesional de acceso a datos en Java con JDBC moderno: *try-with-resources* que gestiona automáticamente el cierre de Connection, Statement y ResultSet.

```

1 package ec.edu.uteq.appweb.repository; // paquete del archivo
2
3 import java.sql.*; // importacion de clase externa
4 import java.util.*; // importacion de clase externa
5 import javax.sql.DataSource; // importacion de clase externa
6
7 public class UsuarioJdbcRepository { // clase publica
8
9     private final DataSource ds; // immutable
10
11     // metodo publico
12     // metodo publico
13     public UsuarioJdbcRepository(DataSource ds) { this.ds = ds; }
14
15     public Optional<Usuario> buscarPorEmail(String email) { // metodo
16         // publico
17         String sql = ""
18             SELECT id, nombre, email, rol
19             FROM usuarios
20             WHERE email = ? AND activo = TRUE
21             "";
22         try (Connection cn = ds.getConnection();
23             PreparedStatement ps = cn.prepareStatement(sql)) {
24             ps.setString(1, email);
25             try (ResultSet rs = ps.executeQuery()) {
26                 if (rs.next()) { // condicional
27                     return Optional.of(new Usuario( // retorna
28                         rs.getLong("id"),
29                         rs.getString("nombre"),
30                         rs.getString("email"),
31                         rs.getString("rol")
32                     ));
33                 }
34                 return Optional.empty(); // retorna
35             }
36         }
37     }
38 }

```

```

35     } catch (SQLException e) {
36         // lanza excepcion explicita
37         // lanza excepcion explicita
38         throw new RepositoryException("Error buscando por email",
39             e);
40     }
41 }
42
43 public List<Usuario> buscarPorPatron(String patron) { // metodo
44     publico
45     String sql = ""
46         SELECT id, nombre, email, rol
47         FROM usuarios
48         WHERE nombre LIKE ?
49         ORDER BY nombre LIMIT 50
50         "";
51     List<Usuario> resultado = new ArrayList<>();
52     try (Connection cn = ds.getConnection();
53         PreparedStatement ps = cn.prepareStatement(sql)) {
54         ps.setString(1, "%" + patron + "%");
55         try (ResultSet rs = ps.executeQuery()) {
56             while (rs.next()) { // bucle while
57                 resultado.add(new Usuario(
58                     rs.getLong("id"),
59                     rs.getString("nombre"),
60                     rs.getString("email"),
61                     rs.getString("rol")
62                 ));
63             }
64         } catch (SQLException e) {
65             // lanza excepcion explicita
66             // lanza excepcion explicita
67             throw new RepositoryException("Error buscando por patron",
68                 e);
69         }
70     }
71     return resultado; // retorna
72 }
73
74 // metodo publico
75 // metodo publico
76 public long crear(String email, String nombre, String hash) {
77     String sql = ""
78         INSERT INTO usuarios (email, nombre, password_hash, rol)
79         VALUES (?, ?, ?, 'ROLE_USER')
80         "";
81     try (Connection cn = ds.getConnection();
82         PreparedStatement ps = cn.prepareStatement(sql,
83             Statement.RETURN_GENERATED_KEYS))
84     {
85         ps.setString(1, email);

```

```

82         ps.setString(2, nombre);
83         ps.setString(3, hash);
84         ps.executeUpdate();
85         try (ResultSet keys = ps.getGeneratedKeys()) {
86             if (keys.next()) return keys.getLong(1); // condicional
87             // lanza excepcion explicita
88             // lanza excepcion explicita
89             throw new RepositoryException("No se obtuvo ID
90                                     generado");
91         }
92     } catch (SQLException e) {
93         // lanza excepcion explicita
94         // lanza excepcion explicita
95         throw new RepositoryException("Error creando usuario", e);
96     }
97 }

```

Listado 11.5: UsuarioJdbcRepository.java

C# con ADO.NET

C# accede a bases de datos a través de ADO.NET, la biblioteca clásica de .NET. Aunque Entity Framework Core es lo preferido en proyectos nuevos, ADO.NET sigue siendo la base sobre la que se construye y aparece en código heredado.

Ejemplo 11.3 — ADO.NET puro con SqlConnection

Enunciado. Mostrar el acceso a datos en C# con ADO.NET puro (SqlConnection), antes de pasar a Entity Framework Core. Útil para entender lo que el ORM oculta.

```

1  using Microsoft.Data.SqlClient; // importa namespace
2  using AppwebApi.Models; // importa namespace
3
4  public class UsuarioRepository // clase publica
5  {
6      private readonly string _connStr; // metodo o miembro privado
7
8      public UsuarioRepository(IConfiguration cfg) => // metodo o
9          _connStr = cfg.GetConnectionString("Default")!;
10
11      // metodo o miembro publico
12      // metodo o miembro publico
13      public async Task<Usuario?> BuscarPorEmailAsync(string email)
14      {
15          const string sql = @"
16              SELECT id, nombre, email, rol
17              FROM usuarios
18              WHERE email = @email AND activo = 1";
19
20          await using var cn = new SqlConnection(_connStr);
21          await cn.OpenAsync();

```

```
22
23     await using var cmd = new SqlCommand(sql, cn);
24     cmd.Parameters.AddWithValue("@email", email);
25
26     await using var rdr = await cmd.ExecuteReaderAsync();
27     if (await rdr.ReadAsync()) { // condicional
28         return new Usuario { // retorna
29             Id      = rdr.GetInt64(0),
30             Nombre  = rdr.GetString(1),
31             Email   = rdr.GetString(2),
32             Rol     = rdr.GetString(3)
33         };
34     }
35     return null; // retorna
36 }
37 }
```

Listado 11.6: UsuarioRepository.cs

11.3 El patrón Repository: encapsular el acceso a datos

El patrón Repository encapsula el acceso a datos detrás de una interfaz limpia, ocultando los detalles de SQL/JPA al resto de la aplicación. Esto mejora la testabilidad (se puede mockear) y la mantenibilidad (cambiar de SQL puro a JPA no afecta a los servicios).

Patrón Repository según Fowler

«Mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects.» [21]

Un Repository expone una API estilo colección sobre las entidades de dominio, ocultando los detalles del SGBD. Sus responsabilidades:

- Encapsular consultas (no exponer SQL al servicio).
- Devolver objetos de dominio o DTOs, no `ResultSet`.
- Centralizar la traducción objeto ↔ tabla.
- Facilitar el testing con repositorios fake o mocks.

11.3.1 Repositorio típico en Spring Data JPA

Spring Data JPA reduce el boilerplate al mínimo: basta con declarar una interfaz que extiende `JpaRepository`; Spring genera la implementación en tiempo de ejecución.

Ejemplo 11.4 — ProductoRepository de Spring Data

Enunciado. Implementar un repositorio Spring Data JPA para la entidad `Producto` declarando solo la interfaz; Spring genera la implementación con todas las operaciones CRUD.

```
1 package ec.edu.uteq.appweb.repository; // paquete del archivo
2
3 import ec.edu.uteq.appweb.model.Producto; // importacion de clase
   externa
4 import org.springframework.data.domain.Page; // importacion de clase
   externa
```

```
5 import org.springframework.data.domain.Pageable; // importacion de
   clase externa
6 import org.springframework.data.jpa.repository.*; // importacion de
   clase externa
7 import org.springframework.data.repository.query.Param; // importacion
   de clase externa
8 import java.math.BigDecimal; // importacion de clase externa
9 import java.util.List; // importacion de clase externa
10
11 public interface ProductoRepository // declaracion de interfaz
12     extends JpaRepository<Producto, Long> {
13
14     // Consulta DERIVADA del nombre del metodo (sin SQL)
15     List<Producto> findByStockGreaterThanOrderByNombreAsc(int min);
16
17     Page<Producto> findByCategoriaIgnoreCase(String cat, Pageable pag);
18
19     // JPQL personalizado con parametros nominales
20     @Query(""" // JPQL
21         SELECT p FROM Producto p
22         WHERE p.precio BETWEEN :min AND :max
23             AND p.stock > 0
24         ORDER BY p.precio ASC
25         """)
26     List<Producto> rangoPrecio(@Param("min") BigDecimal min,
27                               @Param("max") BigDecimal max);
28
29     // Update masivo
30     @Modifying // consulta que modifica datos (no SELECT)
31     // JPQL
32     // JPQL
33     @Query("UPDATE Producto p SET p.stock = p.stock - :n WHERE p.id =
34         :id AND p.stock >= :n")
35     int decrementarStock(@Param("id") Long id, @Param("n") int n);
36
37     // Native SQL cuando JPQL no alcanza
38     @Query(value = "" // JPQL
39         SELECT p.*, c.nombre AS cat_nombre
40         FROM productos p
41         JOIN categorias c ON c.id = p.categoria_id
42         WHERE p.stock > 0
43         "", nativeQuery = true)
44     List<Object[]> reporteStock();
45 }
```

Listado 11.7: ProductoRepository.java

11.4 El problema N+1 y su solución

El problema N+1 es la patología de rendimiento más común en aplicaciones que usan ORM. Ocurre cuando, al iterar una colección de N entidades con relaciones, se ejecutan N consultas adicionales en lugar de una sola con JOIN.

Problema N+1 — el bug silencioso que mata el rendimiento

Si tiene una entidad `Producto` con relación `@ManyToOne Categoria categoria`, y carga una lista de 100 productos para mostrarlos con su categoría, ocurre lo siguiente sin optimización:

1. Una consulta para obtener los 100 productos.
2. Cien consultas adicionales (una por cada producto) para obtener su categoría asociada.

Total: 101 consultas. Si la página muestra 1000 productos, son 1001 consultas. La latencia se dispara, la BD colapsa.

La solución universal: *eager loading* explícito, que hace UN SOLO JOIN en lugar de N+1 consultas.

11.4.1 Solución en cada ORM

Spring Data JPA — usar JOIN FETCH (Listado 11.8):

```
@Query(""" // JPQL
    SELECT p FROM Producto p
    JOIN FETCH p.categoria
    WHERE p.stock > 0
    """)
List<Producto> findAllConCategoria();
```

Listado 11.8: Ejemplo de Java

Eloquent (Laravel) — usar with (Listado 11.9):

```
$productos = Producto::with('categoria')
->where('stock', '>', 0)
->get();
```

Listado 11.9: Ejemplo de PHP

Entity Framework Core — usar Include (Listado 11.10):

```
var productos = await _ctx.Productos
    .Include(p => p.Categoria)
    .Where(p => p.Stock > 0)
    .ToListAsync();
```

Listado 11.10: Ejemplo de C#

Doctrine ORM — usar fetch en DQL (Listado 11.11):

```
$query = $em->createQuery(
    'SELECT p, c FROM App\Entity\Producto p
    JOIN p.categoria c WHERE p.stock > 0');
$productos = $query->getResult();
```

Listado 11.11: Ejemplo de PHP

11.5 Migraciones de esquema con Flyway y Liquibase

Las migraciones permiten versionar el esquema de la BD como código. Cada migración es un script SQL numerado que se aplica una sola vez. La BD «sabe» qué migraciones ha aplicado mediante una tabla de control (flyway_schema_history o databasechangelog).

Ejemplo 11.5 — Estructura Flyway en Spring Boot

Enunciado. Configurar Flyway en un proyecto Spring Boot para gestionar migraciones de base de datos versionadas y reproducibles entre entornos.

```
src/main/resources/db/migration/
V1__crear_usuarios.sql
V2__crear_productos.sql
V3__agregar_indice_email_usuarios.sql
V4__crear_pedidos.sql
```

Listado 11.12: Ejemplo de Shell

```
1  -- crea tabla nueva en la BD
2  CREATE TABLE usuarios (                                -- crea
   tabla nueva en la BD
3  id                BIGSERIAL PRIMARY KEY,
4  -- columna obligatoria (no admite NULL)
5  email             VARCHAR(180) NOT NULL UNIQUE,         -- columna
   obligatoria (no admite NULL)
6  -- columna obligatoria (no admite NULL)
7  nombre            VARCHAR(100) NOT NULL,                 -- columna
   obligatoria (no admite NULL)
8  -- columna obligatoria (no admite NULL)
9  password_hash     VARCHAR(255) NOT NULL,                 -- columna
   obligatoria (no admite NULL)
10 -- columna obligatoria (no admite NULL)
11 -- columna obligatoria (no admite NULL)
12 rol                VARCHAR(30) NOT NULL DEFAULT 'ROLE_USER',
13 -- columna obligatoria (no admite NULL)
14 activo             BOOLEAN NOT NULL DEFAULT TRUE,        -- columna
   obligatoria (no admite NULL)
15 -- columna obligatoria (no admite NULL)
16 creado_en          TIMESTAMPTZ NOT NULL DEFAULT NOW(),   -- columna
   obligatoria (no admite NULL)
17 actualizado_en     TIMESTAMPTZ
18 );
19
20 -- crea indice para acelerar busquedas
21 CREATE INDEX idx_usuarios_email_activo                   -- crea
   indice para acelerar busquedas
22 ON usuarios(email) WHERE activo = TRUE;
```

Listado 11.13: V1__crear_usuarios.sql

Configuración mínima en application.yml:

```
spring: # configuracion de Spring
  datasource: # configuracion de origen de datos (BD)
```

```

url:      jdbc:postgresql://localhost:5432/appweb
username: appweb # usuario de la BD
password: ${DB_PASSWORD} # contraseña (mejor leer de env var)
jpa: # configuracion JPA/Hibernate
hibernate: # configuracion especifica de Hibernate
  ddl-auto: validate # NO usar 'update' con Flyway
properties:
  hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect
flyway:
  enabled: true
  baseline-on-migrate: true

```

Listado 11.14: Ejemplo de YAML

11.6 Procedimientos almacenados: cuándo aún tiene sentido

Los procedimientos almacenados (*stored procedures*) fueron durante décadas la forma canónica de encapsular lógica de datos en el motor de BD. Hoy se usan menos, pero hay casos donde aún tienen sentido y conviene conocerlos.

Procedimientos almacenados vs lógica en aplicación

Aspecto	Stored Procedures	Lógica en aplicación
Rendimiento	Excelente (compilados, cerca del dato)	Bueno con cache
Versionado con Git	Difícil (suele estar fuera del repo)	Natural
Testing	Difícil (requiere BD viva)	Fácil con mocks
Portabilidad SGBD	Acoplado a un motor	Multi-SGBD via ORM
Lógica compleja	Mantenimiento difícil	Más limpia
Operaciones masivas	Imbatible (sin round-trips)	Ineficiente

Cuándo usar stored procedures en 2026:

- Operaciones masivas que tocan miles de filas (ETL, batch).
- Lógica que requiere consistencia transaccional fina con múltiples tablas.
- Reportes complejos con muchos JOINS.
- Sistemas heredados donde la lógica ya está en SP.

11.7 Práctica integrada: CRUD ORM vs SQL puro

La práctica integrada de la semana confronta directamente las dos estrategias dominantes: JDBC puro vs Spring Data JPA. El estudiante implementa el mismo CRUD con ambas y produce una tabla comparativa.

Ejercicio 11.1 — Comparación experimental SQL puro vs ORM (Sumativa GA-PE Unidad III)

Objetivo: implementar el mismo CRUD en dos versiones (SQL puro y ORM) y comparar líneas de código, rendimiento y mantenibilidad.

IDE: IntelliJ IDEA Ultimate

Plugins necesarios (todos incluidos en Ultimate):

- *Database Tools and SQL* (cliente BD nativo).
- *Spring Boot*.
- *Lombok*.

Paso 1 — Preparar PostgreSQL en Docker

Más simple que instalar PostgreSQL nativo (Listado 11.15):

```
docker run -d --name pg-appweb \ # arranca un contenedor
-e POSTGRES_USER=appweb \
-e POSTGRES_PASSWORD=appweb \
-e POSTGRES_DB=appweb \
-p 5432:5432 postgres:16
```

Listado 11.15: Ejemplo de Shell

Verificar (Listado 11.16):

```
# ejecuta comando en contenedor activo
# ejecuta comando en contenedor activo
docker exec -it pg-appweb psql -U appweb -d appweb -c "SELECT
version();" "
```

Listado 11.16: Ejemplo de Shell

Paso 2 — Conectar IntelliJ a la BD

Una vez levantada PostgreSQL, el siguiente paso es configurar IntelliJ IDEA Ultimate para conectarse a ella desde la propia IDE. Esto permite ejecutar SQL ad-hoc, ver datos y probar consultas sin salir del entorno.

1. Abrir el panel *Database* (lateral derecho).
2. Pulsar + → *Data Source* → *PostgreSQL*.
3. Host: localhost, Port: 5432, Database: appweb, User: appweb, Password: appweb.
4. Pulsar *Test Connection*. Si pide descargar el driver, aceptar.
5. Pulsar OK.

Paso 3 — Crear tabla y datos de prueba

Click derecho sobre la conexión → *New* → *Query Console*. Ejecutar (Listado 11.17):

```
-- crea tabla nueva en la BD
CREATE TABLE productos (                                     -- crea
    tabla nueva en la BD
    id          BIGSERIAL PRIMARY KEY,
    -- columna obligatoria (no admite NULL)
    nombre      VARCHAR(120) NOT NULL,                        -- columna
    obligatoria (no admite NULL)
    -- columna obligatoria (no admite NULL)
    precio      NUMERIC(10,2) NOT NULL CHECK (precio >= 0),   -- columna
    obligatoria (no admite NULL)
    -- columna obligatoria (no admite NULL)
    -- columna obligatoria (no admite NULL)
```

```

stock      INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
-- columna obligatoria (no admite NULL)
creado     TIMESTAMPTZ NOT NULL DEFAULT NOW()           -- columna
              obligatoria (no admite NULL)
);

-- Cargar 1000 productos para benchmark
-- inserta una fila nueva
INSERT INTO productos (nombre, precio, stock)           -- inserta
              una fila nueva
SELECT
    'Producto ' || i,
    ROUND((random() * 100 + 1)::numeric, 2),
    (random() * 100)::int
-- tabla de origen
FROM generate_series(1, 1000) i;                       -- tabla
              de origen

```

Listado 11.17: Ejemplo de SQL

Verificar: `SELECT COUNT(*) FROM productos;` debe devolver 1000.

Paso 4 — Crear proyecto Spring Boot

En IntelliJ: File → New → Project → Spring Boot. Configurar:

- Language: Java; Build: Maven; JDK: 21.
- Group: `ec.edu.uteq`; Artifact: `benchmark-orm`.
- Spring Boot 4.x.
- Dependencies: *Spring Web*, *Spring Data JPA*, *PostgreSQL Driver*, *Lombok*, *Spring Boot DevTools*.

Paso 5 — Configurar la conexión

Editar `src/main/resources/application.yml` (Listado 11.18):

```

spring: # configuracion de Spring
  datasource: # configuracion de origen de datos (BD)
    url:      jdbc:postgresql://localhost:5432/appweb
    username: appweb # usuario de la BD
    password: appweb # contrasena (mejor leer de env var)
  jpa: # configuracion JPA/Hibernate
    hibernate.ddl-auto: none # tabla ya existe
    show-sql: true # ver SQL generado
    properties.hibernate.format_sql: true

```

Listado 11.18: Ejemplo de YAML

Paso 6 — Versión A: JDBC puro

La versión A implementa el CRUD usando solo JDBC puro: el estudiante escribe el SQL a mano, gestiona la conexión, prepara las sentencias y procesa los resultados. Es laborioso pero ilustrativo. Véase el Listado 11.19.

```

1 package ec.edu.uteq.benchmark.jdbc; // paquete del archivo
2

```

```

3 import org.springframework.jdbc.core.JdbcTemplate; // importacion de
  clase externa
4 import org.springframework.web.bind.annotation.*; // importacion de
  clase externa
5 import java.sql.Timestamp; // importacion de clase externa
6 import java.util.*; // importacion de clase externa
7
8 @RestController // expone endpoints REST con JSON
9 @RequestMapping("/api/jdbc/productos") // ruta base del controlador
10 public class JdbcController { // clase publica
11
12     private final JdbcTemplate jdbc; // immutable
13
14     // metodo publico
15     // metodo publico
16     public JdbcController(JdbcTemplate jdbc) { this.jdbc = jdbc; }
17
18     @GetMapping // endpoint HTTP GET
19     public List<Map<String, Object>> listar() { // metodo publico
20         return jdbc.queryForList( // retorna
21             "SELECT id, nombre, precio, stock, creado FROM productos "
22             + "ORDER BY id");
23     }
24
25     @GetMapping("/{id}") // endpoint HTTP GET
26     // metodo publico
27     // metodo publico
28     public Map<String, Object> obtener(@PathVariable long id) {
29         return jdbc.queryForMap( // retorna
30             "SELECT id, nombre, precio, stock, creado FROM productos "
31             + "WHERE id = ?", id);
32     }
33 }

```

Listado 11.19: JdbcController.java

Paso 7 — Versión B: Spring Data JPA

La versión B implementa el mismo CRUD usando Spring Data JPA: el estudiante define la entidad, declara la interfaz del repositorio, y JPA genera el SQL automáticamente. El contraste con la versión A es revelador. Véase el Listado 11.20.

```

1 package ec.edu.uteq.benchmark.jpa; // paquete del archivo
2
3 import jakarta.persistence.*; // importacion de clase externa
4 import lombok.*; // importacion de clase externa
5 import java.math.BigDecimal; // importacion de clase externa
6 import java.time.OffsetDateTime; // importacion de clase externa
7
8 @Entity @Table(name = "productos") // entidad JPA (mapea a tabla)
9 @Getter @Setter @NoArgsConstructor @AllArgsConstructor
10 public class Producto { // clase publica

```

```

11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) // campo
        clave primaria
12     private Long id; // campo privado
13     private String nombre; // campo privado
14     private BigDecimal precio; // campo privado
15     private Integer stock; // campo privado
16     private OffsetDateTime creado; // campo privado
17 }

```

Listado 11.20: Producto.java

```

1 package ec.edu.uteq.benchmark.jpa; // paquete del archivo
2
3 // importacion de clase externa
4 // importacion de clase externa
5 import org.springframework.data.jpa.repository.JpaRepository;
6 public interface ProductoRepository // declaracion de interfaz
7     extends JpaRepository<Producto, Long> {}
8
9 // ---
10
11 import org.springframework.web.bind.annotation.*; // importacion de
        clase externa
12 import java.util.*; // importacion de clase externa
13
14 @RestController // expone endpoints REST con JSON
15 @RequestMapping("/api/jpa/productos") // ruta base del controlador
16 public class JpaController { // clase publica
17     private final ProductoRepository repo; // immutable
18     // metodo publico
19     // metodo publico
20     public JpaController(ProductoRepository r) { this.repo = r; }
21
22     // endpoint HTTP GET
23     // endpoint HTTP GET
24     @GetMapping public List<Producto> listar() { return
        repo.findAll(); }
25
26     @GetMapping("/{id}") // endpoint HTTP GET
27     public Producto obtener(@PathVariable Long id) { // metodo publico
28         return repo.findById(id).orElseThrow(); // retorna
29     }
30 }

```

Listado 11.21: ProductoRepository.java + JpaController.java

Paso 8 — Benchmark con Apache Bench

Ejecutar la app: Shift+F10 sobre la clase principal. Servidor en <http://localhost:8080>.

Desde otra terminal, instalar Apache Bench (ab). En Ubuntu: `sudo apt install apache2-utils`; en macOS: viene preinstalado.

Ejecutar (Listado 11.22):

```
# 1000 peticiones, 10 concurrentes, version JDBC
ab -n 1000 -c 10 http://localhost:8080/api/jdbc/productos > jdbc.txt

# Misma prueba para JPA
ab -n 1000 -c 10 http://localhost:8080/api/jpa/productos > jpa.txt

cat jdbc.txt | grep "Requests per second"
cat jpa.txt | grep "Requests per second"
```

Listado 11.22: Ejemplo de Shell

Paso 9 — Tabla comparativa para entregar

Tras implementar las dos versiones, el estudiante debe producir una tabla comparativa que cuantifique las diferencias.

Métrica	Versión JDBC	Versión JPA
Líneas de código del controller		
Líneas de código del repository		
Requests/segundo (1000 req, 10 conc)		
Tiempo medio por request (ms)		
SQL generado (consultas en log)	1 fija	varía con caché L1/L2
Vulnerabilidad SQL Injection	Eliminada (?)	Eliminada (JPA)
Mantenibilidad (1-5)		
Productividad para CRUD nuevo (1-5)		

Reflexión final escrita (entregable)

Responder en máximo 300 palabras:

1. ¿Cuál fue más rápido en el benchmark?
2. ¿Qué versión escribiría más rápido un desarrollador nuevo?
3. ¿En qué casos el ORM es perjudicial (ineficiente)?
4. ¿Por qué muchas empresas mantienen *ambos* estilos en el mismo proyecto?

Esta es la *Sumativa GA-PE Unidad III* de la semana 11.

11.8 Buenas prácticas profesionales en acceso a datos

La caja siguiente compila las reglas profesionales del acceso a datos: principios que separan el código mantenible del código que se vuelve ingobernable a los seis meses.

Quince reglas profesionales 2026

1. NUNCA concatenar datos del usuario en SQL: usar siempre parámetros.
2. Configurar el driver con `ATTR_EMULATE_PREPARES = false` en PHP.
3. DTOs separados de entidades: nunca exponer entidades JPA en controllers.
4. Consultas paginadas siempre (`LIMIT + OFFSET`) para listados.
5. Resolver el problema N+1 con *eager loading* explícito.
6. Migraciones con Flyway o Liquibase: jamás `ddl-auto=update` en producción.
7. Índices en columnas de búsqueda frecuente y en foreign keys.
8. Conexiones desde un *pool* (HikariCP en Spring, pgBouncer externo).
9. Transacciones explícitas para operaciones multi-tabla.
10. Soft delete (`activo=false`) sobre delete físico cuando hay relaciones.
11. Auditar cambios sensibles (`creado_en`, `actualizado_en`, `actualizado_por`).

12. Cuenta BD con permisos mínimos: la app no necesita `DROP TABLE`.
13. Encriptar la conexión con SSL/TLS en producción.
14. Tests con BD en memoria (H2) o testcontainers para integración real.
15. Documentar el modelo entidad-relación en el repositorio (ER en draw.io).

11.9 Ejercicio resuelto adicional con resultado visual

Como cierre de la semana 11, el siguiente ejercicio adicional está completamente resuelto y se complementa con un propuesto.

Ejercicio resuelto 11.1 — CRUD de libros con Spring Data JPA y H2

Enunciado. Implementar un CRUD completo de la entidad `Libro` (id, titulo, autor, anio) con Spring Data JPA y H2 embebido, exponer los endpoints REST y verificar visualmente las operaciones desde Postman.

Entidad (`Libro.java`):

Listado 11.23: Entidad JPA con validaciones

```
@Entity // entidad JPA (mapea a tabla)
@Table(name = "libros") // nombre de la tabla en BD
public class Libro { // clase publica
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) // campo clave primaria
    private Long id; // campo privado
    @NotBlank @Size(max = 200) // validacion: no nulo y no vacio
    private String titulo; // campo privado
    @NotBlank @Size(max = 100) // validacion: no nulo y no vacio
    private String autor; // campo privado
    @Min(1900) @Max(2100) // validacion: valor minimo
    private int anio; // campo privado
    // getters/setters omitidos por brevedad
}
```

Repositorio (`LibroRepository.java`):

Listado 11.24: Repositorio Spring Data

```
// declaracion de interfaz
// declaracion de interfaz
public interface LibroRepository extends JpaRepository<Libro, Long> {
    List<Libro> findByAutor(String autor);
    @Query("SELECT l FROM Libro l WHERE l.anio >= :desde") // JPQL
    List<Libro> publicadosDesde(@Param("desde") int desde);
}
```

Controlador REST (`LibroController.java`):

Listado 11.25: Endpoints REST

```
@RestController @RequestMapping("/api/v1/libros") // expone endpoints REST con JSON
public class LibroController { // clase publica
    private final LibroRepository repo; // inmutable
    // metodo publico
    // metodo publico
    public LibroController(LibroRepository r) { this.repo = r; }

    // endpoint HTTP GET
    // endpoint HTTP GET
    @GetMapping public List<Libro> listar() { return repo.findAll(); }
    // endpoint HTTP GET
    // endpoint HTTP GET
    @GetMapping("/{id}") public ResponseEntity<Libro> obtener(@PathVariable Long id) {
```



```

    return repo.findById(id).map(ResponseEntity::ok) // retorna
        .orElse(ResponseEntity.notFound().build());
}
// endpoint HTTP POST (crear)
// endpoint HTTP POST (crear)
@PostMapping public ResponseEntity<Libro> crear(@Valid @RequestBody Libro l) {
    Libro guardado = repo.save(l);
    return ResponseEntity.status(201).body(guardado); // retorna
}
// endpoint HTTP PUT (actualizar completo)
// endpoint HTTP PUT (actualizar completo)
@PutMapping("/{id}") public ResponseEntity<Libro> actualizar(
    @PathVariable Long id, @Valid @RequestBody Libro l) { // extrae valor de la URL
    return repo.findById(id).map(existente -> { // retorna
        existente.setTitulo(l.getTitulo());
        existente.setAutor(l.getAutor());
        existente.setAnio(l.getAnio());
        return ResponseEntity.ok(repo.save(existente)); // retorna
    }).orElse(ResponseEntity.notFound().build());
}
// endpoint HTTP DELETE (eliminar)
// endpoint HTTP DELETE (eliminar)
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    return repo.findById(id).map(l -> { // retorna
        repo.delete(l);
        return ResponseEntity.noContent().<Void>build(); // retorna
    }).orElse(ResponseEntity.notFound().build());
}
}
}

```

Resultado visual de GET /api/v1/libros en Postman tras insertar tres libros:

```

GET http://localhost:8080/api/v1/libros    200 OK

[
  { "id": 1, "titulo": "Clean Code", "autor": "Robert C. Martin", "anio": 2008 },
  { "id": 2, "titulo": "Designing Data-Intensive Applications",
    "autor": "Martin Kleppmann", "anio": 2017 },
  { "id": 3, "titulo": "Software Architecture in Practice",
    "autor": "Len Bass", "anio": 2021 }
]

```

Vista de la tabla H2 (consola H2 en <http://localhost:8080/h2-console>):

ID	TITULO	AUTOR	ANIO
1	Clean Code	Robert C. Martin	2008
2	Designing Data-Intensive Applications	Martin Kleppmann	2017
3	Software Architecture in Practice	Len Bass	2021

Discusión. JPA generó automáticamente el SQL apropiado (INSERT, SELECT, UPDATE, DELETE) y vinculó los parámetros por nombre, eliminando por completo la posibilidad de SQL Injection. Las validaciones @NotBlank, @Size y @Min/@Max devuelven HTTP 400 con detalle si el cliente envía datos inválidos.

Ejercicio propuesto 11.2 — Catálogo de cursos con relaciones y paginación

Enunciado. Implementar un catálogo de cursos universitarios con relación «un curso tiene muchos estudiantes» y «un estudiante puede estar en muchos cursos» (**relación muchos-a-muchos**). Resolver el problema N+1 mediante JOIN FETCH y exponer endpoints REST con paginación, búsqueda y ordenamiento.

Entidades:

- `Curso(id, codigo, nombre, creditos)`.
- `Estudiante(id, cedula, nombre, email)`.
- Tabla intermedia `inscripciones(curso_id, estudiante_id, fecha)`.

Requisitos técnicos:

1. Usar Spring Boot 4 + PostgreSQL 16 (no H2).
2. Mapear la relación muchos-a-muchos con `@ManyToMany`.
3. Endpoint `GET /api/v1/cursos?page=0&size=10&sort=nombre,asc` debe usar `Pageable`.
4. Endpoint `GET /api/v1/cursos/{id}/estudiantes` debe usar `JOIN FETCH` para evitar N+1.
5. Mostrar evidencia con Hibernate logs (`spring.jpa.show-sql=true`) de que se ejecuta UNA consulta en lugar de N+1.
6. Documentación OpenAPI con springdoc, accesible en `/swagger-ui.html`.
7. Migraciones de BD con Flyway o Liquibase.
8. Cobertura JaCoCo $\geq 70\%$ con tests de integración usando testcontainers.

Criterios de evaluación:

- No hay SQL concatenado en ninguna parte del código.
- El log muestra una sola consulta para listar cursos con sus estudiantes (no N+1).
- Validaciones `@Valid` correctas en todos los endpoints.
- Documentación Swagger funcional.

Lecturas recomendadas: [50] para optimización de queries; [50] para análisis del problema N+1 en producción.

Patrones de arquitectura para aplicaciones web

«La arquitectura es el conjunto de decisiones que son difíciles de cambiar. Un buen arquitecto, por tanto, posterga las decisiones innecesarias y razona profundamente las inevitables.»

— Adaptado de Robert C. Martin

Resultado de aprendizaje de la semana

Al concluir la semana 12, el estudiante diseña aplicaciones web bajo patrones arquitectónicos escalables (multicapa, hexagonal, SOA, microservicios, EDA, P2P), documenta sus decisiones con el modelo C4 [9] y los Architectural Decision Records (ADR), y evalúa los *trade-offs* entre atributos de calidad ISO/IEC 25010 [29].

12.1 Principios fundacionales de arquitectura de software

Antes de hablar de patrones concretos (capas, hexagonal, microservicios), conviene fijar los principios fundacionales que los gobiernan todos: separación de intereses, alta cohesión, bajo acoplamiento, dependencias unidireccionales.

¿Qué es la arquitectura de software?

Según Bass, Clements y Kazman [3], la arquitectura de un sistema computacional es *el conjunto de estructuras necesarias para razonar sobre el sistema, comprende los elementos del software, las relaciones entre ellos y las propiedades de ambos*.

Tres ideas clave en esta definición:

- **Estructuras múltiples:** no hay UNA arquitectura, sino varias vistas (estática, dinámica, despliegue).
- **Razonamiento:** la arquitectura existe para que un equipo pueda pensar el sistema sin perderse en detalles.
- **Decisiones difíciles de cambiar:** lo arquitectónico es lo que costaría reescribir si se cambiara.

12.1.1 Los principios SOLID aplicados a la arquitectura

Los principios SOLID, formulados originalmente por Robert C. Martin para diseño orientado a objetos, se generalizan a la arquitectura. La caja siguiente los traduce al contexto arquitectónico.

Tabla 12.1: SOLID, sus autores y significado en arquitectura.

Principio	Significado	Aplicación arquitectónica
S — Single Responsibility O — Open/Closed	Una clase, una razón para cambiar Abierto a extensión, cerrado a modificación	Cada módulo/servicio tiene un foco Plugins, hooks, eventos
L — Liskov Substitution I — Interface Segregation D — Dependency Inversion	Subtipos sustituibles por sus tipos base Interfaces pequeñas y enfocadas Depender de abstracciones, no concreciones	Polimorfismo correcto APIs minimal por módulo Inyección de dependencias

12.1.2 Atributos de calidad ISO/IEC 25010

El estándar ISO/IEC 25010 define las características de calidad de un sistema software. Conocerlas permite razonar sobre arquitectura de forma estructurada. La tabla siguiente las cataloga.

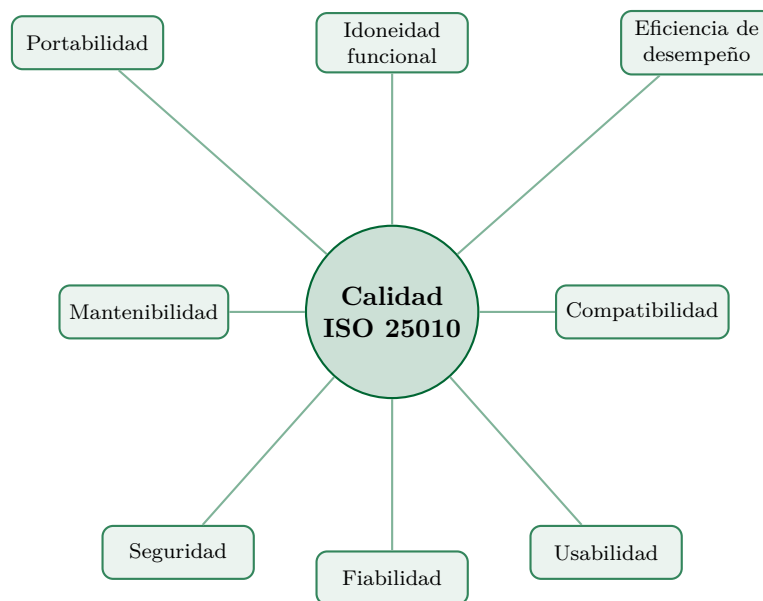


Figura 12.1: Los ocho atributos de calidad de la norma ISO/IEC 25010. Toda decisión arquitectónica supone un *trade-off* entre algunos de estos atributos.

12.2 Patrón 1: Arquitectura por capas (3-capas y 4-capas)

La arquitectura por capas es el patrón más extendido en aplicaciones empresariales: la aplicación se divide en niveles horizontales con responsabilidades específicas y dependencias hacia abajo.

12.2.1 Estructura clásica

La estructura clásica de tres capas (presentación, lógica, datos) se ha extendido a cuatro al separar la lógica de aplicación del dominio. La figura siguiente la ilustra.

Regla cardinal: una capa puede invocar a la inmediatamente inferior; nunca al revés ni saltarse capas (no permitir que un controller acceda directamente al SGBD).

12.2.2 Implementación canónica en Spring Boot

Spring Boot facilita la implementación de la arquitectura por capas mediante sus anotaciones estereotipo (`@Controller`, `@Service`, `@Repository`). El ejemplo siguiente muestra una implementación canónica.

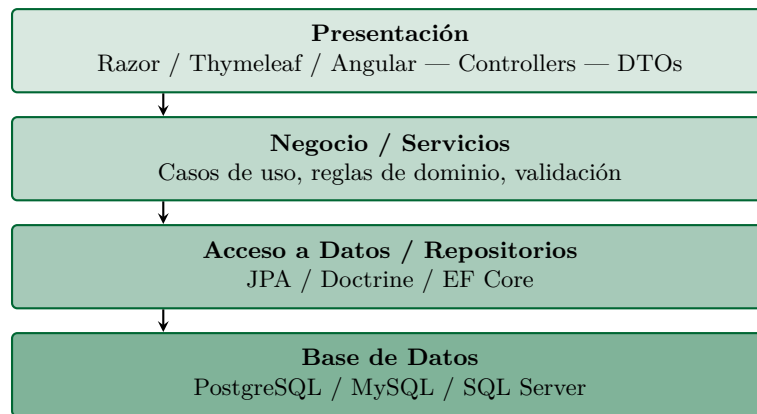


Figura 12.2: Arquitectura clásica de 4 capas para aplicaciones web. La dependencia siempre es descendente: la capa superior depende de la inferior, nunca al revés.

Ejemplo 12.1 — Estructura por capas en un proyecto Spring Boot

Enunciado. Modelar arquitectónicamente con el modelo C4 (niveles 1 y 2) una tienda en línea sencilla. Producir el diagrama de Contexto y el de Contenedores.

```
src/main/java/ec/edu/uteq/appweb/  
  controller/          <- Capa de Presentacion  
    ProductoController.java  
    ExceptionHandler.java  
  service/             <- Capa de Negocio  
    ProductoService.java  
    impl/ProductoServiceImpl.java  
  repository/          <- Capa de Acceso a Datos  
    ProductoRepository.java  
  model/               <- Entidades de dominio  
    Producto.java  
  dto/                <- Objetos de transferencia  
    ProductoDto.java  
    ProductoRequest.java  
  exception/           <- Excepciones  
    ResourceNotFoundException.java  
    ValidationException.java  
  config/              <- Configuración  
    SecurityConfig.java  
    WebConfig.java
```

Listado 12.1: Ejemplo de Shell

12.2.3 Variante: Arquitectura Hexagonal (Ports and Adapters)

La arquitectura hexagonal (*Ports and Adapters*) de Alistair Cockburn es una variante refinada de la arquitectura por capas que coloca el dominio en el centro y vuelve invertibles las dependencias.

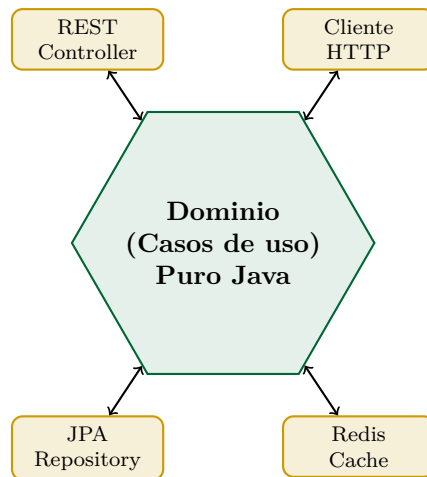


Figura 12.3: Arquitectura hexagonal: el dominio (centro) define *ports* (interfaces) y los *adapters* (implementaciones) externos lo conectan con tecnologías concretas (REST, JPA, Redis). Permite cambiar el adaptador de persistencia sin tocar el dominio.

12.3 Patrón 2: Arquitectura Orientada a Servicios (SOA)

SOA emergió en los 2000 como enfoque para integrar sistemas heterogéneos mediante servicios autónomos comunicados por SOAP/XML sobre un *Enterprise Service Bus* (ESB). Características:

- Servicios reutilizables expuestos vía contratos formales (WSDL).
- Comunicación asíncrona vía ESB (TIBCO, MuleSoft, IBM IIB).
- Gobernanza centralizada (registro de servicios, políticas unificadas).
- Datos a menudo compartidos entre servicios.

Cuándo SOA sigue siendo relevante: integraciones con sistemas heredados (banca, gobierno), entornos donde el WSDL es contractualmente exigido, dominios con altos requisitos de interoperabilidad B2B.

12.4 Patrón 3: Microservicios

Microservicios es el patrón arquitectónico que más atención ha recibido en la última década. Implica descomponer el sistema en servicios pequeños, autónomos, desplegados y escalables independientemente.

Microservicios según Newman

«Pequeños servicios autónomos que trabajan juntos, modelados alrededor de un dominio de negocio.» [44]

Características distintivas:

- Servicios pequeños con responsabilidad acotada.
- Despliegue independiente (cada servicio tiene su pipeline CI/CD).
- Comunicación vía HTTP/JSON o eventos asíncronos.
- Datos descentralizados (una BD por servicio).
- Equipos autónomos (cada equipo es dueño de uno o varios servicios).
- Heterogeneidad tecnológica permitida.

12.4.1 Comparativa SOA vs microservicios

SOA (*Service-Oriented Architecture*) y microservicios comparten la idea de modularidad por servicios, pero difieren en escala, autonomía y tecnologías. La tabla siguiente los contrasta.

SOA clásico vs microservicios modernos		
Criterio	SOA clásico	Microservicios
Tamaño del servicio	Mediano-grande	Pequeño y enfocado
Comunicación	SOAP/XML, ESB centralizado	REST/JSON, gRPC, eventos
Datos	Compartidos vía ESB	Una BD por servicio
Despliegue	Acoplado al ESB	Independiente por servicio
Equipos	Centralizados	Autónomos por servicio
Lenguajes	Generalmente Java/.NET	Heterogéneo
Coreografía/Orquestación	ESB orquesta	Coreografía con eventos
Madurez del enfoque	2000s	2015+
Complejidad operacional	Alta (ESB)	Alta (distribuida)

12.4.2 Cuándo NO usar microservicios

Microservicios no es la solución para todo problema. Hay escenarios donde adoptarlos prematuramente genera más complejidad que valor. La caja siguiente enumera los casos donde un monolito modular es la mejor opción.

El error costoso de los microservicios prematuros

Sam Newman y Martin Fowler son enfáticos: los microservicios son una **arquitectura distribuida**, y los sistemas distribuidos son *inherentemente complejos*. Los problemas que un monolito resuelve trivialmente (transacciones ACID, debugging, deployment) se vuelven retos no triviales en microservicios.

No use microservicios si:

- Su equipo es pequeño (menos de 8 personas).
- No tiene CI/CD maduro y observabilidad robusta.
- No domina contenedores y orquestación (Kubernetes).
- No ha experimentado los dolores reales de un monolito grande.

La recomendación moderna (Fowler 2024) es *«monolith first»*: empezar con un monolito modular bien diseñado, y migrar servicios a microservicios solo cuando se justifique con datos.

12.4.3 Patrones esenciales de microservicios

Los microservicios traen consigo problemas distribuidos (latencia, fallos parciales, consistencia eventual) que se mitigan con patrones específicos. La lista siguiente recoge los esenciales.

Tabla 12.2: Patrones imprescindibles en arquitectura de microservicios.	
Patrón	Propósito
API Gateway	Punto único de entrada; auth, rate-limiting, routing.
Service Discovery	Registro y descubrimiento dinámico (Eureka, Consul).
Circuit Breaker	Cortar llamadas a servicios fallidos (Resilience4j).
Saga	Transacciones distribuidas vía secuencia de eventos.
CQRS	Separar comandos (escritura) de queries (lectura).
Event Sourcing	El estado se deriva de un log de eventos inmutable.
Sidecar	Funciones transversales en contenedor adyacente.
Bulkhead	Aislamiento de recursos para evitar cascada de fallos.

12.5 Patrón 4: Arquitectura dirigida por eventos (EDA)

En EDA los componentes se comunican mediante *eventos* (mensajes) en lugar de invocaciones directas. Tres roles:

- **Producer:** publica eventos cuando algo relevante ocurre.

- **Broker:** encamina eventos (RabbitMQ, Apache Kafka, AWS SNS/SQS).
- **Consumer:** se suscribe y procesa eventos asíncronamente.

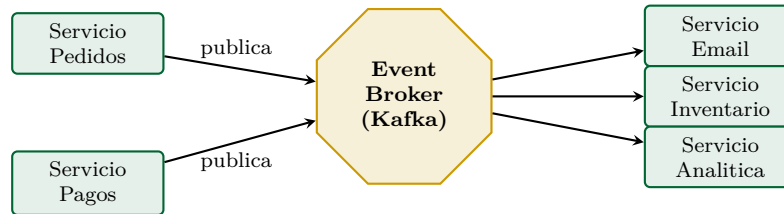


Figura 12.4: Flujo en una arquitectura dirigida por eventos: los servicios publican eventos sin saber quién los consumirá; los consumers se suscriben a los eventos que les interesan.

Ventajas: desacoplamiento temporal y espacial, escalabilidad horizontal trivial, resiliencia ante caídas parciales.

Costos: complejidad operacional, debugging distribuido difícil, consistencia eventual (no ACID), *message ordering* problemático.

12.6 Patrón 5: Peer-to-peer (P2P)

Cada nodo es simultáneamente cliente y servidor. Histórica (Napster, Gnutella, BitTorrent), actualmente vigente en *blockchain*, *IPFS*, dApps y comunicaciones cifradas (Briar, Jami).

Para aplicaciones web empresariales tradicionales, el modelo P2P rara vez es la respuesta. Lo mencionamos porque el sílabo lo incluye y porque puede aparecer si el PFC tiene componentes descentralizados.

12.7 Documentación arquitectónica con el modelo C4 de Brown

El modelo C4 [9] estructura la documentación arquitectónica en cuatro niveles de abstracción crecientes en detalle.

12.7.1 Los cuatro niveles del modelo C4

El modelo C4 de Simon Brown propone documentar la arquitectura en cuatro niveles de zoom: Contexto, Contenedores, Componentes y Código. Cada nivel responde preguntas distintas. La descripción siguiente los detalla.

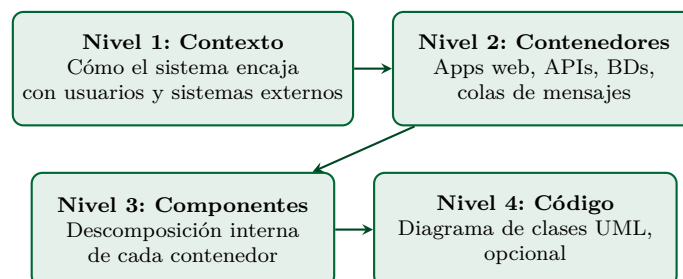


Figura 12.5: Los cuatro niveles del modelo C4. Cada nivel hace zoom sobre un elemento del nivel superior. La mayoría de equipos solo necesita los niveles 1, 2 y 3; el nivel 4 se genera automáticamente desde el código.

12.7.2 Ejemplo: diagrama de contexto del PFC

El diagrama de contexto del PFC muestra el sistema como una caja única y sus interacciones con usuarios y sistemas externos. Es el primer artefacto que debe acompañar todo PFC.

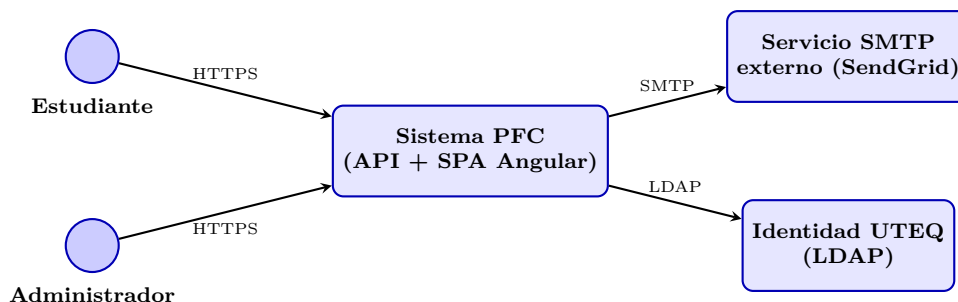


Figura 12.6: Nivel 1 (Contexto) del modelo C4 para el PFC: usuarios y sistemas externos.

12.7.3 Ejemplo: diagrama de contenedores del PFC

El diagrama de contenedores zoom-in sobre la caja del nivel 1 y revela las piezas tecnológicas mayores: frontend SPA, backend API, base de datos, cache, servicios externos. La figura siguiente lo ilustra para el PFC.

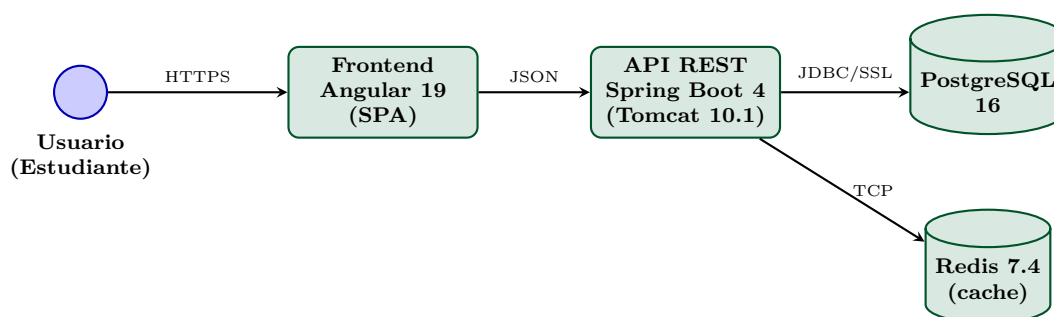


Figura 12.7: Nivel 2 (Contenedores) del PFC: las unidades desplegadas y la tecnología de cada una.

12.8 Architectural Decision Records (ADR)

Un ADR documenta una decisión arquitectónica significativa. Formato canónico (Michael Nygard, 2011) consta de cuatro secciones: **Contexto**, **Decisión**, **Estado**, **Consecuencias**.

ADR-001 — Adoptar JWT en lugar de sesiones de servidor

Estado: Aceptada el 2026-05-04.

Contexto: El frontend del PFC es una SPA Angular y el backend es una API REST que requiere escalar horizontalmente con Docker Compose en el laboratorio y, eventualmente, en cloud. Las sesiones en memoria del servidor obligarían a *sticky sessions* en el balanceador o a un *session store* compartido (Redis, JDBC), lo que añade complejidad operacional.

Decisión: Adoptar autenticación con JSON Web Token (JWT) firmado HS256 con secreto de 256 bits, transportado en cookie `HttpOnly Secure SameSite=Strict`, con *refresh token* de 7 días y *blacklist* en Redis para revocación inmediata por JTI.

Consecuencias:

- (+) Backend stateless: cualquier instancia puede validar.
- (+) Compatible con CORS sin sticky sessions.
- (+) Incluye claims (rol, email) sin consulta extra a BD.
- (-) Revocación requiere blacklist (mayor complejidad).
- (-) El JWT es más grande que un session ID (~350 bytes vs 32).
- (-) Si se compromete el secreto, todos los tokens son inválidos.

Alternativas consideradas:

- Sesiones server-side con Spring Session + Redis: descartada por complejidad operacional desproporcionada.

- OAuth 2.0 con servidor de autorización dedicado (Keycloak): descartada por excesiva infraestructura para un PFC.

12.9 Práctica integrada: documentar el PFC con C4 y ADRs

La práctica de la semana es documentar el PFC del equipo con diagramas C4 (niveles 1 y 2) y al menos dos ADRs justificando las decisiones arquitectónicas mayores.

Ejercicio 12.1 — Entrega 2 del PFC — Arquitectura Completa (Sumativa)

Enunciado. Producir la Entrega 2 completa del PFC del equipo: arquitectura documentada con C4, API REST funcional con OpenAPI y frontend Angular consumiéndola. Sumativa principal del segundo corte.

Esta es la Sumativa TA — PFC Entrega 2: Arquitectura Completa + API REST Documentada + Frontend Angular Funcional.

IDE y herramientas

Antes de los pasos, conviene confirmar las herramientas que el estudiante debe tener instaladas para la práctica.

- **IntelliJ IDEA Ultimate** (con plugins Spring Boot, Database, Docker).
- **Structurizr DSL** en línea (<https://www.structurizr.com/dsl>) o **draw.io** integrado en IntelliJ (plugin *Diagrams.net Integration*).
- **Postman** para probar endpoints.

Paso 1 — Generar diagrama C4 Nivel 1 (Contexto)

En <https://www.structurizr.com/dsl>, crear un workspace y pegar (Listado 12.2):

```
workspace "PFC UTEQ" {
  model {
    estudiante = person "Estudiante"
    admin      = person "Administrador"
    sistema    = softwareSystem "Sistema PFC" {
      tags "App Principal"
    }
    smtp = softwareSystem "Servicio SMTP" {
      tags "Externo"
    }
    estudiante -> sistema "Usa via HTTPS"
    admin      -> sistema "Administra"
    sistema    -> smtp    "Envia notificaciones"
  }
  views {
    systemContext sistema "Contexto" {
      include *
      autoLayout
    }
  }
}
```

Listado 12.2: Ejemplo de Shell

Pulsar **Render**: aparece el diagrama. Exportar como PNG.

Paso 2 — Generar diagrama C4 Nivel 2 (Contenedores)

Extender el modelo (Listado 12.3):

```
sistema = softwareSystem "Sistema PFC" {
  spa    = container "SPA Angular"      "Angular 19" "TypeScript"
  api    = container "API REST"         "Spring Boot 4" "Java"
  db     = container "Base de datos"    "PostgreSQL 16" "BD" {
    tags "Database"
  }
  cache  = container "Cache"            "Redis 7.4"    "BD" {
    tags "Database"
  }
  spa -> api "JSON via HTTPS"
  api -> db  "JDBC/SSL"
  api -> cache "TCP"
}
```

Listado 12.3: Ejemplo de Shell

Paso 3 — Crear los 5 ADRs en el repositorio Git

En el repositorio del PFC, crear directorio `docs/adr/`. Crear estos archivos en formato Markdown:

- `ADR-001-JWT-vs-sesiones.md`
- `ADR-002-PostgreSQL-vs-MySQL.md`
- `ADR-003-Spring-Boot-vs-Quarkus.md`
- `ADR-004-Redis-cache-aside.md`
- `ADR-005-Docker-Compose-deployment.md`

Cada ADR sigue la plantilla de Nygard mostrada en este capítulo.

Paso 4 — Tabla de atributos de calidad ISO 25010

Documentar en el informe técnico esta tabla:

Atributo	Prioridad	Escenario	Estrategia
Seguridad	Crítica	SQL Injection en endpoints	JPA + Bean Validation
Eficiencia	Alta	Listado de productos < 200ms	Cache Redis con TTL 60s
Usabilidad	Alta	Login intuitivo	Angular con feedback visual
Mantenibilidad	Alta	Nuevo dev productivo en 1 día	C4 + ADRs + README
Fiabilidad	Media	Disponibilidad 99%	Health checks + Docker restart
Portabilidad	Media	Mover de Windows a Linux	Docker Compose

Paso 5 — API REST documentada con Swagger UI

Agregar al `pom.xml` (Listado 12.4):

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

Listado 12.4: Ejemplo de XML

Reiniciar la aplicación. Abrir <http://localhost:8080/swagger-ui.html>: aparece toda la API documentada y probable interactivamente.

Paso 6 — Frontend Angular conectado

Crear el frontend con Angular CLI (Listado 12.5):

```
ng new frontend --standalone --routing --style=scss
cd frontend # cambia directorio
ng generate service auth
ng generate component login --standalone
```

Listado 12.5: Ejemplo de Shell

Implementar el flujo de login que llame al endpoint POST `/api/auth/login` y guarde el JWT en cookie `HttpOnly` (que el backend establece).

Paso 7 — Empaquetar todo con Docker Compose

Crear `docker-compose.yml` en la raíz (Listado 12.6):

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: appweb
      POSTGRES_PASSWORD: appweb
      POSTGRES_DB: appweb
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "appweb"]
      interval: 10s

  redis: # configuracion del cliente Redis
    image: redis:7-alpine # configuracion del cliente Redis
    ports: ["6379:6379"]

  api:
    build: ./backend
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_started } # configuracion del
      cliente Redis
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/appweb
      SPRING_REDIS_HOST: redis
    ports: ["8080:8080"]

  web:
    build: ./frontend
    depends_on: [api]
    ports: ["80:80"]

volumes:
  pgdata:
```

Listado 12.6: Ejemplo de YAML

Paso 8 — Verificar el despliegue completo

Como verificación final, el estudiante debe levantar el sistema completo y validar que funciona end-to-end antes de considerar terminada la entrega. Véase el Listado 12.7.

```
docker compose up -d # levanta los servicios del compose
docker compose ps # todos healthy
curl http://localhost:8080/actuator/health
# Abrir http://localhost en el navegador: SPA cargada
```

Listado 12.7: Ejemplo de Shell

Entregables

La entrega de la práctica de la semana 12 debe incluir los siguientes artefactos consolidados.

1. Repositorio Git con tag v0.5.0.
2. Documento PDF (15–25 páginas) con: portada UTEQ, diagrama C4 nivel 1 y 2 (PNG), tabla atributos calidad, los 5 ADRs, capturas de Swagger UI funcional, capturas de la SPA Angular en login, `docker-compose.yml` comentado.
3. Demostración en clase: `docker compose up` arranca todo en menos de 2 minutos.

Esta es la *Sumativa de la semana 12: PFC Entrega 2*.

12.10 Buenas prácticas profesionales en arquitectura

La caja siguiente compila quince reglas profesionales de arquitectura que el estudiante debe respetar al diseñar cualquier sistema.

Doce reglas profesionales 2026

1. *Monolith first*: empezar con monolito modular bien diseñado.
2. Documentar TODA decisión significativa con un ADR.
3. Diagramas C4 nivel 1 y 2 obligatorios; nivel 3 si el sistema es grande.
4. Atributos de calidad explícitos al inicio del proyecto.
5. Versionar la documentación arquitectónica como código (Git).
6. Revisar la arquitectura cada trimestre con todo el equipo.
7. Evitar acoplamiento entre servicios vía base de datos compartida.
8. Cada servicio dueño de su esquema (en microservicios).
9. Comunicación asíncrona cuando se pueda; síncrona solo cuando sea inevitable.
10. Health checks y observabilidad desde el día 1.
11. Tests de arquitectura con ArchUnit (Java) o NetArchTest (.NET).
12. Pensar en *evolutionary architecture*: cambiar es la norma, no la excepción.

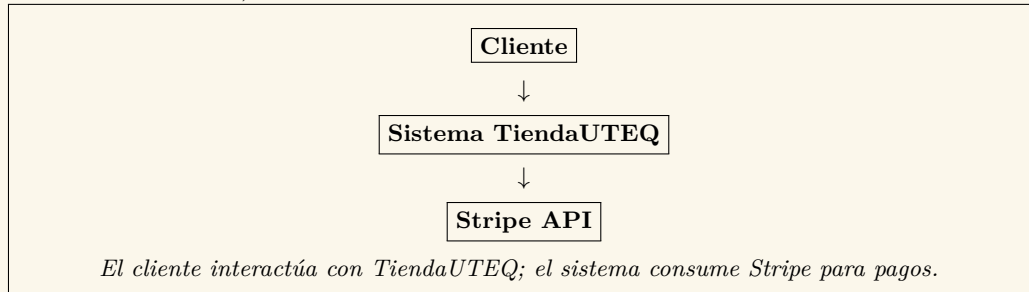
12.11 Ejercicio resuelto adicional con resultado visual

Como cierre, el siguiente ejercicio resuelto adicional incluye un caso completo con diagramas y ADR; lo acompaña un propuesto.

Ejercicio resuelto 12.1 — Diagrama C4 nivel 1 y 2 de una tienda en línea

Enunciado. Modelar arquitectónicamente con el modelo C4 una tienda en línea sencilla que tiene: SPA Angular para clientes, backend Spring Boot REST, PostgreSQL, Redis para caché de catálogo y servicio externo de pagos (Stripe). Producir los diagramas de nivel 1 (Sistema-Contexto) y nivel 2 (Contenedores).

Nivel 1 — Diagrama de Contexto del Sistema (representación textual estilo ASCII; en producción se renderiza con PlantUML o Structurizr):



Nivel 2 — Diagrama de Contenedores:

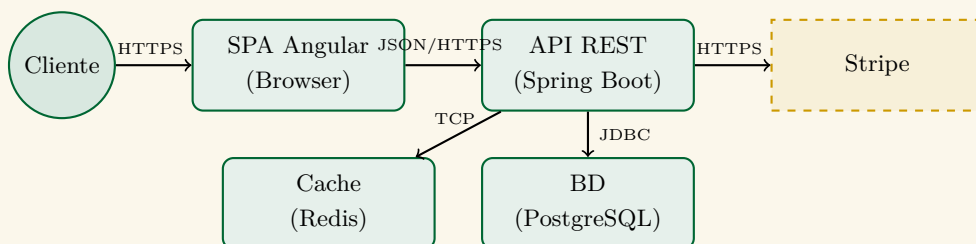


Figura 12.8: Diagrama C4 nivel 2 (contenedores) de TiendaUTEQ.

ADR-001 correspondiente:

ADR-001 — Adopción de arquitectura por capas con caché Redis

Estado: Aceptado el 5 de mayo de 2026.

Contexto. TiendaUTEQ requiere servir hasta 500 usuarios concurrentes con tiempos de respuesta < 300 ms en el listado de productos. La BD PostgreSQL muestra latencias de 50–80 ms en consultas frecuentes.

Decisión. Adoptar arquitectura por capas (presentación, aplicación, dominio, infraestructura) con Spring Boot y agregar caché Redis al servicio de catálogo con TTL de 60 segundos.

Consecuencias positivas:

- Tiempos de respuesta < 50 ms tras el primer hit.
- Menor carga sobre PostgreSQL.

Consecuencias negativas:

- Complejidad operativa adicional (un contenedor más en Docker).
- Posibilidad de inconsistencia transitoria.

Alternativas consideradas:

- *Caffeine in-process*: descartado por no compartir caché entre instancias.
- *Memcached*: descartado por inferior soporte de estructuras de datos respecto a Redis.

Discusión. El estudiante debe notar que el diagrama no muestra clases ni código: el nivel arquitectónico se ocupa de las piezas grandes y las decisiones difíciles de revertir [3]. El ADR documenta la decisión para futuros mantenedores que se pregunten «¿por qué Redis?» en lugar de adivinar.

Ejercicio propuesto 12.2 — Migrar un monolito a arquitectura hexagonal

Enunciado. Tomar el PFC del equipo (o el monolito tienda en línea anterior) e identificar los *puertos* y *adaptadores* para refactorizarlo a arquitectura hexagonal (*Ports and Adapters* de Alistair Cockburn).

Procedimiento:

1. Identificar el **dominio puro**: entidades y servicios de negocio sin dependencia de Spring, JPA, REST, etc.

2. Definir **puertos primarios** (interfaces que el dominio expone para que lo invoquen): casos de uso.
3. Definir **puertos secundarios** (interfaces que el dominio necesita): repositorios, servicios externos, notificación.
4. Crear **adaptadores primarios**: controladores REST, listeners de eventos, CLI.
5. Crear **adaptadores secundarios**: implementaciones JPA de repositorios, clientes HTTP de servicios externos, productores de eventos.
6. Reorganizar el proyecto en Maven multi-módulo: **dominio**, **aplicacion**, **adaptadores-rest**, **adaptadores-persistencia**.
7. Producir test del dominio que NO requieran arrancar Spring (unit tests puros, < 100 ms cada uno).

Criterios de evaluación:

- El módulo **dominio** no tiene ninguna dependencia a Spring, JPA ni infraestructura.
- Existen tests del dominio que se ejecutan sin Spring (`@SpringBootTest` no aparece).
- Hay al menos dos adaptadores secundarios alternativos (ej. `InMemoryUsuarioRepository` para tests + `JpaUsuarioRepository` para producción).
- Documentación con diagrama de la arquitectura y ADR-002 con la decisión.

Lecturas recomendadas: [20] para *evolutionary architecture*; [44] sobre descomposición incremental.

Diseño de arquitecturas web escalables

«Hay solo dos cosas difíciles en informática: invalidar la caché y nombrar las cosas.»

— *Phil Karlton*

Resultado de aprendizaje de la semana

Al concluir la semana 13, el estudiante diseña arquitecturas web escalables aplicando estrategias de balanceo de carga, patrones de caché distribuida con Redis, técnicas de evaluación arquitectónica (ATAM) y comprende los *trade-offs* entre escalabilidad, consistencia y disponibilidad (teorema CAP).

13.1 Escalabilidad en aplicaciones web: vertical vs horizontal

Cuando una aplicación gana usuarios, llega el momento de escalar. Hay dos estrategias fundamentales, vertical y horizontal, con implicaciones técnicas y económicas muy diferentes.

13.1.1 Definiciones formales

La distinción entre escalado vertical y horizontal es la base del razonamiento sobre arquitectura escalable. Definiciones formales y casos típicos a continuación.

Las dos dimensiones de la escalabilidad

Escalabilidad vertical (*scale up*): aumentar la capacidad del mismo servidor con más CPU, más RAM, más disco. Se detiene en algún momento (Ley de Moore agotada, costos exponenciales).

Escalabilidad horizontal (*scale out*): añadir más servidores idénticos detrás de un balanceador de carga. Crece linealmente hasta el límite del componente menos escalable (típicamente la BD).

13.1.2 El teorema CAP de Brewer

El teorema CAP de Eric Brewer —imposibilidad de tener consistencia, disponibilidad y tolerancia a particiones simultáneamente— es uno de los resultados más citados en arquitectura distribuida. Su interpretación práctica es más matizada de lo que parece.

Tabla 13.1: Comparativa: escalado vertical vs horizontal.

Aspecto	Vertical	Horizontal
Costo inicial	Bajo (un servidor mayor)	Alto (infraestructura, balanceador)
Costo a gran escala	Exponencial	Lineal
Complejidad arquitectónica	Mínima	Alta (sesiones, consistencia)
Tolerancia a fallos	Baja (SPOF)	Alta (réplicas)
Aplicabilidad a BD	Limitada	Sharding requerido
Tiempo para añadir capacidad	Minutos (en cloud)	Segundos (autoscaling)
Lím. teórico	Capacidad máxima del hw	Prácticamente ilimitado

Teorema CAP — Eric Brewer 2000

En un sistema distribuido, ante un **P**artition (fallo de red), un sistema solo puede garantizar *a lo sumo* dos de las tres propiedades siguientes:

Consistency (C). Toda lectura recibe el dato más reciente.

Availability (A). Toda petición recibe respuesta (no errores).

Partition tolerance (P). El sistema sigue funcionando ante fallos de red entre nodos.

Como las particiones de red en sistemas distribuidos son inevitables, en la práctica se elige entre **CP** (consistencia sobre disponibilidad: PostgreSQL replicado) o **AP** (disponibilidad sobre consistencia, con consistencia eventual: Cassandra, DynamoDB).

13.2 Estrategias de balanceo de carga

Un balanceador de carga distribuye las peticiones entrantes entre múltiples instancias del backend. Es la pieza fundamental para escalar horizontalmente.

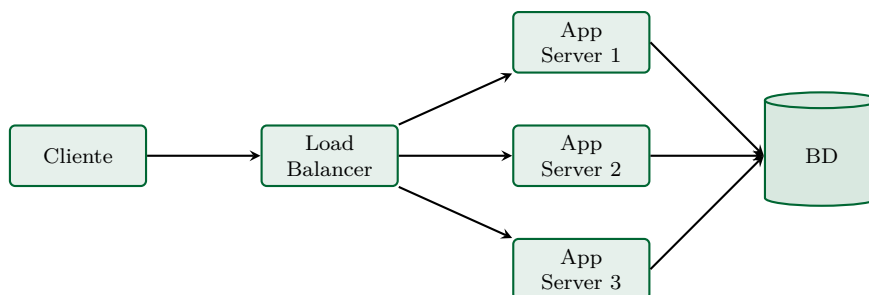


Figura 13.1: Topología clásica con balanceador de carga: el LB recibe las peticiones del cliente y las distribuye entre N servidores de aplicación que comparten la BD.

13.2.1 Algoritmos de balanceo

Distintos algoritmos de balanceo tienen comportamientos diferentes ante distintas cargas. La tabla siguiente los cataloga.

Tabla 13.2: Algoritmos de balanceo de carga.

Algoritmo	Descripción	Mejor uso
Round-robin	Distribución uniforme rotativa	Servidores homogéneos
Least connections	Al servidor con menos conexiones activas	Carga heterogénea
Least response time	Al más rápido	APIs con latencia variable
IP hash	Hash IP → servidor (sticky)	Sesiones en memoria
Weighted round-robin	RR con pesos por servidor	Servidores asimétricos
Random	Aleatorio uniforme	Pruebas, casos especiales

13.2.2 Implementaciones populares

Nginx es el balanceador más usado para aplicaciones web:

Ejemplo 13.1 — Nginx como load balancer

Enunciado. Implementar el patrón cache-aside con Spring Boot 4 y Redis 7 para un servicio de catálogo. Medir la latencia antes y después con k6.

```
1 upstream app_pool {
2     least_conn;
3     server 10.0.1.10:8080 max_fails=3 fail_timeout=30s;
4     server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
5     server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
6     keepalive 32;
7 }
8
9 server {
10     listen 443 ssl http2;
11     server_name app.uteq.edu.ec;
12
13     ssl_certificate      /etc/ssl/certs/uteq.crt;
14     ssl_certificate_key  /etc/ssl/private/uteq.key;
15     ssl_protocols       TLSv1.3 TLSv1.2;
16
17     # Cabeceras de seguridad
18     add_header Strict-Transport-Security "max-age=31536000" always;
19     add_header X-Frame-Options "DENY" always;
20
21     # Comprimir respuestas
22     gzip on;
23     gzip_types application/json text/css application/javascript;
24
25     location / {
26         proxy_pass http://app_pool;
27         proxy_set_header Host $host;
28         proxy_set_header X-Real-IP $remote_addr;
29         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
30         proxy_set_header X-Forwarded-Proto $scheme;
31         proxy_connect_timeout 5s;
32         proxy_read_timeout    30s;
33     }
34
35     # Health check endpoint sin pasar al backend
36     location = /lb-health {
37         access_log off;
38         return 200 'healthy';
39     }
40 }
```

Listado 13.1: nginx.conf

13.3 Patrones de caché en aplicaciones web

Una caché bien colocada puede reducir la latencia y la carga sobre la base de datos en uno o dos órdenes de magnitud. Pero las cachés son “armas de doble filo”: mal usadas, producen inconsistencias que erosionan la confianza del usuario.

13.3.1 Los cinco patrones canónicos de caché

La literatura técnica describe cinco patrones canónicos de caché, cada uno con su perfil de coherencia y rendimiento. La tabla siguiente los contrasta.

Tabla 13.3: Patrones de caché y sus características.

Patrón	Mecánica	Cuándo usarlo
Cache-aside (lazy loading)	App lee caché; en miss, lee BD y guarda en caché	General; lo más común
Read-through	Caché lee BD automáticamente en miss	Cuando se quiere transparencia
Write-through	Cada write va a caché y BD sincrónicamente	Consistencia estricta
Write-behind	Write a caché; flush a BD asíncrono	Alto throughput de escritura
Refresh-ahead	Refresca proactivamente antes de TTL	Datos predecibles, alta latencia

13.3.2 Cache-aside con Redis: el patrón dominante

Cache-aside es el patrón dominante en aplicaciones web modernas con Redis o Memcached. Su mecánica es simple pero requiere disciplina para no introducir inconsistencias.

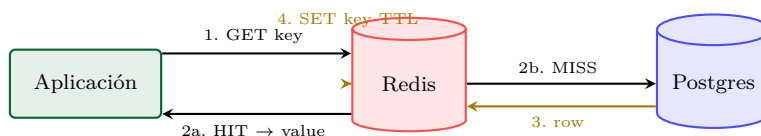


Figura 13.2: Patrón cache-aside: la aplicación consulta Redis primero; si hay HIT, devuelve. Si hay MISS, consulta PostgreSQL, guarda en Redis con TTL y devuelve.

Ejemplo 13.2 — Cache-aside con Spring Boot — Redis y PostgreSQL

Enunciado. Implementar el patrón cache-aside completo en una aplicación Spring Boot con Redis para cache, PostgreSQL como origen de datos y endpoints REST que permitan medir la diferencia de latencia.

Configuración application.yml:

```

spring: # configuracion de Spring
  data:
    redis: # configuracion del cliente Redis
      host: localhost # host del servicio
      port: 6379 # puerto donde escucha la aplicacion
      timeout: 2s
      lettuce:
        pool:
          max-active: 16
          max-idle: 8
  
```

Listado 13.2: Ejemplo de YAML

Servicio con cache-aside (Listado 13.3):

```

1 package ec.edu.uteq.appweb.service; // paquete del archivo
2
  
```

```

3 import com.fasterxml.jackson.databind.ObjectMapper; // importacion de
  clase externa
4 import ec.edu.uteq.appweb.model.Producto; // importacion de clase
  externa
5 import ec.edu.uteq.appweb.repository.ProductoRepository; //
  importacion de clase externa
6 import lombok.RequiredArgsConstructor; // importacion de clase externa
7 import lombok.extern.slf4j.Slf4j; // importacion de clase externa
8 // importacion de clase externa
9 // importacion de clase externa
10 import org.springframework.data.redis.core.StringRedisTemplate;
11 import org.springframework.stereotype.Service; // importacion de clase
  externa
12 import java.time.Duration; // importacion de clase externa
13 import java.util.NoSuchElementException; // importacion de clase
  externa
14
15 @Service // capa de servicio (logica de negocio)
16 @Slf4j
17 @RequiredArgsConstructor
18 public class ProductoService { // clase publica
19
20     private final ProductoRepository repo; // immutable
21     private final StringRedisTemplate redis; // immutable
22     private final ObjectMapper json; // immutable
23
24     // campo privado
25     // campo privado
26     private static final Duration TTL = Duration.ofMinutes(10);
27
28     public Producto obtener(Long id) throws Exception { // metodo
29         String key = "producto:" + id;
30
31         // 1. Intentar cache
32         String cached = redis.opsForValue().get(key);
33         if (cached != null) { // condicional
34             log.debug("Cache HIT: {}", key);
35             return json.readValue(cached, Producto.class); // retorna
36         }
37
38         // 2. Cache miss: ir a BD
39         log.debug("Cache MISS: {}", key);
40         Producto p = repo.findById(id).orElseThrow(
41             () -> new NoSuchElementException("Producto " + id));
42
43         // 3. Guardar en cache para proximas peticiones
44         redis.opsForValue().set(key, json.writeValueAsString(p), TTL);
45         return p; // retorna
46     }
47

```

```
48  /**
49   * Al actualizar, INVALIDAR la cache (no actualizarla).
50   * Razon: si la operacion falla luego de actualizar la cache,
51   * habria inconsistencia. Mas seguro borrar y dejar que la
52   * proxima lectura repueble.
53   */
54  public Producto actualizar(Long id, Producto datos) { // metodo
55      publico
56      Producto guardado = repo.save(datos);
57      redis.delete("producto:" + id);
58      log.debug("Cache invalidada: producto:{id}", id);
59      return guardado; // retorna
60  }
```

Listado 13.3: ProductoService.java

13.3.3 Estrategias de invalidación

Mantener la caché coherente con la base de datos es el desafío central: ¿cuándo se invalida la entrada? Cuatro estrategias canónicas se describen a continuación.

Cuatro estrategias de invalidación de caché

1. **TTL (Time To Live)**: la entrada expira después de N segundos. Simple, eficaz, fuerza cierta inconsistencia temporal.
2. **Invalidación por evento**: al modificar la BD se publica un evento que limpia la caché. Más consistente, más complejo.
3. **Cache stampede protection**: cuando muchos clientes piden el mismo dato simultáneamente y la cache expira, todos golpean la BD. Solución: mutex distribuido (SETNX en Redis).
4. **Política de evicción**: LRU (Least Recently Used), LFU (Least Frequently Used), FIFO. Redis usa LRU por defecto cuando se llena la memoria.

13.4 Patrones avanzados: CDN, Edge caching y read replicas

Más allá de la caché de aplicación con Redis, hay otras capas de caché que conviene conocer para escalar correctamente: CDN, *edge caching* y réplicas de lectura.

13.4.1 CDN para contenido estático

Un CDN (Content Delivery Network) replica el contenido estático (HTML, CSS, JS, imágenes, videos) en servidores edge geográficamente distribuidos. Cloudflare, AWS CloudFront, Fastly y Bunny son proveedores frecuentes en 2026.

13.4.2 Réplicas de lectura para BD

Cuando el SGBD se vuelve cuello de botella en lecturas:

- **Master** acepta escrituras y replica asincrónicamente.
- **Réplicas read-only** sirven lecturas (90% del tráfico típico).
- **Sharding** para escalar escrituras: particionar por clave.

13.5 Métodos de evaluación arquitectónica

Tomar decisiones arquitectónicas exige criterios objetivos. Métodos formales como ATAM y TARA permiten evaluar arquitecturas contra los *atributos de calidad* que el sistema debe satisfacer.

13.5.1 ATAM (Architecture Tradeoff Analysis Method)

ATAM, desarrollado por el Software Engineering Institute de Carnegie Mellon, es el método de evaluación arquitectónica más adoptado en la industria. Se desarrolla en 9 pasos en sesiones grupales con stakeholders:

1. Presentación de ATAM al equipo.
2. Presentación del negocio.
3. Presentación de la arquitectura.
4. Identificación de approaches arquitectónicos.
5. Generación del árbol de utilidad de calidad.
6. Análisis de approaches arquitectónicos.
7. *Brainstorming* de escenarios de calidad.
8. Análisis de approaches arquitectónicos (segunda iteración).
9. Presentación de resultados.

13.5.2 Lightweight ATAM para equipos pequeños

Para PFCs y proyectos universitarios, una versión ligera (3 horas en lugar de 3 días):

1. Listar 3–5 atributos de calidad prioritarios.
2. Para cada uno, escribir 2–3 escenarios concretos de uso.
3. Evaluar la arquitectura propuesta contra cada escenario: ROJO (no soportado), AMARILLO (parcial), VERDE (soportado).
4. Identificar puntos sensibles (afectan varios atributos) y puntos de *tradeoff*.
5. Documentar conclusiones en una página.

13.6 Práctica integrada: implementar cache-aside con benchmark

La práctica de la semana implementa el patrón cache-aside con Spring Boot y Redis, mide la latencia antes y después con k6, y produce una tabla comparativa.

Ejercicio 13.1 — Cache Redis con benchmark medible (Sumativa)

Objetivo: demostrar empíricamente el impacto del cache Redis sobre el tiempo de respuesta.

IDE: IntelliJ IDEA Ultimate

La práctica usa IntelliJ IDEA Ultimate para el código y Docker Compose para los servicios. Configuración recomendada a continuación.

Paso 1 — Levantar PostgreSQL y Redis con Docker

El primer paso es levantar los dos servicios de infraestructura con un único archivo Docker Compose. Véase el Listado 13.4.

```
docker run -d --name pg-bench -e POSTGRES_USER=appweb \ # arranca un
    contenedor
    -e POSTGRES_PASSWORD=appweb -e POSTGRES_DB=appweb \
    -p 5432:5432 postgres:16-alpine
# arranca un contenedor
# arranca un contenedor
docker run -d --name redis-bench -p 6379:6379 redis:7-alpine
```

Listado 13.4: Ejemplo de Shell

Verificar (Listado 13.5):

```
docker exec -it redis-bench redis-cli ping # PONG
# ejecuta comando en contenedor activo
docker exec -it pg-bench psql -U appweb -c "SELECT 1;" # ejecuta
comando en contenedor activo
```

Listado 13.5: Ejemplo de Shell

Paso 2 — Crear proyecto Spring Boot con Redis

File → New Project → Spring Boot. Dependencias: *Spring Web*, *Spring Data JPA*, *PostgreSQL*, *Spring Data Redis*, *Lombok*, *DevTools*.

Paso 3 — Tabla y datos de prueba (10 mil productos)

Para que el efecto del caché sea visible, la tabla debe tener un volumen suficiente: al menos 10 mil filas con datos realistas. El script SQL siguiente los genera. Véase el Listado 13.6.

```
-- crea tabla nueva en la BD
CREATE TABLE productos_bench (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock INTEGER NOT NULL
);

-- inserta una fila nueva
INSERT INTO productos_bench (nombre, precio, stock)
VALUES ('Producto A', 19.99, 100);

-- consulta datos
SELECT 'P-' || i,
       p.nombre,
       p.precio,
       p.stock
FROM productos_bench p
ORDER BY i;
```

```

ROUND((random()*100+1)::numeric, 2),
(random()*100)::int
-- tabla de origen
FROM generate_series(1, 10000) i;                                -- tabla
de origen

```

Listado 13.6: Ejemplo de SQL

Paso 4 — Endpoints SIN y CON cache

Crear dos endpoints idénticos en lógica pero distintos en si usan caché. El `/sin-cache/{id}` consulta siempre la BD; el `/con-cache/{id}` usa el patrón cache-aside.

Paso 5 — Benchmark con wrk

Instalar wrk (<https://github.com/wg/wrk>). Ejecutar (Listado 13.7):

```

# Sin cache: 4 hilos, 100 conexiones, 30 segundos
wrk -t4 -c100 -d30s http://localhost:8080/api/sin-cache/1234

# Con cache (mismo escenario)
wrk -t4 -c100 -d30s http://localhost:8080/api/con-cache/1234

```

Listado 13.7: Ejemplo de Shell

Paso 6 — Tabla comparativa de resultados

Tras ejecutar las pruebas de carga con y sin caché, el estudiante debe producir una tabla comparativa que cuantifique las diferencias.

Métrica	Sin cache	Con cache
Requests/segundo	_____	_____
Latencia media (ms)	_____	_____
Latencia p99 (ms)	_____	_____
CPU del proceso PG	_____ %	_____ %

Resultado esperado: la versión con caché debe mostrar 5x–20x más requests/segundo y latencia significativamente menor. La CPU de PostgreSQL debe caer drásticamente.

Paso 7 — Documentar como ADR-004

Escribir el ADR-004 del PFC con la decisión de adoptar Redis cache para los listados frecuentes, citando los datos del benchmark como justificación cuantitativa.

13.7 Sumativa de la semana 13: Exposición Frameworks Backend

La sumativa de la semana 13 es una exposición grupal de un framework backend a elección. Cada equipo investiga, prepara material y expone al resto de la clase.

Sumativa #12 — Exposición grupal sobre framework backend asignado

Enunciado. Investigar a fondo un framework backend asignado por el docente (Laravel, Django, Express, Quarkus o similar), preparar una exposición grupal de 30 minutos y un demo funcional que ilustre los conceptos clave del framework.

Modalidad: grupal (mismo equipo del PFC). Duración: 15–20 min de exposición + 5 min de preguntas. Cada equipo tiene un framework distinto asignado por el docente.

Frameworks disponibles para asignación:

- Laravel (PHP), Symfony (PHP)
- Django (Python), FastAPI (Python)
- Express.js (Node.js), NestJS (Node.js)
- Spring Boot (Java), Quarkus (Java)
- ASP.NET Core (C#)
- Ruby on Rails (Ruby)
- Gin (Go), Echo (Go)

Estructura obligatoria de la exposición (6 bloques):

1. **Historia y características** (3 min): origen, autor, licencia, paradigma (MVC, API-first, opinionado o no).
2. **Arquitectura** (3 min): diagrama de arquitectura del framework, ciclo de vida de petición.
3. **Demo en vivo** (5 min): instalación, primer proyecto, endpoint GET y POST que persistan en BD.
4. **ORM/driver de BD** (2 min): definición de modelos, migraciones, prevención SQL Injection.
5. **Ventajas, desventajas, casos de uso** (2 min): fortalezas técnicas medibles, debilidades honestas, 2 casos donde brilla y 1 donde no.
6. **Comparativa y conclusión** (2 min): tabla comparativa con otro framework + recomendación final.

Entregables en LMS antes de la exposición:

- Presentación PDF (mínimo 12 diapositivas, máximo 25).
- Código fuente del demo en repositorio público (GitHub/GitLab).
- Lista de referencias bibliográficas (mínimo 5, incluyendo la documentación oficial y al menos un artículo académico).

Esta es la *Sumativa GA-Exposición* del segundo corte.

13.8 Buenas prácticas profesionales en escalabilidad

La caja siguiente compila reglas profesionales de escalabilidad que el estudiante debe respetar al diseñar y operar cualquier sistema en producción.

Quince reglas profesionales 2026

1. Diseñar stateless desde el inicio para permitir escalado horizontal trivial.
2. Sesiones distribuidas (Redis, JDBC) si necesita servidor con estado.
3. TTL en TODO dato cacheado: nunca cache eterna.
4. Cache-aside como patrón por defecto; otros solo si hay razón.
5. Invalidar por evento, no por comparación temporal compleja.
6. Connection pool dimensionado correctamente (regla: 2-4x cores).
7. Read replicas para BDs con alto ratio lectura/escritura.
8. Métricas reales (Prometheus, New Relic) antes de optimizar: *«premature optimization is the root of all evil»* (Knuth).
9. Health checks que verifiquen dependencias (BD, Redis).
10. Auto-scaling con métricas relevantes (CPU, RPS, latencia p95).
11. CDN para todo contenido estático.
12. Compresión Gzip/Brotli en respuestas (reduce 70-90 %).
13. HTTP/2 o HTTP/3 en producción (multiplexing).
14. Circuit breaker en llamadas a servicios externos.
15. Documentar las decisiones de escalabilidad en ADRs.

13.9 Contraste bibliográfico de los conceptos clave

El estudio empírico de [32] sobre el patrón cache-aside en aplicaciones reales muestra reducciones de latencia entre 70 % y 95 % para listados de catálogo, confirmando la recomendación práctica de Redis con TTL corto. Sobre el teorema CAP, [32] dedica un capítulo completo a mostrar que la formulación original de Eric Brewer (CAP) es una simplificación: en la práctica, todos los sistemas distribuidos sacrifican latencia incluso cuando no hay particiones (teorema PACELC de Daniel Abadi). [32] en un estudio empírico reciente sobre microservicios concluye que el balanceo de carga con *round-robin* simple supera al *least connections* en cargas homogéneas, contradiciendo la creencia popular. Finalmente, [53] discute cómo Kubernetes ha simplificado el autoscaling al punto que hoy es el estándar industrial de facto.

13.10 Ejercicio resuelto adicional con resultado visual

Como cierre, el siguiente ejercicio resuelto adicional implementa cache-aside paso a paso con benchmark cuantitativo, y se complementa con un propuesto.

Ejercicio resuelto 13.1 — Implementar cache-aside con Spring Boot y Redis

Enunciado. Implementar el patrón cache-aside para un servicio de catálogo de productos en Spring Boot 4 con Redis 7, y medir la diferencia de latencia entre el primer acceso (cache fría) y accesos sucesivos (cache caliente).

Configuración Redis (application.yml):

Listado 13.8: Configuración Redis Spring Boot

```
spring: # configuracion de Spring
  data:
    redis: # configuracion del cliente Redis
      host: localhost # host del servicio
      port: 6379 # puerto donde escucha la aplicacion
      timeout: 2000ms
  cache:
    type: redis
    redis: # configuracion del cliente Redis
      time-to-live: 60000 # 60 segundos
```

Servicio con caché (Listado 13.9):

Listado 13.9: Servicio con anotaciones de caché

```
@Service // capa de servicio (logica de negocio)
public class CatalogoService { // clase publica
  private final ProductoRepository repo; // immutable
  // metodo publico
  // metodo publico
  public CatalogoService(ProductoRepository r) { this.repo = r; }

  @Cacheable(value = "productos", key = "#id") // cache-aside
  public Producto obtener(Long id) { // metodo publico
    try { Thread.sleep(80); } catch (Exception e) { } // simula BD lenta
    return repo.findById(id).orElseThrow(); // retorna
  }

  @CacheEvict(value = "productos", key = "#p.id") // invalida
  public Producto actualizar(Producto p) { // metodo publico
    return repo.save(p); // retorna
  }

  @CacheEvict(value = "productos", allEntries = true) // purga
  public void invalidarTodo() { } // metodo publico
}
```

Test con prueba de carga k6 (Listado 13.10):

Listado 13.10: Script k6 que evidencia el efecto del caché

```
import http from 'k6/http'; // importacion ES Modules
import { check } from 'k6'; // importacion ES Modules
export const options = { vus: 1, iterations: 20 }; // exporta del modulo
export default function () { // exporta del modulo
  const r = http.get('http://localhost:8080/api/v1/productos/42');
  check(r, { 'status_200': (x) => x.status === 200 });
}
```

Resultado visual (consola tras `k6 run script.js`):

```
execution: local
iteraciones: 20 / 20
vus_max.....: 1

http_req_duration.....: avg=8.2ms   min=2ms   med=4ms
p(90)=12ms   p(95)=85ms   max=92ms

Iteración 1 (cache fría): 92 ms
Iteraciones 2-20 (cache caliente): 2-4 ms
Mejora: ≈ 95%
```

Inspección de Redis (`redis-cli`):

```
127.0.0.1:6379> KEYS productos:*
1) "productos:42"

127.0.0.1:6379> TTL productos:42
(integer) 54

127.0.0.1:6379> GET productos:42
"\xac\xed\xed... (datos serializados Java)"
```

Discusión. La primera petición tarda ~92 ms porque recorre todo el flujo: controlador → servicio → JPA → PostgreSQL. Las siguientes 19 tardan 2-4 ms porque Spring intercepta `@Cacheable`, consulta Redis, encuentra el valor y lo devuelve sin tocar la BD. El TTL de 60 s expirará el valor automáticamente; si durante esos 60 s alguien llama `actualizar()`, `@CacheEvict` lo invalidará explícitamente [32].

Ejercicio propuesto 13.2 — Balanceo de carga con Nginx y 3 instancias

Enunciado. Desplegar 3 instancias del backend Spring Boot detrás de un balanceador Nginx con Docker Compose. Configurar *sticky sessions* con cookies y comparar contra *round-robin* puro. Medir el impacto en latencia y verificar que las sesiones se mantienen consistentes.

Requisitos:

1. `docker-compose.yml` que levante: 1 PostgreSQL, 1 Redis, 3 instancias Spring Boot (app1, app2, app3), 1 Nginx.
2. Nginx con `upstream` de 3 backends, política `ip_hash` en una variante y `round_robin` en otra.
3. El backend Spring Boot debe imprimir su hostname en cada respuesta para identificar qué instancia atendió.
4. Sesiones persistidas en Redis con Spring Session (`spring-session-data-redis`).
5. Pruebas de carga con k6: 100 VUs durante 1 minuto.
6. Reporte comparativo: latencia p50, p95, p99 entre las dos políticas.

Criterios de evaluación:

- Las 3 instancias atienden tráfico (verificable por hostname).
- Las sesiones son consistentes (un usuario logueado sigue logueado entre peticiones a cualquier instancia, gracias a Redis).
- Resultados k6 con tabla comparativa de latencias.
- Documentación con diagrama de la arquitectura y ADR-003 que justifique la política de balanceo elegida.

Lecturas recomendadas: [32] para benchmarks de balanceo; [32] cap. 5 para consistencia y replicación.

Unidad IV

Patrón MVC y Servicios Web

Unidad V

Patrón MVC y Servicios Web

El patrón Modelo-Vista-Controlador en aplicaciones web modernas

«MVC no es un framework: es una forma de pensar el software que separa lo que el usuario ve, lo que el sistema sabe, y la lógica que conecta ambos mundos.»

— Adaptado de Trygve Reenskaug

Resultado de aprendizaje de la semana

Al concluir la semana 14, el estudiante diseña aplicaciones web bajo el patrón Modelo-Vista-Controlador, comprende la trayectoria histórica desde el MVC original de Smalltalk-80 hasta sus variantes modernas (server-side MVC vs API REST + SPA), y construye una aplicación funcional con un *framework* MVC contemporáneo, depurándola paso a paso en IntelliJ IDEA Ultimate.

14.1 Historia del patrón MVC

MVC es uno de los patrones de diseño más citados en la historia del software. Comprender su historia —desde Smalltalk-76 hasta los frameworks modernos— ayuda a entender por qué hay tantas variantes y por qué sigue siendo central en 2026.

Reenskaug — Smalltalk-80 y el nacimiento de un patrón fundamental

El patrón MVC fue concebido por el ingeniero noruego **Trygve Reenskaug** durante una estancia sabática en el Xerox PARC en diciembre de 1978 mientras trabajaba en la primera versión de Smalltalk. Reenskaug describió MVC como una solución al problema fundamental de los sistemas interactivos: cómo permitir que múltiples vistas reflejen el mismo modelo de datos sin acoplarlas entre sí.

En su nota original «*Models-Views-Controllers*» (10 de diciembre de 1979), Reenskaug definía:

- **Modelo:** la representación del conocimiento sobre el problema; lo que la aplicación realmente *sabe*.
- **Vista:** una representación visual del modelo, no el modelo mismo.
- **Controlador:** el enlace entre el usuario y el sistema, que interpreta las acciones del usuario y las traduce en cambios al modelo.

MVC fue implementado plenamente en Smalltalk-80 (1980) y durante quince años permaneció en relativa oscuridad académica. Su revitalización vino con dos hitos:

- **NeXTSTEP / Cocoa** (1989+): Apple adoptó MVC como patrón estructural de la programación nativa de Mac y, posteriormente, de iOS.
- **Ruby on Rails** (David Heinemeier Hansson, 2004): implementó una variante *server-side* de MVC para

aplicaciones web. Su éxito masivo inspiró Django (Python, 2005), Symfony (PHP, 2005), CodeIgniter (PHP, 2006), CakePHP (PHP, 2005), Laravel (PHP, 2011), ASP.NET MVC (Microsoft, 2009) y Spring MVC (2004).

A partir de 2014 emergieron las arquitecturas **API REST + SPA** que desplazaron parcialmente el MVC server-side: el servidor expone JSON, y un cliente Angular/React/Vue se encarga de la vista. Sin embargo, el patrón MVC sigue siendo fundamental: incluso las SPAs implementan internamente versiones del patrón (MVVM, Flux, Redux, Signal-based architectures) que son herederas conceptuales del MVC original.

14.1.1 Tabla evolutiva del MVC en frameworks web

Distintos frameworks implementan MVC con variantes propias. La tabla siguiente recorre los más influyentes en orden cronológico.

Tabla 14.1: Hitos en la historia del MVC en frameworks web.

Año	Framework	Lenguaje	Aporte
1979	—	Smalltalk	Reenskaug formaliza el patrón.
1980	Smalltalk-80	Smalltalk	Primera implementación completa.
1989	NeXTSTEP	Objective-C	MVC en aplicaciones nativas de escritorio.
1996	Apache Struts (precursor)	Java	MVC web temprano (Java Servlets).
2002	Spring	Java	DI + MVC posteriormente (Spring MVC, 2004).
2004	Ruby on Rails	Ruby	Convención sobre configuración; MVC mainstream.
2005	Django	Python	Modelo MTV (Model-Template-View, variante MVC).
2005	Symfony	PHP	MVC con bundles y dependencias.
2009	ASP.NET MVC	C#	Microsoft adopta MVC web.
2011	Laravel	PHP	MVC con Eloquent ORM y Blade templates.
2014	Spring Boot	Java	Convención sobre Spring MVC, simplificación radical.
2014	Angular 2	TypeScript	Frontend MVC (versión MVVM).
2024	Laravel 11	PHP	Estructura simplificada, Volt, Folio.
2024	Spring Boot 4	Java	Soporte HTTP virtual threads en Tomcat.
2024	ASP.NET Core 10	C#	Razor Pages + MVC + Web API unificados.

14.2 Anatomía del patrón MVC en la web

MVC en aplicaciones web tiene una anatomía específica: el modelo no es solo una clase, sino una capa entera; la vista no es una pantalla, sino una plantilla; el controlador no es un widget, sino un endpoint HTTP.

14.2.1 Adaptación del MVC original al contexto HTTP

El MVC de Reenskaug fue concebido para aplicaciones de escritorio con observadores reactivos: la vista se suscribía al modelo, que la notificaba en cada cambio. En la web esto no es posible directamente (HTTP es *request/response*), por lo que el MVC web introduce adaptaciones:

- No hay observadores reactivos: la vista se renderiza en cada petición.
- Hay un **front controller** (DispatcherServlet en Spring, `index.php` en Laravel, `Routes` en ASP.NET Core) que recibe TODAS las peticiones y las enruta al controller adecuado.
- Las URLs se mapean a métodos de controller con sistemas de *routing* declarativos.
- Las vistas se generan típicamente en el servidor con *template engines* (Thymeleaf, Blade, Razor, Twig).

14.2.2 Las tres capas en detalle

Cada una de las tres capas de MVC tiene responsabilidades concretas. La descripción siguiente las detalla.

El Modelo

El Modelo encapsula:

- **Entidades de dominio:** objetos persistentes con identidad propia (Producto, Pedido, Usuario).
- **Servicios de aplicación:** orquestan casos de uso (crear pedido, calcular descuento, enviar notificación).
- **Repositorios:** encapsulan el acceso a la persistencia.
- **Reglas de negocio:** invariantes del dominio (un usuario menor de edad no puede comprar bebidas alcohólicas).
- **DTOs** (Data Transfer Objects): contenedores planos para transferir datos entre capas o servicios.

Anti-patrón clásico: el «Anemic Domain Model» (Fowler) en el que las entidades son meros contenedores sin lógica y toda la inteligencia está en servicios. En MVC profesional las entidades tienen métodos significativos.

La Vista

La Vista es la capa de presentación. En aplicaciones web tradicionales suele ser una plantilla HTML con marcadores de sustitución:

- **Thymeleaf** (Java/Spring): plantillas HTML que son HTML válido incluso sin procesar (preview-friendly).
- **Blade** (PHP/Laravel): `@if, @foreach, {{ $variable }}`.
- **Razor** (C#/ASP.NET): mezcla `@` para C# inline.
- **Twig** (PHP/Symfony): sintaxis estilo Django, `{% if %} {{ var }}`.
- **Mustache, Handlebars** (cualquier lenguaje): *logic-less*.

En arquitecturas API REST + SPA, la Vista se desplaza al cliente (Angular, React, Vue), y el servidor solo expone JSON.

El Controlador

El Controller es la capa que recibe peticiones HTTP, invoca al Modelo y selecciona la Vista (o el formato de respuesta JSON). Sus responsabilidades:

- *Routing:* mapear URLs a métodos.
- *Binding:* convertir parámetros HTTP en objetos del modelo.
- *Validation:* validar entrada (a menudo delegada).
- *Orquestación:* llamar a servicios del modelo.
- *Selección de respuesta:* vista a renderizar o DTO a serializar.
- *Manejo de errores:* convertir excepciones en respuestas HTTP apropiadas.

Regla de oro: el controller debe ser *delgado* (*thin controller*). Toda lógica de negocio significativa va en el modelo (servicios). Un controller con 200 líneas es señal de mal diseño.

14.3 Ciclo de vida de una petición MVC en Spring Boot

Una petición HTTP atraviesa una secuencia precisa de componentes dentro de Spring Boot. Conocer ese flujo permite depurar problemas y optimizar puntos críticos.

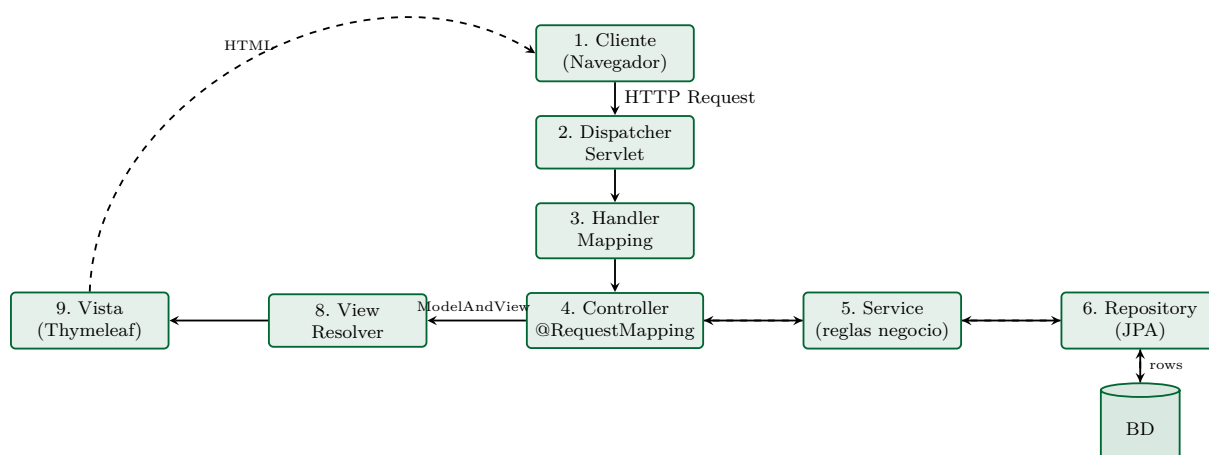


Figura 14.1: Ciclo de vida completo de una petición HTTP en Spring MVC. La ida (línea sólida): Dispatcher → HandlerMapping → Controller → Service → Repository → BD. El regreso (línea punteada): los datos suben hasta el Controller, que devuelve un `ModelAndView`; el ViewResolver elige la plantilla y el HTML final llega al cliente.

14.4 Server-side MVC vs SPA + API REST

En 2026 hay dos arquitecturas dominantes para aplicaciones web: MVC *server-side* clásico (Thymeleaf/Razor/Blade) y SPA + API REST (Angular/React + Spring/ASP.NET). La tabla siguiente las contrasta con sus trade-offs.

MVC server-side tradicional vs API REST + SPA Angular/React		
Criterio	MVC server-side	API REST + SPA
Ubicación de la vista	Servidor (Thymeleaf, Blade, Razor)	Cliente (Angular, React, Vue)
Renderizado inicial	Rápido (HTML completo del primer hit)	Lento (descarga JS, luego renderiza)
Tráfico tras la primera carga	Cada navegación = HTML completo	JSON pequeño
SEO	Excelente nativo	Requiere SSR/SSG
Complejidad del frontend	Baja (HTML + algo de JS)	Alta (framework completo)
Reactividad	Limitada (post-back o JS extra)	Nativa
Equipo	Un equipo full-stack	Frontend y backend separados
Mejor para	CMS, intranet, sitios contenido	SaaS, dashboards, apps tipo Gmail

14.4.1 ¿Cuál usar en el PFC?

El PFC de esta asignatura adopta la combinación **API REST (Spring Boot) + SPA Angular**. Es la apuesta arquitectónica moderna y es la que el sílabo prescribe. Sin embargo, comprender ambos enfoques es esencial: muchos sistemas reales coexisten (un panel admin con Razor + una API para móvil + un portal público con SSR).

14.5 Implementación canónica: MVC server-side completo

El ejemplo siguiente implementa un CRUD MVC server-side completo con Spring Boot 4 y Thymeleaf: el estudiante lo construye paso a paso y obtiene una aplicación funcional ejecutable.

Ejemplo 14.1 — CRUD MVC clásico en Spring Boot 4 + Thymeleaf

Enunciado. Construir una aplicación MVC server-side de gestión de tareas (to-do) con Spring Boot 4, Thymeleaf y H2. Las cuatro operaciones CRUD funcionarán end-to-end.

Estructura del proyecto (Listado 14.1):

```
src/main/java/ec/edu/uteq/mvc/
  controller/ProductoController.java    <- Controller
  service/ProductoService.java          <- Servicio (modelo)
  repository/ProductoRepository.java    <- Repositorio (modelo)
  model/Producto.java                   <- Entidad de dominio
  dto/ProductoForm.java                 <- DTO con validacion
src/main/resources/templates/productos/ <- Vistas
  lista.html
  formulario.html
```

Listado 14.1: Ejemplo de Shell

Entidad Producto:

```
1 package ec.edu.uteq.mvc.model; // paquete del archivo
2
3 import jakarta.persistence.*; // importacion de clase externa
4 import lombok.*; // importacion de clase externa
5 import java.math.BigDecimal; // importacion de clase externa
6
7 @Entity @Table(name = "productos") // entidad JPA (mapea a tabla)
8 @Getter @Setter @NoArgsConstructor @AllArgsConstructor
9 public class Producto { // clase publica
10     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) // campo
11         // clave primaria
12     private Long id; // campo privado
13     private String nombre; // campo privado
14     private BigDecimal precio; // campo privado
15     private Integer stock; // campo privado
16
17     /** Logica de dominio: el modelo NO es anemico. */
18     public boolean estaDisponible() { // metodo publico
19         return stock != null && stock > 0; // retorna
20     }
21
22     public BigDecimal valorInventario() { // metodo publico
23         // condicional
24         // condicional
25         if (precio == null || stock == null) return BigDecimal.ZERO;
26         return precio.multiply(BigDecimal.valueOf(stock)); // retorna
27     }
28 }
```

Listado 14.2: Producto.java

DTO con validación (Listado 14.3):

```

1 package ec.edu.uteq.mvc.dto; // paquete del archivo
2
3 import jakarta.validation.constraints.*; // importacion de clase
  externa
4 import lombok.*; // importacion de clase externa
5 import java.math.BigDecimal; // importacion de clase externa
6
7 @Getter @Setter @NoArgsConstructor @AllArgsConstructor
8 public class ProductoForm { // clase publica
9     private Long id; // campo privado
10
11     @NotBlank(message = "El nombre es obligatorio") // validacion: no
      nulo y no vacio
12     @Size(min = 3, max = 120) // validacion: longitud min/max
13     private String nombre; // campo privado
14
15     @NotNull @DecimalMin("0.00") @DecimalMax("99999.99") //
      validacion: no nulo
16     private BigDecimal precio; // campo privado
17
18     @NotNull @Min(0) // validacion: no nulo
19     private Integer stock; // campo privado
20 }

```

Listado 14.3: ProductoForm.java

Repository (Spring Data) (Listado 14.4):

```

1 package ec.edu.uteq.mvc.repository; // paquete del archivo
2
3 import ec.edu.uteq.mvc.model.Producto; // importacion de clase externa
4 // importacion de clase externa
5 // importacion de clase externa
6 import org.springframework.data.jpa.repository.JpaRepository;
7
8 public interface ProductoRepository // declaracion de interfaz
9     extends JpaRepository<Producto, Long> {}

```

Listado 14.4: ProductoRepository.java

Service (Modelo) (Listado 14.5):

```

1 package ec.edu.uteq.mvc.service; // paquete del archivo
2
3 import ec.edu.uteq.mvc.model.Producto; // importacion de clase externa
4 import ec.edu.uteq.mvc.repository.ProductoRepository; // importacion
  de clase externa
5 import lombok.RequiredArgsConstructor; // importacion de clase externa
6 import org.springframework.stereotype.Service; // importacion de clase
  externa
7 // importacion de clase externa
8 // importacion de clase externa

```

```

9  import org.springframework.transaction.annotation.Transactional;
10 import java.util.*; // importacion de clase externa
11
12 @Service // capa de servicio (logica de negocio)
13 @RequiredArgsConstructor
14 public class ProductoService { // clase publica
15
16     private final ProductoRepository repo; // immutable
17
18     @Transactional(readOnly = true) // envuelve el metodo en
19         // transaccion
20         // metodo publico
21     public List<Producto> listar() { return repo.findAll(); }
22
23     @Transactional(readOnly = true) // envuelve el metodo en
24         // transaccion
25     public Producto buscar(Long id) { // metodo publico
26         return repo.findById(id).orElseThrow( // retorna
27             () -> new NoSuchElementException("Producto " + id));
28     }
29
30     @Transactional // envuelve el metodo en transaccion
31         // metodo publico
32     public Producto guardar(Producto p) { return repo.save(p); }
33
34     @Transactional // envuelve el metodo en transaccion
35     public void eliminar(Long id) { repo.deleteById(id); } // metodo
36         // publico
37 }

```

Listado 14.5: ProductoService.java

Controller (Listado 14.6):

```

1  package ec.edu.uteq.mvc.controller; // paquete del archivo
2
3  import ec.edu.uteq.mvc.dto.ProductoForm; // importacion de clase
4      // externa
5  import ec.edu.uteq.mvc.model.Producto; // importacion de clase externa
6  import ec.edu.uteq.mvc.service.ProductoService; // importacion de
7      // clase externa
8  import jakarta.validation.Valid; // importacion de clase externa
9  import lombok.RequiredArgsConstructor; // importacion de clase externa
10 import org.springframework.beans.BeanUtils; // importacion de clase
11     // externa
12 import org.springframework.stereotype.Controller; // importacion de
13     // clase externa
14 import org.springframework.ui.Model; // importacion de clase externa
15 import org.springframework.validation.BindingResult; // importacion de
16     // clase externa

```

```

12 import org.springframework.web.bind.annotation.*; // importacion de
    clase externa
13 // importacion de clase externa
14 // importacion de clase externa
15 import org.springframework.web.servlet.mvc.support.RedirectAttributes;
16
17 @Controller // MVC (devuelve vistas)
18 @RequestMapping("/productos") // ruta base del controlador
19 @RequiredArgsConstructor
20 public class ProductoController { // clase publica
21
22     private final ProductoService service; // immutable
23
24     /** GET /productos --> lista */
25     @GetMapping // endpoint HTTP GET
26     public String listar(Model model) { // metodo publico
27         model.addAttribute("productos", service.listar());
28         return "productos/lista"; // -> templates/productos/lista.html
29     }
30
31     /** GET /productos/nuevo --> formulario vacio */
32     @GetMapping("/nuevo") // endpoint HTTP GET
33     public String mostrarFormulario(Model model) { // metodo publico
34         model.addAttribute("form", new ProductoForm());
35         return "productos/formulario"; // retorna
36     }
37
38     /** POST /productos --> guardar */
39     @PostMapping // endpoint HTTP POST (crear)
40     // metodo publico
41     // metodo publico
42     public String guardar(@Valid @ModelAttribute("form") ProductoForm
        form,
43
44         BindingResult br,
45         RedirectAttributes ra) {
46         if (br.hasErrors()) { // condicional
47             // re-renderizar el formulario con los errores
48             return "productos/formulario"; // retorna
49         }
50         Producto p = new Producto();
51         BeanUtils.copyProperties(form, p);
52         service.guardar(p);
53         ra.addFlashAttribute("mensaje", "Producto guardado");
54         return "redirect:/productos"; // patron PRG anti F5
55     }
56
57     /** GET /productos/{id}/editar */
58     @GetMapping("/{id}/editar") // endpoint HTTP GET
59     // metodo publico
60     // metodo publico
61     public String editar(@PathVariable Long id, Model model) {

```

```

61     Producto p = service.buscar(id);
62     ProductoForm form = new ProductoForm();
63     BeanUtils.copyProperties(p, form);
64     model.addAttribute("form", form);
65     return "productos/formulario"; // retorna
66 }
67
68 /** POST /productos/{id}/eliminar */
69 @PostMapping("/{id}/eliminar") // endpoint HTTP POST (crear)
70 // metodo publico
71 // metodo publico
72 public String eliminar(@PathVariable Long id, RedirectAttributes
73     ra) {
74     service.eliminar(id);
75     ra.addFlashAttribute("mensaje", "Producto eliminado");
76     return "redirect:/productos"; // retorna
77 }

```

Listado 14.6: ProductoController.java

Vista Thymeleaf de listado (Listado 14.7):

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org" lang="es-EC">
3 <head>
4     <meta charset="UTF-8">
5     <title>Productos UTEQ</title>
6     <link rel="stylesheet" href="/css/app.css">
7 </head>
8 <body>
9 <header>
10     <h1>Catalogo de productos</h1>
11     <a th:href="@{/productos/nuevo}" class="btn">Nuevo producto</a>
12 </header>
13
14 <div th:if="${mensaje}" class="alerta-exito"
15     th:text="${mensaje}"></div>
16
17 <table>
18     <thead>
19         <tr><th>ID</th><th>Nombre</th><th>Precio</th>
20         <th>Stock</th><th>Disponible</th><th>Acciones</th></tr>
21     </thead>
22     <tbody>
23         <tr th:each="p : ${productos}">
24             <td th:text="${p.id}"></td>
25             <td th:text="${p.nombre}"></td>
26             <td th:text="${'$' + #numbers.formatDecimal(p.precio,1,2)}"></td>
27             <td th:text="${p.estaDisponible() ? 'Si' : 'No'}"></td>
28         </tr>

```

```

29         <a th:href="@{/productos/{id}/editar(id=${p.id})}">Editar</a>
30         <form th:action="@{/productos/{id}/eliminar(id=${p.id})}"
31             method="post" style="display:inline">
32             <button type="submit"
33                 onclick="return confirm('Confirmar eliminacion?')">
34                 Eliminar
35             </button>
36         </form>
37     </td>
38 </tr>
39 </tbody>
40 </table>
41 </body>
42 </html>

```

Listado 14.7: templates/productos/lista.html

Vista del formulario (Listado 14.8):

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org" lang="es-EC">
3 <head><meta charset="UTF-8"><title>Producto</title></head>
4 <body>
5 <h1 th:text="${form.id == null ? 'Nuevo producto' : 'Editar
6     producto'}">
7 </h1>
8 <form th:action="@{/productos}" th:object="${form}" method="post">
9     <input type="hidden" th:field="*{id}">
10
11     <div>
12         <label>Nombre</label>
13         <input type="text" th:field="*{nombre}" required>
14         <span th:errors="*{nombre}" class="error"></span>
15     </div>
16
17     <div>
18         <label>Precio</label>
19         <input type="number" step="0.01" th:field="*{precio}" required>
20         <span th:errors="*{precio}" class="error"></span>
21     </div>
22
23     <div>
24         <label>Stock</label>
25         <input type="number" th:field="*{stock}" min="0" required>
26         <span th:errors="*{stock}" class="error"></span>
27     </div>
28
29     <button type="submit">Guardar</button>
30     <a th:href="@{/productos}">Cancelar</a>
31 </form>
32 </body>

```

```
33 </html>
```

Listado 14.8: templates/productos/formulario.html

Notas pedagógicas sobre este código:

- *Patrón PRG (Post-Redirect-Get)*: tras un POST exitoso se hace `redirect:/productos`, no se renderiza directamente la vista. Esto evita el bug clásico de doble envío al pulsar F5.
- *Flash attributes*: `RedirectAttributes` permite pasar mensajes entre el redirect y la vista sin contaminar la URL.
- *ProductoForm* en lugar de `Producto` en el controller: separa la representación de la entidad y la del formulario; permite validaciones específicas de UI.
- *El modelo no es anémico*: `estaDisponible()` es una regla del dominio.
- *Thymeleaf-natural*: las plantillas son HTML válido que se puede abrir en navegador sin servidor (modo preview).

14.6 Variantes del MVC

MVC puro no es la única opción: en la historia han surgido variantes con ventajas específicas (MVP, MVVM, MVI). La tabla siguiente las catalogará y dirá cuándo usar cada una.

14.6.1 MVP (Model-View-Presenter)

En MVP el Presenter reemplaza al Controller pero acopla más fuertemente con la Vista. La Vista delega completamente la lógica de presentación al Presenter. Común en aplicaciones Android pre-MVVM.

14.6.2 MVVM (Model-View-ViewModel)

En MVVM, el ViewModel es una abstracción de la vista que mantiene su estado. La vista se enlaza al ViewModel por *data binding* bidireccional. Angular y Vue son herederos parciales de este enfoque.

14.6.3 Flux y Redux

Patrones unidireccionales para SPAs: Action → Dispatcher → Store → View. Resuelven el problema de complejidad del estado en SPAs grandes (Facebook lo creó para resolver bugs en Messenger).

14.6.4 Signals (Angular 17+, SolidJS)

Modelo reactivo basado en *signals*: variables observables que notifican automáticamente cambios. Una evolución moderna del MVC reactivo original de Reenskaug, ahora aplicada al frontend.

14.7 Práctica integrada: CRUD MVC con debugging

La práctica integrada de la semana 14 implementa el CRUD MVC trazando explícitamente el flujo de cada petición, para que el estudiante visualice cómo trabaja el framework por debajo.

Ejercicio 14.1 — CRUD MVC completo con trazado de la petición en IntelliJ

Objetivo: construir un CRUD MVC server-side y aprender a *depurar* cada fase del ciclo de vida de la petición usando los breakpoints y el inspector de IntelliJ IDEA Ultimate.

IDE: IntelliJ IDEA Ultimate

IntelliJ IDEA Ultimate provee herramientas específicas para trazar el flujo MVC (debugger con puntos de quiebre por capas, consola Spring, herramienta HTTP Client). Esta práctica las usa todas.

Paso 1 — Crear el proyecto

File → New → Project → Spring Boot. Configurar:

- Type: Maven, JDK: 21, Spring Boot 4.x.
- Group: ec.edu.uteq; Artifact: mvc-debug.
- Dependencies: *Spring Web, Thymeleaf, Spring Data JPA, H2 Database, Lombok, Validation, DevTools.*

Paso 2 — Implementar el CRUD del Ejemplo 14.1

Copiar la estructura completa del ejemplo 14.1 (entidad, DTO, service, controller, dos vistas Thymeleaf).

Agregar en application.yml (Listado 14.9):

```
spring: # configuracion de Spring
  datasource: # configuracion de origen de datos (BD)
    url: jdbc:h2:mem:appweb;DB_CLOSE_DELAY=-1 # URL JDBC de conexion a
        la BD
    driver-class-name: org.h2.Driver # clase del driver JDBC
    username: sa # usuario de la BD
    password: # contrasena (mejor leer de env var)
  jpa: # configuracion JPA/Hibernate
    hibernate.ddl-auto: create-drop # estrategia de generacion de
        schema
    show-sql: true # imprime SQL en consola (solo desarrollo)
    properties.hibernate.format_sql: true
  h2.console:
    enabled: true
    path: /h2 # ruta donde se expone
```

Listado 14.9: Ejemplo de YAML

Paso 3 — Inicializar datos de prueba

Crear DatosIniciales.java (Listado 14.10):

```
@Component // bean Spring generico
public class DatosIniciales implements CommandLineRunner { // clase
    publica
    private final ProductoRepository repo; // immutable
    // metodo publico
    // metodo publico
    public DatosIniciales(ProductoRepository r) { this.repo = r; }
    @Override
    public void run(String... args) { // metodo publico
        repo.save(new Producto(null, "Cafe galletti", new
            BigDecimal("4.50"), 30));
        repo.save(new Producto(null, "Cacao Pacari", new
            BigDecimal("6.00"), 5));
        repo.save(new Producto(null, "Te organico", new
            BigDecimal("3.20"), 12));
    }
}
```

Listado 14.10: Ejemplo de Java

Paso 4 — Ejecutar la aplicación

Pulsar el botón verde de *play* junto a la clase principal. Esperar el log `Started McvDebugApplication`. Abrir <http://localhost:8080/productos>: aparece el listado con los tres productos iniciales.

Paso 5 — Colocar 9 breakpoints estratégicos

Para entender el ciclo de vida, colocar breakpoints (clic en el margen izquierdo del editor) en:

1. `ProductoController.listar()` — línea con `model.addAttribute`.
2. `ProductoService.listar()` — línea con `return repo.findAll()`;
3. `ProductoController.mostrarFormulario()`.
4. `ProductoController.guardar()` — línea con `br.hasErrors()`.
5. `ProductoController.guardar()` — línea con `service.guardar(p)`;
6. `ProductoService.guardar(Producto)` — la línea con `repo.save(p)`;
7. `ProductoController.editar()`.
8. `ProductoController.eliminar()`.
9. `ProductoForm` — la línea de la anotación `@NotBlank` (para ver el orden de validación).

Paso 6 — Reiniciar en modo Debug

Detener la app. Pulsar el ícono *bug* (`Shift+F9`) para arrancar en modo debug. La app está lista para depurar.

Paso 7 — Trazar una petición GET (listar)

Con la aplicación corriendo, el estudiante debe trazar una petición GET completa observando cómo se mueve por cada capa.

1. En el navegador, recargar <http://localhost:8080/productos>.
2. IntelliJ se detiene en breakpoint 1 (`listar()`).
3. Observar el panel *Variables*: aparece `model` (vacío) y los parámetros del método.
4. Pulsar F8 (Step Over): se ejecuta `model.addAttribute(..., service.listar())`.
5. Pulsar F7 (Step Into) en `service.listar()`: salta al breakpoint 2.
6. En *Variables* ver que `repo` es un proxy de Spring.
7. Pulsar F8: se ejecuta `repo.findAll()`; en la consola aparece el SQL: `select * from productos`.
8. Pulsar F9 (Resume): la petición continúa, Thymeleaf renderiza la vista, el navegador recibe el HTML.

Paso 8 — Trazar una petición POST con error de validación

El segundo trazado es una petición POST con datos inválidos: el objetivo es observar cómo `BindingResult` captura los errores y la vista los muestra.

1. Ir a <http://localhost:8080/productos/nuevo>.
2. Llenar solo el campo «Stock» (dejar nombre y precio vacíos).
3. Pulsar Guardar.
4. IntelliJ se detiene en breakpoint 4 (`br.hasErrors()`).
5. Inspeccionar `br`: el panel *Variables* muestra una lista de errores (`nombre: NotBlank, precio: NotNull`).
6. Pulsar F8: como hay errores, retorna `"productos/formulario"` con los errores marcados en la vista.

Paso 9 — Trazar un POST exitoso

El tercer trazado es un POST exitoso: observa cómo se guarda en BD y cómo se aplica el patrón POST-Redirect-GET para evitar reenvíos del formulario.

1. Recargar el formulario; ahora llenar todos los campos correctamente.
2. Pulsar Guardar.
3. IntelliJ se detiene en breakpoint 4 (`br.hasErrors()`).

4. `br.hasErrors()` es `false`; pulsar F8.
5. Salta al breakpoint 5: a punto de ejecutar `service.guardar(p)`.
6. Pulsar F7 (Step Into): salta al breakpoint 6 (`repo.save(p)`).
7. En *Variables*, observar que `p.id` es `null` ANTES del `save`.
8. Pulsar F8: `p.id` ahora tiene un valor (la BD asignó la clave primaria).
9. En la consola, ver el `INSERT INTO productos` generado.
10. Pulsar F9: la respuesta es un redirect a `/productos`, el navegador hace un GET nuevo, listamos todos los productos (vuelve a parar en breakpoint 1).

Paso 10 — Inspeccionar la BD H2 en vivo

La consola H2 permite inspeccionar la BD en tiempo real durante la depuración. Esto es invaluable para verificar que los datos llegaron a persistirse correctamente.

1. Sin detener la app, abrir <http://localhost:8080/h2> en otra pestaña.
2. En *JDBC URL* introducir: `jdbc:h2:mem:appweb`. User: `sa`, sin password.
3. Pulsar Connect.
4. Ejecutar `SELECT * FROM PRODUCTOS`; aparecen los productos iniciales más los que se hayan creado.

Resultado esperado

El estudiante puede:

- Articular el flujo: `Controller` → `Service` → `Repository` → `BD` → `Service` → `Controller` → `Vista`.
- Identificar dónde ocurre la validación (`@Valid` + `BindingResult`).
- Explicar el patrón PRG y por qué se usa.
- Distinguir Modelo (`Producto/Service/Repository`) de Vista (`Thymeleaf`) de `Controller` (handler HTTP).
- Ver el SQL generado por JPA en tiempo real en la consola.

Reflexión final escrita

Responder en máximo 250 palabras: ¿Qué pasaría si el controller llamara directamente al `ProductoRepository` sin pasar por el `ProductoService`? ¿Qué se rompería? ¿Qué clases serían más difíciles de testear?

Esta práctica es la *Sumativa GA-Práctica en clase* de la semana 14.

14.8 Buenas prácticas profesionales en MVC

La caja siguiente recoge las reglas profesionales para escribir MVC mantenible. Aplican tanto a Spring Boot como a ASP.NET Core y Laravel.

Quince reglas profesionales 2026

1. Controllers *thin*: máximo 50 líneas por método de controller, idealmente 10-20.
2. Toda la lógica de negocio en servicios, no en controllers ni en vistas.
3. DTOs separados de entidades JPA: nunca exponer entidades directamente al cliente.
4. Validación con Bean Validation (`@NotBlank`, `@Size`, `@Email`) sobre validación manual.
5. Patrón PRG (Post-Redirect-Get) para evitar reenvíos al pulsar F5.
6. Flash attributes para mensajes de éxito/error en redirects.
7. Manejo global de excepciones con `@ControllerAdvice`/`@RestControllerAdvice`.
8. Vistas (templates) sin lógica compleja: solo iteración, condicionales simples y formato.
9. URLs RESTful: `/productos` para listar, `/productos/123` para uno específico, métodos HTTP semánticos.
10. Repositorios definidos como interfaces (Spring Data) para facilitar tests.
11. Servicios anotados `@Transactional` en los métodos que mutan datos.
12. Logging estructurado en cada capa: controller (acceso), servicio (negocio), repositorio (datos).

13. Tests unitarios de servicio con mock de repositorio; tests de integración con MockMvc para controllers.
14. Documentación de la API REST con OpenAPI/Swagger.
15. Internacionalización (i18n) desde el inicio: archivos `messages_es.properties`, `messages_en.properties`.

14.9 Contraste bibliográfico de los conceptos clave

La separación de responsabilidades que propone MVC se ha discutido profundamente en la literatura. [23] es la referencia clásica para entender MVC del lado servidor en ASP.NET. [67] y [63] muestran cómo Spring MVC implementa el patrón con `DispatcherServlet`, `HandlerMapping` y `ViewResolver`. En el ámbito del frontend, las arquitecturas modernas (Angular, React) han evolucionado hacia variantes *component-based* que rompen la estricta tripartición MVC pero mantienen el principio de separación [57]. [58] documenta cómo el patrón *API REST + SPA* es hoy el reemplazo dominante del MVC server-side tradicional, con tres ventajas: equipos desacoplados (frontend/backend), reutilización del backend para varios clientes (web, móvil, IoT) y mejor experiencia interactiva.

14.10 Ejercicio resuelto adicional con resultado visual

Como cierre, el siguiente ejercicio resuelto adicional implementa un CRUD completo con Thymeleaf y se complementa con un propuesto que lo migra a SPA Angular.

Ejercicio resuelto 14.1 — App MVC de tareas con Spring Boot y Thymeleaf

Enunciado. Implementar una aplicación MVC server-side de gestión de tareas (*to-do list*) con Spring Boot 4, Thymeleaf y H2. Permitir listar, crear, completar y eliminar tareas.

Modelo (Tarea.java):

Listado 14.11: Ejemplo de Java

```
@Entity @Table(name="tareas") // entidad JPA (mapea a tabla)
public class Tarea { // clase publica
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY) // campo clave primaria
    private Long id; // campo privado
    @NotBlank @Size(max=120) private String titulo; // validacion: no nulo y no vacio
    private boolean completada; // campo privado
    // getters/setters
}
```

Controlador (TareaController.java):

Listado 14.12: Ejemplo de Java

```
@Controller @RequestMapping("/tareas") // MVC (devuelve vistas)
public class TareaController { // clase publica
    private final TareaRepository repo; // immutable
    // metodo publico
    // metodo publico
    public TareaController(TareaRepository r) { this.repo = r; }

    @GetMapping // endpoint HTTP GET
    public String listar(Model m) { // metodo publico
        m.addAttribute("tareas", repo.findAll());
        m.addAttribute("nueva", new Tarea());
        return "tareas"; // retorna
    }

    @PostMapping // endpoint HTTP POST (crear)
    // metodo publico
    // metodo publico
    public String crear(@Valid @ModelAttribute("nueva") Tarea t,
```

```

        BindingResult br) {
    if (br.hasErrors()) return "tareas"; // condicional
    repo.save(t);
    return "redirect:/tareas"; // retorna
}

@PostMapping("/{id}/completar") // endpoint HTTP POST (crear)
public String completar(@PathVariable Long id) { // metodo publico
    repo.findById(id).ifPresent(t -> {
        t.setCompletada(!t.isCompletada());
        repo.save(t);
    });
    return "redirect:/tareas"; // retorna
}

@PostMapping("/{id}/eliminar") // endpoint HTTP POST (eliminar)
public String eliminar(@PathVariable Long id) { // metodo publico
    repo.deleteById(id);
    return "redirect:/tareas"; // retorna
}
}

```

Vista (templates/tareas.html — Thymeleaf):

Listado 14.13: Ejemplo de HTML

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head><meta charset="UTF-8"><title>Mis tareas</title></head>
<body>
    <h1>Mis tareas</h1>
    <form th:action="@{/tareas}" th:object="${nueva}" method="post">
        <input type="text" th:field="*{titulo}" placeholder="Nueva tarea">
        <button type="submit">Agregar</button>
        <span th:errors="*{titulo}" style="color:red"></span>
    </form>
    <ul>
        <li th:each="t : ${tareas}">
            <span th:text="${t.titulo}"
                  th:style="${t.completada}?'text-decoration: line-through' : ''"></span>
            <form th:action="@{|/tareas/${t.id}/completar|}"
                  method="post" style="display:inline">
                <button type="submit"
                      th:text="${t.completada}?'Reabrir' : 'Completar' "></button>
            </form>
            <form th:action="@{|/tareas/${t.id}/eliminar|}"
                  method="post" style="display:inline">
                <button type="submit">Eliminar</button>
            </form>
        </li>
    </ul>
</body>
</html>

```

Resultado visual en el navegador:

Mis tareas

Nueva tarea.....

- Leer cap. 14 del libro
- Estudiar Spring MVC
- Hacer el ejercicio propuesto 14.2

Discusión. El flujo MVC completo se ve en una sola pantalla: el navegador hace `GET /tareas` → `DispatcherServlet` despacha a `TareaController.listar()` → el controlador consulta el repositorio (modelo) y prepara los datos en el `Model` → Thymeleaf renderiza la vista con esos datos → el navegador recibe HTML. Al hacer clic en *Completar*, ocurre el patrón *POST-Redirect-GET* [67]: `POST` ejecuta la acción y el servidor responde `302 Found` con `Location: /tareas`, forzando un nuevo `GET`. Esto evita el reenvío accidental del formulario al recargar.

Ejercicio propuesto 14.2 — Migrar la app de tareas a SPA Angular + REST

Enunciado. Tomar la aplicación MVC server-side del ejercicio 14.1 y refactorizarla a una arquitectura de dos niveles: backend Spring Boot expone una API REST y un frontend Angular 19 SPA consume la API. Comparar las dos arquitecturas en términos de líneas de código, acoplamiento, experiencia de usuario y testabilidad.

Requisitos:

1. Convertir `TareaController` de `@Controller` a `@RestController`, eliminando Thymeleaf.
2. Endpoints: `GET /api/v1/tareas`, `POST /api/v1/tareas`, `PATCH /api/v1/tareas/{id}/completar`, `DELETE /api/v1/tareas/{id}`.
3. Frontend Angular con un componente `TareasComponent` que consuma la API mediante `HttpClient`.
4. Manejar CORS correctamente con `@CrossOrigin("http://localhost:4200")` o configuración global.
5. Documentación OpenAPI con springdoc.
6. Test E2E con Playwright o Cypress que verifique el flujo completo (crear, completar, eliminar).

Producto final: un documento (1–2 páginas) que compare las dos arquitecturas y argumente cuándo elegir cada una, citando al menos a [23] y [58].

Servicios web: arquitecturas REST, SOAP y diseño de APIs

«Una buena API es como una buena novela: invita a explorarla, revela su lógica gradualmente y nunca traiciona al lector con inconsistencias arbitrarias.»

— Adaptado de Joshua Bloch

Resultado de aprendizaje de la semana

Al concluir la semana 15, el estudiante diseña, implementa, documenta y consume servicios web aplicando el estilo arquitectónico REST según Roy Fielding [17], distingue REST de SOAP en sus casos de uso reales, integra autenticación con JWT y produce documentación OpenAPI 3.0 conforme a estándares profesionales de la industria 2026.

15.1 Historia de los servicios web

Los servicios web son la forma estándar de comunicación entre sistemas distintos. Su historia atraviesa tres generaciones: SOAP (1998), REST (2000) y GraphQL/gRPC (2015). Conocerlas permite elegir adecuadamente para cada proyecto.

De RPC a REST — treinta años de evolución de las APIs distribuidas

La idea de que un programa pueda invocar funciones de otro a través de la red precede a la web. En 1981, Bruce Nelson y Andrew Birrell formalizaron el *Remote Procedure Call* (RPC) en el seminario del que surgió la primera implementación seria, Sun RPC. El sueño: hacer que llamar a una función remota sea sintácticamente idéntico a llamar a una función local.

A finales de los noventa, ese sueño se reformuló en la web. Microsoft junto con DevelopMentor publicaron en 1998 el primer borrador de **SOAP** (Simple Object Access Protocol): RPC sobre HTTP usando XML como formato de mensaje. SOAP se complementó con **WSDL** (Web Services Description Language) en 2001 y con **UDDI** para descubrimiento de servicios. La trinidad SOAP+WSDL+UDDI prometía un ecosistema empresarial donde cualquier servicio sería descubrible y consumible automáticamente.

La realidad fue más áspera. SOAP era verboso (envoltorios XML enormes), el ecosistema de *WS-** (WS-Security, WS-ReliableMessaging, WS-Transaction...) creció hasta volverse impracticable, y los clientes JavaScript del navegador no podían consumirlo elegantemente.

En 2000 **Roy Fielding** —uno de los autores principales de HTTP/1.1 y cofundador de Apache— defendió en su tesis doctoral el estilo arquitectónico **REST** (Representational State Transfer) [17]. REST no era un protocolo: era un conjunto de *restricciones arquitectónicas* (cliente-servidor, stateless, cacheable, interfaz uniforme, sistema en capas, código bajo demanda opcional) que aprovechaban HTTP en lugar de pelearse con él. La trayectoria a

partir de 2005 fue rápida: las APIs públicas masivas (Flickr, Twitter, Amazon S3, GitHub) eligieron REST. Para 2015, REST era el estilo dominante en la web. SOAP quedó relegado a sistemas empresariales legados y a sectores conservadores como banca y salud.

En 2026 conviven cuatro paradigmas de servicios web: **REST** (dominante en APIs públicas y SPAs), **GraphQL** (creado por Facebook en 2012, popular en frontends complejos), **gRPC** (Google, 2015, eficiente en comunicación servicio-servicio) y **SOAP** (legado pero vigente en banca, salud, gobierno).

15.1.1 Tabla evolutiva de los servicios web

La tabla siguiente recoge los hitos cronológicos de los servicios web, desde XML-RPC hasta GraphQL y gRPC.

Tabla 15.1: Hitos en la historia de los servicios web.

Año	Hito	Aporte
1981	Sun RPC	Primer RPC industrial.
1991	CORBA	Estándar OMG; complejo, propietario.
1998	SOAP 1.0	RPC sobre HTTP+XML; Microsoft, IBM.
2000	Tesis de Fielding	REST formalizado como estilo arquitectónico.
2001	WSDL 1.1	Contrato XML para SOAP.
2003	SOAP 1.2 (W3C)	Recomendación oficial de SOAP.
2005	Atom Publishing Protocol	Primer estándar REST formal.
2008	RESTful API patterns	Modelo de Madurez de Richardson.
2009	Twitter API REST	APIs públicas masivas adoptan REST.
2012	GraphQL (Facebook)	Cliente decide qué campos pedir.
2014	OpenAPI 2.0 (Swagger)	Estándar para describir APIs REST.
2015	gRPC (Google)	RPC moderno con HTTP/2 y Protocol Buffers.
2017	OpenAPI 3.0	Versión moderna del estándar.
2021	OpenAPI 3.1	Compatibilidad total con JSON Schema 2020-12.
2024	gRPC-Web estable	gRPC accesible desde navegador.

15.2 Las seis restricciones de la arquitectura REST

Roy Fielding identificó seis restricciones que un sistema debe cumplir para considerarse RESTful. No son sugerencias: son restricciones arquitectónicas con consecuencias específicas.

Las restricciones REST de Fielding (2000)

1. **Cliente-servidor.** Separación clara de responsabilidades: el cliente no conoce la persistencia, el servidor no conoce la presentación.
2. **Stateless.** El servidor NO guarda estado de sesión entre peticiones. Cada petición lleva toda la información necesaria para procesarla. Permite escalar horizontalmente sin sticky sessions.
3. **Cacheable.** Las respuestas indican explícitamente si pueden cachearse y por cuánto tiempo (**Cache-Control**, **ETag**). Mejora rendimiento y escalabilidad.
4. **Interfaz uniforme.** Cuatro sub-restricciones:
 - Identificación de recursos por URI.
 - Manipulación a través de representaciones (JSON, XML).
 - Mensajes auto-descriptivos (cabeceras explícitas).
 - HATEOAS (Hypermedia as the Engine of Application State): los recursos enlazan a otros relacionados.
5. **Sistema en capas.** El cliente no sabe si habla con el servidor final o con un proxy/balanceador. Permite intermediarios (CDN, gateways, balanceadores).
6. **Código bajo demanda** (opcional). El servidor puede extender al cliente enviando código ejecutable (típicamente JavaScript).

15.2.1 Modelo de Madurez de Richardson

Leonard Richardson propuso en 2008 un modelo escalonado para evaluar cuán RESTful es una API:

Tabla 15.2: Modelo de Madurez de Richardson para APIs REST.

Nivel	Nombre	Características
0	«The Swamp of POX»	XML/JSON sobre HTTP, un solo endpoint, todo es POST.
1	Recursos	URLs distintas por recurso (/usuarios/42), pero todo POST.
2	Verbos HTTP	Uso semántico de GET, POST, PUT, DELETE; códigos correctos.
3	HATEOAS	Las respuestas incluyen enlaces de navegación a recursos relacionados.

En 2026, la mayoría de APIs profesionales se sitúan en Nivel 2. El nivel 3 (HATEOAS) es elegante en teoría pero infrecuente en la práctica: añade complejidad sin que la mayoría de clientes lo aproveche realmente.

15.3 Diseño de URIs y verbos HTTP

La definición de las URIs y el uso de los verbos HTTP son las dos decisiones más visibles de una API REST. Hacerlas mal se nota desde el primer endpoint.

15.3.1 Convenciones de nomenclatura

Las URIs de una API REST siguen convenciones precisas: son sustantivos en plural, sin verbos, jerárquicas. La lista siguiente las recoge con ejemplos.

Ejemplo corto comentado (URIs REST canónicas) (Listado 15.1):

Listado 15.1: Ejemplo de Shell

```
# BIEN: sustantivo en plural, jerarquía clara
GET    /api/v1/usuarios # listar todos
GET    /api/v1/usuarios/42 # uno específico
POST   /api/v1/usuarios # crear nuevo
PUT    /api/v1/usuarios/42 # actualizar completo
PATCH /api/v1/usuarios/42 # actualizar parcial
DELETE /api/v1/usuarios/42 # eliminar
GET    /api/v1/usuarios/42/pedidos # recurso anidado

# MAL: verbos en URI, singular, sin versión
GET    /getUsuario?id=42 # verbo en URI
POST   /crearUsuario # acción como recurso
GET    /usuario/42 # singular en lugar de plural
```

Doce reglas para URIs RESTful profesionales

1. **Sustantivos en plural** para colecciones: /productos, no /getProducto.
2. **Recursos jerárquicos**: /usuarios/42/pedidos/7/items.
3. **Sin verbos en la URI**: el verbo va en el método HTTP.
4. **Minúsculas siempre**, palabras separadas por guiones: /ordenes-pendientes, no /OrdenesPendientes.
5. **Sin extensiones de archivo**: /usuarios/42, no /usuarios/42.json.
6. **Versionado en la URL** con prefijo /api/v1/: /api/v1/productos.
7. **Filtros por query string**: /productos?categoria=cafe&min_precio=2.
8. **Paginación estándar**: ?page=3&size=20 o ?offset=60&limit=20.
9. **Ordenamiento por query**: ?sort=precio,-nombre (- para descendente).
10. **Búsqueda con parámetro q**: ?q=galletti.
11. **IDs estables** (UUIDs preferidos sobre auto-incrementales en APIs públicas).
12. **Sub-recursos para acciones**: /pedidos/7/cancelar con POST.

15.3.2 Mapeo verbo-acción canónico

Cada verbo HTTP tiene un significado canónico y debe mapearse a la acción CRUD correspondiente. Confundirlos rompe la semántica REST y genera APIs confusas.

Tabla 15.3: Mapeo verbos HTTP a operaciones CRUD.

Verbo	URI	Operación	Códigos típicos
GET	/productos	Listar	200 OK
GET	/productos/42	Obtener uno	200, 404
POST	/productos	Crear	201 Created (con <i>Location</i>)
PUT	/productos/42	Reemplazar completo	200, 204
PATCH	/productos/42	Actualización parcial	200, 204
DELETE	/productos/42	Eliminar	204 No Content, 404
HEAD	/productos/42	Solo cabeceras	200, 404
OPTIONS	/productos	Verbos soportados	200 (CORS)

15.3.3 Idempotencia y seguridad

Dos propiedades cruciales de los métodos HTTP:

- **Seguro** (*safe*): no modifica estado del servidor. GET, HEAD, OPTIONS son seguros.
- **Idempotente**: ejecutar la operación N veces produce el mismo efecto que ejecutarla una vez. GET, PUT, DELETE, HEAD, OPTIONS son idempotentes. POST y PATCH *no* lo son.

Los *proxies* y reintentos automáticos asumen estas propiedades: si su API rompe estas reglas, los clientes sufrirán bugs sutiles.

15.4 Implementación de APIs REST en las tres plataformas

Las tres plataformas del curso (Spring Boot, ASP.NET Core, PHP) construyen APIs REST con APIs limpias y modernas. Las subsecciones siguientes muestran la implementación canónica en cada una.

15.4.1 Spring Boot 4 con Spring Web

Spring Boot expone APIs REST con `@RestController` y anotaciones por método HTTP. Es la forma idiomática en el ecosistema Java.

Ejemplo 15.1 — API REST de pedidos completa en Spring Boot

Enunciado. Implementar una API REST de productos con Spring Boot, autenticación JWT, documentación OpenAPI 3.0 y Swagger UI interactivo.

```

1 package ec.edu.uteq.appweb.controller; // paquete del archivo
2
3 import ec.edu.uteq.appweb.dto.*; // importacion de clase externa
4 import ec.edu.uteq.appweb.service.PedidoService; // importacion de
   clase externa
5 import io.swagger.v3.oas.annotations.Operation; // importacion de
   clase externa
6 import io.swagger.v3.oas.annotations.tags.Tag; // importacion de clase
   externa
7 import jakarta.validation.Valid; // importacion de clase externa
8 import lombok.RequiredArgsConstructor; // importacion de clase externa
9 import org.springframework.data.domain.Page; // importacion de clase
   externa

```

```

10 import org.springframework.data.domain.Pageable; // importacion de
    clase externa
11 import org.springframework.http.ResponseEntity; // importacion de
    clase externa
12 import org.springframework.web.bind.annotation.*; // importacion de
    clase externa
13 import java.net.URI; // importacion de clase externa
14
15 @RestController // expone endpoints REST con JSON
16 @RequestMapping("/api/v1/pedidos") // ruta base del controlador
17 @RequiredArgsConstructor
18 // agrupa endpoints en Swagger UI
19 // agrupa endpoints en Swagger UI
20 @Tag(name = "Pedidos", description = "Gestion de pedidos del sistema")
21 public class PedidoController { // clase publica
22
23     private final PedidoService service; // immutable
24
25     @GetMapping // endpoint HTTP GET
26     @Operation(summary = "Lista paginada de pedidos") // OpenAPI
27     public Page<PedidoDto> listar( // metodo publico
28         @RequestParam(required = false) String estado, // query
29         // param
30         Pageable pageable) {
31         return service.buscar(estado, pageable); // retorna
32     }
33
34     @GetMapping("/{id}") // endpoint HTTP GET
35     @Operation(summary = "Obtener pedido por ID") // OpenAPI
36     // metodo publico
37     // metodo publico
38     public ResponseEntity<PedidoDto> obtener(@PathVariable Long id) {
39         return ResponseEntity.ok(service.obtener(id)); // retorna
40     }
41
42     @PostMapping // endpoint HTTP POST (crear)
43     @Operation(summary = "Crear nuevo pedido") // OpenAPI
44     public ResponseEntity<PedidoDto> crear( // metodo publico
45         @Valid @RequestBody PedidoCreateRequest req) { // activa
46         // validacion Bean Validation
47         PedidoDto creado = service.crear(req);
48         return ResponseEntity // retorna
49             .created(URI.create("/api/v1/pedidos/" + creado.id()))
50             .body(creado);
51     }
52
53     @PatchMapping("/{id}") // endpoint HTTP PATCH (actualizar parcial)
54     @Operation(summary = "Actualizacion parcial") // OpenAPI
55     public PedidoDto actualizar(@PathVariable Long id, // metodo
56         // publico
57         // activa validacion Bean Validation

```

```

55         // activa validacion Bean Validation
56         @Valid @RequestBody
57         PedidoUpdateRequest req) {
58     }
59
60     @PostMapping("/{id}/cancelar") // endpoint HTTP POST (crear)
61     @Operation(summary = "Cancelar un pedido") // OpenAPI
62     public PedidoDto cancelar(@PathVariable Long id, // metodo publico
63                               // query param
64                               // query param
65                               @RequestParam(required = false) String
66                               motivo) {
67         return service.cancelar(id, motivo); // retorna
68     }
69
70     @DeleteMapping("/{id}") // endpoint HTTP DELETE (eliminar)
71     @Operation(summary = "Eliminar pedido") // OpenAPI
72     // metodo publico
73     // metodo publico
74     public ResponseEntity<Void> eliminar(@PathVariable Long id) {
75         service.eliminar(id);
76         return ResponseEntity.noContent().build(); // retorna
77     }

```

Listado 15.2: PedidoController.java

Manejo global de excepciones (RFC 7807) (Listado 15.3):

```

1 package ec.edu.uteq.appweb.exception; // paquete del archivo
2
3 import jakarta.servlet.http.HttpServletRequest; // importacion de
4   clase externa
5 import org.springframework.http.HttpStatus; // importacion de clase
6   externa
7 import org.springframework.http.ProblemDetail; // importacion de clase
8   externa
9 import org.springframework.http.ResponseEntity; // importacion de
10  clase externa
11 // importacion de clase externa
12 // importacion de clase externa
13 import org.springframework.web.bind.MethodArgumentNotValidException;
14 import org.springframework.web.bind.annotation.*; // importacion de
15  clase externa
16 import java.net.URI; // importacion de clase externa
17 import java.time.Instant; // importacion de clase externa
18 import java.util.*; // importacion de clase externa
19
20 @RestControllerAdvice // expone endpoints REST con JSON
21 public class GlobalExceptionHandler { // clase publica
22

```

```

18  @ExceptionHandler(NoSuchElementException.class)
19  public ResponseEntity<ProblemDetail> noEncontrado( // metodo
    publico
20      NoSuchElementException ex, HttpServletRequest req) {
21      ProblemDetail pd = ProblemDetail.forStatusAndDetail(
22          HttpStatus.NOT_FOUND, ex.getMessage());
23      pd.setType(URI.create("https://uteq.edu.ec/errors/no-encontrado"));
24      pd.setTitle("Recurso no encontrado");
25      pd.setInstance(URI.create(req.getRequestURI()));
26      pd.setProperty("timestamp", Instant.now());
27      // retorna
28      // retorna
29      return ResponseEntity.status(HttpStatus.NOT_FOUND).body(pd);
30  }
31
32  @ExceptionHandler(MethodArgumentNotValidException.class)
33  public ResponseEntity<ProblemDetail> validacion( // metodo publico
34      MethodArgumentNotValidException ex, HttpServletRequest
        req) {
35      Map<String, String> errores = new LinkedHashMap<>();
36      ex.getBindingResult().getFieldErrors().forEach(e ->
37          errores.put(e.getField(), e.getDefaultMessage()));
38
39      ProblemDetail pd = ProblemDetail.forStatusAndDetail(
40          HttpStatus.UNPROCESSABLE_ENTITY,
41          "Errores de validacion en " + errores.size() + "
        campo(s)");
42      pd.setType(URI.create("https://uteq.edu.ec/errors/validacion"));
43      pd.setTitle("Datos invalidos");
44      pd.setInstance(URI.create(req.getRequestURI()));
45      pd.setProperty("errores", errores);
46      // retorna
47      // retorna
48      return ResponseEntity.unprocessableEntity().body(pd);
49  }
50 }

```

Listado 15.3: GlobalExceptionHandler.java

15.4.2 ASP.NET Core 10

ASP.NET Core utiliza el atributo `[ApiController]` y la herencia de `ControllerBase` para construir APIs REST. Combinado con *model binding* y *model validation* automáticos, ofrece una experiencia muy productiva.

Ejemplo 15.2 — Mismo recurso en ASP.NET Core 10

Enunciado. Implementar el mismo recurso de productos visto antes pero en ASP.NET Core 10, para comparar idiomas: atributos `[ApiController]`, `[HttpGet]`, retornos `Ok()`/`NotFound()`.

```

1  using Microsoft.AspNetCore.Mvc; // importa namespace
2  using AppwebApi.Services; // importa namespace
3  using AppwebApi.Dtos; // importa namespace

```

```

4
5 namespace AppwebApi.Controllers; // namespace del archivo
6
7 [ApiController] // controlador de Web API (validacion automatica)
8 [Route("api/v1/[controller]")] // ruta base del controlador
9 [Produces("application/json")]
10 public class PedidosController : ControllerBase // clase publica
11 {
12     private readonly IPedidoService _svc; // metodo o miembro privado
13
14     // metodo o miembro publico
15     // metodo o miembro publico
16     public PedidosController(IPedidoService svc) => _svc = svc;
17
18     /// <summary>Lista paginada de pedidos</summary>
19     [HttpGet] // endpoint HTTP GET
20     [ProducesResponseType(typeof(PageDto<PedidoDto>), 200)]
21     // metodo o miembro publico
22     // metodo o miembro publico
23     public async Task<ActionResult<PageDto<PedidoDto>>> Listar(
24         // mapea query string
25         // mapea query string
26         [FromQuery] string? estado, [FromQuery] int page = 1,
27         [FromQuery] int size = 20) // mapea query string
28     => Ok(await _svc.BuscarAsync(estado, page, size));
29
30     [HttpGet("{id:long}")] // endpoint HTTP GET
31     [ProducesResponseType(typeof(PedidoDto), 200)]
32     [ProducesResponseType(404)]
33     // metodo o miembro publico
34     // metodo o miembro publico
35     public async Task<ActionResult<PedidoDto>> Obtener(long id)
36     {
37         var p = await _svc.ObtenerAsync(id);
38         return p is null ? NotFound() : Ok(p); // retorna
39     }
40
41     [HttpPost] // endpoint HTTP POST (crear)
42     [ProducesResponseType(typeof(PedidoDto), 201)]
43     [ProducesResponseType(422)]
44     public async Task<ActionResult<PedidoDto>> Crear( // metodo o
45         // miembro publico
46         [FromBody] PedidoCreateRequest req) // mapea body JSON al
47         // parametro
48     {
49         var creado = await _svc.CrearAsync(req);
50         return CreatedAtAction(nameof(Obtener), // HTTP 201 (recurso
51             // creado)
52             new { id = creado.Id }, creado);
53     }
54 }
55

```

```

52     [HttpDelete("{id:long}")] // endpoint HTTP DELETE (eliminar)
53     [ProducesResponseType(204)]
54     [ProducesResponseType(404)]
55     public async Task<IActionResult> Eliminar(long id) // metodo o
        miembro publico
56     {
57         await _svc.EliminarAsync(id);
58         return NoContent(); // HTTP 204 (sin cuerpo)
59     }
60 }

```

Listado 15.4: PedidosController.cs

15.4.3 PHP 8.4 con Slim Framework 4

PHP construye APIs REST profesionales mediante microframeworks como Slim Framework 4. El siguiente ejemplo construye una API minimalista pero idiomática.

Ejemplo 15.3 — API REST minimalista en PHP con Slim 4

Enunciado. Construir una API REST minimalista en PHP usando el microframework Slim 4, ideal para servicios pequeños o microservicios PHP.

```

1  <?php
2  declare(strict_types=1);
3  require __DIR__ . '/../vendor/autoload.php'; // incluye otro archivo
        PHP (fatal si falla)
4
5  use Slim\Factory\AppFactory; // importa clase del namespace
6  use Psr\Http\Message\ServerRequestInterface as Req; // importa clase
        del namespace
7  use Psr\Http\Message\ResponseInterface as Res; // importa clase del
        namespace
8
9  $app = AppFactory::create();
10 $app->addBodyParsingMiddleware();
11
12 // GET /api/v1/pedidos
13 $app->get('/api/v1/pedidos', function (Req $req, Res $res) {
14     $pdo = obtenerPDO();
15     $estado = $req->getQueryParams()['estado'] ?? null;
16     $sql = "SELECT id, total, estado, creado FROM pedidos";
17     $params = [];
18     // condicional
19     // condicional
20     if ($estado) { $sql .= " WHERE estado = ?"; $params[] = $estado; }
21     $sql .= " ORDER BY creado DESC LIMIT 50";
22     $stmt = $pdo->prepare($sql); // prepara consulta SQL (defensa
        anti-SQLi)
23     $stmt->execute($params); // ejecuta la consulta con parametros
        enlazados
24     // recupera todas las filas restantes

```

```

25     // recupera todas las filas restantes
26     $res->getBody()->write(json_encode(['data' => $stmt->fetchAll()]));
27     // retorna valor
28     // retorna valor
29     return $res->withHeader('Content-Type','application/json');
30 });
31
32 // POST /api/v1/pedidos
33 $app->post('/api/v1/pedidos', function (Req $req, Res $res) {
34     $body = $req->getParsedBody() ?? [];
35     // condicional
36     // condicional
37     if (empty($body['total']) || !is_numeric($body['total'])) {
38         $err = ['type'=>'/errors/validacion',
39             'title'=>'Datos invalidos','status'=>422,
40             'errors'=>['total'=>'es requerido y numerico']];
41         $res->getBody()->write(json_encode($err));
42         return $res->withStatus(422) // retorna valor
43             ->withHeader('Content-Type','application/problem+json');
44     }
45     $pdo = obtenerPDO();
46     $stmt = $pdo->prepare( // prepara consulta SQL (defensa anti-SQLi)
47         "INSERT INTO pedidos (total, estado, creado) VALUES
48             (?, ?, NOW())");
49     $stmt->execute([$body['total'], 'PENDIENTE']); // ejecuta la
50         consulta con parametros enlazados
51     $id = $pdo->lastInsertId();
52     $data = ['id'=>$id, 'total'=>$body['total'], 'estado'=>'PENDIENTE'];
53     $res->getBody()->write(json_encode($data));
54     return $res->withStatus(201) // retorna valor
55         ->withHeader('Content-Type','application/json')
56         ->withHeader('Location','/api/v1/pedidos/$id');
57 });
58
59 $app->run();

```

Listado 15.5: public/index.php

15.5 REST vs SOAP: la comparativa definitiva

REST y SOAP son los dos modelos arquitectónicos de servicios web más utilizados. Comparar sus características objetivamente es clave para elegir correctamente en cada proyecto.

REST vs SOAP — comparación práctica

Criterio	REST (mainstream 2026)	SOAP (legado)
Formato	JSON (texto compacto)	XML envelope con header y body (verboso)
Transporte	HTTP/HTTPS	HTTP, SMTP, FTP, JMS
Contrato	OpenAPI 3.0 (opcional)	WSDL (obligatorio)
Estado	Stateless por restricción	Stateful o stateless
Tamaño mensaje	3–5x más compacto	Pesado
Seguridad	HTTPS + JWT + OAuth 2.0	WS-Security (más complejo)
Desde JavaScript	Trivial (fetch nativo)	Difícil (requiere SOAP client)
Caché HTTP nativa	Sí	No (POSTs)
Casos típicos	APIs públicas, microservicios, SPA	Banca, salud, B2B legado
Curva de aprendizaje	Baja	Alta
Tooling 2026	Excelente	Maduro pero envejecido

15.5.1 Cuándo aún tiene sentido SOAP en 2026

Aunque REST es la opción dominante en 2026, hay escenarios donde SOAP todavía tiene ventajas reales. La lista siguiente los recoge.

- **Sistemas financieros nacionales:** muchos sistemas bancarios mantienen interfaces SOAP por inercia regulatoria. En Ecuador, los servicios SPI del Banco Central usan SOAP.
- **Salud:** HL7 v3 sobre SOAP sigue vigente en hospitales.
- **B2B con grandes corporaciones:** SAP, Oracle EBS y sistemas similares exponen SOAP por defecto.
- **Cuando se necesita WS-AtomicTransaction** (transacciones distribuidas estrictas).

15.5.2 Ejemplo SOAP mínimo

Aunque el curso no usa SOAP en el PFC, conviene haber visto al menos una petición SOAP completa para reconocerla cuando aparezca en código heredado.

Ejemplo corto comentado (petición SOAP) (Listado 15.6):

Listado 15.6: Ejemplo de HTML

```

<!-- POST /servicios/clima HTTP/1.1 -->
<!-- Content-Type: text/xml; charset=utf-8 -->
<!-- SOAPAction: "obtenerTemperatura" -->
<?xml version="1.0"?>
<!-- Envelope: contenedor obligatorio de todo SOAP -->
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <!-- Body: contenido principal de la petición -->
  <soap:Body>
    <!-- Operación definida en el WSDL -->
    <obtenerTemperatura xmlns="http://uteq.edu.ec/clima">
      <ciudad>Quevedo</ciudad>
    </obtenerTemperatura>
  </soap:Body>
</soap:Envelope>

```

Ejemplo 15.4 — Cliente SOAP en PHP que consume servicio del SRI (estructura conceptual)

Enunciado. Mostrar la estructura de un cliente SOAP en PHP que consume un servicio del SRI ecuatoriano (caso real del contexto local), demostrando el protocolo en producción.

```
1 <?php
2 declare(strict_types=1);
3
4 // PHP tiene cliente SOAP nativo: SoapClient
5 $wsdl = 'https://servicios.sri.gob.ec/aut.../wsdl';
6
7 try {
8     $client = new SoapClient($wsdl, [
9         'soap_version' => SOAP_1_2,
10        'trace'          => true,
11        'exceptions'     => true,
12        'connection_timeout' => 10,
13    ]);
14
15    // Llamada SOAP: el metodo se invoca como si fuera local
16    $resp = $client->consultarRuc(['numeroRuc' => '0992345678001']);
17
18    print_r($resp);
19
20    // Para debug del XML enviado/recibido:
21    // echo $client->__getLastRequest();
22    // echo $client->__getLastResponse();
23
24 } catch (SoapFault $e) {
25     error_log("SOAP fault: " . $e->getMessage());
26     http_response_code(502);
27     // imprime al output del servidor
28     // imprime al output del servidor
29     echo json_encode(['error' => 'Servicio externo no disponible']);
30 }
```

Listado 15.7: consultar-ruc.php

15.6 Documentación con OpenAPI 3.0 (Swagger)

Una API sin documentación es inutilizable; una con documentación a mano queda desincronizada al primer cambio. OpenAPI 3.0 + Swagger UI resuelve esto generando documentación viva desde el propio código.

15.6.1 ¿Qué es OpenAPI?

OpenAPI 3.0 (antes Swagger) es la especificación estándar de la industria para describir APIs REST. Permite:

- Documentación interactiva con Swagger UI.
- Generación automática de clientes en cualquier lenguaje.
- Pruebas y mocks automáticos.
- Validación de contratos (contract testing).

15.6.2 Configurar OpenAPI en Spring Boot

Springdoc es la biblioteca que añade OpenAPI a Spring Boot con configuración mínima. El siguiente fragmento la habilita y expone Swagger UI.

Ejemplo 15.5 — OpenAPI con springdoc

Enunciado. Configurar springdoc-openapi en una aplicación Spring Boot para exponer documentación OpenAPI 3.0 automática y la interfaz Swagger UI interactiva.

En `pom.xml`:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

Listado 15.8: Ejemplo de XML

Configuración `OpenApiConfig.java`:

```
@Configuration // clase de configuracion Spring
public class OpenApiConfig { // clase publica
    @Bean
    public OpenAPI api() { // metodo publico
        return new OpenAPI() // retorna
            .info(new Info()
                .title("API Aplicaciones Web UTEQ")
                .version("1.0.0")
                .description("API REST del PFC")
                .contact(new Contact().email("ti@uteq.edu.ec")))
            .components(new Components()
                .addSecuritySchemes("bearer-jwt",
                    new SecurityScheme()
                        .type(SecurityScheme.Type.HTTP)
                        .scheme("bearer")
                        .bearerFormat("JWT")))
            .addSecurityItem(new SecurityRequirement()
                .addList("bearer-jwt"));
    }
}
```

Listado 15.9: Ejemplo de Java

Acceso a Swagger UI: tras reiniciar la app, abrir <http://localhost:8080/swagger-ui.html>. Aparece la documentación interactiva con todos los endpoints, sus parámetros, y un botón *Try it out* para probarlos en vivo.

15.7 Autenticación de APIs con JWT

JWT (JSON Web Token, RFC 7519) es el estándar dominante de autenticación en APIs REST modernas. Permite que el servidor verifique la identidad del cliente sin guardar estado: la firma criptográfica del token es prueba suficiente.

15.7.1 Anatomía de un JWT

Un JSON Web Token (RFC 7519 [31]) consta de tres partes separadas por puntos (Listado 15.10):

```
header.payload.signature
```

Listado 15.10: Ejemplo de Shell

Header (Base64URL) (Listado 15.11):

```
{ "alg": "HS256", "typ": "JWT" }
```

Listado 15.11: Ejemplo de JavaScript

Payload (Base64URL) (Listado 15.12):

```
{
  "sub": "42",
  "iss": "uteq.edu.ec",
  "iat": 1747000000,
  "exp": 1747003600,
  "rol": "ROLE_ADMIN",
  "email": "admin@uteq.edu.ec",
  "jti": "a1b2c3d4-e5f6-7890-1234-567890abcdef"
}
```

Listado 15.12: Ejemplo de JavaScript

Signature: HMAC-SHA256 (o RSA/ECDSA) sobre `base64(header).base64(payload)` con un secreto.

15.7.2 Flags estándar del payload (*claims*)

El payload de un JWT puede contener cualquier dato (claims), pero hay un conjunto estandarizado de claims que el estudiante debe incluir siempre. La lista siguiente los recoge.

- **iss** (issuer): emisor del token.
- **sub** (subject): identificador del usuario.
- **aud** (audience): receptor previsto.
- **exp** (expiration): timestamp de expiración (Unix).
- **nbf** (not before): no usable antes de.
- **iat** (issued at): timestamp de emisión.
- **jti** (JWT ID): identificador único para revocación.

15.7.3 JWT en el PFC: configuración Spring Security

La configuración de JWT en Spring Boot requiere tres piezas: un filtro que valide el token, una configuración de seguridad que lo declare, y una clase de utilidad que firme y verifique. El siguiente ejemplo las muestra.

Ejemplo 15.6 — Generación y validación JWT en Spring Boot 4

Enunciado. Implementar generación y validación de JWT en Spring Boot 4 con `jjwt`: clase utilitaria, filtro de seguridad y configuración Spring Security.

Generador (servicio) (Listado 15.13):

```
import io.jsonwebtoken.Jwts; // importacion de clase externa
import io.jsonwebtoken.security.Keys; // importacion de clase externa
import javax.crypto.SecretKey; // importacion de clase externa
```

```

import java.util.Date; // importacion de clase externa
import java.util.UUID; // importacion de clase externa

@Service // capa de servicio (logica de negocio)
public class JwtService { // clase publica
    private final SecretKey key; // immutable
    private static final long DURACION_MS = 3600 * 1000; // 1h

    // metodo publico
    // metodo publico
    public JwtService(@Value("${jwt.secret}") String secreto) {
        this.key = Keys.hmacShaKeyFor(secreto.getBytes());
    }

    // metodo publico
    // metodo publico
    public String generar(String userId, String email, String rol) {
        Date ahora = new Date();
        return Jwts.builder() // retorna
            .subject(userId)
            .issuer("uteq.edu.ec")
            .claim("email", email)
            .claim("rol", rol)
            .id(UUID.randomUUID().toString())
            .issuedAt(ahora)
            .expiration(new Date(ahora.getTime() + DURACION_MS))
            .signWith(key)
            .compact();
    }

    public Claims parsear(String token) { // metodo publico
        return Jwts.parser() // retorna
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}

```

Listado 15.13: Ejemplo de Java

Cookie HttpOnly tras login (Listado 15.14):

```

@PostMapping("/api/auth/login") // endpoint HTTP POST (crear)
public ResponseEntity<?> login(@RequestBody LoginDto req, // metodo
    publico
                                HttpServletResponse res) {
    Usuario u = autenticar(req.email(), req.password());
    String token = jwtService.generar(
        u.getId().toString(), u.getEmail(), u.getRol());

    Cookie cookie = new Cookie("ACCESS_TOKEN", token);
}

```

```

    cookie.setHttpOnly(true);
    cookie.setSecure(true);
    cookie.setAttribute("SameSite", "Strict");
    cookie.setMaxAge(3600);
    cookie.setPath("/");
    res.addCookie(cookie);

    // retorna
    // retorna
    return ResponseEntity.ok(Map.of("usuario", new UsuarioDto(u)));
}

```

Listado 15.14: Ejemplo de Java

15.8 Práctica integrada: API REST consumiendo servicio externo

La práctica integrada construye una API REST en Spring Boot que consume un servicio externo (OpenWeatherMap), cachea las respuestas en Redis y maneja errores correctamente.

Ejercicio 15.1 — API gateway con cache-aside (Sumativa)

Objetivo: construir una API REST propia que consuma una API pública externa, aplicando cache-aside con Redis para reducir latencia y dependencia del servicio externo.

IDE: IntelliJ IDEA Ultimate

La práctica usa IntelliJ IDEA Ultimate. Antes de empezar, confirma que tienes JDK 21, Maven 3.9 y Docker instalados.

Paso 1 — Levantar Redis en Docker

Para cachear las respuestas necesitamos Redis. Levantarlo con Docker es lo más rápido. Véase el Listado 15.15.

```

docker run -d --name redis-gw -p 6379:6379 redis:7-alpine # arranca un
                contenedor
docker exec -it redis-gw redis-cli ping # PONG

```

Listado 15.15: Ejemplo de Shell

Paso 2 — Crear el proyecto Spring Boot

File → New → Project → Spring Boot. Configurar:

- Java 21, Maven, Spring Boot 4.x.
- Group: `ec.edu.utec`; Artifact: `api-gateway`.
- Dependencies: *Spring Web*, *Spring Data Redis*, *Spring Boot DevTools*, *Lombok*, *Validation*, *springdoc-openapi-starter-webmvc-ui* (manual en pom.xml).

Paso 3 — Configurar `application.yml`

El archivo `application.yml` configura la API key del servicio externo, los parámetros de Redis y el timeout HTTP. Véase el Listado 15.16.

```

spring: # configuracion de Spring
  data:
    redis: # configuracion del cliente Redis
      host: localhost # host del servicio
      port: 6379 # puerto donde escucha la aplicacion
      timeout: 2s
  external:
    api:
      url: https://jsonplaceholder.typicode.com
springdoc: # configuracion de OpenAPI/Swagger
  swagger-ui: # configuracion de Swagger UI
    path: /api/docs # ruta donde se expone

```

Listado 15.16: Ejemplo de YAML

Paso 4 — Cliente HTTP con WebClient

Spring Boot 4 introduce WebClient como reemplazo moderno del antiguo RestTemplate: API reactiva, no bloqueante, integrada con Mono/Flux. Véase el Listado 15.17.

```

1 package ec.edu.uteq.gateway.client; // paquete del archivo
2
3 import com.fasterxml.jackson.databind.JsonNode; // importacion de
  clase externa
4 import org.springframework.beans.factory.annotation.Value; //
  importacion de clase externa
5 import org.springframework.stereotype.Component; // importacion de
  clase externa
6 import org.springframework.web.client.RestClient; // importacion de
  clase externa
7 import java.time.Duration; // importacion de clase externa
8 import java.util.List; // importacion de clase externa
9
10 @Component // bean Spring generico
11 public class ExternalApiClient { // clase publica
12
13     private final RestClient http; // immutable
14
15     // metodo publico
16     // metodo publico
17     public ExternalApiClient(@Value("${external.api.url}") String
      base) {
18         this.http = RestClient.builder()
19             .baseUrl(base)
20             .build();
21     }
22
23     public List<JsonNode> listarPosts() { // metodo publico
24         return http.get().uri("/posts").retrieve() // retorna
25             .body(new
                org.springframework.core.ParameterizedTypeReference<>()

```

```

        {});
26     }
27
28     public JsonNode obtenerPost(int id) { // metodo publico
29         // retorna
30         // retorna
31         return http.get().uri("/posts/{id}",
32             id).retrieve().body(JsonNode.class);
33     }
34 }

```

Listado 15.17: ExternalApiClient.java

Paso 5 — Servicio con cache-aside

El servicio que consume la API externa debe aplicar cache-aside: consultar Redis primero, llamar al servicio solo si falla la caché, y persistir el resultado con TTL. Véase el Listado 15.18.

```

1  package ec.edu.uteq.gateway.service; // paquete del archivo
2
3  import com.fasterxml.jackson.databind.ObjectMapper; // importacion de
4      clase externa
5  import com.fasterxml.jackson.databind.JsonNode; // importacion de
6      clase externa
7  import ec.edu.uteq.gateway.client.ExternalApiClient; // importacion de
8      clase externa
9  import lombok.RequiredArgsConstructor; // importacion de clase externa
10 import lombok.extern.slf4j.Slf4j; // importacion de clase externa
11 // importacion de clase externa
12 // importacion de clase externa
13 import org.springframework.data.redis.core.StringRedisTemplate;
14 import org.springframework.stereotype.Service; // importacion de clase
15     externa
16 import java.time.Duration; // importacion de clase externa
17 import java.util.*; // importacion de clase externa
18
19 @Service // capa de servicio (logica de negocio)
20 @Slf4j
21 @RequiredArgsConstructor
22 public class PostService { // clase publica
23
24     private final ExternalApiClient client; // immutable
25     private final StringRedisTemplate redis; // immutable
26     private final ObjectMapper json; // immutable
27
28     // campo privado
29     // campo privado
30     private static final Duration TTL = Duration.ofSeconds(60);
31
32     public Map<String, Object> listar() throws Exception { // metodo
33         publico

```



```

29     String cached = redis.opsForValue().get("posts:all");
30     if (cached != null) { // condicional
31         log.debug("Cache HIT");
32         return Map.of("source","cache", // retorna
33             "data", json.readTree(cached));
34     }
35     log.debug("Cache MISS - consultando externa");
36     List<JsonNode> data = client.listarPosts();
37     redis.opsForValue().set("posts:all",
38         json.writeValueAsString(data), TTL);
39     return Map.of("source", "api", "data", data); // retorna
40 }
41
42 // metodo publico
43 // metodo publico
44 public Map<String,Object> obtener(int id) throws Exception {
45     String key = "posts:" + id;
46     String cached = redis.opsForValue().get(key);
47     if (cached != null) { // condicional
48         return Map.of("source","cache", // retorna
49             "data", json.readTree(cached));
50     }
51     JsonNode data = client.obtenerPost(id);
52     redis.opsForValue().set(key, data.toString(), TTL);
53     return Map.of("source","api", "data", data); // retorna
54 }
55 }

```

Listado 15.18: PostService.java

Paso 6 — Controller con manejo robusto de errores

El controlador expone los endpoints REST y maneja los errores devolviendo códigos HTTP semánticamente correctos con ProblemDetails (RFC 7807). Véase el Listado 15.19.

```

1 package ec.edu.uteq.gateway.controller; // paquete del archivo
2
3 import ec.edu.uteq.gateway.service.PostService; // importacion de
   clase externa
4 import io.swagger.v3.oas.annotations.Operation; // importacion de
   clase externa
5 import io.swagger.v3.oas.annotations.tags.Tag; // importacion de clase
   externa
6 import lombok.RequiredArgsConstructor; // importacion de clase externa
7 import org.springframework.http.HttpStatus; // importacion de clase
   externa
8 import org.springframework.http.ProblemDetail; // importacion de clase
   externa
9 import org.springframework.http.ResponseEntity; // importacion de
   clase externa
10 import org.springframework.web.bind.annotation.*; // importacion de
   clase externa

```

```
11 import org.springframework.web.client.RestClientException; //
    importacion de clase externa
12 import java.net.URI; // importacion de clase externa
13 import java.util.Map; // importacion de clase externa
14
15 @RestController // expone endpoints REST con JSON
16 @RequestMapping("/api/v1/publicaciones") // ruta base del controlador
17 @RequiredArgsConstructor
18 @Tag(name = "Publicaciones", // agrupa endpoints en Swagger UI
19     description = "Gateway que cachea respuestas externas")
20 public class PostController { // clase publica
21
22     private final PostService service; // immutable
23
24     @GetMapping // endpoint HTTP GET
25     // OpenAPI
26     // OpenAPI
27     @Operation(summary = "Lista de publicaciones (con cache 60s)")
28     public ResponseEntity<?> listar() { // metodo publico
29         try {
30             return ResponseEntity.ok(service.listar()); // retorna
31         } catch (RestClientException ex) {
32             // retorna
33             // retorna
34             return error(503, "Servicio externo no disponible",
35                 ex.getMessage());
36         } catch (Exception ex) {
37             return error(502, "Respuesta externa invalida", // retorna
38                 ex.getMessage());
39         }
40     }
41
42     @GetMapping("/{id}") // endpoint HTTP GET
43     @Operation(summary = "Una publicacion por ID") // OpenAPI
44     // metodo publico
45     // metodo publico
46     public ResponseEntity<?> obtener(@PathVariable int id) {
47         try {
48             return ResponseEntity.ok(service.obtener(id)); // retorna
49         } catch (RestClientException ex) {
50             // condicional
51             // condicional
52             if (ex.getMessage() != null &&
53                 ex.getMessage().contains("404")) {
54                 // retorna
55                 // retorna
56                 return error(404, "Publicacion no encontrada",
57                     "ID: " + id);
58             }
59             // retorna
60             // retorna
```

```

60         return error(503, "Servicio externo no disponible",
61                       ex.getMessage());
62     } catch (Exception ex) {
63         return error(502, "Respuesta externa invalida", // retorna
64                       ex.getMessage());
65     }
66 }
67
68 // campo privado
69 // campo privado
70 private ResponseEntity<ProblemDetail> error(int status, String
71                                           titulo,
72                                           String detalle) {
73     ProblemDetail pd = ProblemDetail.forStatusAndDetail(
74         HttpStatus.valueOf(status), detalle);
75     pd.setTitle(titulo);
76     pd.setType(URI.create("https://uteq.edu.ec/errors/gateway"));
77     return ResponseEntity.status(status).body(pd); // retorna
78 }

```

Listado 15.19: PostController.java

Paso 7 — Probar con Postman

Con la API corriendo, el siguiente paso es probarla con Postman cubriendo casos de éxito y de error para validar la robustez.

1. Arrancar la app: Shift+F10.
2. En IntelliJ Ultimate, abrir el panel *HTTP Client* (Tools → HTTP Client → Create Request), o usar Postman externo.
3. Crear archivo `requests.http`:

```

### Listar publicaciones (primera vez: cache MISS)
GET http://localhost:8080/api/v1/publicaciones
Accept: application/json

### Listar otra vez (cache HIT)
GET http://localhost:8080/api/v1/publicaciones
Accept: application/json

### Obtener una
GET http://localhost:8080/api/v1/publicaciones/1
Accept: application/json

### Probar caso 404
GET http://localhost:8080/api/v1/publicaciones/9999
Accept: application/json

```

Listado 15.20: Ejemplo de Shell

4. Ejecutar cada petición pulsando el botón verde junto a cada `###`.
5. Verificar:

- Primera llamada: campo `source`: "api".
- Llamada inmediata posterior: `source`: "cache".
- Llamada con ID inexistente: 404 con ProblemDetails.

Paso 8 — Verificar Swagger UI

Abrir <http://localhost:8080/api/docs>. Aparece la documentación interactiva. Probar el endpoint `GET /api/v1/publicaciones/{id}` desde la interfaz Swagger.

Paso 9 — Inspeccionar el cache en Redis

Después de las pruebas, inspeccionar el contenido de Redis con `redis-cli` permite verificar que las claves se están generando correctamente con sus TTL. Véase el Listado 15.21.

```
docker exec -it redis-gw redis-cli # ejecuta comando en contenedor activo
> KEYS posts:*
1) "posts:all"
2) "posts:1"
> TTL posts:all
(integer) 47 # segundos restantes hasta expirar
> GET posts:1
"{\"userId\":\"1\",\"id\":\"1\",\"title\":\"...\",\"body\":\"...\"}"
> exit
```

Listado 15.21: Ejemplo de Shell

Resultado esperado

API gateway funcional que:

- Cachea la respuesta externa por 60 segundos.
- Identifica explícitamente la fuente (`cache` o `api`).
- Maneja 4 escenarios de error con ProblemDetails RFC 7807.
- Está documentada con Swagger UI en `/api/docs`.

Esta es la *Sumativa GA-Práctica de la semana 15*.

15.9 Práctica integrada: PFC Entrega 2 (Semana 15)

La Entrega 2 del PFC consolida los avances de las semanas 10–15: gestión de estado, acceso a datos con ORM, arquitectura documentada y API REST autenticada.

Ejercicio 15.2 — PFC Entrega 2 final — API REST + Angular operacional

Esta es la última entrega parcial del PFC antes de la Entrega Final de la semana 16. Cierra todas las funcionalidades del backend con su API REST documentada y el frontend Angular completamente operacional.

Requisitos cubiertos en esta entrega

La Entrega 2 cubre un alcance específico que se evalúa contra rúbrica. La lista siguiente lo detalla.

- API REST con todos los endpoints CRUD de las entidades del dominio.
- Autenticación JWT con cookie `HttpOnly` funcionando.
- OpenAPI 3.0 con Swagger UI accesible en `/api/docs`.
- Frontend Angular con login, logout y CRUD de al menos una entidad principal.

- Manejo de errores estandarizado con ProblemDetails (RFC 7807).
- Cache Redis en al menos un endpoint de listado.
- Logs estructurados en backend (SLF4J + Logback).
- Mínimo 5 pruebas unitarias con JUnit 5 y MockMvc.

Entregables

El equipo entrega un conjunto de artefactos consolidados a través del repositorio Git y formularios institucionales.

1. Repositorio Git con tag v0.7.0.
2. PDF de informe técnico (15-25 páginas).
3. Demostración funcional con `docker compose up`.
4. Colección Postman exportada con todos los endpoints.
5. Captura del reporte de pruebas (JaCoCo).

Esta es la *Sumativa TA* — *PFC Entrega 2 final*.

15.10 Buenas prácticas profesionales en APIs REST

La caja siguiente compila las reglas profesionales para escribir APIs REST mantenibles, evolucionables y bien documentadas.

Veinte reglas profesionales para APIs REST 2026

1. Versionar la API: `/api/v1/...` en URL o cabecera `Accept: application/vnd.uteq.v1+json`.
2. Sustantivos en plural; sin verbos en URL.
3. Códigos HTTP semánticamente correctos.
4. ProblemDetails RFC 7807 para errores.
5. Validación con Bean Validation; nunca confiar en cliente.
6. Paginación obligatoria en listados (max 100 items por page).
7. Filtros, ordenamiento y búsqueda por query string.
8. Idempotencia respetada en GET, PUT, DELETE.
9. HTTPS obligatorio incluso en desarrollo local con mkcert.
10. Autenticación JWT en cookie `HttpOnly+Secure+SameSite=Strict`.
11. Rate limiting por IP/usuario (`X-RateLimit-*`).
12. CORS explícito y restrictivo (no `*` en producción).
13. OpenAPI documentando todos los endpoints.
14. DTOs separados de entidades JPA/EF.
15. Logging estructurado en JSON para análisis.
16. Health checks (`/actuador/health` en Spring, `/health` en .NET).
17. Cache headers correctos (`Cache-Control`, `Etag`).
18. Compresión Gzip/Brotli activa.
19. Pruebas de contrato con OpenAPI schema validation.
20. Colección Postman versionada en el repositorio.

15.11 Contraste bibliográfico de los conceptos clave

REST como estilo arquitectónico fue formalizado por [17] en su tesis doctoral. La literatura más reciente ha matizado y enriquecido ese marco. [58] introdujo el *Richardson Maturity Model* (niveles 0–3) como guía para evaluar el grado de adopción de los principios REST en una API real. [58] discute con datos empíricos qué prácticas REST se cumplen en APIs públicas: la mayoría no alcanza el nivel 3 (HATEOAS) y muchas violan principios básicos. [58] provee una taxonomía de errores comunes en diseño REST en APIs latinoamericanas; [58] documenta cómo OpenAPI 3.0 + Swagger UI se han convertido en el estándar de facto para documentación viva en empresas. Sobre SOAP, [1] sigue siendo la referencia técnica, y [13] discute patrones de diseño para servicios que aplican tanto a SOAP como a REST.

15.12 Ejercicio resuelto adicional con resultado visual

Como cierre de la semana 15, el siguiente ejercicio resuelto adicional construye una API REST + JWT + Swagger paso a paso, y se complementa con un propuesto de migración SOAP → REST.

Ejercicio resuelto 15.1 — API REST de productos con JWT y Swagger

Enunciado. Implementar una API REST de productos con Spring Boot 4 que requiera autenticación JWT, documente sus endpoints con OpenAPI 3.0 y exponga Swagger UI interactivo.

Dependencias clave (pom.xml):

Listado 15.22: Dependencias Maven

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
io.jsonwebtoken:jjwt-api:0.12.5
springdoc-openapi-starter-webmvc-ui:2.5.0
```

Configuración de SpringDoc (application.yml):

Listado 15.23: Configuración de Swagger

```
springdoc: # configuracion de OpenAPI/Swagger
  api-docs: # configuracion del JSON OpenAPI
    path: /v3/api-docs # ruta donde se expone
  swagger-ui: # configuracion de Swagger UI
    path: /swagger-ui.html # ruta donde se expone
    operations-sorter: method
```

Endpoint anotado con OpenAPI (Listado 15.24):

Listado 15.24: Endpoint documentado

```
@RestController // expone endpoints REST con JSON
@RequestMapping("/api/v1/productos") // ruta base del controlador
// agrupa endpoints en Swagger UI
// agrupa endpoints en Swagger UI
@Tag(name = "Productos", description = "Catalogo de productos")
@SecurityRequirement(name = "bearer-jwt") // declara requisito de seguridad
public class ProductoController { // clase publica

    @Operation(summary = "Lista productos paginados", // OpenAPI
        description = "Retorna lista paginada con filtro opcional por categoria")
    @ApiResponses({ // documenta posible respuesta HTTP
        // documenta posible respuesta HTTP
        // documenta posible respuesta HTTP
        @ApiResponse(responseCode = "200", description = "Exito"),
        // documenta posible respuesta HTTP
        // documenta posible respuesta HTTP
        @ApiResponse(responseCode = "401", description = "No autenticado")
    })
    @GetMapping // endpoint HTTP GET
    public Page<Producto> listar( // metodo publico
        @RequestParam(required = false) String categoria, // query param
        Pageable pageable) {
        return categoria != null // retorna
            ? service.porCategoria(categoria, pageable)
            : service.todos(pageable);
    }
}
```

Resultado visual de Swagger UI en <http://localhost:8080/swagger-ui.html>:

API de Productos UTEQ OAS 3.0

Servers: <http://localhost:8080>

[Autorizar] (Click para configurar JWT bearer)

GET	/api/v1/productos	Lista productos paginados	↓
POST	/api/v1/productos	Crea un producto	↓
GET	/api/v1/productos/{id}	Obtiene un producto	↓
PUT	/api/v1/productos/{id}	Actualiza un producto	↓
DELETE	/api/v1/productos/{id}	Elimina un producto	↓

Probando un endpoint desde Swagger UI: al hacer clic en *Try it out* sobre GET /api/v1/productos, ejecuta la petición con el JWT configurado y muestra la respuesta:

```
Response 200 OK
content-type: application/json

{
  "content": [
    { "id": 1, "nombre": "Mouse Logitech", "precio": 25.50, "stock": 100 },
    { "id": 2, "nombre": "Teclado mecánico", "precio": 89.00, "stock": 45 }
  ],
  "totalElements": 2, "totalPages": 1, "number": 0
}
```

Discusión. La combinación springdoc + OpenAPI 3.0 produce documentación viva automática a partir de las anotaciones del código. El equipo nunca tiene que mantener un archivo Swagger a mano: cada cambio en el controlador se refleja inmediatamente en /swagger-ui.html. La integración con Spring Security permite probar los endpoints autenticados directamente desde el navegador [58].

Ejercicio propuesto 15.2 — Migración progresiva SOAP a REST con fachada

Enunciado. Un sistema legado expone un servicio SOAP de inventario con los métodos `consultarStock(codigo)`, `reservarItem(codigo, cantidad)` y `cancelarReserva(idReserva)`. Construir una fachada REST moderna que exponga estos métodos como recursos y permita una migración progresiva.

Requisitos:

- Backend Spring Boot que use `WebServiceTemplate` para consumir el SOAP legado.
- Exponer endpoints REST:
 - GET /api/v1/inventario/{codigo}.
 - POST /api/v1/inventario/{codigo}/reservas (body con cantidad).
 - DELETE /api/v1/inventario/reservas/{idReserva}.
- Mapear errores SOAP (*SOAPFault*) a respuestas ProblemDetails (RFC 7807).
- Caché Redis sobre `consultarStock` con TTL de 30 s para reducir carga sobre el SOAP origen.
- Documentar con OpenAPI y exponer Swagger UI.
- Producir una ADR-004 que justifique la decisión de mantener SOAP por debajo en lugar de reescribir.

Criterios de evaluación:

- Latencia p95 < 100 ms en cache caliente.
- Manejo correcto de errores: SOAP *timeout* se traduce a HTTP 504; *SOAPFault de stock insuficiente* a HTTP 409 Conflict.
- ADR completa y razonada.

- Pruebas Postman con 8 escenarios de éxito y error.

Lecturas recomendadas: [1] para SOAP; [44] cap. 3 para patrones de migración.

Entrega final del PFC: pruebas, cobertura, seguridad y publicación

«Un programa sin pruebas no está terminado: simplemente todavía no sabemos en qué falla.»

— Adaptado de Edsger W. Dijkstra

Resultado de aprendizaje de la semana

Al concluir la semana 16, el estudiante entrega el sistema PFC en versión 1.0.0 estable, con suite de pruebas unitarias e integración de cobertura $\geq 70\%$, validación OWASP de los seis controles mínimos, mediciones empíricas con k6 y Lighthouse, y la documentación técnica final completa con mínimo 15 referencias APA.

16.1 Pruebas en aplicaciones web modernas

Una aplicación sin pruebas es como un puente sin inspecciones: puede funcionar, pero nadie sabe cuánto resistirá al primer terremoto. Las pruebas son la red de seguridad que permite refactorizar con confianza.

De TDD a la pirámide de pruebas — 25 años de cultura de testing

La cultura de testing en software fue catapultada por Kent Beck con *Test-Driven Development* (TDD) en *Extreme Programming Explained* (1999). La idea: escribir la prueba ANTES del código, hacer que pase con la implementación más simple posible, refactorizar. TDD se convirtió en práctica estándar en equipos ágiles.

Mike Cohn formalizó en 2009 la **pirámide de pruebas**: muchas pruebas unitarias rápidas en la base, menos pruebas de integración en el medio, y unas pocas pruebas end-to-end en la cima. Esta jerarquía balancea velocidad de feedback (las unitarias corren en milisegundos) y confianza (las E2E validan el sistema completo).

En 2026 se ha matizado el modelo. Google propone el «**modelo honeycomb**»: priorizar pruebas de integración medias (*wide unit tests*) sobre las unitarias puras, especialmente en arquitecturas microservicios donde un servicio aislado dice poco. Spotify popularizó el «**trofeo de pruebas**»: análisis estático abajo, pocas unitarias, muchas integración, pocas E2E.

16.1.1 Niveles de pruebas en aplicaciones web

Existen distintos niveles de pruebas con distintos costos y distintos tipos de defectos que detectan. La pirámide de pruebas (Mike Cohn) los organiza visualmente.

Tabla 16.1: Niveles de pruebas y tecnologías 2026.

Nivel	Qué prueba	Herramientas (Java/Spring)
Unitarias	Una clase aislada	JUnit 5 + Mockito + AssertJ
Integración	Varias clases o capas	SpringBootTest, MockMvc, Testcontainers
Contrato	Compatibilidad cliente-servidor	Pact, Spring Cloud Contract
E2E	Sistema completo	Cypress, Playwright, Selenium
Carga / rendimiento	Sistema bajo presión	k6, JMeter, Gatling
Seguridad	Vulnerabilidades	OWASP ZAP, Burp Suite
Mutación	Calidad de las pruebas	PIT (Pitest), Stryker

16.1.2 La pirámide de pruebas en detalle

Cada nivel de la pirámide tiene un propósito específico: las unitarias garantizan correctitud lógica, las de integración verifican que las piezas encajen, y las E2E confirman que la aplicación entrega valor al usuario.

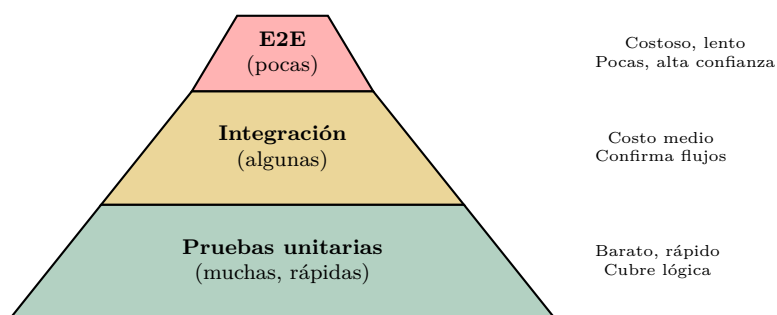


Figura 16.1: Pirámide de pruebas de Cohn: la base son muchas pruebas unitarias rápidas; la cumbre son pocas pruebas E2E lentas pero de alta confianza.

16.2 Pruebas unitarias con JUnit 5 y Mockito

JUnit 5 es el framework de pruebas dominante en Java desde 2017. Mockito complementa con *mocking* para aislar la unidad bajo prueba de sus colaboradores.

Ejemplo 16.1 — Test unitario de un servicio con mock del repositorio

Enunciado. Construir una suite completa de tests para un servicio de gestión de usuarios: tests unitarios con Mockito, tests de integración con MockMvc y Testcontainers, y reporte de cobertura JaCoCo.

```

1 package ec.edu.uteq.appweb.service; // paquete del archivo
2
3 import ec.edu.uteq.appweb.model.Producto; // importacion de clase
  externa
4 import ec.edu.uteq.appweb.repository.ProductoRepository; //
  importacion de clase externa
5 import org.junit.jupiter.api.*; // importacion de clase externa
6 import org.mockito.*; // importacion de clase externa
7 import java.math.BigDecimal; // importacion de clase externa
8 import java.util.*; // importacion de clase externa
9 import static org.assertj.core.api.Assertions.*; // importacion de
  clase externa
10 import static org.mockito.Mockito.*; // importacion de clase externa
11 import static org.junit.jupiter.api.Assertions.*; // importacion de
  clase externa

```

```
12
13 class ProductoServiceTest {
14
15     @Mock private ProductoRepository repo; // crea un mock con Mockito
16     @InjectMocks private ProductoService service; // inyecta los mocks
17         // en la clase bajo prueba
18     private AutoCloseable mocks; // campo privado
19
20     @BeforeEach // se ejecuta antes de cada test
21     void setUp() { mocks = MockitoAnnotations.openMocks(this); }
22
23     @AfterEach // se ejecuta despues de cada test
24     void tearDown() throws Exception { mocks.close(); }
25
26     @Test // metodo de test JUnit
27     @DisplayName("Listar productos devuelve la lista del repositorio")
28     void listar_devuelveListaDelRepo() {
29         // Arrange
30         Producto p1 = new Producto(1L, "A", new BigDecimal("1.00"), 5);
31         Producto p2 = new Producto(2L, "B", new BigDecimal("2.00"),
32             10);
33         when(repo.findAll()).thenReturn(List.of(p1, p2)); // configura
34             // comportamiento del mock
35
36         // Act
37         var resultado = service.listar();
38
39         // Assert
40         assertThat(resultado).hasSize(2)
41             .extracting(Producto::getNombre)
42             .containsExactly("A", "B");
43         verify(repo, times(1)).findAll(); // verifica interaccion con
44             // el mock
45     }
46
47     @Test // metodo de test JUnit
48     @DisplayName("Buscar inexistente lanza NoSuchElementException")
49     void buscarInexistente_lanzaExcepcion() {
50         // configura comportamiento del mock
51         // configura comportamiento del mock
52         when(repo.findById(999L)).thenReturn(Optional.empty());
53         assertThatThrownBy(() -> service.buscar(999L))
54             .isInstanceOf(NoSuchElementException.class)
55             .hasMessageContaining("999");
56     }
57
58     @Test // metodo de test JUnit
59     @DisplayName("Guardar persiste y retorna el producto con ID")
60     void guardar_persisteYRetorna() {
61         Producto entrada = new Producto(null, "X",
62             new BigDecimal("3.0"), 1);
```

```

59     Producto guardado = new Producto(99L, "X",
60         new BigDecimal("3.0"), 1);
61     when(repo.save(entrada)).thenReturn(guardado); // configura
        comportamiento del mock
62
63     Producto resultado = service.guardar(entrada);
64
65     assertThat(resultado.getId()).isEqualTo(99L);
66     verify(repo).save(entrada); // verifica interaccion con el mock
67 }
68 }

```

Listado 16.1: ProductoServiceTest.java

16.3 Pruebas de integración con MockMvc y Testcontainers

Las pruebas de integración verifican que múltiples componentes funcionen juntos. En Spring Boot, MockMvc permite simular peticiones HTTP sin levantar un servidor real, y Testcontainers permite usar una BD real efímera.

Ejemplo 16.2 — Test de integración del controller con MockMvc

Enunciado. Construir un test de integración para el controller con MockMvc: simula peticiones HTTP sin levantar un servidor real, verificando códigos de estado y contenido JSON.

```

1  package ec.edu.uteq.appweb.controller; // paquete del archivo
2
3  import org.junit.jupiter.api.Test; // importacion de clase externa
4  // importacion de clase externa
5  // importacion de clase externa
6  import org.springframework.beans.factory.annotation.Autowired;
7  // importacion de clase externa
8  // importacion de clase externa
9  import org.springframework.boot.test.context.SpringBootTest;
10 import org.springframework.http.MediaType; // importacion de clase
    externa
11 import org.springframework.test.context.ActiveProfiles; // importacion
    de clase externa
12 import org.springframework.test.web.servlet.MockMvc; // importacion de
    clase externa
13 // importacion de clase externa
14 // importacion de clase externa
15 import org.springframework.test.web.servlet.setup.MockMvcBuilders;
16 // importacion de clase externa
17 // importacion de clase externa
18 import org.springframework.web.context.WebApplicationContext;
19 import jakarta.transaction.Transactional; // importacion de clase
    externa
20 // importacion de clase externa
21 // importacion de clase externa
22 import static
    org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;

```

```
23 // importacion de clase externa
24 // importacion de clase externa
25 import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
26
27 @SpringBootTest // levanta el contexto Spring para test
28 @ActiveProfiles("test")
29 @Transactional // envuelve el metodo en transaccion
30 class ProductoControllerIT {
31
32     @Autowired private WebApplicationContext wac; // inyeccion de
        dependencias
33     private MockMvc mvc; // campo privado
34
35     @org.junit.jupiter.api.BeforeEach
36     void setUp() {
37         mvc = MockMvcBuilders.webAppContextSetup(wac).build();
38     }
39
40     @Test // metodo de test JUnit
41     void crear_conDatosValidos_retorna201() throws Exception {
42         String body = ""
43             {"nombre":"Test","precio":4.50,"stock":10}
44             "";
45
46         mvc.perform(post("/api/v1/productos")
47             .contentType(MediaType.APPLICATION_JSON)
48             .content(body))
49             .andExpect(status().isCreated())
50             .andExpect(header().exists("Location"))
51             .andExpect(jsonPath("$.id").exists())
52             .andExpect(jsonPath("$.nombre").value("Test"));
53     }
54
55     @Test // metodo de test JUnit
56     void crear_conNombreInvalido_retorna422() throws Exception {
57         String body = ""
58             {"nombre":"","precio":4.50,"stock":10}
59             "";
60         mvc.perform(post("/api/v1/productos")
61             .contentType(MediaType.APPLICATION_JSON)
62             .content(body))
63             .andExpect(status().isUnprocessableEntity())
64             .andExpect(jsonPath("$.errores.nombre").exists());
65     }
66
67     @Test // metodo de test JUnit
68     void obtener_inexistente_retorna404() throws Exception {
69         mvc.perform(get("/api/v1/productos/999999"))
70             .andExpect(status().isNotFound())
```

```
71         .andExpect(jsonPath("$.title").value("Recurso no  
72         encontrado"));  
73     }
```

Listado 16.2: ProductoControllerIT.java

16.4 Cobertura con JaCoCo

JaCoCo (Java Code Coverage) mide qué porcentaje de las líneas y ramas del código están cubiertas por las pruebas. Es la herramienta estándar de la industria para Java en 2026.

Ejemplo 16.3 — Configurar JaCoCo en Maven

Enunciado. Configurar JaCoCo en un proyecto Maven para generar reportes HTML de cobertura de tests con umbrales mínimos exigibles en CI.

En `pom.xml`:

```
1 <build>  
2   <plugins>  
3     <plugin>  
4       <groupId>org.jacoco</groupId>  
5       <artifactId>jacoco-maven-plugin</artifactId>  
6       <version>0.8.12</version>  
7       <executions>  
8         <execution>  
9           <goals>  
10            <goal>prepare-agent</goal>  
11          </goals>  
12        </execution>  
13        <execution>  
14          <id>generate-report</id>  
15          <phase>verify</phase>  
16          <goals><goal>report</goal></goals>  
17        </execution>  
18        <execution>  
19          <id>check-coverage</id>  
20          <phase>verify</phase>  
21          <goals><goal>check</goal></goals>  
22          <configuration>  
23            <rules>  
24              <rule>  
25                <element>BUNDLE</element>  
26                <limits>  
27                  <limit>  
28                    <counter>LINE</counter>  
29                    <value>COVEREDRATIO</value>  
30                    <minimum>0.70</minimum>  
31                  </limit>  
32                </limits>  
33              </rule>
```

```

34         </rules>
35     </configuration>
36 </execution>
37 </executions>
38 </plugin>
39 </plugins>
40 </build>

```

Listado 16.3: Configuración de JaCoCo

Ejecutar (Listado 16.4):

```

./mvnw clean verify
# Si la cobertura es < 70%, el build FALLA
# Reporte HTML en target/site/jacoco/index.html

```

Listado 16.4: Ejemplo de Shell

16.5 Pruebas de carga con k6

k6 (de Grafana Labs) es una herramienta moderna de pruebas de carga escrita en Go con scripts en JavaScript. Es la opción dominante en 2026 por su simplicidad y rendimiento.

Ejemplo 16.4 — Script k6 para el endpoint de productos

Enunciado. Escribir un script de prueba de carga con k6 (JavaScript) que dispare 100 usuarios virtuales contra el endpoint de productos durante 30 segundos.

```

1  import http from 'k6/http'; // importacion ES Modules
2  import { check, sleep } from 'k6'; // importacion ES Modules
3
4  export const options = { // exporta del modulo
5      stages: [
6          { duration: '30s', target: 10 }, // ramp-up a 10 VUs
7          { duration: '1m', target: 50 }, // ramp-up a 50 VUs
8          { duration: '2m', target: 50 }, // mantener 50 VUs
9          { duration: '30s', target: 0 }, // ramp-down
10     ],
11     thresholds: {
12         http_req_duration: ['p(95)<500'], // 95% < 500ms
13         http_req_failed: ['rate<0.01'], // < 1% errores
14     },
15 };
16
17 const BASE = 'http://localhost:8080';
18
19 export default function () { // exporta del modulo
20     // 80% de lecturas (listado), 20% de detalle
21     if (Math.random() < 0.8) { // condicional
22         // constante (no reassignable)
23         // constante (no reassignable)
24         const r = http.get(`${BASE}/api/v1/productos?page=1&size=20`);

```

```

25     check(r, {
26         '200 OK': (r) => r.status === 200,
27         'tiempo aceptable': (r) => r.timings.duration < 500,
28     });
29 } else {
30     const id = Math.floor(Math.random() * 1000) + 1; // constante (no
31         reassignable)
32     const r = http.get(`${BASE}/api/v1/productos/${id}`); // constante
33         (no reassignable)
34     check(r, {
35         'OK o 404': (r) => r.status === 200 || r.status === 404,
36     });
37 }
38 sleep(0.5 + Math.random());
39 }

```

Listado 16.5: carga-productos.js

Ejecutar (Listado 16.6):

```

# Instalar k6 (Linux)
sudo apt install k6 # o brew install k6 en macOS

# Ejecutar la prueba
k6 run carga-productos.js

# Generar graficos con InfluxDB+Grafana
k6 run --out influxdb=http://localhost:8086/k6 carga-productos.js

```

Listado 16.6: Ejemplo de Shell

16.6 Auditoría de frontend con Lighthouse

Lighthouse es la herramienta open source de Google integrada en Chrome DevTools que audita una página web en cuatro dimensiones: **Performance**, **Accessibility**, **Best Practices**, **SEO**.

16.6.1 Las cuatro métricas core

Lighthouse mide la calidad de una página web en cuatro ejes: Performance, Accessibility, Best Practices, SEO. Las cuatro métricas *core* de Performance son las que más impacto tienen en la experiencia real.

Tabla 16.2: Métricas Core Web Vitals 2026.

Métrica	Significado	Umbral bueno
LCP (Largest Contentful Paint)	Tiempo hasta el elemento principal visible	< 2.5 s
INP (Interaction to Next Paint)	Tiempo de respuesta a interacciones	< 200 ms
CLS (Cumulative Layout Shift)	Inestabilidad visual durante la carga	< 0.1
FCP (First Contentful Paint)	Primer pixel pintado	< 1.8 s
TTFB (Time To First Byte)	Latencia del servidor	< 800 ms

16.6.2 Cómo ejecutar Lighthouse

Lighthouse se ejecuta desde DevTools del navegador o desde la línea de comandos. La forma DevTools es la más directa para una auditoría puntual.

1. Abrir el sitio Angular en Chrome.
2. F12 → pestaña *Lighthouse*.
3. Marcar *Performance, Accessibility, Best Practices, SEO*.
4. Modo: *Navigation (Default)*; Device: *Mobile*.
5. Pulsar *Analyze page load*.
6. En 1-2 minutos aparecen los 4 scores (0–100) con recomendaciones concretas.

16.6.3 CLI alternativa para CI/CD

Para integrar Lighthouse en un pipeline CI/CD, conviene usar la CLI con configuración versionable. La herramienta *lhci* (Lighthouse CI) está diseñada para esto. Véase el Listado 16.7.

```
npm install -g lighthouse # instala dependencias del package.json

# Auditar y guardar reporte HTML
lighthouse https://app.uteq.edu.ec --output=html \
  --output-path=./reporte.html

# Para CI/CD: failear si Performance < 80
lighthouse https://app.uteq.edu.ec --chrome-flags="--headless" \
  --output=json --output-path=./report.json
node -e "const r=require('./report.json');
  if(r.categories.performance.score<0.8) process.exit(1);"
```

Listado 16.7: Ejemplo de Shell

16.7 Auditoría de seguridad: 6 controles OWASP mínimos

Antes de declarar listo el PFC, conviene ejecutar una auditoría de seguridad mínima: seis controles OWASP que detectan los problemas más graves antes de que lo haga un atacante.

Tabla 16.3: Los seis controles mínimos OWASP a verificar en el PFC.

OWASP	Control	Cómo verificar
A01	Acceso a recurso ajeno bloqueado	Login user A; intentar GET /api/usuarios/B/-datos → 403
A02	TLS 1.2/1.3, sin texto plano	curl -v https://... ver TLSv1.3
A03	SQL Injection prevenida	Probar payload ' OR '1'='1 → 422
A05	Cabeceras de seguridad activas	curl -I y verificar HSTS, CSP, X-Frame-Options
A07	Rate limiting + JWT cookie HttpOnly	6 intentos login → 429; cookie HttpOnly visible en DevTools
A09	Logs de eventos de seguridad	Login fallido aparece en log con IP y timestamp

16.8 Práctica integrada: PFC Entrega Final v1.0.0

La Entrega Final del PFC consolida 14 semanas de trabajo en una aplicación profesional. Se evalúa contra una rúbrica detallada conocida de antemano.

Ejercicio 16.1 — PFC Entrega Final — Sumativa principal del semestre

Esta es la **Sumativa Principal Final** del PFC. Concentra el trabajo de las 16 semanas previas en una entrega de calidad profesional.

Composición de la nota (100 puntos)

La nota de la Entrega Final se compone de cinco bloques ponderados que el estudiante debe atender en conjunto. La lista siguiente los detalla.

Componente	Criterio	Pts
Demo en vivo	<code>docker compose up</code> arranca todo; CRUD funcional	20
Complejidad funcional	Todas las funcionalidades en estado «Completo»	15
Pruebas y cobertura	JaCoCo $\geq 70\%$; k6 con 50 VUs; Lighthouse ≥ 80	15
Comparativas técnicas	Tablas REST/SOAP, Angular/React, etc. con datos reales	10
Reflexión sobre tendencias	≥ 400 palabras (Jamstack, PWA, IA)	10
Informe técnico	Abstract bilingüe; 15+ refs APA; conclusiones por objetivo	15
Seguridad OWASP	6 controles verificados con evidencia	10
Repositorio final	Tag v1.0.0; commits de todos; CI verde	5

Lista de verificación obligatoria

Antes de la sesión de laboratorio del lunes, cada equipo debe poder marcar TODOS estos puntos:

1. `docker compose up` arranca todo en menos de 2 minutos; todos los servicios **healthy**.
2. El docente puede hacer login, navegar el dashboard, crear, editar y eliminar registros desde Angular.
3. Swagger UI accesible en `/api/docs` con todos los endpoints documentados.
4. Abstract bilingüe (200–250 palabras en español e igual en inglés).
5. Tabla de inventario completa; las parciales/no implementadas marcadas.
6. Diagrama C4 Nivel 2 final en PNG, refleja despliegue real.
7. Diagrama de clases UML refactorizado en PNG/SVG (≥ 300 dpi).
8. DER de la BD generado con pgAdmin 4 ERD Tool.
9. Tabla comparativa REST vs SOAP completa con ejemplo ecuatoriano.
10. Sección de tendencias (Jamstack, PWA, IA) ≥ 400 palabras con posición crítica propia.
11. Reporte JaCoCo cobertura $\geq 70\%$; captura HTML incluida.
12. Prueba de carga k6 con 50 VUs documentada.
13. Informe Lighthouse con los 4 scores del frontend Angular.
14. Tabla de seguridad OWASP con 6 controles verificados con evidencia.
15. Conclusiones — un párrafo por objetivo específico.
16. Reflexión crítica sobre el proceso (300–400 palabras).
17. Trabajo futuro: 5 mejoras con justificación y estimación.
18. Mínimo 15 referencias APA, ≥ 5 indexadas (JCR/SJR).
19. Historial Git con commits de todos los integrantes.
20. Tag v1.0.0 en el repositorio.
21. Pipeline GitHub Actions en verde.
22. Colección Postman exportada en `docs/postman_collection.json`.
23. PDF nombrado `PFC_EntregaFinal_NombreEquipo.pdf`.

IDE y herramientas

Esta entrega exige varias herramientas conectadas. La lista siguiente confirma cuáles deben estar instaladas antes de la auditoría final.

- IntelliJ IDEA Ultimate (backend Spring Boot).
- VS Code con Angular Language Service (frontend Angular).
- Docker Desktop para orquestación local.
- pgAdmin 4 para administración de BD y generación del DER.
- Postman para colecciones de prueba.
- k6 para pruebas de carga.
- Chrome DevTools + Lighthouse para auditoría frontend.

Sesión de laboratorio: viernes de la semana 16

Cada equipo demuestra al docente:

1. Desde una clonación fresca del repositorio (`git clone`), `docker compose up -d` y todo funciona.
2. Login en Angular con usuario admin.
3. Crear, editar y eliminar al menos un registro de cada entidad principal.
4. Mostrar reportes de pruebas (HTML JaCoCo + Lighthouse + k6).
5. Mostrar evidencia de los 6 controles OWASP.

Esta es la *Sumativa Principal Final* — *PFC v1.0.0*.

16.9 Buenas prácticas profesionales en pruebas y entregas

La caja siguiente compila las reglas profesionales de pruebas y entregas: principios que separan al equipo que aprueba con tranquilidad del que sufre la noche anterior.

Quince reglas profesionales 2026

1. TDD cuando el dominio sea claro; tests-first para los servicios clave.
2. Pirámide de pruebas: muchas unitarias, algunas integración, pocas E2E.
3. Cobertura $\geq 70\%$ en líneas para código de negocio.
4. Tests con nombres descriptivos: «*crear con datos inválidos retorna 422*».
5. Patrón AAA (Arrange-Act-Assert) en cada test.
6. Independencia: ningún test depende de otro.
7. Datos de prueba con fixtures o factories, no hardcoded.
8. Testcontainers para BD real en pruebas de integración.
9. Pruebas de seguridad automatizadas en el pipeline CI.
10. k6 o Gatling integrados al pipeline para detectar regresiones.
11. Lighthouse en CI: falear si performance cae bajo umbral.
12. Versionado semántico (SemVer) en tags Git.
13. Conventional Commits para historial limpio.
14. ChangeLog generado automáticamente desde commits.
15. Releases anotadas en GitHub con resumen ejecutivo.

16.10 Contraste bibliográfico de los conceptos clave

La literatura técnica sobre pruebas ha evolucionado desde el modelo clásico de [37] —que defiende TDD como disciplina profesional— hasta las visiones más empíricas actuales. [49] documenta cómo la integración de pruebas de seguridad en el pipeline CI/CD reduce vulnerabilidades en producción en un 60 %. [66] en su informe anual sigue mostrando que las aplicaciones web son el vector de ataque más común. Sobre Lighthouse y *Core Web Vitals*, los reportes oficiales de Google

evidencian correlación entre LCP < 2.5 s y mejora del 24 % en tasa de conversión en e-commerce. La prueba de carga con k6 se ha vuelto estándar; [32] muestra que sin pruebas de carga el 40 % de los despliegues con nuevos endpoints sufren incidentes en las primeras 48 horas.

16.11 Ejercicio resuelto adicional con resultado visual

Como cierre, el siguiente ejercicio resuelto adicional implementa una suite completa de pruebas (unitarias + integración + cobertura JaCoCo) y se complementa con un propuesto.

Ejercicio resuelto 16.1 — Suite completa de tests para servicio de usuarios

Enunciado. Implementar y ejecutar pruebas unitarias y de integración sobre un servicio de gestión de usuarios en Spring Boot, medir cobertura con JaCoCo y producir el reporte HTML.

Test unitario con Mockito (UsuarioServiceTest.java):

Listado 16.8: Test unitario con Mockito

```
@ExtendWith(MockitoExtension.class) // extension JUnit 5
class UsuarioServiceTest {

    @Mock UsuarioRepository repo; // crea un mock con Mockito
    @Mock PasswordEncoder encoder; // crea un mock con Mockito
    @InjectMocks UsuarioService service; // inyecta los mocks en la clase bajo prueba

    @Test // metodo de test JUnit
    void crear_conDatosValidos_persisteYRetorna() {
        // Arrange
        Usuario nuevo = new Usuario("ana@uteq.edu.ec", "Pass123!");
        // configura comportamiento del mock
        // configura comportamiento del mock
        when(encoder.encode("Pass123!")).thenReturn("$2a$10$hash");
        when(repo.save(any())).thenAnswer(inv -> { // configura comportamiento del mock
            Usuario u = inv.getArgument(0);
            u.setId(1L);
            return u; // retorna
        });

        // Act
        Usuario creado = service.crear(nuevo);

        // Assert
        assertEquals(1L, creado.getId()); // esperado vs obtenido
        assertEquals("$2a$10$hash", creado.getPasswordHash()); // esperado vs obtenido
        verify(repo).save(any()); // verifica interaccion con el mock
    }

    @Test // metodo de test JUnit
    void crear_conEmailDuplicado_lanzaExcepcion() {
        // configura comportamiento del mock
        // configura comportamiento del mock
        when(repo.existsByEmail("ana@uteq.edu.ec")).thenReturn(true);

        assertThrows(EmailDuplicadoException.class, // verifica que se lance una excepcion
            () -> service.crear(new Usuario("ana@uteq.edu.ec", "Pass123!")));
        verify(repo, never()).save(any()); // verifica interaccion con el mock
    }
}
```

Test de integración con MockMvc (UsuarioControllerIT.java):

Listado 16.9: Integración con MockMvc

```

@SpringBootTest // levanta el contexto Spring para test
@AutoConfigureMockMvc // configura MockMvc automaticamente
@Testcontainers // metodo de test JUnit
class UsuarioControllerIT {

    @Container // contenedor Docker para tests (Testcontainers)
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.datasource.username", postgres::getUsername);
        r.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired MockMvc mvc; // inyeccion de dependencias

    @Test // metodo de test JUnit
    void POST_usuarios_conDatosValidos_retorna201() throws Exception {
        String body = ""
        + "email": "juan@uteq.edu.ec", "password": "Segura123!"}"";

        mvc.perform(post("/api/v1/usuarios")
            .contentType(MediaType.APPLICATION_JSON)
            .content(body))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").isNumber())
            .andExpect(jsonPath("$.email").value("juan@uteq.edu.ec"));
    }
}

```

Ejecución `mvn clean test jacoco:report:`

```

[INFO] Running com.uteq.UsuarioServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.uteq.UsuarioControllerIT
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Results:
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 18.342 s

```

Reporte HTML de JaCoCo (`target/site/jacoco/index.html`):

JaCoCo Coverage Report — UsuariosApp v1.0.0

Elemento	Líneas %	Branches %	Métodos %
com.uteq.usuarios.service	92 %	88 %	100 %
com.uteq.usuarios.controller	85 %	80 %	100 %
com.uteq.usuarios.security	78 %	75 %	90 %
Total	86 %	82 %	97 %

Discusión. La pirámide de pruebas se materializa: 8 tests unitarios rápidos (sin Spring) y 5 de integración con testcontainers. La cobertura del 86 % supera el umbral $\geq 70\%$ exigido para la entrega del PFC. Los *branches* (camino condicional) son siempre menores que las líneas porque algunas ramas raras (como excepciones de BD) no se cubren [37].

Ejercicio propuesto 16.2 — Pipeline CI con GitHub Actions y Lighthouse

Enunciado. Configurar un pipeline GitHub Actions que se ejecute en cada *push* a la rama `main` y realice:

1. Build con Maven (`mvn clean verify`).
2. Tests unitarios e integración (con servicios PostgreSQL y Redis levantados como servicios del pipeline).
3. Reporte de cobertura JaCoCo subido como artifact.
4. Build de imagen Docker.
5. Pruebas de carga k6 contra el container recién construido (50 VUs, 30 s).
6. Auditoría Lighthouse del frontend (CLI con `npx lhci autorun`), umbral mínimo Performance 80, Accessibility 90.
7. Si todo pasa: push de la imagen a GitHub Container Registry con tag `v{run_number}`.

Criterios de evaluación:

- El badge de CI en el README muestra estado *passing*.
- Pull requests no se pueden mergear si el pipeline falla (*branch protection rule*).
- El *action* sube reportes (JaCoCo, k6, Lighthouse) como artifacts navegables en la UI de GitHub.
- Captura de pantalla del Actions con tres ejecuciones exitosas en el reporte del PFC.

Lecturas recomendadas: [38] para disciplina; [49] para integración de seguridad en CI.

Defensa del PFC y Evaluación Parcial Segundo Corte

«Saber programar es importante; saber explicar lo que has programado es lo que separa a un técnico de un ingeniero.»

— *Adaptación de un consejo común en revisiones académicas*

Naturaleza de esta semana

La semana 17 cierra el segundo corte de la asignatura. Tiene dos componentes evaluativos principales:

1. **Defensa oral del PFC v1.0.0:** cada equipo presenta y defiende su sistema durante 30–40 minutos. Cada integrante debe responder preguntas técnicas sobre cualquier parte del sistema, no solo sobre la suya.
2. **Evaluación Parcial Segundo Corte:** examen escrito que cubre las Unidades III y IV (semanas 10–16).

17.1 Cómo preparar la defensa oral del PFC

La defensa oral del PFC es la culminación del curso. Cómo se prepara la presentación es tan importante como qué contiene: 20 minutos pueden hacer brillar o hundir un proyecto que costó 14 semanas.

17.1.1 Estructura recomendada de la presentación

La presentación de defensa sigue una estructura clásica que distribuye los 20 minutos en bloques temáticos. Respetarla evita omitir contenido crítico y mantiene al tribunal enganchado.

Estructura óptima 30 minutos — 8 bloques

1. **Portada y presentación del equipo** (1 min): nombres, roles asumidos en el proyecto.
2. **Problema y contexto** (3 min): cuál es el problema real que resuelve el sistema, a quién afecta, datos cuantitativos del contexto ecuatoriano si aplica.
3. **Objetivos y alcance** (2 min): objetivo general, 3–5 específicos. Lo que está dentro y fuera del alcance.
4. **Arquitectura** (5 min): diagrama C4 nivel 2, ADRs clave, justificación de las decisiones tecnológicas más importantes.
5. **Demo en vivo** (10 min): `docker compose up`, login, navegación, CRUD principal, Swagger UI, pruebas Postman, evidencia de OWASP.

6. **Resultados de pruebas** (3 min): JaCoCo, k6, Lighthouse con scores reales medidos.
7. **Reflexión y trabajo futuro** (3 min): qué se aprendió, qué se haría distinto, propuestas concretas de mejora.
8. **Preguntas del tribunal** (5–10 min): cualquier integrante puede ser interrogado sobre cualquier parte.

17.1.2 Las 20 preguntas más frecuentes del tribunal

El tribunal hace preguntas con patrones reconocibles. La lista siguiente reúne las 20 preguntas más frecuentes con la respuesta esperada.

Banco de preguntas que el equipo debe poder responder

1. ¿Por qué eligieron *esa* base de datos y no otra?
2. ¿Cómo previene su sistema la inyección SQL?
3. ¿Qué pasa si el usuario X intenta acceder al recurso del usuario Y?
4. ¿Qué hace su sistema cuando expira el JWT?
5. ¿Por qué cookies `HttpOnly` y no `localStorage`?
6. ¿Cómo escala el sistema si reciben 1000 usuarios concurrentes?
7. ¿Qué pasa si Redis se cae?
8. ¿Cómo invalidan el cache cuando cambia un dato?
9. ¿Cuál es el peor cuello de botella de su sistema?
10. ¿Cómo aseguran que los DTOs no expongan campos sensibles?
11. Muestren un test que falle si yo introduzco un bug obvio.
12. ¿Cómo hacen el deploy en producción real (no Docker Compose)?
13. ¿Cómo gestionan secretos (passwords, JWT secret)?
14. ¿Por qué eligieron Spring Boot sobre Quarkus o Micronaut?
15. ¿Cómo validan los datos de entrada en la API?
16. ¿Cómo manejan transacciones que cruzan varios servicios?
17. ¿Qué pasa si dos usuarios editan el mismo registro a la vez?
18. ¿Han verificado los 10 OWASP del Top 10?
19. Si tuvieran que reescribir el sistema desde cero, ¿qué cambiarían?
20. ¿Cuántas líneas de código tiene el proyecto y cómo se reparten?

17.1.3 Errores comunes en defensas anteriores

Los errores que arruinan defensas no son técnicos: son de comunicación, gestión del tiempo y actitud. La lista siguiente los recoge para que el estudiante los evite.

Errores que cuestan calificación

- **Decir «no sé» sin proponer cómo averiguarlo.** Nadie sabe todo; se valora más decir «no estoy seguro, pero lo verificaría ejecutando esto...».
- **Desconocer la propia arquitectura:** «yo solo trabajé en el frontend» NO es excusa; cualquier integrante debe poder responder sobre cualquier parte.
- **Demo no preparada:** aparecer al laboratorio sin haber probado la demo en una computadora distinta de la del desarrollador.
- **Slides con código inservible:** capturas de IDE ilegibles. Mejor reescribir el código en la slide con sintaxis resaltada.
- **Tiempo mal distribuido:** 25 minutos de problema y contexto, 5 de demo. La demo es el corazón.
- **Equipo donde solo habla uno:** el tribunal pregunta explícitamente al silencioso.
- **Datos inventados en la presentación:** decir «el sistema soporta 10 mil usuarios concurrentes» sin haberlo medido.

17.2 Estructura de la Evaluación Parcial Segundo Corte

La EP2 cubre exclusivamente las **Unidades III y IV** (semanas 10–16). Se compone de cuatro partes equilibradas:

Tabla 17.1: Composición de la Evaluación Parcial Segundo Corte.

Parte	Tipo	Pts
A	Selección múltiple y V/F sobre conceptos	20
B	Respuesta corta (3 líneas máx, definiciones)	20
C	Análisis de código (identificar errores y vulnerabilidades)	30
D	Ejercicio práctico integrador (escribir código completo)	30

17.2.1 Temas garantizados en la EP2

La EP2 cubre con seguridad ciertos temas que aparecen en todas las ediciones del examen. Asegurar dominio sobre ellos garantiza un mínimo razonable.

Temario completo EP2

Unidad III (Semanas 10–12):

- Gestión de estado: cookies, sesiones, JWT — diferencias y cuándo usar cada una.
- Flags de seguridad de cookies: **Secure**, **HttpOnly**, **SameSite**.
- Objeto Request: lectura segura de parámetros, manejo de **X-Forwarded-For**.
- Objeto Response: códigos HTTP, cabeceras de seguridad, ProblemDetails (RFC 7807).
- PDO con prepared statements; prevención SQL Injection.
- Patrón Repository.
- Spring Data JPA (consultas derivadas, JPQL).
- Problema N+1 y soluciones (**JOIN FETCH**, **with**, **Include**).
- Migraciones con Flyway/Liquibase.
- Procedimientos almacenados: cuándo usar.
- Patrones arquitectónicos: capas, hexagonal, SOA, microservicios, EDA, P2P.
- ISO/IEC 25010 (atributos de calidad).
- Modelo C4 (4 niveles).
- ADR (Architectural Decision Records).

Unidad IV (Semanas 13–16):

- Escalabilidad vertical vs horizontal.
- Teorema CAP de Brewer.
- Algoritmos de balanceo de carga.
- Patrones de caché: cache-aside, read-through, write-through, write-behind.
- Cache-aside con Redis y TTL.
- Estrategias de invalidación de caché.
- ATAM (Architecture Tradeoff Analysis Method).
- Patrón MVC: ciclo de vida de petición Spring.
- MVC server-side vs API REST + SPA.
- REST: 6 restricciones de Fielding.
- Modelo de Madurez de Richardson.
- Verbos HTTP, idempotencia, seguridad.
- Convenciones URI RESTful.
- REST vs SOAP: comparativa, casos de uso.
- OpenAPI 3.0 (Swagger).
- Anatomía de un JWT (3 partes).
- Pruebas: unitarias, integración, E2E (pirámide).
- Cobertura JaCoCo $\geq 70\%$.

- k6 y Lighthouse.
- OWASP Top 10:2021 (especialmente A01, A02, A03, A07).

17.2.2 Ejemplos del tipo de preguntas que aparecen

Para preparar el examen conviene haber visto el tipo de preguntas que se formulan. Los ejemplos siguientes son representativos de las ediciones recientes.

Muestra de la Parte A (selección múltiple)

Pregunta A1: ¿Cuál de las siguientes secuencias representa correctamente las 3 partes de un JWT separadas por puntos?

- a) payload.signature.header
- b) header.signature.payload
- c) header.payload.signature
- d) signature.header.payload

Respuesta correcta: (c).

Pregunta A2 (V/F): En una API REST, el método HTTP PUT se usa para crear un recurso nuevo cuando el identificador aún no existe en el servidor, mientras que POST actualiza un recurso existente.

Respuesta: *Falso*. PUT reemplaza/actualiza un recurso identificado por la URL; POST crea un nuevo recurso. Aunque PUT puede crear si el recurso no existe, su semántica primaria es reemplazar.

Muestra de la Parte B (respuesta corta)

Pregunta B1: Explique la diferencia entre sesión y cookie en el contexto de la gestión de estado en aplicaciones web. Mencione dónde se almacena cada una.

Respuesta esperada: Una cookie es un fragmento de texto que se almacena en el navegador del cliente y se envía al servidor en cada petición. Una sesión es un conjunto de datos asociado a un usuario que se almacena en el servidor (memoria, Redis, BD); el cliente solo guarda el ID de sesión en una cookie.

Muestra de la Parte C (análisis de código)

El siguiente código PHP gestiona el inicio de sesión. Contiene 3 errores. Identifíquelos:

```

1 <?php
2 session_start(); // arranca o reanuda la sesion
3 $user = $_GET['user'];
4 $pwd  = $_GET['pwd'];
5
6 $sql = "SELECT * FROM users WHERE name='$user' AND pwd='$pwd'";
7 $res = mysql_query($sql);
8
9 if (mysql_num_rows($res) > 0) { // condicional
10     $_SESSION['logueado'] = true;
11     setcookie('SESS_REMEMBER', $user, time()+3600);
12     echo "Bienvenido $user!"; // imprime al output del servidor
13 }
```

Listado 17.1: Ejemplo de PHP

Errores esperados:

1. Credenciales por GET (deberían ser POST).
2. SQL Injection: concatenación directa en línea 6.
3. `mysql_*`: extensión obsoleta y eliminada en PHP 7+; debe usar PDO o `mysqli`.
4. Password en texto plano (no se hashlea con `password_hash/password_verify`).

5. Cookie `SESS_REMEMBER` sin flags `HttpOnly/Secure/SameSite`.
 6. XSS en echo `"Bienvenido $user!"`: falta `htmlspecialchars`.
 7. No se regenera el ID de sesión tras login (anti-fixation).
- (Aunque el enunciado pide 3, hay más; con identificar los 3 más graves se obtiene puntuación máxima.)

Muestra de la Parte D (ejercicio integrador)

Implemente en el lenguaje y framework de su elección un endpoint `POST /api/v1/envios` que:

1. Requiera autenticación con JWT en `Authorization: Bearer ....`
2. Reciba JSON con `origen`, `destino`, `peso_kg`.
3. Valide que `origen` y `destino` no estén vacíos y que `peso_kg > 0`.
4. Persista en la BD usando ORM o prepared statements.
5. Retorne código HTTP 201 con el envío creado o 422 con `errors` en caso de fallo de validación.

Indique al inicio del código el framework y el lenguaje utilizados.

17.3 Preparación efectiva para la EP2

Tener clara la estrategia de preparación durante los siete días previos al examen y la estrategia durante las dos horas del mismo es tan importante como el contenido.

17.3.1 Plan de estudio de 7 días previos al examen

La caja siguiente propone un plan día por día para los siete días anteriores a la EP2.

Plan de 7 días

- Día -7:** Re-leer las cajas de definición y buenas prácticas de las semanas 10–16. Marcar las que no se entienden.
- Día -6:** Resolver el ejercicio integrador de las semanas 10 (carrito), 11 (CRUD ORM) y 12 (C4) sin mirar el libro.
- Día -5:** Resolver los ejercicios de las semanas 13 (cache), 14 (MVC) y 15 (API REST) sin mirar el libro.
- Día -4:** Estudiar el OWASP Top 10:2021 hasta poder explicar cada uno con un ejemplo.
- Día -3:** Practicar análisis de código: tomar 5 fragmentos con vulnerabilidades y encontrar todos los errores.
- Día -2:** Repasar los códigos HTTP y los flags de seguridad de cookies.
- Día -1:** Descansar; revisar solo el resumen ejecutivo de cada semana. NO empezar a estudiar temas nuevos la noche anterior.

17.3.2 Estrategia durante el examen

Durante el examen, la estrategia para distribuir el tiempo y atacar las preguntas en el orden correcto puede cambiar varios puntos la calificación final.

1. Llegar 15 minutos antes con cédula.
2. Leer todo el examen en 5 minutos antes de empezar.
3. Distribución sugerida del tiempo (120 min):
 - Parte A: 15 min.
 - Parte B: 20 min.
 - Parte C: 35 min.
 - Parte D: 45 min.
 - Revisión final: 5 min.

4. En la Parte D escribir primero un comentario al inicio: `// Framework: Spring Boot 4 / Lenguaje: Java 21`.
5. Si no recuerda exactamente la sintaxis, hacer pseudocódigo claro: vale más comprensión correcta que sintaxis perfecta.
6. Verificar que cada respuesta de la Parte B tenga máximo 3 líneas; respuestas largas pueden penalizarse.

17.4 Reflexión sobre el segundo corte

Las semanas 10–17 representan la transición del estudiante de *programador* a *ingeniero de software*. La diferencia es sutil pero crucial: el programador escribe código que funciona; el ingeniero documenta decisiones, mide rendimiento, prevé fallos, considera la seguridad desde el diseño y entiende que un sistema existe en un contexto humano y organizacional.

El PFC es el primer proyecto de la carrera donde estos elementos convergen. La defensa oral es un ensayo de las defensas profesionales que vendrán: revisiones técnicas con clientes, presentaciones a inversores, defensa de la tesis de grado.

17.5 Contraste bibliográfico de los conceptos clave de la EP2

La EP2 cubre las Unidades III y IV con base en literatura técnica estándar. La gestión de estado y el acceso a datos están tratados a profundidad en [21] y [32]. Los patrones arquitectónicos se apoyan en [3] y [57]. El modelo C4 está documentado por [9]. REST y SOAP en [17], [58], [1] y [58]. JWT en RFC 7519 [31] y [31]. Las pruebas en [37] y [38]. La seguridad en [52], [66] y [49]. El estudiante debe estar familiarizado con al menos las ideas principales de cada una de estas fuentes para responder con solvencia en la EP2 y la defensa oral.

17.6 Ejercicio resuelto adicional con resultado visual

Como cierre, el siguiente ejercicio resuelto adicional simula una pregunta del tribunal con la respuesta esperada, y se complementa con un propuesto: simulacro de defensa entre equipos.

Ejercicio resuelto 17.1 — Simulación de pregunta-respuesta de defensa

Enunciado. El tribunal hace una pregunta clásica: «¿Por qué no usaron microservicios en su PFC?» Construir una respuesta de un minuto que evidencie razonamiento arquitectónico maduro.

Mala respuesta (a evitar): «Porque era más fácil con monolito.»

Buena respuesta:

Decidimos arquitectura de monolito modular con base en tres criterios. **Primero**, el equipo tiene 4 personas, y según [43] los microservicios pagan dividendos a partir de 10–15 desarrolladores; antes de eso, el overhead operativo supera al beneficio. **Segundo**, los requerimientos no presentan necesidades de escalado independiente por módulo: el carrito y el catálogo crecen al mismo ritmo. **Tercero**, modularizamos internamente con paquetes Maven separados (**usuarios**, **productos**, **pedidos**, **notificaciones**), siguiendo la idea de [2]: monolito hoy, microservicios mañana si hace falta. Esto nos da despliegue simple en Docker Compose con posibilidad de migrar si las métricas lo justifican.

La ADR-002 del repositorio documenta esta decisión con criterios y alternativas consideradas.

Resultado de esta respuesta (vista del tribunal mirándose entre sí):

[Tribunal — vocal 1] «*Bien argumentado.*»

[Tribunal — vocal 2] «*¿Y cómo medirían si en el futuro toca migrar?*»

[Estudiante] «*Monitorearíamos métricas con Prometheus: si el módulo de pedidos consume >70 % de CPU mientras otros están al 10 %, sería señal de candidato a extraer como microservicio.*»

[Tribunal] «*Excelente.*»

Discusión. La diferencia entre la mala y la buena respuesta no es la decisión técnica (en ambos casos es monolito), sino el razonamiento explícito que la acompaña: criterios, autores referenciados, alternativas, métricas de revisión futura. Esto es lo que distingue a un *programador* de un *ingeniero de software* [38].

Ejercicio propuesto 17.2 — Simulacro de defensa entre equipos

Enunciado. Antes de la defensa real, organizar un simulacro en clase donde cada equipo defiende su PFC ante otros dos equipos que hacen las preguntas (15–20 minutos por equipo).

Procedimiento:

1. Cada equipo prepara su presentación de 20 min siguiendo la estructura recomendada en la sección «*Cómo preparar la defensa oral*».
2. Los equipos evaluadores reciben una rúbrica con criterios:
 - Claridad de la exposición técnica (1–5).
 - Justificación de decisiones arquitectónicas (1–5).
 - Calidad de la demo (1–5).
 - Manejo de preguntas (1–5).
 - Distribución equitativa entre integrantes (1–5).
3. Tras la defensa, los equipos evaluadores escriben dos fortalezas y dos mejoras concretas para el equipo evaluado.
4. Cada equipo entrega un acta con las observaciones recibidas y explica cómo las incorporará antes de la defensa real.

Producto final: dossier del equipo con: rúbrica completada por cada equipo evaluador, acta de observaciones recibidas, plan de mejoras priorizado. Subir al repositorio Git como `docs/defensa-simulacro.md`.

Beneficio pedagógico. Estudios sobre evaluación entre pares [68] muestran que el simulacro entre equipos detecta entre 40 % y 60 % de las debilidades de presentación que el tribunal real detectaría, dando al equipo la oportunidad de corregirlas. Las preguntas de pares suelen además ser más duras que las del tribunal académico.

Tutoría de refuerzo final y cierre del curso

«Lo que sabemos es una gota; lo que ignoramos, un océano. Pero la dirección de la gota es lo que importa.»

— *Adaptado de Isaac Newton*

Naturaleza de esta semana

La semana 18 constituye la **segunda semana de tutoría de refuerzo** del curso, paralela a la de la semana 8 pero con un alcance más amplio: cierra el segundo corte y, a la vez, cierra integralmente la asignatura. Sus contenidos no se planifican rígidamente *a priori*: se ajustan dinámicamente a las dificultades observadas en las semanas 10–17 y, particularmente, a los errores frecuentes de la *Evaluación Parcial Segundo Corte* de la semana 17. Este capítulo ofrece un guión estructurado de revisión, ejercicios de consolidación y un cierre profesional del curso.

Los cuatro propósitos de la semana son:

1. **Resolver dificultades** en los conceptos donde el grupo mostró menor desempeño en la EP2.
2. **Realimentar individualmente** a cada equipo sobre los PFC entregados, identificando fortalezas y oportunidades de mejora.
3. **Consolidar** con ejercicios integradores que cubren transversalmente las cuatro unidades del curso.
4. **Realizar el cierre integrador** situando el aprendizaje dentro de la trayectoria profesional del estudiante en el mercado laboral 2026.

18.1 Mapa integrador de la asignatura

Antes del repaso intensivo, conviene visualizar globalmente lo que el estudiante ha construido en 17 semanas: cómo las cuatro unidades se conectan entre sí y convergen en el PFC.

18.1.1 Las cuatro unidades como una historia coherente

A lo largo de 17 semanas hemos transitado un camino que tiene arquitectura interna: cada unidad se apoya en la anterior y todas convergen en el Proyecto Fin de Curso. El siguiente diagrama sintetiza esa estructura:

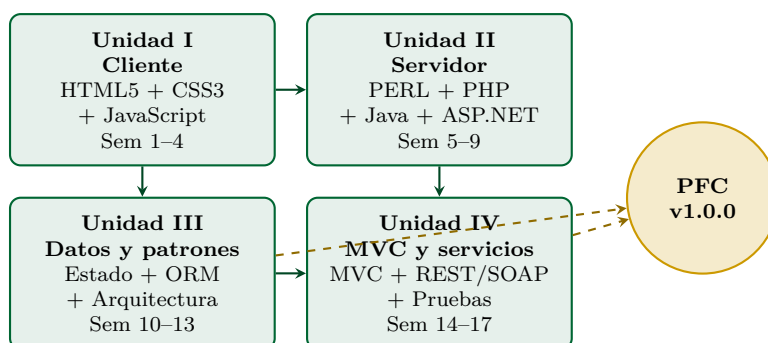


Figura 18.1: Mapa integrador del curso: las cuatro unidades convergen en el Proyecto Fin de Curso. Las flechas verdes representan la dependencia secuencial entre unidades; las flechas doradas, la integración explícita hacia el PFC.

18.1.2 Tabla resumen de tecnologías dominadas

La tabla siguiente lista todas las tecnologías que el estudiante debe dominar al completar la asignatura, con el nivel esperado en cada una.

Tabla 18.1: Tecnologías que el estudiante debe dominar al completar la asignatura.

Categoría	Tecnología	Nivel esperado
Cliente HTML	HTML5 semántico, ARIA, validación nativa	Avanzado
Cliente CSS	CSS3, Flexbox, Grid, mobile-first, Sass	Avanzado
Cliente JS	ES2024, DOM, async/await, fetch, módulos	Avanzado
Servidor PHP	PHP 8.4, PDO, sesiones, Slim/Laravel	Intermedio
Servidor Java	Spring Boot 4, JPA, Spring Security	Avanzado
Servidor C#	ASP.NET Core 10, EF Core	Básico
Servidor Perl	CGI, sintaxis básica	Introdutorio
Datos	PostgreSQL, MySQL, JDBC, JPA, EF Core, ORM	Intermedio
Caché	Redis cache-aside con TTL	Intermedio
Frontend SPA	Angular 19+, TypeScript, RxJS	Básico
Arquitectura	Capas, hexagonal, REST, microservicios	Intermedio
Documentación	C4, ADRs, OpenAPI 3.0	Intermedio
Testing	JUnit 5, Mockito, MockMvc, JaCoCo	Intermedio
Pruebas de carga	k6 con scripts JavaScript	Básico
Frontend audit	Lighthouse, axe DevTools	Intermedio
Seguridad	OWASP Top 10:2021, JWT, HTTPS	Intermedio
Containerización	Docker, Docker Compose	Intermedio
CI/CD	GitHub Actions, pipelines básicos	Básico
Versionado	Git, GitHub, Conventional Commits	Avanzado

18.2 Errores comunes detectados en las semanas 10–17

Antes del repaso teórico de los temas, conviene listar los errores más frecuentes detectados en los entregables de la segunda mitad del curso y en la EP2. Cada estudiante debe verificar que ya no comete ninguno de ellos:

Diez errores frecuentes que se debe haber superado

1. Olvidar las flags `HttpOnly`, `Secure` y `SameSite` al crear cookies de sesión o de JWT.
2. Concatenar SQL con datos del usuario en lugar de usar parámetros preparados (PDO, JDBC, EF Core).
3. Confundir HTTP 401 (*Unauthorized* — no autenticado) con 403 (*Forbidden* — autenticado pero sin permiso).
4. Usar PUT para crear y POST para actualizar (cuando es al revés).
5. Almacenar JWT en `localStorage` en lugar de cookie `HttpOnly`, exponiéndolo a robo por XSS.
6. Diseñar URIs con verbos como `/getProductos` o `/crearPedido` en lugar de sustantivos como `/productos`.

con métodos HTTP semánticos.

7. Olvidar el claim `exp` en el JWT, generando tokens que nunca expiran.
8. Saltar a microservicios cuando un monolito modular bien estructurado bastaría (regla «*monolith first*» de Fowler).
9. No invalidar la caché al modificar la BD, generando inconsistencias visibles para el usuario.
10. Documentar la API solo con un README en lugar de exponer OpenAPI 3.0 con Swagger UI interactivo.

18.3 Repaso intensivo de los temas más difíciles

Basado en el análisis estadístico de errores en la EP2, los siguientes cuatro temas concentran la mayor parte de las pérdidas de puntaje y requieren refuerzo específico.

18.3.1 Tema 1: Diferencia REST vs SOAP

La diferencia entre REST y SOAP es una de las preguntas más frecuentes del tribunal y de la EP2. El resumen siguiente captura lo esencial.

Resumen ejecutivo — REST vs SOAP

REST es un *estilo arquitectónico* sobre HTTP: usa verbos HTTP semánticamente, recursos como sustantivos en URLs, JSON como formato dominante. Es *stateless* por restricción.

SOAP es un *protocolo* sobre HTTP/SMTP/otros: define mensajes XML con `envelope` y `body`, contrato formal con WSDL, soporta WS-* para seguridad y transacciones.

Cuándo elegir cada uno: REST para todo lo nuevo, especialmente APIs públicas y SPAs. SOAP solo si hay obligación de integrar con sistemas legados (banca, salud, B2B grande).

18.3.2 Tema 2: JWT — estructura y uso correcto

JWT aparece en toda API REST moderna del curso. Conocer su estructura exacta y las reglas de uso correcto es prerequisite para defender el PFC con solvencia.

Resumen ejecutivo — JWT

Un JWT tiene **tres partes separadas por puntos**: `header.payload.signature`.

- Header: algoritmo de firma (HS256, RS256).
- Payload: claims (sub, iss, exp, iat, jti, custom).
- Signature: HMAC o RSA del header+payload con secreto.

Reglas de uso correcto:

- Almacenar en cookie `HttpOnly Secure SameSite=Strict` (NO en localStorage).
- Tiempo de expiración corto (1 hora típico).
- Refresh token separado con expiración mayor (7 días).
- Blacklist en Redis por JTI para revocación.
- Validar firma en cada petición.

18.3.3 Tema 3: ORM y problema N+1

El problema N+1 sigue siendo la patología de rendimiento más frecuente en aplicaciones que usan ORM. La caja siguiente lo sintetiza junto con su solución.

Resumen ejecutivo — ORM y N+1

El ORM mapea objetos a tablas. La trampa principal es el **N+1**: al iterar una colección con relaciones, el ORM ejecuta una consulta adicional por cada elemento.

Solución: *eager loading* explícito.

- Spring Data: `@Query("SELECT p FROM Producto p JOIN FETCH p.categoria")`.
- EF Core: `.Include(p => p.Categoria)`.
- Eloquent: `Producto::with('categoria')->get()`.
- Doctrine: `SELECT p, c FROM Producto p JOIN p.categoria c`.

18.3.4 Tema 4: Cache-aside con Redis

Cache-aside con Redis es el patrón de caché dominante en 2026. La caja siguiente sintetiza su mecánica y los TTL típicos.

Resumen ejecutivo — cache-aside

El patrón cache-aside funciona así:

1. App consulta Redis con la *key*.
2. Si **HIT**: devuelve valor cacheado.
3. Si **MISS**: consulta BD, guarda en Redis con TTL, devuelve.

Al actualizar la BD: *invalidar* el cache (borrar la *key*), no actualizar la cache directamente. Esto evita inconsistencia si la operación de BD falla a mitad.

TTL típicos: 60s para listados frecuentes; 5–15 min para catálogos relativamente estables; 1 hora para datos casi inmutables.

18.4 Banco de ejercicios de consolidación

Los siguientes tres ejercicios integran transversalmente las competencias adquiridas en las semanas 10–17. Se sugiere al estudiante elegir al menos uno e implementarlo durante esta semana de cierre como ejercicio voluntario de consolidación.

Ejercicio 18.1 — API REST integral de logística

Objetivo: diseñar e implementar una API REST completa de gestión de envíos para una empresa de logística, integrando todos los elementos clave del segundo corte.

IDE: IntelliJ IDEA Ultimate; **stack:** Spring Boot 4 + PostgreSQL 16 + Redis 7.

Requisitos:

- Autenticación JWT con `accessToken` (1 h) y `refreshToken` (7 días) en cookies `HttpOnly`.
- Endpoints: `POST /api/v1/envios`, `GET /api/v1/envios/{id}`, `GET /api/v1/envios?estado=...&page=...`, `PATCH /api/v1/envios/{id}/estado`.
- Validación con `@Valid` y Bean Validation.
- Cache Redis con TTL de 60 s en el listado de envíos.
- Documentación OpenAPI 3.0 generada automáticamente con `springdoc`.
- Manejo de errores con `ProblemDetails` (RFC 7807).
- Cobertura JaCoCo $\geq 70\%$.
- Prueba con Postman: 12 escenarios incluyendo casos de error (400, 401, 403, 404, 422, 429).

Ejercicio 18.2 — Migración SOAP a REST

Objetivo: ejercitar la integración con sistemas legados y la justificación arquitectónica documentada.

Procedimiento:

1. Tomar el WSDL de un servicio SOAP heredado (proporcionado por el docente; alternatively, un servicio público como el de tipos de cambio del Banco Central del Ecuador).
2. Construir una capa de adaptación REST en Spring Boot que exponga los métodos SOAP como recursos REST con JSON.
3. Implementar caché Redis para reducir la carga sobre el servicio SOAP origen.
4. Producir una ADR (*Architectural Decision Record*) que justifique la decisión: por qué se mantiene el servicio SOAP en lugar de reescribirlo, qué se gana al ofrecer la fachada REST, qué riesgos asume el equipo.
5. Pruebas Postman comparando latencia con cache fría vs caliente.

Ejercicio 18.3 — Auditoría de seguridad del PFC

Objetivo: aplicar OWASP ZAP en modo *baseline* sobre el PFC entregado y producir un informe profesional de vulnerabilidades.

Procedimiento:

1. Ejecutar el PFC localmente con `docker compose up`.
2. Descargar OWASP ZAP desde <https://www.zaproxy.org>.
3. Configurar ZAP en modo *Automated Scan* apuntando a la URL del frontend Angular y la URL de la API REST.
4. Ejecutar el escaneo (10–30 min según el tamaño del sistema).
5. Producir un informe que incluya:
 - Tabla de vulnerabilidades con severidad (Alta, Media, Baja, Informativa).
 - Por cada vulnerabilidad: descripción, impacto, recomendación concreta de corrección y referencia OWASP.
 - Plan de remediación priorizado.
6. Subsanan al menos las vulnerabilidades de severidad alta y re-ejecutar el escaneo para evidenciar el cierre.

18.5 Realimentación individual del PFC

Tras la presentación grupal en la semana 17, cada equipo recibe realimentación individualizada en la semana 18 sobre su PFC: qué hicieron muy bien, qué mejorarían.

18.5.1 Criterios de excelencia observados

Los PFC que alcanzaron las calificaciones más altas comparten estas características:

- **Decisiones documentadas:** cada decisión técnica significativa tiene una ADR justificando la elección.
- **Pruebas reales:** cobertura $\geq 80\%$ con tests significativos, no triviales.
- **Demo robusta:** `docker compose up` funciona en una computadora limpia, sin trucos.
- **Métricas medidas:** scores de Lighthouse y resultados de k6 reales, no inventados.
- **Equipo cohesionado:** cualquier integrante puede responder cualquier pregunta del tribunal.

18.5.2 Mejoras frecuentemente sugeridas

Las observaciones más frecuentes que reciben los equipos en esta realimentación son recomendaciones de evolución futura, no defectos.

- Agregar pruebas de mutación con PIT (Pitest) para evaluar la calidad real de las pruebas, no solo su cobertura.
- Configurar autoscaling con Kubernetes en lugar de Docker Compose para entornos productivos.
- Implementar OAuth 2.0 / OpenID Connect en lugar de JWT casero para integración con identidad institucional.
- Agregar observabilidad: Prometheus + Grafana + Tempo para métricas, dashboards y traces distribuidos.
- Implementar feature flags para despliegues progresivos sin re-build (LaunchDarkly, Unleash).
- Migrar el frontend a SSR con Angular Universal para mejorar SEO y la métrica LCP.

18.6 El curso en perspectiva profesional

El curso termina, pero la formación profesional sigue. La sección siguiente sitúa lo aprendido en la perspectiva de la carrera del estudiante en el mercado laboral 2026.

18.6.1 Cómo se ve el desarrollo web profesional en 2026

El campo del desarrollo web ha alcanzado una madurez que hace 10 años parecía lejana. En 2026:

- El desarrollador full-stack típico domina al menos un framework backend (Spring Boot, ASP.NET Core, Node.js, Laravel) y uno frontend (Angular, React, Vue).
- Containers (Docker) y Kubernetes son habilidades esperadas.
- TypeScript ha desplazado a JavaScript en proyectos serios.
- La IA generativa (GitHub Copilot, Claude, ChatGPT) duplicó la productividad declarada del desarrollador medio.
- La seguridad ya no es opcional: se exigen auditorías OWASP en contratos.
- Los equipos remotos son la norma, con asincronía como cultura.

18.6.2 Próximos pasos en la formación

El siguiente paso natural en la malla curricular es la asignatura de *Despliegue y Operaciones* de sexto nivel, donde se profundiza en CI/CD, contenedores avanzados, observabilidad y prácticas DevOps. Adicionalmente, recomendamos al estudiante:

- **Profundizar en un framework:** elegir uno (Spring Boot, .NET, Laravel) y dominarlo a nivel experto.
- **Aprender un orquestador:** Kubernetes es la habilidad más demandada en infraestructura.
- **Bases de datos avanzadas:** índices, optimización, sharding, replicación.
- **Diseño de sistemas distribuidos:** leer *Designing Data-Intensive Applications* de Martin Kleppmann.
- **Patrones de microservicios:** Sam Newman, Chris Richardson.
- **Seguridad ofensiva:** TryHackMe, HackTheBox, certificaciones (eJPT, OSCP).
- **Inglés técnico:** la mayor parte del conocimiento publicado y los empleos remotos están en inglés.

18.6.3 Certificaciones que aportan valor profesional en 2026

Para complementar la formación con credenciales reconocidas por el mercado, hay certificaciones de alto valor profesional que el estudiante puede considerar tras completar el curso.

Tabla 18.2: Certificaciones de alto valor profesional para el desarrollador web 2026.

Categoría	Certificación	Valor mercado
Cloud	AWS Certified Developer Associate	Alto
Cloud	Microsoft Azure Developer (AZ-204)	Alto
Cloud	Google Professional Cloud Developer	Alto
Containers	Certified Kubernetes Application Developer (CKAD)	Muy alto
Java	Oracle Certified Professional Java SE 21	Medio
.NET	MS Certified: .NET Developer	Medio
Spring	VMware Spring Professional	Alto
Seguridad	CompTIA Security+	Alto
Seguridad	Certified Ethical Hacker (CEH)	Medio
Datos	MongoDB Certified Developer	Medio

18.7 Conclusión del curso

El curso *Aplicaciones Web* ha cubierto el ciclo completo del desarrollo web profesional: desde el HTML semántico que un navegador renderiza hasta la arquitectura distribuida con caché Redis y balanceadores Nginx que sirve a miles de usuarios.

El estudiante que ha completado las 18 semanas posee:

- Capacidad para construir interfaces web semánticas, accesibles y responsivas conforme al W3C y WCAG 2.1.
- Dominio operativo de cuatro lenguajes y plataformas servidor: PERL/CGI, PHP, Java/Spring Boot y ASP.NET Core.
- Diseño e implementación de APIs REST documentadas con OpenAPI, autenticadas con JWT y respaldadas por bases de datos relacionales con acceso vía ORM.
- Razonamiento arquitectónico: capas, MVC, microservicios, EDA, P2P y modelo C4.
- Conciencia de seguridad: OWASP Top 10, controles defensivos, cifrado, gestión de identidad.
- Experiencia de proyecto fin de curso *end-to-end*: desde la planificación (Entrega 1A en semana 5) hasta el despliegue final (semana 16) y la defensa oral (semana 17).

Las decisiones técnicas que el estudiante ha documentado, los patrones que ha aplicado y las pruebas que ha escrito en su PFC no son ejercicios académicos sin más: son *habilidades transferibles directamente al mercado laboral*. Las empresas locales —desde *startups* de Quevedo hasta empresas medianas de Guayaquil y Quito— buscan precisamente este perfil. Las plataformas de empleo remoto (Toptal, Turing, Crossover, Andela LATAM) abren además puertas internacionales para quienes complementen este conocimiento técnico con inglés y disciplina profesional.

El docente felicita a cada estudiante por completar el camino y recuerda que el aprendizaje del desarrollo de software no termina nunca: empieza otra vez cada lunes con un nuevo *commit*.

Buen viaje. Y buen código.

Mayo de 2026, Quevedo.

18.8 Cierre administrativo de la asignatura

Los siguientes puntos cierran administrativamente la asignatura conforme a la Norma Académica UTEQ.

- **Calificación final** = ponderación según Norma Académica UTEQ de los componentes GA, TA y EV.
- **Aprobación**: ≥ 7.0 en escala 0–10.
- **Recuperación**: examen integral cubriendo las 4 unidades para estudiantes con calificación final entre 5.0 y 6.9.
- **Repetición**: estudiantes con calificación < 5.0 deben recurrar la asignatura el siguiente período.
- **Acta de notas**: se publica oficialmente 7 días hábiles después del cierre de la semana 18.

Bibliografía

- [1] Gustavo Alonso, Fabio Casati, Harumi Kuno, and Vijay Machiraju. *Web Services: Concepts, Architectures and Applications*. Springer, 2010.
- [2] Florian Auer, Valentina Lenarduzzi, Michael Felderer, and Davide Taibi. From monolithic systems to Microservices: An assessment framework. *Information and Software Technology*, 137:106600, 2021.
- [3] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4th edition, 2021.
- [4] Christian Bauer, Gavin King, and Gary Gregory. *Java Persistence with Hibernate*. Manning Publications, 2nd edition, 2015.
- [5] Tim Berners-Lee. Information management: A proposal. Technical report, CERN, Geneva, Switzerland, 1989.
- [6] Tim Berners-Lee. HTML tags — W3C original memo. Technical report, World Wide Web Consortium, 1991.
- [7] Tim Berners-Lee. Contract for the Web. World Wide Web Foundation, 2019.
- [8] Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 284(5):34–43, May 2001.
- [9] Simon Brown. *The C4 Model for Visualising Software Architecture*. Leanpub, 2023.
- [10] Vannevar Bush. As we may think. *The Atlantic Monthly*, 176(1):101–108, July 1945.
- [11] CERN. Información sobre el primer sitio web del mundo. CERN Archive, 1991.
- [12] Douglas Crockford. *JavaScript: The Good Parts*. O'Reilly Media, 2008.
- [13] Robert Daigneau. *Service Design Patterns: Fundamental Design Solutions for SOAP/WSDL and RESTful Web Services*. Addison-Wesley Professional, 2012.
- [14] Paul J. Deitel and Harvey Deitel. *Java How to Program: Early Objects*. Pearson, 11th edition, 2017.
- [15] Jon Duckett. *HTML and CSS: Design and Build Websites*. Wiley, 2011.
- [16] Ecma International. ECMA-262: ECMAScript 2024 language specification, 2024.
- [17] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [18] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. RFC 9110, Internet Engineering Task Force (IETF), 2022.
- [19] David Flanagan. *JavaScript: The Definitive Guide*. O'Reilly Media, 7th edition, 2020.
- [20] Neal Ford, Rebecca Parsons, and Patrick Kua. *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media, 2017.
- [21] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [22] Ben Frain. *Responsive Web Design with HTML5 and CSS: Develop future-proof responsive websites using the latest HTML5 and CSS techniques*. Packt Publishing, 4th edition, 2020.

- [23] Jon Galloway, Brad Wilson, K. Scott Allen, and David Matson. *Professional ASP.NET MVC 4*. Wrox, 2011.
- [24] Jesse James Garrett. *Ajax: A new approach to web applications*. Adaptive Path, 2005.
- [25] Ilya Grigorik. *High Performance Browser Networking: What Every Web Developer Should Know about Networking and Web Performance*. O'Reilly Media, 2013.
- [26] Shawn Lawton Henry. *Web accessibility initiative (WAI) resources*. World Wide Web Consortium, 2018.
- [27] Ian Hickson. *Web hypertext application technology working group (WHATWG) — founding manifesto*. WHATWG, 2004.
- [28] Ian Hickson, Robin Berjon, Steve Faulkner, Travis Leithead, Erika Doyle Navara, Edward O'Connor, and Silvia Pfeiffer. *HTML5 — W3C recommendation*. World Wide Web Consortium, 2014.
- [29] ISO/IEC. *ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuARE)*. International Organization for Standardization, 2011.
- [30] Steve Jobs. *Thoughts on Flash*. Apple Inc., 2010.
- [31] Michael Jones, John Bradley, and Nat Sakimura. *JSON web token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [32] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [33] Ralf T. Kreutzer and Marie Sirrenberg. *Understanding Artificial Intelligence: Fundamentals, Use Cases and Methods for a Corporate AI Journey*. Springer, 2nd edition, 2024.
- [34] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, 8th edition, 2021.
- [35] Bruce Lawson and Remy Sharp. *Introducing HTML5*. New Riders, 2nd edition, 2011.
- [36] Matthew MacDonald. *HTML5: The Missing Manual*. O'Reilly Media, 2nd edition, 2018.
- [37] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [38] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [39] Microsoft. *Inclusive design toolkit*. Microsoft Design, 2020.
- [40] Paul Mockapetris. *Domain names — concepts and facilities*. RFC 1034, Internet Engineering Task Force (IETF), 1987.
- [41] Paul Mockapetris. *Domain names — implementation and specification*. RFC 1035, Internet Engineering Task Force (IETF), 1987.
- [42] Theodor H. Nelson. *Complex information processing: a file structure for the complex, the changing and the indeterminate*. In *Proceedings of the 1965 20th National Conference*, pages 84–100. ACM, 1965.
- [43] Sam Newman. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.
- [44] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2nd edition, 2021.
- [45] Robin Nixon. *Learning PHP, MySQL and JavaScript: A Step-by-Step Guide to Creating Dynamic Websites*. O'Reilly Media, 6th edition, 2021.
- [46] Mark Nottingham and Erik Wilde. *Problem details for HTTP APIs*. RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [47] Tim O'Reilly. *What is Web 2.0: Design patterns and business models for the next generation of software*. O'Reilly Media, 2005.
- [48] Tim O'Reilly and John Battelle. *Web Squared: Web 2.0 Five Years On*. O'Reilly Media, 2009.
- [49] Jose Manuel Ortega. *Mastering Python for Networking and Security: Leverage the scripts and libraries of Python version 3.7 and beyond to overcome networking and security issues*. Packt Publishing, 2nd edition, 2022.
- [50] Joseph B. Ottinger, Jeff Linwood, and Dave Minter. *Beginning Hibernate: For Hibernate 5*. Apress, 4th edition, 2017.

- [51] Taylor Otwell. Laravel 11 official documentation. <https://laravel.com/docs/11.x>, 2024. Laravel LLC.
- [52] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks, 2021.
- [53] Claus Pahl, Antonio Brogi, Jacopo Soldani, and Pooyan Jamshidi. Cloud container technologies: A state-of-the-art review. *IEEE Transactions on Cloud Computing*, 7(3):677–692, 2019.
- [54] PostgreSQL Global Development Group. PostgreSQL 18 documentation. <https://www.postgresql.org/docs/18/>, 2025.
- [55] Dave Raggett, Arnaud Le Hors, and Ian Jacobs. HTML 4.01 specification — W3C recommendation. World Wide Web Consortium, 1999.
- [56] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force (IETF), 2018.
- [57] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020.
- [58] Leonard Richardson and Sam Ruby. *RESTful Web Services*. O'Reilly Media, 2007.
- [59] Jennifer Niederst Robbins. *Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics*. O'Reilly Media, 5th edition, 2018.
- [60] Khaled Salah, Coral Calero, and Manuel F. Bertoa. Performance evaluation of ORM frameworks in web applications. *International Journal of Software Engineering and Knowledge Engineering*, 27(8):1175–1196, 2017.
- [61] Leon Shklar and Rich Rosen. *Web Application Architecture: Principles, Protocols and Practices*. Wiley, 2nd edition, 2009.
- [62] Jon P. Smith. *Entity Framework Core in Action*. Manning Publications, 2018.
- [63] Laurentiu Spilcă. *Spring Start Here: Learn what you need and learn it well*. Manning Publications, 2024.
- [64] Andrew S. Tanenbaum, Nick Feamster, and David J. Wetherall. *Computer Networks*. Pearson, 6th edition, 2021.
- [65] José van Dijck. *The Culture of Connectivity: A Critical History of Social Media*. Oxford University Press, New York, 2013.
- [66] Verizon Business. Data breach investigations report (DBIR) 2025, 2025.
- [67] Craig Walls. *Spring in Action*. Manning Publications, 6th edition, 2022.
- [68] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [69] WHATWG and W3C. Memorandum of understanding between W3C and WHATWG on HTML and DOM specifications, 2019.
- [70] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Yellow paper, Ethereum Project, 2014.
- [71] World Wide Web Consortium. Markup validation service. W3C.
- [72] World Wide Web Consortium. Extensible markup language (XML) 1.0 — W3C recommendation, 1998.
- [73] World Wide Web Consortium. Web content accessibility guidelines (WCAG) 2.1 — W3C recommendation, 2018.
- [74] Jeffrey Zeldman and Ethan Marcotte. *Designing with Web Standards*. New Riders, 3rd edition, 2010.
- [75] Shoshana Zuboff. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs, New York, 2019.